# Basic Model

## Contents

# BasicModel

## Basic Model of Cognition

*The basic model of cognition relies on conceptual hyperplanes and perceptual prototypes to synthesize and analyze the input space. It uses a high-dimensional embedding to characterize mental space and integrates symbolic computation. Its three major operations are intersection (analysis: the top-down division of the presented unity into symbolic generalities), union (synthesis: the bottom-up aggregation of atoms into perceptual particulars), and equality (where symbols are elements that map across perceptual and conceptual domains). See Philosophy for the analytic/synthetic orientation.*

## Documentation

| Document | Description |
|---|---|
| Architecture | Pipeline design, layer types, invertible LDU factorisation |
| Runtime architecture | Live ownership map, non-modular mechanisms, and componentization sequence |
| Spaces | Runtime spaces: Input, Part/Perceptual, Modal, Conceptual, Whole/Symbolic, Output, and Symbo |
| STM | Short-term memory: shift/reduce slots, routing conditioning, ltmConsolidation |
| Ergodic | Gradient energy sensor, adaptive exploration, factor-level noise injection |
| Training | Two-phase training, SBOW embeddings, masked prediction modes |
| Params | XML configuration reference and migration notes |
| BasicModel | Cognitive science foundations |
| Language | Grammar layers, signal routing, category codebook, and syntax output |
| Mereology | Parthood as the fundamental operation, the five mereological relations, ImpenetrableLayer |
| Logic | Subsymbolic and symbolic logic operations |
| Reasoning | Truth methods, bidirectional reasoning, contemplative awareness stubs |
| Philosophy | Kant's analysis/synthesis, Ramsey's theoretical roles, Buddhist epistemology (pramana, tetralemma |
| MachineMinds | What machine minds are, feel, and know |
| Lexicon | Word-vector store: surface forms, codebook binding, LBG splitting |
| SymbolFirewall | Governing principle: the boundary between subsymbolic and symbolic computation |
| Installation | Setup, Makefile targets, environment variables |

## Overview

BasicModel is a parameterized neural architecture that answers the question "what is the truest thing we can say of this experience?".

Model configurations are specified in XML. See doc/Architecture.md for the full mathematical treatment.

## Files

| File | Description |
|---|---|
| bin/train.py | Two-phase training orchestrator: embeddings (Phase 1) then model (Phase 2), local or remote |
| bin/Models.py | Main entry point: model factory, training loop, `--compare`, optional `--report` |
| bin/Layers.py | Layer library: SigmaLayer, PiLayer, ErgodicLayer, LinearLayer, TruthLayer |
| bin/Spaces.py | Space classes: InputSpace, PartSpace, ModalSpace, ConceptualSpace, WholeSpace, OutputSpa |

| File | Description |
|---|---|
| bin/Language.py | Grammar layers and SymbolSpace: parsing, composition, category codebook |
| bin/embed.py | Word vector training: CBOW/SBOW with negative sampling, `WordVectors` (gensim-compatib |
| bin/serve.py | HTTP server: OpenAI-compatible `/chat/completions` endpoint for WikiOracle integration |
| data/ | XML model configurations |
| doc/Architecture.md | Algorithm details: Sigma/Pi layers, ergodic exploration, gradient energy sensor |
| doc/Componentization.md | Software ownership, consolidation targets, and extraction gates |
| doc/Params.md | Full XML parameter reference |
| doc/Training.md | Embedding pretraining, CBOW/SBOW, masked prediction, `<trainEmbedding>` modes |
| doc/Installation.md | Setup, Makefile targets, train.py options, remote training |
| test/ | Unit tests |

## Quick Start

```
# Set up virtual environment
make install

# Run a single model
make simple          # data/simple.xml
make ergodic         # data/ergodic.xml

# Compare two models side-by-side
make compare         # defaults: data/simple.xml vs data/ergodic-only.xml
make compare XML1=data/simple.xml XML2=data/ergodic.xml

# Run tests
make test

# Generate PDF documentation
make doc
```

## XML Configuration

Models are configured via XML files in `data/`. Training and data parameters live in nested sub-elements:

```
<model>
  <architecture>
    <ergodic>false</ergodic>
    <weightsPath>output/BasicModel.ckpt</weightsPath>

    <data>
      <dataset>xor</dataset>        <!-- xor | phrases | mnist | tomatoes | text | inline -->
      <dataType>numeric</dataType>  <!-- embedding | numeric -->
    </data>

    <training>
      <numTrials>1</numTrials>
      <numEpochs>20</numEpochs>
      <batchSize>10</batchSize>
      <learningRate>0.001</learningRate>
      <certainty>false</certainty>
      <autoload>true</autoload>
      <autosave>false</autosave>
```

```
    </training>
  </architecture>

  <InputSpace> ... </InputSpace>
  <PartSpace> ... </PartSpace>
  <ConceptualSpace> ... </ConceptualSpace>
  <WholeSpace> ... </WholeSpace>
  <OutputSpace> ... </OutputSpace>
</model>
```

See doc/Params.md for the full parameter reference.

### Output

Runs produce an HTML report only when `Models.py` is invoked with `--report` (the `make compare` target does this). Reports are timestamped under `output/` and may include:

- **Error per Epoch** – training and test loss curves
- **Accuracy per Digit** – per-class breakdown for digit/classification configs

In compare mode, overlay plots show combined loss and accuracy across models when report generation is enabled.

## Installation and Usage

### Prerequisites

- **Python 3.12** with a virtual environment at `.venv/`
- **PyTorch** with MPS (Apple Silicon) or CUDA
- **pandoc** (optional, for PDF generation via `make doc`)

### Conceptual Orientation

The installable artifact is not only an LLM training stack. It runs BasicModel's explicit architecture: LLM-like prediction/reconstruction objectives, Formal Concept Analysis-like part/whole concept order, and DisCoCat-like typed grammar composition. The generated PDF includes the detailed explanation in Architecture.md, Mereology.md, and Language.md.

### Setup

```
make install   # creates .venv with dependencies
```

Makefile targets invoke the project virtual environment (`.venv/bin/python` on Unix, `.venv/Scripts/python.exe` on Windows), so no manual activation is required.

---

### Makefile Targets

#### Training

| Target | Description |
|---|---|
| `make train` | Full training (Phase 1 embeddings + Phase 2 model), logged to `output/logs/`; pins `--data text` |
| `make train_micro` | Smoke run: max 1,000 docs, one random shard, ten batches, logged; also pins `--data text` and ca |
| `make preflight` | Fast FineWeb launch/config/data/checkpoint gates |
| `make preflight_full` | Preflight plus one real N=64 optimizer step |

**Models**

| Target | Description |
|---|---|
| `make run` | Run `Models.py` with the config in `XML1` |
| `make xor` | `data/MM_xor.xml` |
| `make simple` | `data/simple.xml` |
| `make ergodic` | `data/ergodic.xml` |
| `make tomatoes` | `data/tomatoes.xml` |
| `make mnist` | `data/mnist.xml` |
| `make compare` | Side-by-side compare using `XML1` and `XML2`; writes an HTML report |
| `make SigmaPi` / `SymPercept` / `SPNN` | Run the legacy demos under `bin/etc/` |

**Testing and Documentation**

| Target | Description |
|---|---|
| `make test` | Unit tests (forces `BASICMODEL_DEVICE=cpu`); the canonical, deterministic gate |
| `make testp` | Same suite, parallel via pytest-xdist (`TEST_JOBS`); faster iteration |
| `make test_all` | Same suite plus slow-gated tests (`RUN_SLOW=1`) |
| `make bench` | Training benchmarks (baseline) |
| `make bench_local` | Reconstruction fidelity/timing bench, local (cpu+eager for reproducibility) |
| `make bench_sync` | rsync the working tree to the ArborStudio remote (excludes venvs, caches, large regenerable data) |
| `make bench_remote` | `bench_sync` then run the reconstruction bench on ArborStudio's native path |
| `make bench_pull` | rsync bench result JSON/profile files back from ArborStudio to `output/` |
| `make doc` | Generate `BasicModel.pdf` via pandoc |

PDF chapters in order: `README.md`, `doc/Installation.md`, `doc/Architecture.md`, `doc/Componentization.md`, `doc/BasicModel.md`, `doc/Spaces.md`, `doc/STM.md`, `doc/Language.md`, `doc/Mereology.md`, `doc/Logic.md`, `doc/Reasoning.md`, `doc/Training.md`, `doc/Ergodic.md`, `doc/MachineMinds.md`, `doc/Params.md`.

`make bench_local` / `bench_remote` use `data/MM_20M_grammar.xml` (a compact grammar fixture) rather than the FineWeb default. Bench variables:

| Variable | Default | Description |
|---|---|---|
| `EPOCHS` | `3` | Epoch count passed to `recon_bench.py` |
| `REMOTE_COMPILE` | `auto` | `MODEL_COMPILE` value used on `bench_remote` |
| `ARBOR_USER` | `arogers` | SSH user for the ArborStudio remote |
| `ARBOR_HOST` | `ArborStudio.local` | ArborStudio hostname (LAN) |
| `ARBOR_KEY` | `~/.ssh/id_ed25519_arborstudio` | SSH key for the remote |
| `ARBOR_DEST` | `~/WikiOracle/basicmodel` | Remote working directory |
| `ARBOR_TUNNEL` | `0` | Set to `1` to route off-LAN via the wikiOracle.org reverse tunnel ins |

**Utilities**

| Target | Description |
|---|---|
| `make clean` | Remove generated files (`BasicModel.pdf` and `output/*`) |
| `make all` | Default; alias for `make xor` |

## Makefile Variables

Override on the command line, e.g. `make run XML1=data/ergodic.xml`.

| Variable | Default | Description |
|---|---|---|
| `MODEL` | `data/MM_20M_fineweb.xml` | XML config for training |
| `XML1` | `data/simple.xml` | Primary config for `make run` / `compare` |
| `XML2` | `data/ergodic-only.xml` | Secondary config for `make compare` |

---

## train.py Options

`train.py` orchestrates Phase 1 (embeddings) followed by Phase 2 (model training).

### General

| Flag | Default | Description |
|---|---|---|
| `--model`, `-m` | `data/MM_20M_fineweb.xml` | XML config |
| `--data` | *(from XML)* | Override dataset/data source selector |
| `--max-docs` | *(from XML)* | Override `maxDocs` |
| `--num-shards` | *(from XML)* | Override `numShards` |
| `--num-epochs` | *(from XML)* | Override training epochs |
| `--batch-size` | *(from XML)* | Override `batchSize`; forwarded as `BASIC_BATCH_SIZE` |
| `--max-tokens` | *(from XML)* | Override token budget |
| `--batches` | *(from XML)* | Cap Phase 2 batches |
| `--test [N]` | off | Run evaluation passes; optional `N` caps test batches |
| `--compile-target` | `gpu` | `gpu` runs normal training with accelerator torch.compile defaults; `m` |
| `--compile-mode` | *(env/runtime default)* | Forwarded as `MODEL_COMPILE_MODE` |
| `--mlx-output` | `output/mlx/<model>.pte` | Destination for `--compile-target mlx` |
| `--random-shards` | off | Pick random shard indices |
| `--force-embeddings` | off | Retrain embeddings even if file exists |
| `--latent-vector-size` | *(model/vector size)* | Phase 1 SBOW latent size before optional projection |
| `--embed-lr` | *(embed default)* | Phase 1 SBOW learning rate |
| `--log [FILE]` | off | Capture stdout/stderr to `.log` in `output/logs/`. Omit filename fo |
| `--profile` | off | Run Phase 2 under `cProfile`; writes `.prof` to `output/profiles/` |

### Remote Execution (SSH)

| Flag | Default | Description |
|---|---|---|
| `--host` | *(none)* | Remote host; enables remote run |
| `--user` | `arogers` | SSH user |
| `--key-file` | *(none)* | Optional SSH key file; uses SSH config/agent when omitted |
| `--remote-dir` | `~/WikiOracle/basicmodel` | Working directory on remote |

---

## Environment Variables

| Variable | Description |
|---|---|
| BASIC_DATASET | Override dataset/data source selector |
| BASIC_MAX_DOCS | Override maxDocs |
| BASIC_NUM_SHARDS | Override numShards |
| BASIC_NUM_EPOCHS | Override numEpochs |
| BASIC_BATCH_SIZE | Override batchSize |
| BASIC_MAX_TOKENS | Override token budget |
| BASIC_MAX_BATCHES | Cap Phase 2 batches |
| BASIC_RANDOM_SHARDS | Set to 1 to pick random shard indices |
| BASIC_RUN_TEST | Request evaluation passes; optional value caps test batches |
| BASICMODEL_DEVICE | Force device, e.g. cpu. Used by make test and the bench_* targets |
| BASICMODEL_PYTHON | Python executable for train.py subprocesses; overrides the in-project .venv lookup |
| MODEL_COMPILE | torch.compile backend selector (auto, none, inductor, eager, aot_eager) |
| MODEL_COMPILE_MODE | torch.compile mode (default, reduce-overhead, max-autotune, max-autotune-n |
| PYTORCH_MPS_HIGH_WATERMARK_RATIO | MPS memory limit; set to 0.0 by train.py |
| BASICMODEL_MPS_IOBUF | MPS inductor fusion cap (max_fusion_unique_io_buffers, default 24) — keeps f |
| BASICMODEL_MPS_FUSE | Optional MPS inductor max_fusion_size (nodes per fusion); unset leaves the torc |
| PYTHONUNBUFFERED | Set to 1 by train.py for real-time log streaming |
| RUN_SLOW | Set to 1 to also run slow-gated (>30s) tests; used by make test_all |
| TEST_JOBS | pytest-xdist worker count for make testp (e.g. auto) |

Additional env vars read directly by Models.py/util.py (not surfaced as train.py flags): BASIC_SEED (overrides the training seed), BASIC_CHECKPOINT_EVERY_BATCHES (checkpoint frequency in batches, default off), BASIC_PROFILE (enable cProfile around Phase 2, also set by train.py --profile), MODEL_AMP (autocast mode: off/fp16/bf16), MODEL_DEBUG (gates verbose debug asserts/stat prints).

---

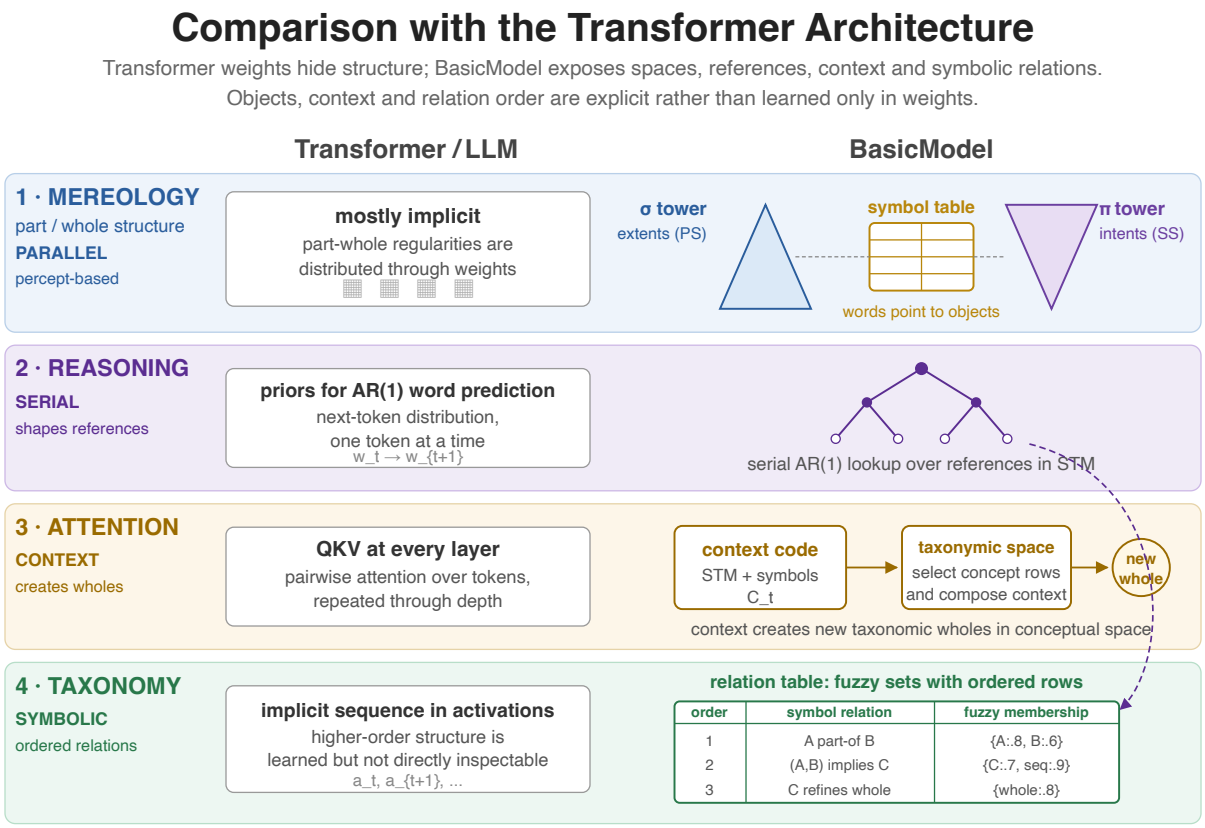
### Private Remote Training

There is no Makefile.local include mechanism today: the bench_local/bench_sync/ bench_remote/bench_pull targets hardcode one machine-specific SSH target (ARBOR_HOST/ARBOR_USER/ARBOR_KEY/ARBOR_DEST, all overridable on the command line — see the Bench Variables table above) directly in the Makefile. The public train.py --host mode remains available for generic SSH execution when a caller supplies the host, user, optional key, and remote directory explicitly.

## Architecture

This document describes the cognitive and mathematical architecture. For the live software ownership map—including configuration, training, STM, losses, checkpoints, reconstruction, and proposed consolidation boundaries—see Runtime Architecture and Componentization.

## The three pieces: Mereology, Attention, Thought (2026-06-11)




**Terminology (2026-06-21 convention).** One noun per-space: a **percept** is a perceptual thing (PartSpace/WholeSpace, dimensionally-embedded, extensional; *part* and *whole* are its two subtypes); a **concept** is a ConceptualSpace relation tying one part-percept to one whole-percept (the Concept codebook); a **symbol** is a SymbolSpace 0-D reference to a concept. The CS "symbol table" is therefore the **Concept codebook** below.

The architecture decomposes into three pieces; the first two run in parallel, the third is serial:

1. **The mereological towers (and the Concept codebook).** In an LLM the mereology is completely subsymbolic — implicit in the weights, never surfaced. In BasicModel it is percept-based and symbolic: two towers — the $\sigma$ tower ascending bottom-up (part-percept extents, the PS codebook) and the $\pi$ tower descending top-down (whole-percept intents, the SS codebook) — linked by the word/object Concept codebook (`bin/References.py`), whose rows are the concepts (part↔whole references).
2. **Attention.** In an LLM, attention is QKV per subsymbolic layer. In BasicModel: **bases of relevance guide attention; attention determines the contents of awareness.** Relevance is carried as weights on the percepts; ATTENTION is the single selection process (at CS) that reads the integrated priority out; AWARENESS is what the selection admits. (The priority-map synthesis — Fecteau & Munoz 2006; Bisley & Goldberg 2010 — with attention as the readout, never the sources; attention distinct from awareness per Koch & Tsuchiya 2007.) Relevance factors as **ORIGIN** × **AXIS**, realized as **ONE QUADRATIC PRIMING SURFACE per space with two write channels** (the simplified law): **bottom-up things are primed in virtue of BEING SEEN** (perception itself writes the surface: fired rows bump, the surface decays toward neutral) and **top-down things are primed in virtue of being DESIRED or HATED** (signed intent: desire boosts, hate suppresses with floor 0 — suppression, never a veto). "Top-down" is therefore a DIRECTION of writes on the one surface, not a second mechanism — matching biased competition (Desimone & Duncan: the template

IS conceptual content) while keeping the two channels' automatic/strategic dissociation distinct (§C). `readingAttention` is HARD-CODED over the same surface: the reading scope is the span of the hottest-primed word-whole. Two AXES, orthogonal to origin ("Parse time" §C): HORIZONTAL (which parts within the level) and VERTICAL (which properties — fixing Rosch's Basic Level and, through it, which objects exist at all).

3. **Thought.** In an LLM, thought is the computation of priors for autoregressive word prediction. In BasicModel, thought is a **subsequent isolation of attention over that space, enabled by references** — the serial pass: referential lookup, shift/reduce composition, story selection. Because thought is the only process that invokes the referential taxonomy *qua references*, it is the only process licensed to **shape the references**.

This yields the codebook update law (GrammarOpsPass §6d, implemented): **percepts are shaped by the parallel pass; references are shaped by the serial pass.** STE is untouched in both modes; the partition governs row updates — parallel mode may not shape references, serial mode may not shape non-references (`Spaces.reference_update_mask`; the `update_mask_fn` chokepoint on `VectorQuantize`). Rational by construction — though possibly an overly optimistic arrangement with respect to human thinking.

See SymbolFirewall.md for the governing principle this update law (and the codebook/meronomy ownership model generally) instances: all computation is composed over typed, symbol-attached units — read/write masks, no anonymous global residual stream.

## Relation to LLMs, Formal Concept Analysis, and DisCoCat

BasicModel is best read as an explicit decomposition of functions that a transformer LLM usually folds into attention heads, feed-forward blocks, and an unrestricted residual stream. The LLM comparison in this document is therefore architectural, not merely benchmark-oriented:

- **LLMs:** a conventional LLM learns fluent priors over token sequences with latent attention and hidden residual state. BasicModel keeps the language-model goal of prediction and generation, but separates perception, concept formation, attention, grammar, truth, and reconstruction into named stores and typed operations. The wager is that some behavior now implicit in LLM weights should become inspectable state.
- **Formal Concept Analysis:** the PartSpace/WholeSpace towers and Concept codebook form a neural, fuzzy analogue of a formal context. Part-percepts play the role of objects or extents, whole-percepts play the role of attributes or intents, and concepts are the order-bearing links between them. The resulting meronymic structure is not a classical binary FCA lattice, because support and trust are graded and trainable, but it uses the same extent/intension discipline as its organizing constraint.
- **DisCoCat:** the grammar path is a DisCoCat-like composition engine: typed grammatical reductions decide how word/object meanings compose into sentence meanings in vector space. BasicModel differs from standard categorical compositional distributional semantics by making the reductions bidirectional, tying them to part/whole codebooks, and feeding their results into truth, reconstruction, and memory.

In short: LLMs are the operational baseline, Formal Concept Analysis supplies the lattice and extension/intension reading of concepts, and DisCoCat supplies the grammar-to-vector-composition reading of sentence meaning.

### Addressable attention — the typed `.where`

Global attention (`GlobalAttention`, `bin/Spaces.py`; gated `<globalAttention>`) ranges over a **typed addressable space**: one distribution competes across every store at once and emits a typed `.where` = `(space-id, bracket)` plus a soft-read $\sum_k \alpha_k \cdot \text{key}_k$. Six stores (the `SPACE_*` ids):

| id | store |
|---|---|
| INPUT | the staged input window (per-span percept content) |
| STM | the live short-term-memory rows |
| LTM | the consolidated truth store (rows + trust value) |
| PART | the PartSpace codebook (part-percepts) |
| WHOLE | the WholeSpace codebook (whole-percepts + meronomy/taxonomy) |
| SYMBOL | the SymbolSpace codebook (symbols, 1:1 with concepts) |

PART/WHOLE appear whenever their tower has a codebook; SYMBOL only under <symbolTower>. Pointing .where at a codebook/LTM store is recall; at the input window it is reading — one mechanism, the type tag distinguishes them. Under <globalAttentionConsume> the soft-read is fed back into the head as a zero-init gated residual, so the output loss trains the retrieval.

The symbol codebook is a **reference**, not a learned copy: it tracks the concept codes so the two cannot diverge or dissociate. Its training objective is kept separate from reconstruction (the source concepts are shaped by their own pass): the symbol's IDENTITY (the codebook row) stays EMA-only/detached, while the symbol's VALUE (the signed 0-D activation from the symbolic phase) is grad-bearing. (Two-phase update, 2026-07-02: the activations' gradient path is the conceptual SBOW over the settled slab parked at the post-pump cutover – the once-built SS leg itself is a state-contract sync whose product no loss consumes; pre-P3 the in-loop leg carried the gradient.)

These six stores are the substrate for the four foundations of mindfulness; the mapping (and the trust-sign-as-vedana / luminosity-as-joy reading of LTM) is in Philosophy.md.

**Status (2026-05-29 update):** further architectural pivots landed on top of the 2026-05-27 substrate refactor:

- **PerceptStore → RadixLayer.** The radix-trie input encoder is now a first-class Layer subclass in bin/Layers.py; PartSpace.reverse invokes RadixLayer.reverse for the structural decode.
- **MetaLayer → SymbolizeLayer.** The binary GrammarLayer that promotes a freshly-seen percept to a symbolic prototype is now SymbolizeLayer; no semantic change.
- **Auto-META moves PS → CS.** The cross-codebook bind (META entry: PS chunk-id ↔ SS prototype-id) fires from ConceptualSpace._maybe_autobind_meta at stage 0; PS no longer holds a back-ref to SS.
- **Clean-stack STM.** ConceptualSpace.forward bypasses sigma_in / sigma_cs on forward — folded = primary at stage 0, folded = sym at $k > 0$. The Stage-10 additive composition is retired; per-stage space-role attribution is trivially invertible.
- **basis= kwarg for grammar reverses.** UnionLayer.reverse(parent, basis=None) / IntersectionLayer.reverse(parent, basis=None) accept a Codebook / Basis object (typically WholeSpace.subspace.what) instead of a raw W tensor; bin/Language.py::unreduce() dispatches accordingly.
- **LSE soft-max kernels.** Ops._disjunction_kernel / Ops._conjunction_kernel default to LogSumExp smooth variants when monotonic=False; the hard branch is retained for monotonic-mode and exact idempotency tests.
- **LBG-style SS codebook splitting.** Gray (1990) EMA + per-row variance tracking; rows whose running variance exceeds a threshold split along the top-variance eigendirection.

**Status (2026-05-27):** the **substrate refactor** has landed end-to-end. PS is a single-arg input processor (synthesis front end + sigma fold). CS is a STM container + grammatical CPU (no atomic forward fold; sigma_percept retired). SS owns the unified word lexicon codebook with paired (orth, semantic) rows. The CKY Chart and STM shift-reduce parsers retire entirely; LanguageLayer (signal router) is the canonical parser. LiftLayer / LowerLayer are binary GrammarLayer subclasses with internal Sigma / Pi (no longer borrowing substrate folds). GrammarLayer gains an optional butterfly cascade mode

for cross-position mixing — closes the XOR convergence target. Two operating modes selected by `<serial>`: **SERIAL/GRAMMATICAL** (per-word PS with grammar dispatch over STM) and **PARALLEL** (T = `<subsymbolicOrder>` iterations of PS over CS). The `<parserBackend>`, `<routerKind>`, `<chartTau>`, `<chartTopK>`, `<chartNoiseEps>` XML knobs are retired; `<symbolicOrder>` is now the symbolic / relational loop budget.

## Symbolic weights, reconstruction, parse-time, attention (2026-06-30)

A design pass clarifying four coupled pieces (PS = PartSpace, WS = WholeSpace, CS = ConceptualSpace, SS = SymbolSpace).

### A. Symbolic weights (the two-phase forward; reworked 2026-07-02, forward composition superseded 2026-07-10)

**Implemented 2026-07-02**, superseding the forward-transform parts of the prior sparse-layer conceptual-embedding design, as a dedicated `SparseLayer` (`bin/Layers.py`), NOT a SigmaLayer option: SigmaLayer's atanh-entry contract expects logit-domain codes in $[-1, 1]$, while these maps consume *presences/activations* – a different input domain deserves its own class. The substrate contract is the transpose autoencoder pair: forward = $\tanh(Wx)$, reverse = $\tanh(W^T y)$; export-safe scatter-add kernel by default, `torch.sparse.mm` opt-in. Edges append host-side at mint (`add_edge`, idempotent, tail-preserving value growth) and are removed by pruning rounds (`remove_edges`; `ConceptualSpace.prune_concept_links` keeps the closest links using within-tower relations only). The iterated-loop rework (v3, landed 2026-07-03) kept this substrate and collapsed the per-order families into ONE square untyped `ConceptualAttentionLayer` subclass; its *forward composition* was in turn superseded by the 2026-07-10 dual-towers rev-2 feedforward sigma-pyramid (below) – the `SparseLayer` substrate itself (edges, COO forward/reverse, `add_edge` / `remove_edges`) is unchanged by either rework.

**The forward is TWO PHASES with one terminal cutover.** Phase A – the purely continuous PS/WS↔CS pump: `subsymbolicOrder` iterations of the 2-stream bind (no symbol leg in the loop, no quantization inside the pump). The C→P / C→S handoffs carry the per-tower WINDOWS of the MIXED carrier (`combine.views`, the demux feedback): the part-stream returns to PS for further $\sigma$ synthesis, the whole-stream to WS for further $\pi$ analysis – the mix goes UP (the next stage's contribution), the un-mix goes DOWN. Phase B – ONE late cutover at the bandwidth seam (`cs_symbolic_phase`): the settled field is SNAPPED to the ORDER-0 block of the conceptual codebook (`cs_snap_order0`: differentiable normalized-sum presence – slot-mean projection onto the unit atom direction in hypercube-diagonal $\sqrt{D}$ units, magnitude-preserving, NOT cosine – with an EMA identity trace of the winning rows while training), then the concept pyramid runs ($K$ = `symbolicOrder` rungs over the `ConceptualAttentionLayer`, below) and its outputs feed the SS leg, the head-side losses (conceptual SBOW on the settled slab), and the concept table – they are NEVER substituted back into the subsymbolic carrier (the `<sparseReplace>` knob is retired; non-replacement is structural). Quantization sits exactly at the seam because 0-D symbols lack the bandwidth to carry subsymbolic content.

**The `ConceptualAttentionLayer` is SYMBOLIC-ONLY and owns BOTH readings of concept structure (dual-towers rev 2 – landed 2026-07-12).** The weighted reading is ONE square untyped $[N \times N{+}1]$ store over the stacked concept inventory (`ConceptualAttentionLayer`, bin/Layers.py) – named for what it IS, bottom-up attention over the concept inventory (see "Parse time" sec C); `SparseLayer` is how it works (it subclasses the substrate above). The per-order role-split families and their dyadic capacities are RETIRED. Edges are UNTYPED fuzzy set-membership degrees: population at mint writes one edge per SYMBOLIC constituent (row = the relation, col = the constituent; raw-code refs stay record-only), plus a trailing-bias-column edge (col $N$) for relations bounded above by the EVERYTHING pole (bias-bounded chain links, un-refined asserted concepts; a concrete whole retires it). Self-edges raise (the Quine atom $x = \{x\}$); longer cycles are deliberate – a documented fact of un-ramsified taxonomies and of human minds, which nothing in the current codebase observes or damps at runtime (the diagnostic that used to watch them, `cs_groundedness_probe`, is retired – see below).

The forward is a FEEDFORWARD SIGMA-PYRAMID (`cs_forward_content`), ONE hop per order rung,

NOT a settling recurrence: $a^0$ is the order-0 snap presence (`cs_snap_order0`) padded to the inventory; for each rung $k = 1..K$ ($K$ = `symbolicOrder`, the pyramid-depth budget – the maximum possible conceptual order, not a forced ramsification, so a depth-$d$ vine simply completes at rung $d$), one feedforward hop $\tanh(W[a \mid 1])$ is gathered onto that rung's `order_slice(k)` rows, then admits only the per-batch top-`caps[k]` winners by $|\cdot|$ magnitude (`_order_caps`; optionally rank-boosted by `_relevance_priority` spreading through $|W|$, sec C) – non-admitted rows read 0 and carry nothing to the next rung. No fixed point, no re-injection: each rung is a strict feedforward pass over the PREVIOUS rung's admitted rows only. `_cs_wave_qe` is retired (set `None` every call, `# wave retired`); there is no settling residual left to report.

`_order_caps()` sizes the per-rung taper. While the sparse concept transform is active (`_sparse_active`: `symbolicOrder > 0` in parallel mode), it is a tile-based taper `[base, base>>1, .., 1]` ($K$+1 entries, `base = min(outputShape[0], nVectors)`, shrunk until the taper fits the inventory) – inventory rows past `sum(caps)` stay inert. Off that path, the caps fall back to the pre-rev-2 `(n_snap, n_pool)` 50/50 split (`n_snap = max(1, N // 2)`), a fossil of the superseded design below.

> **Historical note (forward composition superseded 2026-07-10).** The v3 iterated-symbolic-loop design (landed 2026-07-03) ran the store as an ITERATED WAVE with an additive source term every step, $a^{i+1} = \tanh(W[a^i \mid 1] + s)$, $i = 0..K-1$, over a row space split 50/50 into a SNAP block (rows $[0, n_{\text{snap}})$, order-0 concepts, no in-edges) and a RELATION POOL (the rest, minted relations, first-come with a loud overflow warning). A fresh probe on `MM_sparse_concept` found the wave DARK end-to-end – sign-then-clamp annihilation at the order-0 rectifier, scale-blindness between the settled field and the codebook, and a capacity gap where `<nVectors>` never reached the per-stage store (the dual-towers rev-2 design's "Why"). Rev 2's correction: the concept base is the 8-tile corpus-callosum frame, not the codebook inventory, so attention is a top-K taper over that frame rather than a settling recurrence – replacing the wave with the feedforward pyramid above closes the darkness (no trivial fixed point left to go dark). The `(n_snap, n_pool)` split above is the only surviving trace of the v3 design, kept as the `_order_caps` off-path fallback.

Alongside this weighted reading, the store holds the DISCRETE relation table: ordered role-tagged constituent records (`embed_pair` stores the sec-4c ordered pair [whole, part], whole first: whole $\Rightarrow$ part / if $\rightarrow$ then; `discretize_row` is exact on the binary-ordered subset). A thin shared `ConceptAllocator` owns global concept ids, order derivation (bookkeeping only under v3 – nothing migrates by order), the raise/retire/singleton sets, and the idempotency caches; ConceptualSpace keeps orchestration only. The signed bounded activation $a$ IS the 0-D symbol: the once-built SS leg is $a\times$ the row-aligned identity row (codebook rows stay EMA-only; the leg syncs the SS state contract, while the activations' GRADIENT path is the conceptual SBOW over the settled slab parked at the cutover). Per-pass $\sigma/\pi$ stacks (canonical, always built; `<subsymbolicNoop>` marks identity slots) give the pump DISTINCT layers per pass – depth IS mereological order – and the per-percept snap-residual (`snap_settle_qe`) is read as a report-only SETTLE SIGNAL for later adaptive work.

**Relation-table entry contract** At the sparse-entry level, one entry binds one concept row index to one symbol column index, plus its signed membership weight. A concept definition is therefore not limited to one symbol: the same concept index may occur in multiple entries, each paired with a different symbol index. Those repeated entries collectively form the concept's sparse, set-like definition; different concept indices define different concepts.

The same representation is sufficient for a vine, with an important qualification: one entry alone does not encode a total order. Each vine link is the ordered pair [whole = current, part = rest], where `rest` references the next relation concept. The discrete role-tagged records preserve the whole/part distinction exactly, while recursive nesting and wave iteration make the sequence order operational. In short: repeated entries for one concept define a set; recursively nested relation concepts define a vine.

**Groundedness and cycles – REMOVED (2026-07-10).** `cs_groundedness_probe` does not exist in the current codebase (zero grep hits). It was the KRIPKE grounded/ungrounded reading of the v3 iterated wave, in two runs (source-driven, then source-released) – a diagnostic over settling dynamics that the feedforward

pyramid does not have. The design pass that replaced the wave accepted the loss outright ("the wave was dark anyway," per the rev-2 design's "Theory"). The posture on cycles it used to report stands as a design statement even without the probe: loops are a documented FACT – of un-ramsified taxonomies and of human minds – and nothing in the current codebase observes or damps them at runtime. Solutions are invited – for both human and machine minds.

The lattice poles are VECTORS of the presence domain: **NOTHING** = $[0, 0, ...]$ (bottom; a part contributing $W \cdot 0 = 0$ – no edge) and **EVERYTHING** = $[1, 1, ...]$ (top; a whole realized as the trailing bias column). Order-0 concepts (word A-symbols, the span knit, fresh pole-pair objects) RESERVE their order-0 codebook row – the snap reads them; their part/whole decomposition lives in the PS/WS codebooks plus the ordered reference store (store by reference, never duplicate codes). The word≡object META is the sec-4c ORDERED PAIR [whole = word-symbol, part = object-symbol] – roles are positional slots of an ordered pair, not containment claims; the typed read-out (`meta_word_object`) recovers (word, object) by INTERSECTING the pair with the word-symbol class rather than trusting slot order. The JOINT/sentence concept (`create_joint_concept`) is the ordered Gallistel CHAIN over the row's word-symbols – each link the pair [whole = current, part = rest], bias-bounded – one head per sentence TYPE, so word order distinguishes sentence types. A 1:1 tie between SYM refs is the SINGLETON principle (Alec 2026-07-02): the unit-set $\{x\}$ – a whole containing exactly one symbolic part (`singleton_concept`, min-support exempt) – is the constructive primitive behind if→then ($\{x\} \Rightarrow x$) and the recursion vine, and it is stored structure that `resolve_identities` never collapses (only ties between concrete raw codes resolve away). Sequencing depth under strict ramsification TRUNCATED the chain's weighted reading (same-order link references were dropped at the order cap) – the defect that motivated the successor design (the iterated-symbolic-loop, landed 2026-07-03), which FIXES it: iteration over the one untyped square `ConceptualAttentionLayer` replaces stratification, so a link of any order simply arrives one rung later (originally a wave hop, now a feedforward pyramid rung – the 2026-07-10 historical note above); see "Parse time" sec C.

### B. Reconstruction (parts → `.what`, wholes → `.where`)

InputSpace maps two views of the *same* data, segmented differently: a **universal view** to WS (which it analyses) and an **atomic view** to PS (which it synthesises). WS yields **low-fidelity** information covering the **whole** space; PS yields **high-fidelity** information over a **smaller** area (`_paint_reconstruction`: the universal view paints the background, the atomic view is averaged in where it has support). For verbal reconstruction of text the **parts (PS) should reconstruct the `.what`** and the **wholes (WS) should reconstruct the `.where`** — approximately the inverse of what happens at parse time. The separate `what_scale` / `where_scale` / `when_scale` reconstruction channels already exist to carry this.

For text, it would be foolish to insist on an *absolute* `.where` from WS: the parts already know each word's size, so under a perfect tiling the placement is just the running sum of part sizes (serial mode computes this as an AR1 increment over the previous `.where`; the for-loop is time). WS may still supply **type** information for the tiling — *word*, *space*, *word*, *punct*, … — even where it does not supply coordinates.

#### Tiling, subspace sizes, and consciousness

Whether the parts/wholes **perfectly tile** the input (a partition — so order alone reconstructs, with no gaps between parts) is **NOT** entailed by parallel-vs-serial mode. It is entailed by the **relative sizes of the InputSpace subspace and the PS/WS subspaces**:

- In **serial mode** they are *forced* to match — the for-loop traverses **all** of input space — so the tiling is always perfect.
- In **parallel mode**, if they do not match because InputSpace is *larger* than PS/WS, that bounded mismatch is exactly where the two attentions **select what is most relevant to consciousness**: PS/WS cannot hold all of InputSpace, so attention picks the salient subset to surface. (When InputSpace $\leq$ PS/WS — as in the current test fixtures — the tiling is again a partition and order suffices.)

**C. Parse time (relevance = origin × axis)**

At parse time CS receives the overcomplete input representations from the PS and WS mereological towers. **Bases of relevance — carried as weights on the percepts — integrate into a priority signal; attention is the selection at CS that reads it out; awareness is what the selection admits.** Relevance factors as ORIGIN × AXIS: two origins (perceptual salience/novelty; symbolic history) crossed with two axes (horizontal: which parts; vertical: which properties/level).

- **Salience/novelty, HORIZONTAL (significant particles, PS)** — within the level the vertical axis fixed, WHICH items? The level may fix the word boundary of *wheelhouse*, but what makes *wheel* + *house* its building blocks rather than the equally-segmentable *wheelhou* + *se*? **Greedy longest-match** is the current easy approximation (`RadixLayer.longest_match`); particle salience should guide both parsing and reconstruction here. Under the simplified law the signal is SEEN-priming: rows that fire are primed by being perceived (bump + exponential decay toward neutral) — presence primes; no separate novelty computation.
- **Salience/novelty, VERTICAL (significant properties, WS)** — a property can be stimulus-significant too (a novel type-run, an unexpected region); property salience weights the wholes and thereby participates in fixing the **Basic Level** of analysis (E. Rosch): the size of parts and wholes. The scope handoff is the vertical axis's EFFECT ON PERCEPTION: the word-level isolates the regions that are type *word*, *punctuation*, and *space*, forming a **complete tiling** (the WS→PS `_passback_scope_where` handoff) — the level of analysis governs which particular objects are chosen at all.
- **The single quadratic surface (SS and every space)** — ONE priming vector over each space's rows, with TWO WRITE CHANNELS: **SEEN** (bottom-up — fired rows bump, the surface decays exponentially toward neutral; `prime_seen`) and **DESIRED/HATED** (top-down — signed intent; desire boosts, hate suppresses with floor 0, never a veto; `prime_desire`). The two channels ARE the automatic/strategic dissociation (Posner & Snyder 1975; Neely 1977, the prime-target expectancy experiments): **AUTOMATIC** — fast, capacity-free, inhibitionless priming through use — versus **STRATEGIC** — slow, capacity-limited, goal-set (the SINGLE intent of GrammarOpsPass §5 — capacity limited by construction), able to suppress. They also dissociate from each other behaviorally (selection/reward history captures attention even AGAINST current goals: Awh, Belopolsky & Theeuwes 2012) — both channels WRITE, neither vetoes. **`readingAttention` is HARD-CODED over this surface**: the reading scope is the span of the hottest-primed word-whole (`_primed_reading_step`, the learned producer's contract) — the symbols map onto the wholes that isolate words.

**Priming diffusion (`<primingSpread>`, default 0.25, LIVE by default, Alec 2026-07-12).** Before the SEEN bump, `prime_seen` moves an `s = <primingSpread>` fraction of each connected row's standing priming energy to its neighbors via `_priming_edges` (the concept store's untyped edges, ConceptualSpace-only; `None` elsewhere → no diffusion) — a conservative transfer (dst gains exactly what src loses), not amplification, so successive SEEN events propagate energy further out into the connected symbols before decay + bump apply. `0` restores pure decay+bump (the pre-diffusion behavior).

**Frozen concepts (`freeze_concept / _frozen_concepts`, Alec 2026-07-11).** A concept's relational structure can be FROZEN: no FORMING of new edges, no FORGETTING of existing ones, no WEIGHT drift on them (`_refresh_frozen_values_hook` zeros backprop gradient on the frozen rows' edge values) — the codebook row/content stays live and keeps tracking perception; only its DEFINITION is fixed.

**The readout site is the concept pyramid's per-order top-K** (`cs_forward_content`; the FF pyramid is COMPOSITION, not attention — the selection over it is where attention acts). The CS surface projects directly onto the inventory rows as the ranking score (`_relevance_priority` = boost − 1; rank = $|cand| \cdot$ (1+score), spreading upward through edge magnitudes, admitted rows only) — selection changes, activations never distort. The pyramid's ADMITTED rows write back through `prime_seen`: awareness primes. Gated `<architecture><relevance>` (default false, byte-identical); `<primingDecay>` sets the seen decay.

**The bases interact (cross-basis priming).** The psychological literature is unambiguous that symbolic activation primes the subsymbolic layers: automatic spreading activation vs. strategic expectancy in

semantic priming (Neely 1977 — the heat vs. intent split, exactly); word-level activation feeding back to letter perception (McClelland & Rumelhart's interactive activation); conceptual templates biasing early sensory competition (Desimone & Duncan's biased competition); learned context guiding spatial attention without awareness (Chun & Jiang's contextual cueing); labels sharpening perception (Lupyan's label feedback). The architectural consequence: **conceptual activation should be the ORIGIN of `readingAttention`** — the reading template built from the currently selected concepts (the pyramid's winners, `_concept_activations`) rather than a free-standing query — making WS's vertical basis the conceptual tower's own downward projection, as biased competition prescribes.

**D. Attention indexing (`.where` / `.when` / codebooks)**

The three addressing roles are disjoint: **`.where` indexes over the input buffer** (positional; period = config-derived `<wherePeriod>`, default 8192 input bytes — the 2026-07-04 encoding pass corrected the earlier "½ · InputSpace" claim here: the pre-change period was actually Σ nVectors, raised-never-lowered at the build seam, and is now decoupled from `nObjects` entirely, with a warn-once raise-to-fit for longer inputs), **`.when` indexes over LTM** (the 4-dim start-ladder band is the SIMILARITY channel; ABSOLUTE addressing rides the exact long-int clock `BasicModel.when_time` — the Option-C hybrid; see Spaces.md "Encodings"), and the **codebooks are content-addressable** (identity is the row index; the cross-codebook `.where` slice registry was retired). Reconstruction re-derives the input tiling from the `.where` band alone — the BLIND decode (Gate 2b, `test_blind_decode.py`; the forward scaffold survives as the explicit debug/regression path and the scaffold-masking curriculum bridges scaffold-fed to blind as training allows).

## Cognitive grounding: dense-perceptual vs sparse-symbolic (2026-07-02)

The design splits representation into a **dense, invertible, subsymbolic** integrator (the corpus-callosum mixing matrix, which mixes part/whole *content* in high dimension) and a **sparse, symbolic** composer (the `ConceptualAttentionLayer`, which composes *scalar activations* of named concepts; it subclasses the `SparseLayer` substrate). This is not an arbitrary engineering choice; the split, and specifically *why sparsity belongs only on the symbolic side*, tracks several convergent findings on human cognition.

- **Complementary Learning Systems** (McClelland, McNaughton & O'Reilly 1995). Neocortex uses slow, dense, *overlapping* distributed codes that extract statistical structure across experiences; hippocampus uses fast, *sparse, pattern-separated* codes to bind individuated episodes. The reason hippocampal codes are sparse is **interference avoidance among individuated bindings**, not compression: dense overlapping codes suffer catastrophic interference when made to hold many discrete conjunctions. This is the cognitive answer to "why sparse, and why only symbolically" — the perceptual manifold wants the dense mixing; the binding of individuated things into reusable conjunctions (the joint/sentence concept over word-symbols) wants sparsity. A **symbol is a scalar handle on a grounded direction** (the atom), which is precisely a hippocampal index into cortex; the transpose decode ($\tanh(W^\top y)$) is pattern **reinstatement**. Caveat: this maps the *sparse-symbolic vs dense-perceptual* axis onto hippocampus-vs-cortex — NOT the ramsified *orders* (abstraction is a separate, graded anterior-temporal / prefrontal axis).
- **Dual-process cognition** (Sloman 1996; Kahneman). A fast, parallel, similarity-driven associative system and a slow, serial, rule-based symbolic one. The subsymbolic (parallel content-mixing) and symbolic (sparse relational) loops instance this split. The sparsity and the serial capacity-limit are the *same fact*: a sparse composer has low fan-in per concept, the computational shadow of working-memory span (Miller's $7 \pm 2$; the STM depth $\approx 8$ the model already carries).
- **The neuro-symbolic interface / systematicity** (Fodor & Pylyshyn 1988). The architecture is a concrete stance on the oldest fight in cognitive science: **content stays connectionist** (dense, continuous, invertible mixing), **structure becomes symbolic** (discrete, reusable, composable edges), and the **interface is the activation readout** — the point where mixed content is read out as a scalar activation of a named thing. The discrete edges buy the compositionality/systematicity that pure distributed codes are accused of lacking, while the dense mixing keeps perception continuous and gradient-trainable.

- **Grounded cognition** (Barsalou 1999; contra amodal symbol systems). Because the *direction* lives in the concept's atom and only the *activation* is abstracted to a scalar, the symbols are not amodal Fodorian tokens — they are grounded pointers-with-magnitude, closer to perceptual-symbol "simulators." Keeping magnitude (the normalized-*sum* presence readout, not cosine) is cognitively load-bearing: graded activation *is* typicality / salience (Rosch's graded membership), which a cosine would discard.
- **Basic-level categories are perceptual, not content-free** (Rosch et al. 1976). "Subsymbolic" ≠ "category-free": the mixing output at order 0 is already carved toward basic-level regions, because that is what perceptual integration *for a categorizing organism* produces. This suggests the EMA-snapped dictionary atoms are the **pre-linguistic Gärdenfors regions** (prototype centers, basic-level, perceptual) and the sparse symbol graph is the *post-linguistic* labelling-and-composition that points at them — three cognitively distinct stages (integrated field → unnamed category region → named composable symbol), not two layers with a bookkeeping detail between.

**Where the mechanism is deliberately cleaner than cognition.** (1) The order-0 boundary is a *default flow*, not a wall: perception is concept-penetrated (top-down / predictive coding), and it is the TOP-DOWN attention channel — goal/emotion-weighted properties fixing the level of analysis, delivered to perception through the scope handoff — that carries that penetration back down; the priming/heat loop is the BOTTOM-UP horizontal channel and touches perception only through what it makes retrievable. The division must never become impermeable. (2) *Invertibility* is instrumental (reconstruction, gradient), not a biological claim; brains approximate and predict, they do not compute exact inverses. The extensional/intensional semantics this grounding implies for a **single** conceptual space is developed in BasicModel.md "Conceptual Space."

**References.** McClelland, McNaughton & O'Reilly (1995), *Why there are complementary learning systems in the hippocampus and neocortex*, Psychological Review 102(3). Sloman (1996), *The empirical case for two systems of reasoning*, Psychological Bulletin 119(1); Kahneman (2011), *Thinking, Fast and Slow*. Miller (1956), *The magical number seven, plus or minus two*, Psychological Review 63(2). Fodor & Pylyshyn (1988), *Connectionism and cognitive architecture*, Cognition 28. Barsalou (1999), *Perceptual symbol systems*, Behavioral and Brain Sciences 22(4). Rosch, Mervis, Gray, Johnson & Boyes-Braem (1976), *Basic objects in natural categories*, Cognitive Psychology 8(3). Gärdenfors (2000), *Conceptual Spaces: The Geometry of Thought*, MIT Press. Olshausen & Field (1996), *Emergence of simple-cell receptive field properties by learning a sparse code*, Nature 381 (sparse *activation* over a dense dictionary — the dictionary/atom side here — as distinct from the sparse *relational graph* of the `ConceptualAttentionLayer`).

## Overview

BasicModel is a bidirectional neural architecture organized as a pipeline of five **spaces** plus a symbol host (`SymbolSpace`), each implementing a distinct representational transformation:

```
Forward:  InputSpace -> PartSpace -> ConceptualSpace -> WholeSpace -> OutputSpace
Reverse:  OutputSpace -> WholeSpace -> ConceptualSpace -> PartSpace -> InputSpace
```

The pre-2026-05-27 "two feedback loops" (S → C symbolic loopback per stage, C → P subsymbolic loopback cross-forward) collapse under the substrate refactor:

- **Subsymbolic loop dissolves.** PS is a single-direction input processor. No recurrent C → P feedback at the substrate level. In PARALLEL mode, iteration happens by passing `CS` to the same `PS.forward(x)` for T refinement passes (the `<subsymbolicOrder>` knob).
- **Symbolic loop generalizes** to pairwise grammar ops over STM, dispatched by the signal router (`LanguageLayer`). `Lift` and `Lower` join the same GrammarLayer dispatch surface as `Intersection`, `Union`, etc.

The forward pass transforms raw input into predictions; the reverse pass reconstructs the original input from the symbolic representation. Both directions are trained simultaneously with a single optimizer minimizing a combined loss:

`totalLoss = (1 - reconRatio) * outputLoss + reconRatio * reconstructionLoss`

The legacy `SubwholeSpace` and `SyntacticSpace` classes have been retired. The subsymbolic role is filled by `PartSpace`; syntax / grammar dispatch lives on `SymbolSubSpace.languageLayer` (the signal router, which subsumes the retired `Chart`). The `MereologicalTree` sidecar that backed `part` / `equals` / `query` is also retired — those operations are pure-geometric clipped-cosine projections over WholeSpace codebook activations.

`PartSpace` and `WholeSpace` (renamed 2026-06-12 from `PerceptualSpace` / the original `SymbolSpace`) both subclass `Space` directly — both views are perceptual, but there is no shared intermediate base class. A thin `PerceptualSpace(Space)` base briefly existed (holding no params/submodules; only `NULL_PERCEPT_KEY` and isinstance sites) but was **removed 2026-07-10** as part of the dual-towers rev-2 pyramid rework (decision 3): PS/WS became symmetric duals with the same `forward(in_sub, CS_out)` signature instead. At the corpus callosum, objects are analysed and synthesized by sending them back through the towers — wholes get split, parts get chunked. In symbolic "mode" the objects sent back are symbols. Terminologically there are objects and references; a reference is a *sign* (a quantized version of the referent) or a *symbol* (an unrelated version of the referent, of much lower dimensionality). The freed name `SymbolSpace` was **reintroduced 2026-06-19** with new semantics — it is now the grammar/word space-role (formerly `WordSpace` / `WordSubSpace`, abbrev `ss`; the WholeSpace stream is now `ws`).

Gated `<mereologyRaise>` (default false, byte-identical off; the cross-tower binding is `ConceptualSpace._autobind_cross_to` the part/whole-ratio read-out is `RunStructureLayer` via `WholeSpace`'s `_mereology_ratio_obs`, threaded read-only, never persisted), the corpus callosum **builds a single meronomy out of the two towers**: a part `A` (PartSpace) and a whole `B` (WholeSpace) carry `.what` codes from different codebooks (incomparable), but their `.where` is comparable, so the callosum links `A isa B` (token `isa` type) when `A.where` is contained in `B.where` with no greater-part/lesser-whole intervening. Word↔object — too unlike to link directly — is bridged by a **second-order meta-object** (synthesized in PartSpace, outside `.where`/`.when`: the MetaSymbol). The correctness signal is the **part/whole ratio** (many-parts→one-whole = under-analysed; one-part→one-whole = over-analysed), which requests further $\sigma$-synthesis / $\pi$-analysis in the offending `.where` — and is the principled fix for the MM_20M mean-collapse.

**Spaces**

| Space | Role |
|---|---|
| **InputSpace** | Lifts raw data into working dimensionality; surface tokenization |
| **PartSpace** | Bottom-up SYNTHESIS branch (Pi/Sigma swap, rev. 2026-06-09): sigma fold + `<synthesis>` front |
| **ConceptualSpace** | STM container + main grammatical CPU + (when sparse-active) the POST-PUMP symbolic phase |
| **WholeSpace** | Top-down ANALYSIS branch: pi fold + `<analysis>`/`<lexer>` knobs; symbol-prototype codebook ow |
| **OutputSpace** | Final prediction |

The cross-space fold contract has changed:

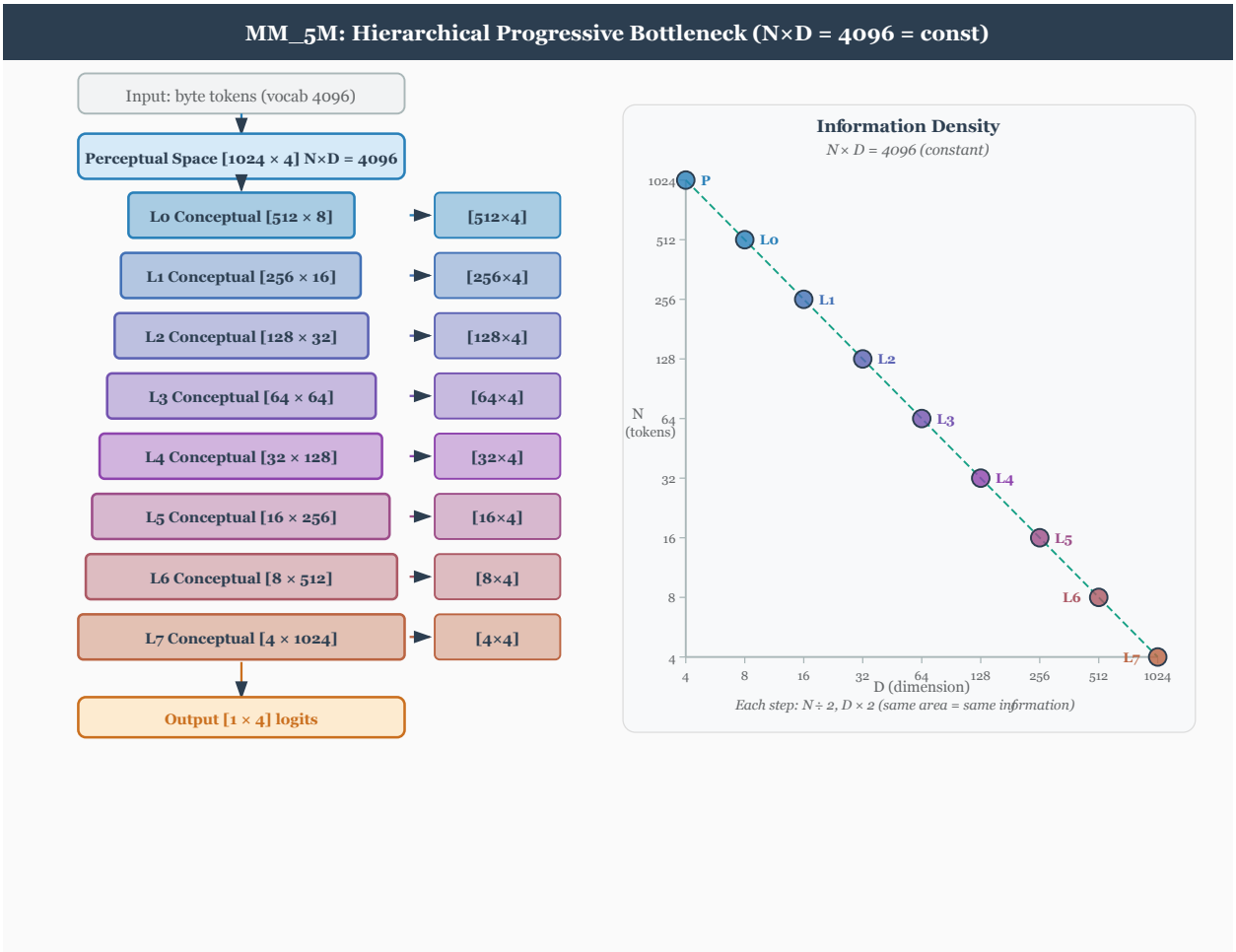
```
PS.forward(x):  return self.sigma(x.materialize())
CS.forward(subspace, word_subspace):
    STM[1..7] = STM[0..6];  STM[0] = folded          # newest at slot 0, shift toward higher indices, o
# POST-PUMP cutover (sparse-active, once per forward, in _forward_body):
#   content, acts = cs.cs_symbolic_phase(last_cs.materialize())   # snap -> concept pyramid (K = symbol
#   last_cs._concept_activations = acts;  SS leg built ONCE;  SBOW parks the settled slab
SS:  no atomic forward operator; hosts write-required grammar ops
     (the CS->SS symbol bind leg is SymbolSpace.forward_concept_to_symbol, .forward()-mediated)
```

The legacy composition `C = sigma_percept(pi_input(IS) + pi_concept(C_prev))` is **retired**. Per-stage feedback is absent at the substrate; grammar dispatch over STM provides the recurrent character via the signal router.

**Attention-to-relation promotion** (gated `<attentionPromotion>`, default off → byte-identical): the pyramid's admitted field is also the DISCOVERY surface for latent taxonomic wholes. The cutover stashes the admitted activations; `ConceptualSpace.Reset(hard)` consumes them (the same compile-safety hoist as `learn_relations_from_stm`) into a bounded candidate cache keyed by context signature — each active order-0 row is a focal member observation over the rest of the active set, with EWMA member/context weights and cosine fold-in of near contexts. Recurrent, contrasted member sets face the SAME acceptance law as sentence learning (learn-score $\geq$ `truthCriterion` AND `truthCriterion` $< 1$, the three-factor product over the Task-6c seams); accepted sets mint a higher-order whole via `synthesize_higher_order` (member edge values from the candidate statistics, top context concepts as weighted `sym_part` intent), which then competes in the pyramid like any other row. Re-support strengthens (Hebbian) instead of re-minting; unsupported wholes decay and retire.

See Spaces.md Section "Sigma / Pi ownership" for the cognitive rationale and the migration trail. See Logic.md Section 8 for the algebraic constraints on sigma/pi.

Dimensions (`nDim`) are read from `TheObjectEncoding`. Codebook sizes (`nVectors`) are likewise on `TheObjectEncoding`; the factory validates `nVectors >= nActive`.




Layer selection by `invertible` (the `<reconstruct>` element / `reconstructEnum` were **RETIRED** in A1, 2026-06-09; reconstruction is now seeded from concepts **unconditionally**, gated only by `reconstructionScale`):

1. **Non-invertible, forward-only layers** (`PiLayer`, `SigmaLayer`): forward-only, no reverse pipeline.
2. **`invertible`**: Single invertible layer (`PiLayer(invertible=True)`, etc.) serves both directions, sharing weights.

3. **Not `invertible`** (but reconstructed): Two layers with separate weights — `forward()` on one, `reverse()` on the other. Avoids the expressivity limitation where a non-invertible layer can't represent the inverse of another. Reverse uses matrix `pinv` (may be numerically unstable from SVD convergence). `<invertible>true</invertible>` avoids this via shared-weight inversion.

#### Single Optimizer with Overlapping Weight Spaces

The forward and reverse passes share a **single Adam optimizer** that minimizes the combined loss. Forward and reverse weight spaces **partially overlap** — neither disjoint (allowing independent optimizers) nor identical (creating destructive interference). Some layers share weights between directions (shared embeddings, the symbolic bottleneck); others are direction-specific (`pi1`/`pi2`, `sigma1`/`sigma2`, `linear1`/`linear2`).

- **Shared weights** receive gradient from both losses, learning representations useful in both directions.
- **Direction-specific weights** specialize without interference.
- **No ping-pong**: separate optimizers on overlapping parameters would pull weights in alternating, conflicting directions each step.

When `invertible=true`, overlap is total: one invertible layer serves both directions and receives the full combined gradient.

Reference: A.M. Rogers, T.T. Shannon, and G.G. Lendaris, "A comparison of DHP based antecedent parameter tuning strategies for fuzzy control," *Proceedings Joint 9th IFSA World Congress and 20th NAFIPS International Conference*, 2001, doi: 10.1109/NAFIPS.2001.944317.

#### Training Loop

Single Adam optimizer with persistent state (momentum/variance accumulate across epochs):

1. Forward pass: input → prediction + `end_state`
2. Compute `outputLoss` from prediction vs. target
3. Reverse pass: `end_state` → reconstructed input
4. Compute `reconstructionLoss` from reconstruction vs. original input
5. Backpropagate combined `totalLoss`
6. If ergodic: run `paramUpdate()` (gradient energy sensor updates `bias`/`var`)
7. Optimizer step (embedding params excluded when `trainEmbedding` is `NONE`, `CBOW`, or `SBOW`)
8. If `trainEmbedding` is `CBOW`, `SBOW`, or `BOTH`: run embedding update step

Ergodic exploration is not epoch-annealed. `ErgodicLayer` starts in pure-exploit mode (`bias=1`, `var=0`) and updates those buffers from observed gradient variance after backward. See Ergodic.md.

See Params.md for all XML parameters. See Training.md for embedding modes.

#### The three cognitive operations (2026-06-14)

Processing decomposes into three operations, in increasing order of abstraction. Each maps to a knob (or, for the first, to the folds themselves):

1. **Granularity of analysis and synthesis** — done *automatically* by the two perceptual views' folds, per pass. PartSpace's **Sigma synthesizes** (union; count-reducing: many atoms → fewer chunks); WholeSpace's **Pi analyses** (intersection; count-increasing: one unity → many parts). How finely the scene is carved, or how coarsely it is chunked, is set by the folds — there is no separate granularity knob. The InputSpace feeds the two views directly: the **Atom** view (`[B, N, D]`, which PartSpace synthesizes bottom-up) and the **Universe** view (`_unity_view`, the whole as one event, which WholeSpace analyses top-down). Optionally (`<mereologyRaise>`), perception builds a meronymic lattice over the towers and **raises abstraction order** as attention requires — see Mereology.md → Order-raising.
2. **Subsymbolic order** (`<subsymbolicOrder>`) — *iterating* the folds: codes are passed back to PartSpace / WholeSpace across `subsymbolicOrder` passes (the CS→PS loop). Synthesis chunks

the codes into higher-order percepts (fewer each pass); analysis re-expands, attention selecting what to expand (a top-k over the priming, applied after the WholeSpace codebook lookup). Symbolic composition is no longer a separate CS→PS passback flag: the recurrent symbolic leg always flows through `WholeSpace.forward(prevCS_forSS)`, and the symbolic-iteration codebook handles higher-order symbolic composition on the CS→SS path.

> **Proposed refinement (mereological-order-raising spec).** This single subsymbolic loop is really **two** moves CS should choose between *per representation*, by reading the **contiguity of `.where`**: a *contiguous* extent → **refine granularity** (chunk finer / tile — drive the radix), same order; a *discontiguous* extent → **raise order** (another $\sigma/\pi$ fold, lifting out of `.where`/`.when`); a *zero* `.where` → null. The number of contiguous runs in a whole's `.where` *is* its part/whole ratio, so the same read also routes integrate-vs-disintegrate (→PartSpace $\sigma$ vs →WholeSpace $\pi$) — this is the three-aspect loop described above. As of 2026-06-16 the contiguity read has its substrate: `.where` is an **endpoint-sum bracket `[start, end]`** (`WhereEncoding.decode_span`), so extent and gaps are read directly off a code (a zero-extent instant vs a span); see doc/Spaces.md. (2026-07-04: `.when` no longer carries the bracket — it is the 4-dim start ladder; exact times/extents ride the clock side-band.)

3. **Symbolic order** (`<symbolicOrder>`) — the symbolic / relational loop budget. In serial mode (`<serial>true</serial>`), words are read **one at a time** from WholeSpace (reading isolated words to ConceptualSpace *is* attention) and processed grammatically in ConceptualSpace's STM and on the PartSpace side. `symbolicOrder` limits how many symbolic / SS loops may run; `<serial>` selects whether the per-word traversal is active.

So: granularity is intrinsic to the folds, subsymbolic order iterates the subsymbolic passes (composing higher-order percepts), symbolic order budgets the relational pump, and `serial` selects the serial grammatical loop over words.

> **Current order semantics.** This section supersedes the older mode-selector wording. The three order axes now have separate semantics, bounds, and composition rules:
>
> - `subsymbolicOrder` — the **analysis/synthesis refinement-pass count and the area of attention**. `T` parallel CS→PS/WS iterations; each pass *refines* (contiguous `.where`) or *raises* (discontiguous), and attention scopes via a `.where` on the dual-input SECOND ARGUMENT (the top-down WS→PS handoff, gated `<mereologyRaise>`). The serial-word reading supplies word `.where`s through the **same** channel.
> - `symbolicOrder` — the **relational pump** budget. It spreads activation through the relation graph to surface *higher-order* (relations-of-relations) features that have **no mereological `.where`** and so can't be primed off `.where` contiguity. `subsymbolicOrder` pumps the mereological substrate; `symbolicOrder` pumps the relational one. `<serial>` separately selects whether traversal is per-word serial or whole-slab parallel.
> - `syntacticOrder` *(NEW — implemented 2026-06-19)* — the **parse-tree composition DEPTH** per sentence, bounded by the word count. `0` = unbounded (byte-identical); a positive value caps the NULL-seal reduce sweep to that many fold levels (static `min(syntacticOrder, cap-1)`; $\leq W$ structural). Inert in parallel mode.
>
> Composition (serial run): `<serial>true</serial>` loops words × `syntacticOrder` bounds the parse-tree depth per sentence × `subsymbolicOrder` pumps per node; the **basic-level stop** is shared (synthesis halts at words, so the tree's leaves are words). `syntacticOrder` **layers over** the serial traversal loop (it bounds depth; it does not replace the parallel-vs-serial switch).
>
> **Where this is headed (historical design note):** the three orders become **pump counts** over one connectionist attention substrate — a cumulative priming hierarchy (mereological entries → relations/concepts → higher-order, each seeing all below) where reading is a learned `.where` attention (text-mode next-word loss) that replaces the serial for-loop.

### Modes of operation

Two operating modes, selected by `<architecture><serial>` (replaced the `conceptualMode` enum; legacy configs that omit `serial` derive the mode from `symbolicOrder > 0`):

| Mode | Trigger | PS.forward argument | Iterations |
|---|---|---|---|
| **SERIAL / GRAMMATICAL** | `<serial>true</serial>` | `IS_t` per word | one per word; PS pus |
| **PARALLEL** | `<serial>false</serial>` | `IS` once, then `CS` for T-1 iterations | T = `<subsymbolicO` |

SERIAL and GRAMMATICAL are not architecturally distinguished — grammar dispatch is a chart / rule-catalog config, not a substrate mode. PS.forward takes a single positional argument in both modes; the argument is whatever input is being processed (IS in SERIAL, IS then CS in PARALLEL refinement).

**Pre-2026-05-27 "two feedback loops" retired.** The legacy S → C symbolic loopback (per-stage) and C → P subsymbolic loopback (cross-forward) collapse under the substrate refactor:

- PS is a single-direction input processor; no recurrent C → P feedback at the substrate. CS state enters PS only via `PS.forward(CS)` in PARALLEL mode's refinement iterations.
- Symbolic loop becomes pairwise grammar ops over STM (the signal router's copy/reduce dispatch). `Lift` and `Lower` are binary GrammarLayer subclasses dispatched alongside `Intersection`, `Union`, etc.

The recurrent character of the architecture lives in (a) STM accumulation across words in SERIAL mode, and (b) the T-pass PARALLEL refinement loop. Cross-call serial-cache (`subspace.serial_cache`) for streaming / autoregressive contexts is preserved; gated by `PartSpace._recurrent_pass_idx == 0`.

### Pipeline as a unit, two-space-role reset

`runBatch` is a pure compute brick: forward → loss → backward → optimizer.step. It does **not** decide when to reset per-row state, does **not** consume `_end_of_stream` for control flow, and (after Section 6 vectorization) does **not** issue any GPU→host sync inside the brick.

Reset lives in `runEpoch`. The same loop drives both byte cursor (AR text byte) and trial cursor (non-AR); `next_tick` is universal dispatch:

```
while not ds.all_done():
    inp, out, hard_eos = ds.next_tick()             # 3-tuple, host-side
    runBatch(inp, out)                               # compute brick
    flush_word_buffers()                             # materialize subspace.word
    dispatch_per_row_reset(hard_eos)                 # hard resets
    dispatch_soft_reset()                            # grammar <start> reductions
    post_tick_compact()                              # truth_layer.compact
```

For AR text byte, `inp` is a byte slab and `hard_eos[b]` flips True when row b's cursor exhausts a doc. For non-AR / numeric data, each tick yields one batch of trials with `hard_eos = [True] * B`.

**Hard reset.** `TheData` walks each document one slab of ≤1024 bytes at a time. `hard_eos` flips True on cursor exhaustion. Full row-state cascade fires for that row only; other rows continue mid-document with state preserved.

**Soft reset.** The active parser signals when a row's parse reduces to `<start>`. `wordSpace._sentence_completed` is drained per-tick: re-arms `_stm_fired[b]`, clears `_last_svo[b*K..]` and parse-stack rows for `b`, but **preserves discourse history** (discourse accumulates across sentences within a document and clears only on hard reset).

**No truncation.** Documents longer than `slab_bytes` span multiple ticks; concatenating per-tick slabs for any row reproduces the original document byte-exact. `valid_mask: [B, K]` handles partial-fill tails via NULL-padding.

**Compute-brick contract.** No `.item()`, no `.tolist()`, no Python conditional on a tensor value, no GPU→host copy inside `runBatch`. The chart's residual `.tolist()` calls retired with the `Chart` class itself in the substrate refactor.

**Two-File Architecture**

| File | Contents |
|---|---|
| **XML config** (e.g. `BasicModel.xml`) | Architecture, hyperparameters |
| **Weights checkpoint** (e.g. `BasicModel.ckpt`) | Full integrated bundle: model parameters, register-buffer state, embeddin |

The 2026-05-12 *integrated-weights* refactor retired the separate `.kv` embedding artifact: embeddings, vocabulary mappings, and the BPE codebook now ride inside the single `.ckpt` bundle alongside the model's other parameters. The bundle layout is:

- `state_dict`: every `nn.Parameter` and `register_buffer` in the module tree (model weights, `wv._vectors`, `TruthLayer.truths`, etc.) — serialised by the normal PyTorch path.
- `vocab_extras`: the WordVectors Python-side mappings that don't live in `state_dict` (`index_to_key`, `counts`, `total_count`).
- `bpe_extras`: the ChunkLayer's pure-Python state (merges list, vocab dict, `id_to_bytes`, growth cursors). Required because `ChunkLayer` stores its merge table as Python dicts/lists, not tensors.

`bin/embed.py` still produces standalone `.kv` artifacts for CBOW/SBOW *pre-training* studies, but those artifacts are no longer part of the runtime artifact set. Cold-start training initialises the vocabulary and BPE codebook from scratch and learns them end-to-end alongside the model weights.

---

## Language System

The grammar dispatch runs through the **signal router** (`LanguageLayer`, `bin/Language.py`) — the single canonical parser. `SymbolSubSpace` owns it directly as `self.languageLayer`. The pre-substrate CKY `Chart` and STM shift-reduce parsers retired in Stage 3 of the substrate refactor.

The signal router represents STM as a slab `[B, N, D]`; per-layer scorers emit per-position copy/reduce scores; `binary_tiling_soft_dp` produces marginals at training (soft superposition), `binary_tiling_viterbi` produces the best tiling at eval. `Grammar.rule_probability(body)` is generalized to the per-position, per-op score head — dormant defaults (fold ops fire, negation ops don't) carry over as initial biases. Single-application enforcement via `_fired_bodies` / `reset_derivation` carries over unchanged.

Grammar ops are GrammarLayer subclasses dispatched by the router as unary (copy-side) or binary (reduce-side):

- **Unary symbolic operators**: `not(S)`, `non(S)`, `swap`, `copy`, `true`, `false`.
- **Binary symbolic operators**: `intersection(S, S)`, `union(S, S)`, `conjunction`, `disjunction`.
- **Mereological operators**: `part(S, S)`, `isEqual(S, S)`, `query(S, S)`. Pure-geometric — the `MereologicalTree` sidecar that formerly stored explicit parent / equality links retired in favor of clipped-cosine parthood on codebook activations. See Mereology.md.
- **`lift` and `lower`**: now binary `GrammarLayer` subclasses (Stage 4 of the substrate refactor). Each owns an internal `SigmaLayer` (`LiftLayer`) or `PiLayer` (`LowerLayer`) for the pairwise math. No longer "substrate-borrowing" — fully self-contained binary grammar ops with `arity=2`, `space_role='CS'`. Typed grammar signatures still determine result order (e.g., `S4 = lift(NP3, VP1)`). See Language.md.
- **Butterfly mode on `GrammarLayer`** (Stage 5): all GrammarLayer subclasses accept `butterfly=True, N=N` for efficient cross-STM pairwise composition via a packed `nn.Parameter[n_levels, N//2, 2D, 2D]` cascade with bit-reversal permutations. Wired into the space folds (`PartSpace.sigma` /

`WholeSpace.pi` post the Pi/Sigma swap, rev. 2026-06-09) by default in butterfly-enabled configs. Closes the XOR convergence target.

Parthood (`part`) is the **fundamental** mereological operation, realized as clipped cosine projection on symbolic activations. The full suite (`whole`, `equal`, `overlap`, `underlap`, `boundary`) composes through `part` on `Basis`. `isEqual(S, S)` is propositional identity on S; delegates to `Basis.equal`.

### Short-Term Memory on ConceptualSpace

`ConceptualSpace.stm` (an instance of `ShortTermMemory`) is a per-batch stack of unquantized CS "ideas" — the working set the signal router dispatches grammar ops over. Capacity defaults to 8 (within Miller's 7±2 band); `<ConceptualSpace><stmCapacity>N</stmCapacity></ConceptualSpace>` overrides.

Post-substrate-refactor, `CS.forward(subspace, word_subspace=None)` is **STM bookkeeping** — shift existing slots toward higher indices, push the new idea onto slot 0 (newest-at-slot-0; `_stm_shift_and_push`). The legacy atomic forward fold (`sigma_percept`) is retired. (The symbolic transform no longer runs inside `forward` — P3 two-phase rework: the pump stays purely subsymbolic and `cs_symbolic_phase` fires ONCE at the post-pump cutover in `_forward_body`; its outputs feed the SS leg and the losses, never the STM content. See Architecture.md sec A.) The mode dispatch:

- **SERIAL / GRAMMATICAL**: one idea pushed per word; STM shifts (newest to slot 0, oldest dropped from the high end). Grammar ops dispatched per word or at sentence boundary.
- **PARALLEL**: T = `<subsymbolicOrder>` iteration outputs written to STM slots simultaneously; no shift.

STM is cleared on hard `Reset` (sentence boundary) and survives soft reset. The signal router consumes `stm.snapshot()` for its slab input. See Spaces.md.

The STM data model, the predict-then-perceive cadence (serial and parallel), the in-STM and inter-sentence predictors, masked-word reconstruction, relative-vs-absolute end-states, and the LTM chain are documented in full in the dedicated STM.md chapter. Note in particular that **serial mode runs WITH attention by design** (the old serial-vs-attention guard was lifted; `MentalModel.xml` is serial + `hasAttention`) — see STM.md Section 4.

### Per-word operational flow (SERIAL mode)

In SERIAL / GRAMMATICAL mode, each word traverses a per-word path:

```
byte stream  ->  PS.forward(IS_t)    # MPHF surface lookup -> PS lexicon
                                     # then pi(x) + sigma(x), no outer tanh
             ->  CS.forward(idea)    # STM shift toward higher indices; push idea onto slot 0
             ->  signal router dispatches grammar ops over STM
                                     # (read-only via CS; write-required via SS)
```

PS's `self.vocabulary` (Embedding) holds the per-word vectors keyed by MPHF, including the authoritative `key_to_index` forward map. Lookup chain: surface → MPHF → PS lexicon row. The 2026-05-27 tied-storage plan (`insert_paired_word`: an orth row copied from PS's per-word vector, paired with a random semantic row via `Codebook.set_part_parent`, both living on WS's codebook) was **retired 2026-06-10** — the lexicon keeps PS-local, untied storage permanently. Decode (row → word) resolves through the INVERSE of `key_to_index`, not positional `index_to_key` (the two coincide for an untied lexicon).

**Category information rides the symbol machinery.** The live category codebook learns role participation for MetaSymbols, and the router uses that context when ranking grammar routes. So per-word symbol handling does not depend on a separate POS tagger. The parser still uses category information for typing reduce candidates ($NP + VP \to S$, etc.); that information is learned through parsing alongside the codebook.

See Logic.md, Mereology.md, and Language.md.

**Shamatha Speech target.** Planned narrow grammar for one-pointed object speech: complete DNF over active percepts, permitting each `conjunction` / `disjunction` only when operands' `where()` supports are connected and `when()` supports are continuous. See Philosophy.md.

---

## Sigma and Pi Layers

For weight matrix $W \in \mathbb{R}^{m \times n}$ and input $x \in \mathbb{R}^n$:

Sigma layer:

$$y_j = Wx + b = b_j + \sum_{i=1}^{n} W_{ji} x_i$$

Pi layer (log-space linear):

$$s_i = \log \frac{1 + x_i}{1 - x_i} = 2 \operatorname{atanh}(x_i)$$

$$z_j = \sum_i W_{ji}\, s_i + b_j$$

$$y_j = \frac{e^{z_j} - 1}{e^{z_j} + 1} = \tanh(z_j / 2)$$

Forward maps $[-1, 1] \to (0, \infty)$ via `_to_mult`, log, linear, exp, `_from_mult`. Domain and range both $[-1, 1]$. Reverse inverts each step: `_to_mult(y)`, log, $W^{-1}(z - b)$, exp, `_from_mult`.

**Motivation.** The classical product form $y_j = b_j \prod_i (1 + W_{ji} x_i)$ becomes, after taking logs, a sum. The code moves into a log-multiplicative domain via atanh, performs a linear op there, returns via tanh. The atanh transform stretches values near $\pm 1$ toward infinity, making the layer sensitive to strong activations.

**Monotonicity of the lift / lower chain.** Under `monotonic=True`, Pi/Sigma select non-negative linear layers, giving $W \geq 0$. Positive matrices are monotone on the positive cone, so lift / lower preserve parthood for activations represented in that cone. Truth-set bivectors remain live for user-supplied truths; they are explicit truth/operator surfaces rather than a space-wide output mode. See Spaces.md.

---

## Dimensionality Constraints

- Input layer output dim = perceptual layer output dim (conceptual operates on both).
- Symbolic layer input dim = perceptual layer input dim (both operate on conceptual output).
- Output layer input dim = sum of symbolic layers' output dims.

---

## Invertible Linear Layer (LDU)

Factors $W = L \cdot D_{\text{embed}} \cdot U$:

- $L$: unit lower-triangular ($nIn \times nIn$, diagonal = 1).
- **D**: diagonal vector of length `rank = min(nIn, nOut)`, embedded into $[nIn, nOut]$ by zero-padding.
- $U$: unit upper-triangular ($nOut \times nOut$).

**Exact inverse via triangular solves:** $W^{-1} = U^{-1} \cdot D^{-1} \cdot L^{-1}$. Each factor inverted by `torch.linalg.solve_triangular`. No SVD; inverse exact when all D entries are nonzero. Parameter count: $nIn^2 + \text{rank} + nOut^2$. Initialized at $L = I, d = 1, U = I$ (identity).

`naive=False` (default) applies L/D/U sequentially without materialising `W_eff` as a full matrix. `naive=True` materialises `W_eff` and its inverse.

Ergodic noise injection at the factor level, plus the `stable=True` clamp and the noise lifecycle, are documented in Ergodic.md.

---

## Ergodic Exploration

See Ergodic.md.

---

## Sentence-level AR (`InterSentenceLayer`)

Within-sentence training is IR-only (BERT-style masked-LM at the subsymbolic (PS); see `doc/Spaces.md` Section "Within-sentence AR retirement"). The **autoregressive** signal in this architecture lives one scale up: between sentences, on a per-sentence representation `s_t`. That's the job of `InterSentenceLayer` (alias `wordSpace.discourse`).

### Sentence representation

`s_t` is the **root SS slot** of the body's final stage: the single vector the start-symbol reduction wrote into. The chart's parse trace already commits to this slot at sentence end; the layer pools `[B, N, D] -> [B, D]` by taking row 0 (root). Width is `sentence_dim = n_dim` (one vector per row), **not** the full `n_symbols * n_dim` flatten that the pre-2026-05-14 contrastive layer used — that broader rep would have blown the predictor's Linear past the allocator budget on large MM_5M-scale configs.

### ARMA(p, q) predictor

`InterSentenceLayer` runs an autoregressive moving-average predictor:

```
s_hat_t = predictor(s_{t-1..t-p}, e_{t-1..t-q})
e_t     = s_t - s_hat_t
loss    = MSE(s_hat_t, s_t)        # accumulated per batch
```

- `p` = AR lag count (default 5) — last p sentence reps.
- `q` = MA lag count (default 2) — last q prediction errors.
- `predictor` = `nn.Sequential(Linear(p*D + q*D, H), Tanh, Linear(H, D))`, with `H = min(1024, 2*sentence_dim)`.

The MA term lets the predictor correct for systematic bias in the AR extrapolation: if the AR model consistently under-predicts the sentence rep, the residual `e_t` carries that signal forward.

Buffers (per row, non-persistent):

- `_s_history`: `[B, p, sentence_dim]` ring of last p sentence reps (most recent at index `-1`).
- `_e_history`: `[B, q, sentence_dim]` ring of last q residuals.
- `_s_count` / `_e_count`: `[B]` long, fill levels (cap at p / q).

`ensure_batch(B)` resizes these on cascade from `SymbolSpace.ensure_batch`; `Reset()` clears them on hard / discourse boundary. Default behaviour is to **not** auto-reset across document boundaries — the AR lags carry information through discourse continuity unless the caller explicitly calls `Reset`.

**Wiring into the training loop**

1. After the body finishes (sentence end), `_forward_per_stage` stashes the SS event on `_current_discourse_s`.
2. In `runBatch`, when training, `discourse.observe(s_tensor)`:
   - Pools `s_t = sigma_S(s_tensor[:, 0, :])`.
   - Computes `s_hat_t = predictor(_s_history, _e_history)`.
   - Returns `MSE(s_hat_t, s_t)` (None on the first call per row when the ring is empty).
   - Computes `e_t = s_t - s_hat_t`, pushes both into the rings.
3. `runBatch` adds the loss to `TheError` under category `"discourse"` with weight `armaScale` (training XSD knob, default 0.1).

**Inference (chat-loop seeding)**

`BasicModel.generate_sentence(seed_text)`:

1. Calls `discourse.predict_next()` for the ARMA-predicted sentence-rep prior `s_hat_{t+1}`.
2. Lifts it through `discourse.cast` to `concept_dim` and stages on `ConceptualSpace._c_prior`.
3. Runs the IR forward — the body's first sigma_percept output gets `_c_prior` summed in as a sentence-level conditioning bias before the codebook lookup. Cleared after the forward consumes it.
4. The post-body perceptual event is decoded by nearest-neighbour against the perceptual codebook, producing the `(slot, original, predicted)` triples for the seed text's masked positions.
5. Commits the produced sentence's SS root to the ARMA ring via `discourse.observe(s_tensor)`.

The IR head plays no role at inference — the prediction lives at the masked subsymbolic (PS) positions, decoded against the (frozen) perceptual codebook.

**Configuration**

| XSD knob | Section | Default | Notes |
|---|---|---|---|
| `<armaP>` | `<SymbolSpace>` | 5 | AR lag count |
| `<armaQ>` | `<SymbolSpace>` | 2 | MA lag count |
| `<armaHiddenDim>` | `<SymbolSpace>` | `2*sentence_dim` (cap 1024) | predictor hidden width |
| `<armaScale>` | `<architecture><training>` | 0.1 | ARMA loss weight added to `TheEr` |
| `<sentencePrediction>` | `<architecture><training>` | false | Gates `InterSentenceLayer` constr |

The retired pre-2026-05-14 knobs (`<sentenceContextWindow>`, `<sentenceCentroidHistory>`, `<sentenceLambda>`, `<sentencePredictionScale>`, `<sentencePredictiveScale>`, `<sentenceContrastiveScale>`) shaped the legacy contrastive cosine machinery (recent-centroid attraction + older-centroid repulsion). They are not parsed; configs that still set them are tolerated silently.

# Runtime Architecture and Componentization

> **Status: architectural assessment, 2026-07-13.** This document describes the live software boundaries in `bin/`, not the cognitive decomposition of perceptual, conceptual, whole, and symbolic space. It is a refactoring guide, not a claim that the proposed components already exist.

The conceptual architecture is substantially more modular than its runtime. Layers, bases, spaces, grammar operations, and typed `SubSpace` carriers give the model a useful domain vocabulary. The non-modularity is concentrated in the mechanisms that *run* those objects: configuration hydration, corpus streaming, per-step state, STM mutation, objective collection, checkpointing, and reconstruction.

That distinction matters. Moving methods out of a large file would reduce file size without fixing ownership. The high-value work is to give each lifecycle one owner, explicit inputs and outputs, and serializable state.

## What is already a useful boundary

The following structures should be preserved and made easier to compose:

- `Layer` subclasses own tensor transformations and, in most cases, expose a forward/reverse contract.
- `Space` subclasses name the cognitive roles and own their codebooks.
- `SubSpace` is the typed carrier for event, activation, basis, and word data.
- `SentenceStreamDataset` already encapsulates the rolling multi-stream cursor better than a conventional shuffled `DataLoader` would.
- `Error` is a promising loss registry: it names, weights, categorizes, and reports objective terms.
- `ModelFactory.validate_config` and the preflight tests establish the right fail-before-training principle.

The recommended refactor wraps these mechanisms; it does not replace them with generic framework abstractions.

## The current ownership problem

`bin/Models.py` is nearly 12,000 lines, but line count is only the symptom. `BasicModel` and `BaseModel` jointly own construction, the training loop, compiled execution, forward and reverse routing, checkpoint schema, vocabulary migration, corpus progress, reconstruction reports, generation, truth storage, and reasoning. The same object also serves as the mutable message bus between phases through attributes such as `_staged_in_sub`, `_staged_concepts_in`, `_prev_cs_for_ps`, `_prev_cs_for_ss`, `_ir_mask_positions`, `_stm_single_S`, and `_current_discourse_s`.

The result is a system with named domain modules but implicit runtime APIs:

- **Launch configuration.** State and policy span `bin/train.py`, `ModelFactory`, `BaseModel.create_from_config`, `util.TheXMLConfig`, and `BASIC_*` environment variables. Effective configuration is reconstructed in stages and can differ between validation, data loading, construction, and training.
- **Corpus execution.** `Data`, `SentenceStreamDataset`, `BasicModel.runEpoch`, and `BasicModel.runBatch` share responsibility. Dataset mode, cursor semantics, reset policy, and device conversion leak into the model.
- **Forward execution.** `BasicModel._begin_step`, `_forward_body`, `_forward_body_per_word`, `_forward_per_stage`, and mutable attributes on several spaces collectively define the path. A stage's true input includes hidden state left by earlier calls.
- **STM and grammar.** `ShortTermMemory`, `SymbolSubSpace`, `ConceptualSpace.forward`, and the `_stm_*` methods on `BasicModel` divide storage, typed metadata, prediction, push/reduce policy, and sentence-boundary reset among different owners.
- **Objectives.** `ModelLoss`, module-level `TheError`, per-`SubSpace` registries, spaces that add terms, and `runBatch` all participate. It is difficult to prove which objectives are active for one launch or which phase owns a term.
- **Checkpoints.** `BaseModel.save_weights/load_weights`, vocabulary and BPE helpers, optimizer setup, RNG globals, and stream progress in the training loop contribute to one artifact. Adding a persistent mechanism requires editing the central serializer and knowing construction order.
- **Reconstruction.** Forward caches on spaces, `_reconstruction_seed`, `_reverse_body`, several `_reverse_*` helpers, and surface rendering in `InputSpace/PartSpace` form an implicit trace of the preceding forward pass. Cleared-cache and shape xfails expose this coupling.
- **Reasoning and serving.** Truth/LTM methods, reasoning adapters, realization, query parsing, and generation methods on `BasicModel` make training-time model state and application behavior share one facade and lifecycle.

There is a second form of coupling in the global runtime singletons: `TheXMLConfig`, `TheData`, `TheDevice`, `TheMessage`, `TheGrammar`, and `TheError`. These are convenient service locators, but they make isolation, multiple models in one process, exact launch manifests, and construction tests harder than necessary.

## Consolidate before componentizing

A mechanism should first be consolidated when it has one lifecycle but several owners. It should become a separate component only after that lifecycle has a stable boundary.

For example, STM should not initially be split into more classes. Its buffer, typed metadata, push/reduce policy, predictor, and reset rules should first be made one coherent runtime API. Only then is it useful to distinguish a storage object from a grammar policy. Conversely, launch resolution already has a natural immutable output and can become a component immediately.

The practical rule is:

1. Identify the authoritative state.
2. Move every mutation behind one owner.
3. Replace parked attributes with an explicit request/result object.
4. Characterize the boundary with tests.
5. Extract the owner without changing tensor math.

## Proposed component boundaries

### 1. `LaunchSpec` and `RuntimeContext` - priority P0

`train.py` should be the CLI/remote transport adapter, not a second configuration system. XML defaults, XML overlay values, CLI flags, and environment overrides should resolve once into an immutable `LaunchSpec`. That spec should contain the dataset limits, training schedule, device, precision, compilation policy, seed, artifact paths, and the model's resolved architecture settings.

`RuntimeContext` should hold the explicitly process-scoped services needed to execute that spec: device, logger, RNG handles, grammar catalog, and project paths. Model and data constructors should receive these values rather than read global singletons after construction.

The first extraction can be conservative: keep `TheXMLConfig` internally, but have a resolver produce a frozen snapshot and require `ModelFactory`, `Data`, and checkpoint metadata to consume the same snapshot. Environment variables may remain an input to the resolver; they should not remain a late-bound input to the model.

**Acceptance seam:** one test renders the fully resolved launch manifest and asserts that validation, data loading, model construction, compilation, and the checkpoint record all see the same values.

### 2. `TrainingSession` and `TensorStep` - priority P0

The host-eager training lifecycle and the compiled tensor computation need a hard boundary.

`TrainingSession` should own:

- epoch/trial iteration and batch budgets;
- stream acquisition and sentence/document resets;
- optimizer construction and stepping;
- checkpoint cadence and progress reporting;
- profiling, preflight, and failure handling.

`TensorStep` should accept a `StepInput` plus explicit recurrent state and return a `StepResult` containing predictions, next state, objective terms, and diagnostics. It should contain only the shape-stable tensor work intended for `torch.compile`.

Today `runBatch` is both of these things. That makes the compile boundary a list of exceptions rather than an interface: host-side vocabulary growth, reporting, stream state, error aggregation, reset dispatch, and tensor forward all meet in one method. Separating the two gives the FineWeb launch a testable unit of progress and makes a compiled step callable without manufacturing a whole training run.

**Acceptance seam:** eager and compiled `TensorStep` return the same named outputs and objective terms for a fixed `StepInput`; `TrainingSession` can be tested with a deterministic fake step.

### 3. `CorpusSource`, `StreamCursor`, and `BatchAdapter` - priority P0

`Data` currently combines dataset selection, downloading, parsing, synthetic fixtures, normalization, split storage, tokenization, target preparation, and loader construction. Preserve `SentenceStreamDataset`, but put a protocol in front of it:

- `CorpusSource` yields stable document/sentence identities and a source manifest.
- `StreamCursor` owns resumable multi-row position and reset state.
- `BatchAdapter` converts a cursor tick into the model's typed `StepInput`.

The distinction between a corpus byte stream and a labeled trial dataset then belongs in adapters rather than in `runEpoch`. A checkpoint stores `StreamCursor.state_dict()` and the `CorpusSource` manifest, not a collection of model-private counters that are later interpreted as cursor movement.

This boundary is essential for a long FineWeb.edu run: exact resumption is a data property, while batch counting is only training telemetry.

**Acceptance seam:** interrupt/resume produces the same next document IDs, bytes, row reset mask, and targets as an uninterrupted run, across a shard boundary.

### 4. `ForwardFrame` and `ModelRuntimeState` - priority P1

The forward path needs explicit dataflow objects. A useful split is:

- `ForwardFrame`: immutable or single-use values for one step, including the input event, masks, per-stage PS/WS/CS carriers, active-row gates, and the reconstruction trace.
- `ModelRuntimeState`: recurrent values that intentionally survive steps, including discourse state, model time, inter-sentence state, and STM state.
- `StepResult`: terminal output, next runtime state, loss terms, attention observations, and optional reconstruction artifacts.

This replaces the model-as-message-bus pattern. In particular, values like `_prev_cs_for_ps`, `_prev_cs_for_ss`, `_staged_in_sub`, and `_serial_row_gate` should be passed, not discovered on sibling objects. Persistent state should be listed and checkpointable; everything else should die with the frame.

Do not attempt this as one rewrite. Start with `_begin_step` / `_end_step`, then thread the existing carrier values through the per-word loop while leaving the underlying `Space.forward` methods unchanged.

**Acceptance seam:** running two model instances interleaved in one process produces the same results as running each alone, with no state crossing between frames.

### 5. `STMCoordinator` - priority P1

The current STM implementation has at least four legitimate concerns, but no single authoritative facade:

- `ShortTermMemory` provides the live idea buffer and rule scorer.
- `SymbolSubSpace` owns typed stack metadata and some backing capacity.
- `ConceptualSpace` predicts, perceives, and pushes events.
- `BasicModel` decides bounded reduction, sentence-relative protection, compaction, row reset, and reverse snapshot use.

First create an `STMCoordinator` API over the existing objects:

```
begin_step(batch, reset_mask)
predict(prior, routing) -> prediction
perceive(event, row_gate)
reduce(policy, protection) -> ReductionTrace
```

```
snapshot() -> STMSnapshot
end_sentence(row_mask)
```

`STMSnapshot` should contain payload, depth, typed references/categories, and ordering convention. Both reconstruction and discourse prediction should consume that object. Once every mutation is behind this API, storage and reduction policy can be separated if doing so still buys clarity.

**Acceptance seam:** the existing serial/parallel, padding no-op, bounded-fold, typed-STM, and cleared-cache reconstruction tests run through the coordinator; no caller reads `_buffer`, `_depth`, or a sibling's transient STM attributes.

### 6. `ObjectiveSet` - priority P1

`Error` already has most of the reporting behavior needed for an `ObjectiveSet`, but it should be instantiated per step/session rather than imported as `TheError`. Spaces may return named `LossTerm` values; they should not write into a process-global registry.

A `LossTerm` should record name, scalar tensor, configured weight, category, producer, normalization denominator, and active-row count. The objective coordinator should be the only code that applies the curriculum and combines terms for backward. This makes an unlabeled FineWeb batch auditable: the launch can fail if the resolved objective set is empty, non-finite, or contains only a term whose active count is zero.

Keep the existing `ModelLoss` math. The componentization target is ownership and observability, not a new loss formula.

**Acceptance seam:** a preflight test asserts the exact active objective names, weights, finite values, and nonzero gradient-bearing total for the production config.

### 7. `CheckpointManager` and component state providers - priority P1

Checkpoint persistence is currently a central method that knows how to reach inside the lexicon, WholeSpace taxonomy, PartSpace ramsification, BPE chunker, optimizer, RNGs, counters, and corpus manifest. This will grow every time a new store, such as the word store, becomes persistent.

Introduce a versioned `CheckpointBundle` with top-level sections:

```
model_state
optimizer_state
runtime_state
data_cursor_state
rng_state
component_state
launch_manifest
schema_version
```

Each stateful component should implement a small provider contract such as `checkpoint_state()` and `restore_checkpoint_state(blob, version)`. The manager owns atomic write, version dispatch, manifest validation, and diagnostics. Components own their schemas and migrations.

This is consolidation as much as extraction: vocabulary and BPE migrations remain close to those components, while the manager stops knowing their private attributes.

**Acceptance seam:** a checkpoint saved after several streamed steps restores parameters, optimizer moments, all RNG streams, cursor position, vocabulary, word-store/taxonomy state, and the next-step loss exactly enough for the project's determinism contract.

#### 8. `ReconstructionPipeline` and `ForwardTrace` - priority P2

Reconstruction is the clearest place where a nominally modular forward/reverse API is undermined by implicit state. The reverse path can require caches, generated rules, STM snapshots, active masks, span metadata, and codebook identity left on objects by the most recent forward.

Create an explicit `ForwardTrace` containing only the information that cannot be recovered from the terminal idea and persistent model state. A `ReconstructionPipeline` should then own three phases:

1. plan/unfold the symbolic or grammatical derivation;
2. reverse the space transformations using the trace;
3. render percept/byte/word identities into the requested surface form.

The trace makes two architectural questions answerable in tests: whether a reverse is a true inverse of a transformation, and whether generation is able to proceed from an idea without replaying a particular forward pass.

The remaining reverse/shape xfails are useful acceptance tests for this boundary. They should remain strict, but be grouped separately from xfails whose missing prerequisite is semantic learning rather than runtime ownership.

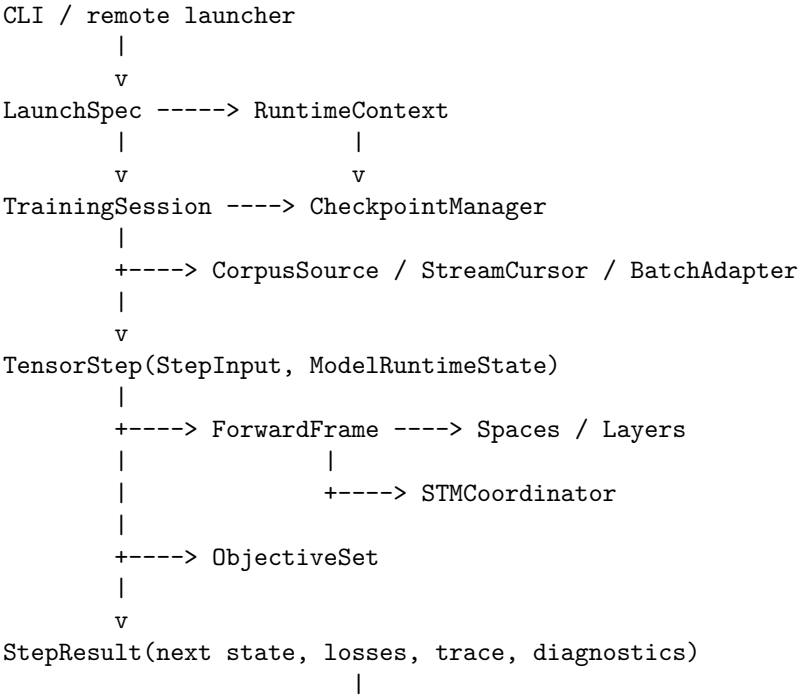
#### 9. `ReasoningRuntime` - priority P2

Truth storage, LTM provisioning, query detection, search/reasoning, and surface realization are application services over a trained model. They should share the model's spaces and stores through explicit read/write interfaces, but they do not need to live on the training model class.

Extracting them later keeps the initial work focused on large-scale training. The useful early change is simply to stop adding new serving methods to `BasicModel`; new reasoning behavior should enter through a dedicated facade.

## Target dependency direction

The desired dependency direction is deliberately one-way:



```
                    +----> ReconstructionPipeline
                    +----> ReasoningRuntime
```

Lower layers must not import the session, launcher, or global objective registry. The launcher should not know the internals of a vocabulary or STM buffer. The checkpoint manager coordinates component state without interpreting it.

## Refactoring sequence for large-corpus readiness

The order below minimizes semantic change and puts the highest-risk FineWeb mechanisms first.

0. **Characterization tests and this ownership map.** Refactoring an implicit API without pinning it only moves the bugs. Gate: the existing full suite, strict-xfail sweep, and production preflight remain green.
1. **Frozen `LaunchSpec` plus launch manifest.** Every later component needs one authoritative configuration. Gate: XML/CLI/environment precedence and manifest parity.
2. **`CorpusSource` plus resumable `StreamCursor`.** A 20M-scale run must be restartable before other cleanup is valuable. Gate: interrupted versus uninterrupted shard-boundary equivalence.
3. **Versioned `CheckpointManager`.** Cursor and launch state need a durable home. Gate: exact next-step resume, including optimizer and RNG state.
4. **`TrainingSession` / `TensorStep` boundary.** This stabilizes the compiled hot path and makes preflight representative. Gate: eager/compiled parity, the zero-host-sync CUDA gate, and bounded smoke training.
5. **`ForwardFrame` and explicit runtime state.** This removes hidden per-step dependencies before STM or reverse behavior changes. Gate: interleaved-model isolation and no-stale-frame tests.
6. **Consolidated `STMCoordinator` and `ObjectiveSet`.** These affect the meaning and gradient of every production step. Gate: typed STM, gated-row no-op, reduction, and active-objective tests.
7. **`ReconstructionPipeline` with explicit trace.** This addresses the main cluster of architectural xfails without destabilizing launch/resume work. Gate: cleared-cache, dimensional, grammar-operation, and idea-only reconstruction tests.
8. **`ReasoningRuntime`.** This is important, but not a prerequisite for safe corpus-scale substrate training. Gate: truth/LTM/reasoning suites through the new facade.

## Tests as component contracts

The test suite should increasingly describe public boundaries rather than private method choreography. Existing tests already provide good raw material:

- `test_fineweb_preflight.py`, `test_stream_dataset.py`, `test_cursor_universal.py`, and `test_data_no_byte_loss.py` define the corpus and cursor contract.
- `test_compiled_step_invoked.py`, `test_ir_fullgraph_compile.py`, `test_brick_no_sync.py`, and `test_cuda_graph_capture.py` define the tensor step's compile boundary.
- the `test_*stm*` files, `test_per_row_hard_reset.py`, and `test_padded_rows_no_op.py` define STM ownership and row-gating behavior.
- `test_reconstruction_roundtrip.py`, `test_explicit_dimensions.py`, and `test_stm_recon_from_cleared_cache.` define the reverse trace contract.
- `test_ltm_consolidation.py`, `test_truth_grounded_reasoning.py`, and `test_reasoning.py` define the later reasoning facade.

During extraction, replace tests of private helper choreography with boundary behavior. Keep a strict xfail only when it names intended, specific behavior; an obsolete call sequence is not a compatibility contract.

## Definition of a successful component

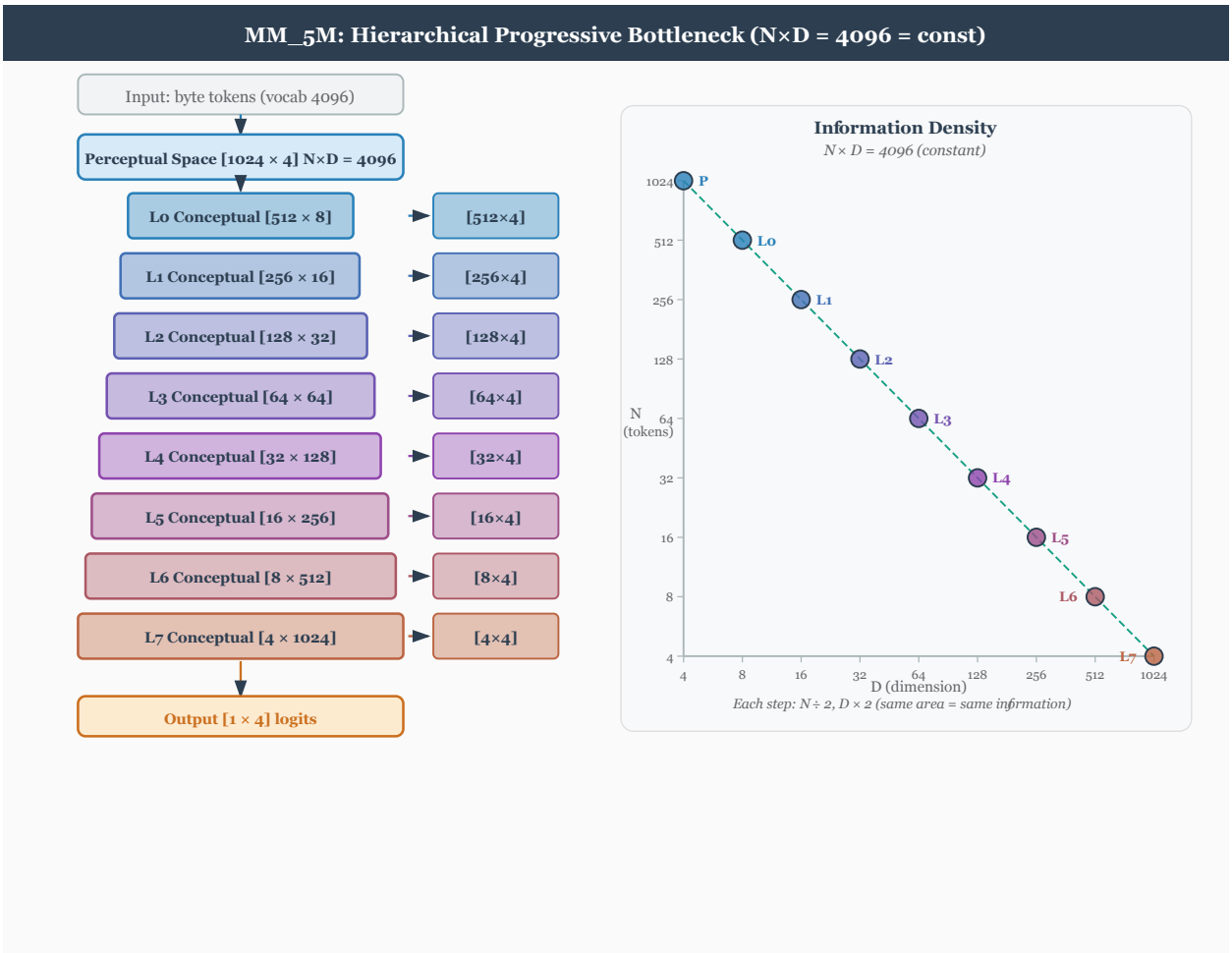
A new boundary is doing real architectural work only if it satisfies all of the following:

- It has one authoritative owner for every state value it mutates.

- Its inputs, outputs, and persistent state can be named without referring to a sibling object's private attributes.
- It can be constructed in a test without initializing unrelated global services.
- Its checkpoint state is explicit and versioned when it survives a step.
- Host-eager and compiled responsibilities do not cross accidentally.
- Its tests assert observable behavior, including negative and resume cases.
- Removing the old forwarding shim would not require callers to know the new component's internals.

The goal is not a larger class count. It is a runtime whose state transitions are as explicit and typed as the cognitive spaces already are.

# Basic Model




***The Basic Model of Cognition is a neural network of four spaces: real space, perceptual space, conceptual space, and symbolic space.***

**Model Entities:**

| Entity | Meaning |
|---|---|
| $x$, $\hat{x}$, $X$ | Actual and predicted input (experiences), input space |
| $y$, $\hat{y}$, $Y$ | Actual and predicted output (actions), output space |
| $p$, $P$ | Percepts, perceptual space |
| $c$, $C$ | Concepts, conceptual space |

| Entity | Meaning |
|---|---|
| $s$, $S$ | Symbols, symbolic space |
| $W_e$ | Experiencing, experiential weights |
| $W_a$ | Acting, action weights |
| $W_k$ | Knowing, knowledge weights |
| $W_t$ | Thinking, thought weights |

***Background***

Based on the Basic Model of Cognition at cognitivesciencesociety.org/framework-cognitive-science; see also The Whole Part.

## Relation to LLMs, Formal Concept Analysis, and DisCoCat

BasicModel is a language-capable architecture, but it is not organized like a standard transformer LLM. A transformer LLM usually represents token context, semantic association, syntactic composition, and memory inside dense hidden state. BasicModel makes those roles explicit: percepts ground the input, concepts bind part/whole support, symbols provide addressable references, and grammar/reasoning layers compose or test those references.

In Formal Concept Analysis terms, a concept has both an **extension** (the percepts that support it) and an **intension** (the attributes, relations, or symbolic edges that define it). BasicModel implements a fuzzy neural version of that split rather than a strict binary concept lattice. In DisCoCat terms, the grammar path composes word/object vectors into sentence-level meaning through typed reductions. BasicModel extends that DisCoCat-style composition with bidirectional reconstruction, truth weighting, and explicit meronymic codebooks.

## Inputs and Outputs

Inputs and outputs derive from a real space that is ineffable. The original theory framed learning as tabula-rasa perturbation at temperature; the current implementation expresses that as initialized weights plus ergodic `bias`/`var` buffers driven by gradient energy. See Ergodic.md.

Inputs scale to `[-1, 1]` via global data min/max. The signed range represents presence and absence symmetrically. Paraphrasing Augustine, *negative is the privation of the positive.*

Action is a function of perceptual space and our expectation, filtered by conceptual projections. Performance errors are defined relative to action, desired effect, and exhaustion. Expectation errors are defined relative to perception and perception-as-reconstructed.

To accommodate a changing input context window, input is augmented with relative spatiotemporal encoding by extending its dimensionality. The attention mechanism treats what and where pathways separately.

## Perceptual Space

Perceptual spaces are composed of perceptual prototype vectors, or ***percepts***.[1] Percepts that do not correspond to objects are *hallucinations.*

Percepts form a Voronoi tessellation using prototype and/or exemplar vectors, related via Self Organizing Map connections, operating in parallel as prototype-theory categories.

Perceptual accuracy is the distance between inputs and prototype vectors — similar to cosine similarity, becoming a simple distance measure for poorly known percepts.

[1] Perceptual space is known as "embedding space" in transformer literature.

Attraction and aversion warp perceptual space: attraction is a pole at the percept's location, aversion is a zero. Perfectly recognized percepts lie on the unit hypersphere within conceptual space. Two percepts can be joined into one concept without loss.

## Conceptual Space

***Concepts*** create boundaries in conceptual space, grounded in the support vectors of perception. Concepts support logical negation; they may refer to negative entities. Conceptual spaces are capable of representing negative entities. Concepts that do not correspond to percepts are *false*.

Concepts collect percepts into fuzzy sets — linear separating hyperplanes partitioning conceptual space, or sets defined as linear combinations of percept activations.

Concepts are relative: meaningful only insofar as they partition a meaningful space. As concepts approach certainty, decision boundary slope becomes infinite. A single unknowing state is common to all knowledge vectors (activation slope zero).

Conceptual spaces are similarity spaces partitioned by decision boundaries imposing positivity and negativity on a hypersphere of knowing.

Within conceptual space, attraction/aversion to concepts expresses as hyperplane bias. Certainty is the slope of the activation function (tuned after other parameters stop changing).

A concept can be split into two percepts without loss.

### Order, extension, and intension (2026-07-02)

Concepts occupy a **ramsified order**. An **order-0 concept** is the output of mixing a part and a whole at the corpus callosum — it is defined **extensionally**, by its perceptual support (the percepts that fall under it), and it is **subsymbolic**: a conceptual-space citizen with no relational definition yet. It is grounded directly. A **higher-order concept** is defined in terms of other symbols (its sparse constituent edges) and so acquires an **intensional** definition (its *sense* — the relations that fix it), while still retaining extensional grounding, inherited transitively through its constituents down to order-0's support and refinable by direct experience. Ascending the orders is therefore a **graded extensional → intensional shift, never a hard switch**, and the grounding becomes increasingly *mediated* (reached through the definitional graph rather than directly from percepts) — which is why superordinate concepts feel more theoretical and less perceptual (Rosch et al. 1976; the basic → superordinate trajectory). This is the reconciliation of the prototype-vs-definition tension: a concept is *both* its region and its definition, because intension and extension are carried by two different structures over the same object. Concretely — **intension is the sparse edge-graph; extension is the atom's position** (Frege's sense / reference; Carnap's intension / extension, realized as graph-over-geometry).

Because every concept, at any order, has an extensional footprint (a grounded atom), specific and general objects share **one conceptual space**: a single dictionary holds each concept as a *point*, and the relational graph is structure laid *over* that space, not a second space. The single-space claim is not a new assumption — it follows from grounding (symbols are a subtype of perceptual objects; see Symbolic Space below). One refinement matters: a concept's home in the space is its **point/atom**, *not* necessarily a convex region. Gärdenfors's convexity holds for *natural* concepts and is the **order-0 / extensional limit**, cleanest at the grounded base; **discontiguous, high-order** objects still live in the same space as points, but their extension is **graph-defined** (the things reachable through their definition), not a convex region — which is exactly why the word↔object bridge needs a MetaSymbol ("no convex set is specific enough," see doc/Mereology.md). So: **one space of points plus a relational graph, with convexity as the basal/extensional reading.** The cognitive grounding for the dense-perceptual / sparse-symbolic split is in doc/Architecture.md "Cognitive grounding."

The 2026-07-02 two-phase rework makes the extensional/intensional seam OPERATIONAL: the forward is a purely continuous PS/WS↔CS pump (extension settling) followed by ONE late snap to the order-0 codebook block – quantization exactly at the bandwidth seam – and then the symbolic composition over the relational graph (intension). Order-0 concepts ARE codebook rows (extension = the atom's position, no edges); order

$\geq 1$ concepts are edge-defined (intension = the sparse graph). Three relational primitives carry the graph (Alec 2026-07-02): the sec-4c ORDERED PAIR $[whole, part]$ (whole $\Rightarrow$ part / if $\rightarrow$ then); the SINGLETON – a whole containing exactly one symbolic part, the unit-set $\{x\}$ Lewis resisted, here the constructive primitive; and the recursion VINE (ordered chains from nested pairs – order from nesting, since each row is set-like). At the storage level, each sparse entry pairs one concept index with one symbol index. Repeating the same concept index across entries accumulates its set-like definition; recursively nesting relation concepts supplies vine order. See the relation-table entry contract in Architecture.md. Where the relation does NOT need order (the word/object meta), the read-out is typed intersection, not slot order. See doc/Architecture.md sec A; the un-ramsified successor (iteration over one untyped square layer, with Kripke groundedness as the cycle diagnostic) was sketched separately during earlier design work.

## Symbolic Space

*Symbolic Spaces* are composed of percepts that reference concepts — a subtype of perceptual spaces mapping onto conceptual spaces.

Symbols have no spatial context: zero-dimensional points within perceptual space. In humans, perceptual space projects onto a 3D cerebellar subspace before symbolic encoding.

Symbols are independent, permanent, and integral (as references). The top-down serial activation differs from bottom-up perceptual activation — it casts shadows.

The formation of symbols increases the dimensionality of conceptual space (outer product); the formation of concepts from percepts decreases the dimensionality of perceptual space (inner product).

Symbols are erroneous if their associated concept is not true. Participation in multiple sets creates structural relations that replace positional information.[2]

## Reasoning

Reasoning in symbolic spaces involves computing with parts and wholes — Venn-diagram computations enabling dynamic Bayesian-type statistics. Learned probabilities can be stored as unidirectional connection weights between concepts, possibly with part-whole structure imposed by corresponding symbols.

## Symmetric Perception: the Single Layer Linear Case

Perception is bidirectional: we see the world and the world sees us. The egocentric/cultural view treats perception as passive; in neural networks this manifests as forgetting the influence of outputs on inputs.

Symmetric Perception finds $f(x)$ minimizing MSE with respect to:

$$\begin{pmatrix} in \\ out \end{pmatrix} \cdot \begin{pmatrix} f(x) & 0 \\ 0 & f^{-1}(x) \end{pmatrix} = \begin{pmatrix} out \\ in \end{pmatrix}$$

In the linear case, $f(x)$ becomes a single invertible matrix $A$. When the mapping is nonlinear and an inverse may not exist:

$$\begin{pmatrix} in \\ out \end{pmatrix} \cdot \begin{pmatrix} f(x) & 0 \\ 0 & g(x) \end{pmatrix} = \begin{pmatrix} out \\ in \end{pmatrix}$$

This resembles $Ax = b$, except $x$ is known and we want weights $A$ mapping one space to another. The standard LLS solution $x = (A^\top A)^{-1} A^\top b$ becomes finding $A$ such that $Ax = b$ and $A^{-1}b = x$.

Retrocausality suggests an additional objective: $A$ should be optimal for both forward and inverse transforms. Approximately: $A$ as a product of two matrices, each performing a perfect surjection: $A \cdot A^{-1} = I$.

[2]For example, {0,1} and {0,1}, if orthogonal, pose no constraint on the location of the 1 in their 2x2 grid. However, {0,1,1} and {0,1} partition a 3x2 space and require two 1's on the same row — a fact present in neither set alone.

$$(\frac{b}{2} * x^{-1}) \cdot (\frac{x}{2} * b^{-1}) = I$$

Best $A$ requires iterative solution (NN learning, ping-pong style) or Gram-Schmidt orthogonalization with respect to $A$ and $A^{-1}$.

## Symmetric Perception: the Single Layer Nonlinear Case

If $f(x)$ and $g(x)$ share weight matrix $W$:

Forward computation and gradient:

$$\hat{y} = f(g^{-1}(x)W)$$
$$e_y = \frac{1}{2}(\hat{y} - y)^2$$
$$\Delta W_y = \eta(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x)$$

Reverse computation and gradient:

$$\hat{x} = g(W^\top f^{-1}(y))$$
$$e_x = \frac{1}{2}(\hat{x} - x)^2$$
$$\Delta W_x = \eta(\hat{x} - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)$$

Since $W$ is shared, summing the partial derivatives gives:

$$\frac{\partial e_y}{\partial W} + \frac{\partial e_x}{\partial W} = \eta(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x) + \eta(\hat{x} - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)$$

Setting gradient to zero, both errors going to zero when $x = i$ and $y = o$. But $xy - iy = ox - yx$ may hold — simultaneous gradient descent ping-pongs. A better cost function multiplies the two error partials — intersection of the spaces satisfying both:

$$\frac{\partial e_y}{\partial W}\frac{\partial e_x}{\partial W} = \eta(\hat{x} - x)(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))\delta_g(W \cdot f^{-1}(y))f^{-1}(y)g^{-1}(x)$$

## Distance Metrics

$$d(x_1, x_2) = \frac{\|x_1\|^n \cdot \|x_2\|^n}{\|x_1 - x_2\|^{2n}}$$

Z-plane form:

$$d(x_1, x_2) = \|x_1 - x_2\| \cdot \frac{(x - z_1)}{(x - z_2)}$$

where $z_1$ is a zero and $z_2$ is a pole.

## Exploration/Exploitation as a function of temperature

$$y = x \cdot \frac{(W + rt)}{\|W\|}$$

where $r$ is a random vector and $t$ is temperature.

### Transfer Functions

Current invertible transfer functions use factorized linear maps rather than the earlier SVD-rotation sketch. `InvertibleLinearLayer` parameterizes the map with triangular factors and a diagonal scale so forward and reverse share the same learned transform without forming a dense inverse on every call. `SigmaLayer` and `PiLayer` wrap those maps for additive and multiplicative composition. See Architecture.md for the implementation contract.

## Spaces

**Architecture relation.** The Space/SubSpace split is where BasicModel most directly departs from conventional LLMs: hidden state is routed through named percept, concept, symbol, STM, and codebook stores instead of one residual stream. PartSpace/WholeSpace plus the Concept codebook provide the fuzzy Formal Concept Analysis reading (extent, intent, and concept order), while SymbolSpace and the grammar host provide the DisCoCat-like composition path from typed syntax to vector meaning.

**2026-06-21 terminology note (one noun per-space).** This doc follows the percept / concept / symbol convention: a **percept** is a PartSpace/WholeSpace thing (dimensionally-embedded, extensional; the two subtypes are **part-percepts** = atoms, $\sigma$, and **whole-percepts** = properties/regions, $\pi$); a **concept** is a ConceptualSpace relation tying one part-percept $\leftrightarrow$ one whole-percept (the Concept codebook); a **symbol** is a SymbolSpace 0-D reference to a concept (the symbol codebook). The prose below reserves "symbol" for genuine SymbolSpace things; WholeSpace content is whole-percepts, and its compact `[0,1]` emission is the symbol that migrates to SymbolSpace. Code identifiers (e.g. `get_symbols`, `subspace.what`) are unchanged — those get a separate code pass.

**2026-06-12 update (part/whole rename — the perceptual split).** `PerceptualSpace` → `PartSpace` and `SymbolSpace` → `WholeSpace`; both now subclass a thin shared `PerceptualSpace(Space)` base (no parameters, no submodules — state-dict keys unchanged). Both views are *perceptual*: PartSpace synthesizes bottom-up over atoms (Sigma), WholeSpace analyses top-down over unity (Pi). XML config sections and grammar-file scopes are renamed to `<PartSpace>` / `<WholeSpace>` to match. The PS/SS shorthand in older notes reads part-side/whole-side. The freed name `SymbolSpace` was **reintroduced 2026-06-19** as the grammar/word space-role (formerly `WordSpace`).

**At the corpus callosum, objects are analysed and synthesized** — by sending them back to PerceptualSpace: wholes get split and parts get chunked. In symbolic "mode", the objects that get sent back are *symbols*.

**Terminology (semiotics).** There are **objects** and **references**. A reference is either a **sign** or a **symbol**. A *sign* is a quantized version of the referent (same space, snapped to a codebook row). A *symbol* is an unrelated version of the referent — an arbitrary code of much lower dimensionality (cf. the zero-banded symbol codes below: whole-percepts of symbols carry `.where = .when = 0`).

**`subspace.what`: atoms vs properties — wholes are types.** On `PartSpace` the `.what` rows are **atoms** (letters and words — the existing lexicon / BPE / MPHF front ends, gathered by `Codebook.lookup`). On `WholeSpace` the `.what` rows are **properties**, and properties are what wholes *are*: a `.what` row is a binary proposition over the field, and only a proposition can bifurcate the input into two clearly-identified regions (`+1` has-type / `-1` complement) the way `materialize_property` does below. Types apply to **runs of characters, not single characters**: the four canonical properties are `isLetter`, `isSpace`, `isDigit`, `isPunctuation` — the existing `LETTER` / `WHITESPACE` / `DIGIT` / `PUNCT` classes of `Layers.char_class_region`, one codebook code each. A whole is a **maximal constant-type run**; words are runs of letters, punct, or digits. `materialize` hands the per-position selection (`subspace._index`, renamed from

_active 2026-06-12) to the codebook so it produces the materialized object: atoms via lookup, or a per-position **region membership** via Codebook.materialize_property (mode="property") — the region that has the property (> 0) and the region that doesn't (<= 0). The continuous sinusoid backend is wired as the worked example; the content-keyed (whitespace/words) backend reuses the meronymic analyzer's span segmentation and is a documented seam. Properties are **low-frequency by construction**: there is too much detail to learn a codebook over the whole input unless the atoms are low-frequency, so the property basis frequency is tied to the input length (the k-th harmonic completes only ~k/2 cycles over the whole field, mirroring EndpointSumWhere's sub-half-period div_term). The learnable property row is the low-frequency atom; the basis enforces the coarseness.

**2026-06-11 update (meronomy cutover — MeronomySpec/MeronomyPlan, Stage 9).** With <architecture><meronomy>on</meronomy> (now the model.xml default) the meronymic slots bind the membership kernels: PartSpace.sigma → SigmaLayer2 and WholeSpace.pi → PiLayer2, each through the K3-wire MeronymicFoldAdapter ($\chi$ at the boundary; the wire keeps carrying signed scalars; the fold computes on memberships, near-identity at init). Additions this page should be read with:

- **CS encoding**: stored reference rows are gauge-signed unit directions (semantic embedding only); certainty is activation magnitude, polarity activation sign; gauge fixed at mint (Spaces.gauge_orient). Reference-half lookup is the pole quotient (embed._pole_aligned_score); token/form codebooks keep full-vector lookup.
- **The callosum**: percepts cross NAMELESS and FACTORED (ConceptualSpace.factor_percept — content selects the row, evidence sets the SIGNED $a \in [-1, +1]$, 2026-07-06 correction: the percept input stays non-negative, but the match against a CONCEPT row is un-clamped abs-argmax, so an anti-aligned row — a "known false" exclusion — is reachable); parallel mode is the 2N mixing matrix; serial mode bypasses it (fusion migrates into the shift).
- **Two codebooks, one table**: the towers ARE the PS/SS codebooks (extent/intent; shared with IS recognition — recognition is tower lookup), linked by the word-keyed binding table (References.ReferenceTable) hosted on WholeSpace (search-then-mint gate; adopt_stage0_evidence stays ground-half memory, not naming). Symbols are atomic, zero-banded codes — whole-percepts of symbols carry .where = .when = 0 (the Mind marker).

**2026-06-09 update (analysis/synthesis orientation — supersedes the ownership notes below).** The corrected orientation (rev. 2026-06-09; see Philosophy.md):

- **InputSpace emits the DUAL VIEW**: forward(x) -> (percepts_in, concepts_in) — the atom view (content [B, N, 1]) for the perceptual branch and the unity view ([B, 1, N]) for the symbolic branch.
- **PS = bottom-up SYNTHESIS**: owns ONE SigmaLayer (self.sigma, additive/union; the Pi/Sigma swap) and the <synthesis> front ends (radix/bpe/byte/lexicon/mphf — was <chunking>).
- **SS = top-down ANALYSIS**: owns ONE PiLayer (self.pi, multiplicative/intersection), the <analysis> division knob (byte/word/raw/sentence/grammatical/meronomy). Stage 0 consumes the unity view as symbolic evidence (per-part coarse means, snapped through the live SS codebook with the asymmetric STE forward leg).
- The meronymic analyzer (bin/perceptual_analyzer.py) is SS-side analysis machinery now; PS <synthesis>analyse was removed.

Sections below that predate this orientation are marked or should be read against it.

**2026-06-02 update (subsymbolic analyzer).** New IdeaSubSpace (bin/Language.py) – the PS-meronymic carrier analogue of SymbolSubSpace (spans, parent/child links, route ids, marker-route replay fields). WholeSpace gains insert_operations (grammar operations in a dedicated operator codebook – _operation_vectors / _operation_positions – separate from the symbol codebook so the symbol/idea/.where namespace is untouched),

resolve_ps_terminal / null_sem (PS-to-SS binding), and operator_superposition. The PS meronymic analyzer lives in bin/perceptual_analyzer.py.

**Status (2026-05-27):** updated for the substrate refactor. PS is a single-arg input processor (self.pi + self.sigma). CS is an STM container + grammatical CPU; no atomic forward fold. SS owns the unified word lexicon codebook with paired (orth, semantic) rows. Grammar dispatch lives on the signal router (LanguageLayer) at SymbolSubSpace.languageLayer; the CKY Chart and STM shift-reduce parsers are retired. LiftLayer / LowerLayer are binary GrammarLayer subclasses with internal Sigma / Pi (no longer substrate-borrowing). GrammarLayer gains an optional butterfly cascade mode.

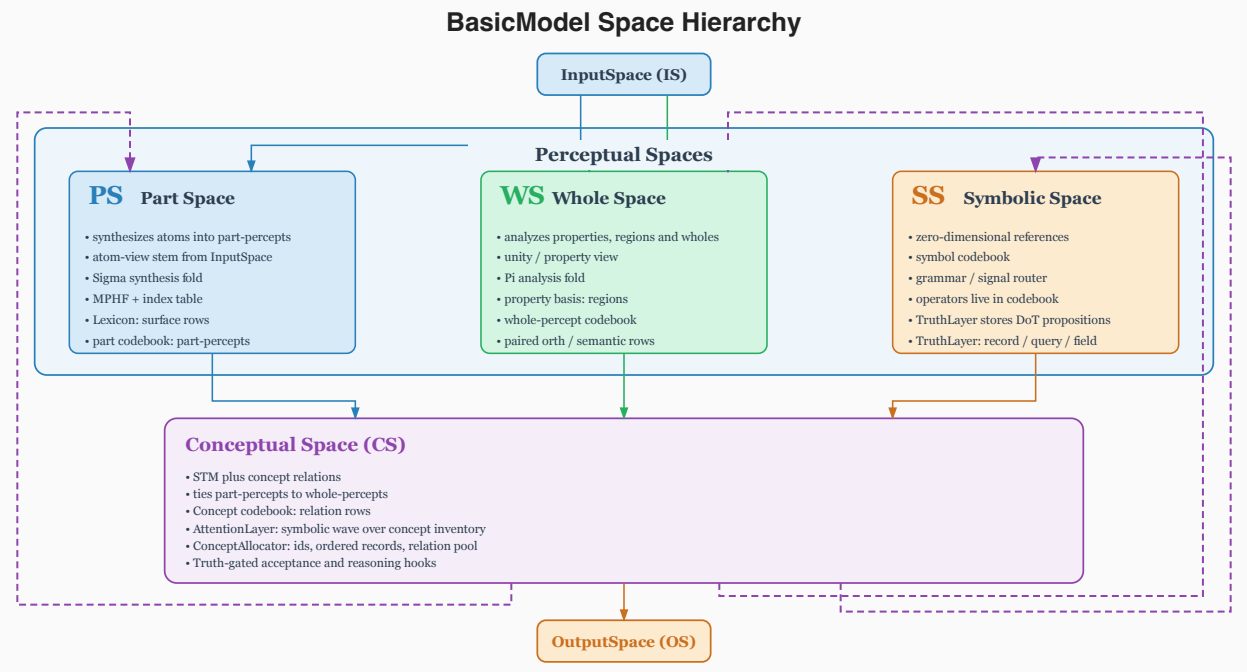
## Overview

BasicModel is a pipeline of five **spaces** plus a grammar host (SymbolSpace), each performing a distinct representational transformation. Data flows forward from raw input to task output; the reverse pass reconstructs the original input from the symbolic representation. The legacy SubwholeSpace and SyntacticSpace classes have been retired — the subsymbolic role is filled by PartSpace itself, and the grammar runs from SymbolSubSpace.languageLayer (the signal router; subsumed the retired Chart).

**SymbolSpace literally subclasses Space** (class SymbolSpace(Space), bin/Language.py) — it is a peer tower alongside PartSpace/WholeSpace (the third .what/.where carrier), not merely an attached "grammar host." It constructs via nn.Module.__init__ directly, deliberately SKIPPING Space.__init__'s object/what/where/when VQ-basis build, and holds a SymbolSubSpace coordinator (the typed-STM stack + grammar dispatch carrier) as self.subspace. SymbolSpace is a transparent container: reads that miss its own attrs fall through __getattr__ to the held coordinator, and writes forward through __setattr__, so symbolSpace.X call sites are unchanged whether X lives on SymbolSpace or on the coordinator it forwards to.

```
Forward:  InputSpace -> PartSpace -> ConceptualSpace -> WholeSpace -> OutputSpace
Reverse:  OutputSpace -> WholeSpace -> ConceptualSpace -> PartSpace -> InputSpace
```

The pre-2026-05-27 "two feedback loops" (S → C per-stage, C → P cross-forward) are retired. The recurrent character lives in (a) STM accumulation across words in SERIAL / GRAMMATICAL mode, and (b) the T-pass PARALLEL refinement loop driven by <subsymbolicOrder> via WS.forward(cs_out=...) / CS.forward(...) iterations — PS itself runs ONCE, at stage 0 only; the per-stage recurrence advances through the prior stage's CS output, not repeated PS calls (see **Sigma / Pi ownership** below).




---

## Base Class: Space

All spaces inherit from `Space`, which manages:

- **Shape management.** `inputShape` / `outputShape` as `[nObjects, nDim]`. Subclasses read dimensions from `TheObjectEncoding`.
- **Codebook / VQ quantization.** When `nVectors > nActive`, a codebook holds candidate vectors; top-k selection gives the bottleneck.
- **Reshape flag.** When in/out object counts differ, the `[B, nIdeas, nDim]` tensor is flattened before the next space and restored on the way back.
- **Attention.** The legacy boolean `hasAttention` is deprecated and inert (kept only as a backward-compat alias); use the `<attention>` element instead (`off` | `primer` | `second-order` | `low-rank`).
- **`set_sigma` propagation.** Ergodic-mode noise level cascades from the top-level model down through every child layer.

### Reset cascade: hard vs. soft

Every space exposes `Reset(batch=None, hard=True)`. The signature is required (legacy zero-arg fallback removed).

| Call form | Scope | Use |
|---|---|---|
| `space.Reset()` | All rows | Whole-state wipe (epoch boundary) |
| `space.Reset(batch=b, hard=True)` | Row `b` only | Document boundary |
| `wordSpace.soft_reset(batch=b)` | Row `b` sentence-scoped state | Grammar `<start>` reduction |

Hard-reset clears: parse stack, `_last_svo`, `_stm_fired`, codebook commit accumulator, discourse history, `serial_cache`, `_ar_embedded`, `_end_of_stream` for the affected rows.

Soft-reset clears per-sentence working buffers (parse stack rows, `_last_svo[b]`, category and reconstruction stacks) and re-arms `_stm_fired[b]`. Does **not** touch discourse history (`InterSentenceLayer` ring buffer) or codebook EMA — those are document-scoped.

Reset is dispatched from `runEpoch`, never from inside `runBatch` (the pure compute brick — see Architecture.md).

---

## Sigma / Pi ownership (Pi/Sigma swap, rev. 2026-06-09)

History: the 2026-05-13 rebalance described "each space owns one operator"; the 2026-05-27 substrate refactor gave PS both folds; Stage 10 made PS pi-only. The **corrected analysis/synthesis orientation (2026-06-09) swaps the folds to their proper sides**: Sigma (sum/union) is synthesis and belongs to the bottom-up PartSpace; Pi (product/intersection) is analysis and belongs to the top-down WholeSpace. CS remains an STM bookkeeper with **no atomic forward operator**; Lift / Lower stay **binary `GrammarLayer` subclasses** with internal Sigma / Pi (no substrate-borrowing).

| Space | Owns |
|---|---|
| **PartSpace** | one `self.sigma` (SigmaLayer — the synthesis fold), the `<synthesis>` front ends, MPHF + index tab |
| **ConceptualSpace** | STM (`ShortTermMemory`, depth ~8) + (when sparse-active) the single untyped square `ConceptualAtt` |
| **WholeSpace** | one `self.pi` (PiLayer — the analysis fold), the `<analysis>` + `<lexer>` knobs, the unified word lexic |

**Composition (per-mode):**

- **SERIAL / GRAMMATICAL** (`<serial>true</serial>`): one iteration per word.

```
PS_t = PS.forward(word_t)                     # single positional arg; no CS feedback into PS
WS_t = WS.forward(ws_universe, cs_out=prevCS_forSS)  # universe every pump; cs_out is the carrier f
CS_t = CS.forward(PS_t, WS_t)                  # STM shift + push
prevCS_forPS, prevCS_forSS = CS_t's cs._subspaceForPS/_subspaceForWS  # carried to the next word
router.dispatch_at_C(STM)           # read-only grammar ops on STM contents
router.dispatch_at_S(STM)           # codebook-write-required ops via SS
```

- **PARALLEL** (`<serial>false</serial>`): T iterations of PS over CS.

```
contribution = PS.forward(IS)      # PS runs ONCE, at stage 0 only -- not
                                   # repeated per stage (single-pass subsymbolic decision)
for t in 1..T = <subsymbolicOrder>:
    # Universe glue contract (dual-towers rev 2): sparse-parallel
    # (symbolicOrder > 0) offers the universe EVERY stage; otherwise it
    # bootstraps stage 0 only and t > 0 stays carrier-driven.
    WS_t = WS.forward(unity if (sparse_parallel or t == 0) else None,
                      cs_out=prevCS_forSS)
    CS_t = CS.forward(contribution, WS_t)   # STM[t] = combine (parallel write; no shift)
    contribution = CS_t             # stage k+1's contribution IS stage k's CS output
router.dispatch_at_C(STM)           # grammar ops after STM population
```

The legacy formula `C = sigma_percept(pi_input(IS) + pi_concept(C_prev))` is retired entirely.

### Lift / Lower in the new ownership

`LiftLayer` and `LowerLayer` are no longer "rule-id annotators over shared substrate." They are first-class binary `GrammarLayer` subclasses (Stage 4 of the substrate refactor):

- `LiftLayer(GrammarLayer)`: `arity=2`, `rule_name="lift"`, `space_role='CS'`. Owns an internal `self._sigma: SigmaLayer` for the pairwise additive (sigma-style) math. `forward(left, right)` delegates to `_sigma.compose`.
- `LowerLayer(GrammarLayer)`: `arity=2`, `rule_name="lower"`, `space_role='CS'`. Owns an internal `self._pi: PiLayer` for the pairwise multiplicative (pi-style) log-domain math. `forward(left, right)` delegates to `_pi.compose`.

Both reverse cleanly via their internal layer's reverse. Both gain butterfly mode for free via `GrammarLayer` base inheritance (Stage 5).

The signal router dispatches them as binary reduce ops at the CS, weighted by `Grammar.rule_probability` (the per-position copy/reduce score head).

**Butterfly mode on `GrammarLayer` (Stage 5)**

`GrammarLayer(butterfly=True, N=N)` allocates an FFT-style **element-pair** cascade over a flattened `[B, M]` view (`M` = `N` padded to the next power of two; `n_levels = log2(M)`), NOT a packed per-node matrix Parameter. Storage is three per-node LDU-triplet `nn.Parameter`s shared by every `GrammarLayer` subclass — `butterfly_L: [n_levels, M//2]`, `butterfly_d: [n_levels, M//2, 2]`, `butterfly_U: [n_levels, M//2]` (sub-diagonal, the two diagonal scalars, super-diagonal of each node's `2 x 2` LDU block) plus a `butterfly_perms` buffer (per-level bit-reversal permutations placing XOR-neighbour elements adjacent). Identity init (`L=0, d0=d1=1, U=0`) makes the cascade identity at construction. `_butterfly_pair_forward` / `_butterfly_pair_reverse` (`GrammarLayer`, `bin/Layers.py`) implement the closed-form `2x2` LDU pair op ONCE at the base class – sign-flip on the off-diagonals + reciprocals on the diagonals, no `torch.linalg.solve`. Every `GrammarLayer` subclass (`SigmaLayer`, `PiLayer`, `LiftLayer` / `LowerLayer`, `IntersectionLayer` / `UnionLayer`, `ConjunctionLayer` / `DisjunctionLayer`, …) reuses this SAME pair op via `self._butterfly_forward` / `self._butterfly_reverse` in butterfly mode; there is no per-subclass `_butterfly_pair_op` override or distinct pairwise math (atanh/einsum/tanh, min/max, etc.) — the subclass identity only matters for the non-butterfly forward/reverse math.

Parameter savings: $O(N \cdot \log N)$ scalars per cascade vs $O(N^2 \cdot D^2)$ for a single big matrix. Wired into the space folds (`PartSpace.sigma` / `WholeSpace.pi` post the Pi/Sigma swap) by the global `<sigmaPi>` mode (default butterfly). Closes the XOR convergence target (`test_mm_xor.py`).

---

## Normalization and Ranges

| Space | Data Contract |
|---|---|
| InputSpace | Measured tensor features scale to presence; signed text embeddings retain sign |
| PartSpace | Modal/demuxed what/where/when encoding. Marked radix/meronomy percept stores expose one-sided |
| ConceptualSpace | Combined/muxed event encoding. Positive concept atoms are scaled by signed, tanh-bounded activation |
| WholeSpace | Whole-percepts (`[0,1]` presence). The compact emission is the symbol that references a concept; one sy |
| OutputSpace | Rescaled from activation range to original data range |

`SyntacticSpace` is retired (see Section "SyntacticSpace — retired" below). Grammar / chart machinery lives on `SymbolSpace` and is attached to `WholeSpace` (the canonical grammar host).

**Data scaling.** `Data` computes global `input_min`/`input_max` and `output_min`/`output_max` at load time. InputSpace uses `Data.normalize(x, "input")` to scale measured presence data to `[0,1]`; signed text embeddings remain in `[-1,1]`. OutputSpace uses `Data.denormalize(x, "output")` to restore the original output range.

**Whole-percept presence.** Whole-percepts live in `[0, 1]`; since conceptual activations range `[-1, 1]`, the mapping is `presence = (activation + 1) / 2`. `SubSpace.get_symbols()` / `set_symbols()` (code identifiers unchanged) perform the conversion.

**Demuxed mode.** When `InputSpace.demuxed=true`, what/where/when components are stored independently in the SubSpace rather than concatenated. ModalSpace routes each component through independent PartSpaces; downstream spaces see an identical muxed tensor via `materialize()`.

---

## Percept Geometry: Positive Unit Hypercube

> **Design.** A percept is a vector of independent **presence** features. Each coordinate is a membership in `[0,1]`: `0` is absent or nothing, `1` is fully present or everything, and intermediate values are partial presence. A percept is one-sided; its opposite is the complement `1-x`, not the signed negation `-x`.

The percept hypercube is the grounded extent side of the architecture. An LLM usually learns such grounding indirectly through token statistics; BasicModel keeps percept presence as an explicit carrier. Formal Concept Analysis enters when perceptual extents are paired with whole/property intents to form concept order. DisCoCat enters later, when grammar composes conceptual meanings rather than raw percept memberships.

Percepts and concepts therefore use different geometries: percepts use the cube for presence, while concepts use a positive atom whose signed scalar activation carries polarity and certainty. Keeping the roles distinct avoids treating a percept's unused negative half as if it were sensory content.

### Presence, Not Signal

A percept feature answers *how present is this?*

| Value | Meaning |
|---|---|
| `0` | nothing or absent; the empty corner |
| `1` | everything or fully present |
| `x in (0,1)` | partial presence |

There is no negative presence. On the percept path:

- `ConceptualSpace.factor_percept` (a staticmethod, `bin/Spaces.py`) keeps the percept INPUT non-negative (`percept.clamp(min=0)`; percepts are one-sided) but the EVIDENCE it returns against a CS concept row is SIGNED in `[-1, +1]` (2026-07-06 correction: abs-argmax selection so an anti-aligned row — a "known false" exclusion — is reachable, not clamped away); a zero percept yields zero evidence (the dot-product tautology).
- The meronymic membership folds (`SigmaLayer2` / `PiLayer2` in `bin/Layers.py`) operate on memberships in `[0,1]` through `log/exp`, flooring the bottom element with `EPS_LOG` before `log(0)`.
- A marked radix/meronomy Codebook (`is_percept_store`) is read through a UNORM straight-through clamp, so forward lookup and reverse decode see the same `[0,1]` rows while the float master remains trainable.

The signed orthographic Lexicon is a separate path. Its sign is form content, so it stays in the projective unit-ball geometry described under Lexicon.

### Complement, Copart, and Negation

A percept's opposite is its complement:

```
antipode_percept(x) = 1 - x
```

Mereologically, the complement of a part is its **copart**, the rest of the whole. With the whole normalized to `1`, the carrier `[part, copart] = [x, 1-x]` has dependent axes; the copart is derived rather than stored.

This is not the catuskoti/tetralemma carrier. That truth representation has two independent axes so it can distinguish BOTH from NEITHER:

| Carrier | Axes | Redundant? | Layer |
|---|---|---|---|
| part/copart `[x, 1-x]` | dependent | yes; collapse to one `[0,1]` coordinate | percept |
| catuskoti `[TRUE, FALSE]` | independent | no | concept/truth |

Complement on `[0,1]` and negation in centered signed coordinates are the same reflection. Let `y = x - 1/2`; then `1-x = 1/2-y`. The operation is shared, but the stored carriers and their semantics remain distinct.

### Sigma/Pi Membership Lattice

The meronymic fold/split operators form a bounded lattice:

| Operator | Role | Identity | Absorber |
|---|---|---|---|
| sigma | synthesis / union | `0`: `x union empty = x` | `1`: `x union everything = everything` |
| pi | analysis / intersection | `1`: `x intersection everything = x` | `0`: `x intersection empty = empty` |

The log-space fold floors `0` because nothing absorbs multiplication and `log(0)` is unbounded. The membership value zero and the operator identity are different roles: zero is sigma's identity but pi's absorber.

These lattice roles are not the same thing as the `SigmaLayer`/`PiLayer` butterfly ownership described above. The membership operators constrain mereological presence; the butterfly folds carry higher-order transformations.

### Percept-to-Concept Seam

The percept origin and the concept origin have different readings:

- percept `0` is observed absence;
- concept activation `0` is uncertainty or no assertion.

At the percept level, absence contributes no positive evidence. Presence enters the order-0 concept snap as a non-negative source term; it is not re-centered or injected as a concept vector. `PerceptDim` and `ConceptDim` remain decoupled. The sparse conceptual feedforward pyramid grows signed structure through learned weights and activations. Negative conceptual content therefore comes from concept operations and signed relations, not from negative percept coordinates.

The signed-to-membership chart `chi(a) = (1+a)/2` and its inverse belong at the truth/catuskoti boundary (`Ops.eval_chart` / `eval_chart_inv`), not at the percept-to-concept seam.

### Geometry Split

The implementation keeps percept and concept/symbol carriers distinct:

| Property | Percepts |
|---|---|
| space | positive unit hypercube `[0,1]^D` |
| coordinate | per-axis presence |
| opposite | complement `1-x` |
| sides | one-sided |
| metric | membership distance; presence MSE for the marked store |
| magnitude | presence per axis |

| Property | Concepts / symbols |
|---|---|
| space | positive concept atom; signed activation or signed Lexicon path |
| coordinate | atom direction plus activation polarity and certainty |
| opposite | negation `-x` |
| sides | two-sided sign is semantic content |
| metric | dot product for concepts; projective lookup for Lexicon rows |
| magnitude | scalar activation carries certainty |

In the sparse conceptual implementation, the stored `ConceptDim` atom is strictly positive (`softplus(atom)`). Sign and magnitude live in the scalar activation: `concept_code = signed_activation * softplus(atom)`. The phrase "signed sphere" describes the activation's role, not a signed stored atom.

A symbol is also not a synonym for a concept or a whole. A concept is a `ConceptualSpace` relation over percepts; a symbol is a reference to a concept. Their epistemic roles remain distinct even when a signed geometry is shared by an implementation path.

### SBOW Situates Concepts, Not Percepts

`conceptual_sbow_loss_codes` in `bin/embed.py` situates concept codes by neighborhood attraction and pairwise negative-sample repulsion. The gradient is tangential: it changes a concept's angle without overwriting its certainty radius.

Percepts are deliberately not SBOW-situated:

1. Percept identity is grounded by nearest-row decode in its perceptual metric; moving a row changes the token or signal it denotes.
2. Byte spans, `.where`/`.when`, and the meronymic tower anchor its structural role.

Distributional movement would therefore break grounding even if the composition algebra remained invertible. Concept codes can move by substitutability because their identity is mediated; percept rows cannot.

### Bounded Encodings

The bounded ranges admit efficient fixed-point storage:

| Format | Range | Intended carrier |
|---|---|---|
| UNORM | `[0,1]` | percept presence |
| SNORM | `[-1,1]` | signed concept/symbol values |
| Q-format fractional | bounded signed or unsigned range | CPU/DSP equivalent |

Training remains in `float32`/`bf16`. Forward-time quantization keeps a float master, applies a saturating function (`tanh` or `clamp`), and uses a straight-through estimator or stochastic rounding. Over a bounded unit range, SNORM16 has a roughly `3e-5` uniform grid and is finer than bf16 across most of the interval; SNORM8 is materially coarser.

### Live Paths and Migration Status

The tracked baseline enables the global meronomy chart, but the actual carrier still depends on the input and synthesis mode:

| Path | Live representation |
|---|---|
| measured tensor (`input_presence=True`) | normalized to `[0,1]` |

| Path | Live representation |
|---|---|
| text embedding (`input_presence=False`) | retained in `[-1,1]` |
| radix/meronomy percept store | UNORM `[0,1]`; presence decode |
| orthographic Lexicon | signed projective unit ball |
| conceptual atom | positive atom; signed activation |

The `[0,1]` percept-store migration is complete: `Codebook.getW()` is the read chokepoint for both forward gather and reverse decode, so no seam adapter is needed. The store lives on the PartSpace subspace, the dataflow container. The remaining open decision is whether the orthographic Lexicon should ever move to `[0,1]`; doing only the input half would break signed embedding reconstruction.

**Mereological Guarantees**

The encoding is hybrid.

**Guaranteed by construction:**

- Byte-position structure - radix longest-match, slot order, the exact id-to-bytes table, `.where`/`.when` brackets, and run containment - does not depend on vector position.
- When a fold is configured as invertible/butterfly, `compose(generate(y)) == y` follows from its LDU/butterfly form. A bare linear fold does not provide that guarantee.

**Anchored by the perceptual metric:**

- Vector-to-token identity uses complement-aware presence MSE on the marked percept store and wrapped/projective lookup on the signed Lexicon path.
- The learned sigma composition determines which whole a part set produces. Radix supplies ordered references through a trie; it is not place-value arithmetic.

Moving percept rows would leave byte structure and configured invertibility intact while redirecting nearest-row decode. The model could compose and decompose consistently yet name the wrong token, which is precisely why percepts remain anchored.

See also Mereology and Logic.

---

## Codebook Similarity Metric

`Codebook` wraps `VectorQuantize`. Similarity metric per space:

| Store | Geometry and metric | Retrieves |
|---|---|---|
| PartSpace radix/meronomy | `[0,1]^d`; presence MSE | percept-presence row |
| Lexicon and unmarked WholeSpace | `[-1,1]^d`; wrapped MSE | form or symbol row |
| Conceptual similarity codebook | unit rows; dot product | SBOW concept row |

The ConceptualSpace entry is the concept dictionary's *situating* metric, not the forward concept-production path. Its input magnitude carries belief certainty, and lookup ranks $\arg\max_i(x \cdot c_i)$.

**Presence and Wrapped-MSE (Perceptual / Symbolic)**

These codebooks store *what something looks like*, so the right notion is coordinate-wise distance. The marked percept store uses `_presence_mse_score`: ordinary unwrapped MSE on `[0,1]^d`, monotonically remapped so identical rows score `1` and complementary corners score `-1`. The signed Lexicon and unmarked stores use

`_wrapped_mse_score`, which first wraps the coordinate difference into the signed unit cell. In both cases, lookup chooses the row with the largest similarity (equivalently, the smallest MSE in the active geometry).

For an ordinary, unwrapped Euclidean codebook, retrieval expands $\|x - c_i\|^2$:

$$\|x - c_i\|^2 = \|x\|^2 + \|c_i\|^2 - 2\,(x \cdot c_i)$$

$\|x\|^2$ is constant across $i$ and drops from argmin:

$$\arg\min_i \|x - c_i\|^2 = \arg\max_i (x \cdot c_i - \tfrac{1}{2}\,\|c_i\|^2)$$

`VectorQuantize` can therefore keep $\|c_i\|^2$ in `_b_norms_sq`:

```
indices = (flat @ codebook.T - 0.5 * b_norms_sq).argmax(dim=-1)
```

One matmul + one broadcast subtract + one argmax. Skips the `sqrt`, the per-row $\|x\|^2$ add, and the cdist autograd plumbing.

### Dot product (Conceptual)

> **Note.** This dot-product metric is the `similarity_codebook`'s retrieval metric (used by the substitutability / SBOW *situating* signal), NOT the forward concept-production path. When the sparse transform is active, a concept code is produced by the snap + feedforward sigma-pyramid (order $k$: cand = $\tanh(W[a \mid 1])$ gathered to `order_slice(k)` with a per-batch top-K taper — one hop per order, no re-injection — then $a \cdot \text{softplus}(atom)$) — there is no `argmax_i (x · c_i)` concept retrieval on the forward path, and the atoms are softplus-positive rather than maintained unit-norm by EMA. See **ConceptualSpace → The symbolic phase**.

ConceptualSpace concepts are *named directions* in belief space. $x \cdot c_i$ gives the *signed strength of belief that $x$ affirms concept $i$*:

- $+1$ fully affirms; 0 orthogonal; $-1$ fully denies

Two consequences:

1. **Codebook must be unit L2-norm.** EMA renormalizes after each update.
2. **Input must NOT be normalized.** The magnitude *is* the certainty signal. Cosine similarity would divide it out.

For *ranking*, $x \cdot c_i$ and $\cos(x, c_i) = (x \cdot c_i)/\|x\|$ are monotone-equivalent (positive constant cancels). Omitting input normalization preserves certainty and costs less:

```
# codebook is unit L2-norm (maintained by EMA)
indices = (flat @ codebook.T).argmax(dim=-1)
```

### Configuring the metric

`use_dot_product` is a class attribute on `Codebook` (default `False`). Set it on a Space subclass to opt in — `ConceptualSpace` does this. The underlying `VectorQuantize.use_cosine_sim` flag is historical; after the April 2026 perf pass, input-side normalization is gone, so the effective meaning is "codebook unit-norm; rank by dot product".

---

## Ramsification table (per-code fold record)

The Pi / Sigma folds carry a reference onto sortable mereological space but do **not** preserve their own *ramsification* — the record of how the code was produced — so a folded code cannot, on its own, be reconstituted. `Codebook.ramsification` is the small adjacent table that fixes this: a `[V, max_order]` `uint8` sidecar, index-aligned with the codebook rows (both perceptual codebooks — `PartSpace.subspace.what` and `WholeSpace.subspace.what`), recording for each code which fold it was routed through at each subsymbolic pass — `FOLD_NEITHER` / `FOLD_SIGMA` / `FOLD_PI`. `invert_ramsified(code, row, sigma, pi)` walks that sequence in reverse pass order, applying `sigma.reverse` / `pi.reverse` per recorded fold, landing back at the codebook row that produced the code.

The table is an **opt-in additive sidecar** (`enable_ramsification`; a plain attr like `part_parents` / `category_ids`, not a Parameter or buffer) — it adds no state_dict keys and cannot move a pinned basin; it resizes with the codebook (`grow_to`). Live per-pass stamping in the subsymbolic pump loop is the deliberate cutover seam (call `record_fold` where `PartSpace.sigma` / `WholeSpace.pi` fire).

**Word abstraction order.** A code's `abstraction_order` is its fold count (non-`NEITHER` passes), and words are subsymbolic at several abstraction levels: a **proper noun** (prototype / token) matches raw at **order 0**; a **regular noun** (type) at **order 1**; a **count noun** (concrete only under a determiner) at **order 2**; higher orders are more abstract. Words need not be nouns, but all benefit from an abstract / discontiguous spatial representation. This connects to the ramsified order hierarchy in `Language.Taxonomy` and the order-typed STM plan.

## Codebook Uniqueness Contract

Every codebook entry is identified by its **row index** and must carry **distinct `.what` (`WhatEncoding`) content**; the old `.where`-keyed uniqueness scheme is retired:

- **`.where` — positional / spatial-extent key (no longer a codebook row key).** The cross-codebook **`.where` slice registry was RETIRED** (modality re-architecture, 2026-06-04; `WhereEncoding`, `Spaces.py`). `allocate_codebook_slice` / `global_max_val` / `reset_codebook_registry` were removed: there is no shared where-space to allocate disjoint slices in. Codebook identity is now the **row index** (the `_index` selection), `.where` keeps only its positional / spatial-extent role, and CS→WS reverse decode is **content-match** (nearest row). Cross-codebook taxonomy is row/position-keyed via WholeSpace's explicit dicts (`category_ids`, `part_parents`).
- **`.what` — distinct prototype content.** Identical `.what` collapses to the same parthood identity (`equal(A, A) = 1`) — a redundant pair the network can't distinguish.

Current enforcement:

| Source | Mechanism |
|---|---|
| WholeSpace codebook | `ImpenetrableLayer` overlap penalty + variance floor; five-relations classifier pushes pairs tow |
| ConceptualSpace codebook | `ImpenetrableLayer` available; not yet wired by default |
| PartSpace Lexicon | Cosine-margin pode/antipode SBOW training |
| InputSpace vocabulary | Shares PartSpace's Lexicon |

`.where` is now a positional / spatial-extent carrier (the slice registry is retired — see above); codebook identity is the **row index**. `.what` uniqueness is **learned** (encouraged by `ImpenetrableLayer` + antipodal quotient) and, together with the distinct row indices, keeps the parthood lattice well-formed.

---

## Lexicon (Projective Unit Ball)

The **Lexicon** (`bin/Layers.py`) backs PartSpace word embeddings and WholeSpace whole-percept prototypes. Each row is a vector $w_i$ in the **projective unit ball** — the closed ball $B^D = \{x : \|x\|_2 \leq 1\}$

with the **negation identification** $w \sim -w$ realizing real projective space $\mathbb{RP}^D$.

**Terminology pin** — three notions sometimes conflated:

- **Pode** of $(a, b)$: midpoint $(a + b)/2$; SBOW positive-pair attractor.
- **Wrapped pode**: midpoint via the $\pm$-quotient, $(a - b)/2$; the midpoint through *negation* of $b$.
- **Antipode** of a single point $p$: furthest point. On the flat torus unique (wrap$(p + 1)$); on $\mathbb{RP}^D$ **not unique** — the maximum-distance set is the orthogonal hyperplane.

So $-w$ is the **negation** of $w$, *not* the antipode.

### Distance and lookup

For $a, b \in B^D$ the projective squared distance is

$$d^2_{\mathbb{RP}}(a, b) = \min(\|a - b\|_2^2,\ \|a + b\|_2^2) = \|a\|_2^2 + \|b\|_2^2 - 2\,|\langle a, b\rangle|.$$

With pode$(a, b) = (a + b)/2$ and wpode$(a, b) = (a - b)/2$, $d_{\mathbb{RP}}(a, b) = 2 \cdot \min(\|a - \text{pode}\,\|,\ \|a - \text{wpode}\,\|)$. The lookup picks whichever rep of $b$ ($b$ or $-b$) is closer to $a$.

Sorting by smallest $d^2_{\mathbb{RP}}$ = sorting by largest score$(x, w_i) = |\langle x, w_i\rangle| - \frac{1}{2}\|w_i\|_2^2$.

Implementation: cache `W_norm2 = W.square().sum(-1)` once per optimizer step; top-k is `(x @ W.T).abs() - 0.5 * W_norm2` followed by `torch.topk` — dense matmul + abs + broadcast subtract. No $V \cdot D$ outer-product.

The `Lexicon` API:

```
lexicon = Lexicon(V, D)
lexicon.project_unit_ball_()          # after optimizer.step()
W_index, W_norm2 = lexicon.lookup_index()

# Projective (RP^D) --- antipode-aware, default.
idx, dist_sq, scores = Lexicon.topk_rp(x, W_index, W_norm2, k=32)

# Plain L2 --- for sites where w and -w are distinct.
idx, dist_sq, scores = Lexicon.topk_l2(x, W_index, W_norm2, k=32)

# Pairwise primitives:
Lexicon.rp_distance_sq(a, b)
Lexicon.rp_similarity(a, b)
Lexicon.rp_pode(a, b)
Lexicon.rp_wrapped_pode(a, b)
Lexicon.rp_closer_rep(a, b)           # sign(<a, b>) * b
```

For $V \gtrsim 10^5$, use `topk_rp_chunked` to bound peak score-tensor size.

### SBOW training: pode (attractor) and antipode (repulsion target)

- **Pode (attractor).** Positive-pair updates pull $a$ and $b$ toward pode$(a, b)$; the gradient picks the closer of $b$ and $-b$ for shorter-arc attraction.
- **Antipode (balancing repulsion target).** Negative-pair updates push the row toward the furthest point. On $\mathbb{RP}^D$ this is a $(D - 1)$-sphere, so SBOW samples a representative orthogonal direction.

Negative-sampling gradient has two regimes by sign$\langle a, b\rangle$: positive case is standard contrastive repulsion along $(a - b)$; negative case pushes $a$ away from $-b$ along $(a + b)$.

After every optimizer step the trainer calls `lexicon.normalize()`, clipping $\|w_i\| \leq 1$. `W_norm2` should be refreshed when weights change.

Torus primitives (`Lexicon.wrap`, `Lexicon.delta`, etc.) and the `torus=True` constructor flag remain as **legacy** static methods (the earlier Lexicon used the flat torus $T^D = [-1, 1)^D$ with wrapped MSE). New code must use the `rp_*` primitives.

---

## InputSpace

**Role.** Receives the raw source buffer and lifts it into the model's internal working dimensionality.

**Text mode forward.** Delegates tokenization to `Lex`, producing a span table of `(start, end, type)`. Each span → a vector with two components:

- `nWhat` dims — token content, encoded via `Basis` / `Codebook` (the word embedding lookup).
- `nWhere` dims (4) — the **2-rung start LADDER** (`WhereEncoding`, 2026-07-09 multi-rung pass; mirrors the `.when` v2 ladder below): $[\sin(p\,\omega_{lf}), \cos(p\,\omega_{lf}), \sin(p\,\omega_{hf}), \cos(p\,\omega_{hf})]$ — two TRUE quadrature pairs over ONE quantity, the byte **START** position $p$ only (the END is NOT in the band; content terminates the tile), with $P_{lf} = $ `<wherePeriod>` (default 8192 input bytes; the full-sentence RANGE) and $P_{hf} = P_{lf}$ / `<whereRungRatio>` (default 32; the fine RESOLUTION rung — one byte at ≈ 0.0245 rad at the default period, above the ≈ 0.02 rad measured reverse-transport noise, the Gate-B closing measurement). `decode` = atan2 per pair + HF branch resolution by LF (the canonical positional identifier across IS / PS / SS taxonomies). The 2026-06-16 **endpoint-sum BRACKET** form (angle = span center, magnitude = span extent, with a `decode_span`) was RETIRED from `.where` on 2026-07-09 in favor of this start-only ladder; `WhereEncoding` has no `decode_span` (the analyzer-side `EndpointSumWhere` in `bin/perceptual_analyzer.py` is a separate codec that keeps the bracket form for its own span key). The period is config-derived: `<architecture><wherePeriod>`, decoupled from `nObjects`; the build seam raise-to-fits with a warn-once for longer inputs (2026-07-04 encoding pass).
- `nWhen` dims (4) — the **2-rung start LADDER** (`WhenStartDurationEncoding`, 2026-07-04 encoding pass): $[\sin(s\,\omega_{lf}), \cos(s\,\omega_{lf}), \sin(s\,\omega_{hf}), \cos(s\,\omega_{hf})]$ — two TRUE quadrature pairs over ONE quantity, the event **onset** $s$, with $P_{lf} =$ `<whenPeriod>` (default $10^6$ ticks) and $P_{hf} = P_{lf}/$`<whenRungRatio>` (default 32; safe branch bound ≈ 35 at the dense-support phase floor). Constant norm $\sqrt{2}$; decode = atan2 per pair + HF branch resolution by LF. **Shared-HF caveat:** at the default short HF, start-fine is a LOCAL phase fingerprint (sharp neighbor discrimination), absolute branch unresolved band-only — **absolute addressing rides the exact long-int clock** (`BasicModel.when_time`), the Option-C hybrid side-band. DURATION left the band (it was write-only: `decode_span` had zero callers, tense rotates the onset only, aspect is retired) — exact extents belong to the record store when aspect is built. Tense is the onset-vs-`now` relation; `shift_time` rotates BOTH pairs coherently, each at its own $\omega$. Both `.where` and `.when` are now START-only 2-rung ladders; the endpoint-sum bracket each once carried in the muxed band is retired from both (`WhereEncoding` / `WhenStartDurationEncoding`) and survives only in the analyzer-side `EndpointSumWhere` span codec.

Result: `[nActive, nWhat + nWhere + nWhen]` tensor.

**Text mode reverse.** Inverts the span encoding: each vector → nearest codebook entry, then spans → characters via the stored offset table.

**Numeric mode.** Tensor data passes through unchanged; `LiftingLayer` projects native input dim (e.g. 784 for MNIST) to `nDim`. Non-embedding inputs are scaled to `[-1, 1]` via the global data min/max.

**Key parameters.**

| Parameter | Description |
|---|---|
| `nActive` | Sequence length |
| `nDim` | Output dim per vector |
| `lexer` | (Moved to `<WholeSpace>`, Phase 4b — lexing is analytic cutting. InputSpace executes the intake; the knob liv |
| `codebook` | Whether input values are discrete |
| `demuxed` | Store what/where/when independently |

2026-05-28: per-Space `<nWhere>` / `<nWhen>` XML knobs are retired. The band is architectural: `architecture.canonical_shape(section)` — (nWhere=4, nWhen=4) on every interior space, (0, 0) on OutputSpace (the 2026-07-04 encoding pass widened `.when` 2 → 4; the 2026-07-09 multi-rung pass widened `.where` 2 → 4 to match, `bin/architecture.py`).

**Invertibility.** Always non-invertible; reverse is a separate reconstruction using the span table.

### Document streaming and `valid_mask`

Documents longer than `nOutput` bytes are not truncated. `TheData` maintains a per-row cursor `(doc_idx[b], offset[b])` and `next_tick()` returns `(input, output, hard_eos)` where `input` is a `[B, nOutput]` slab containing the next ≤ `nOutput` bytes from each row's current document. `hard_eos[b]` is a host-side bool set when row `b`'s cursor exhausts the current document. A short fill at document end NULL-pads the slab tail; `valid_mask: [B, K] bool` flips False for padded positions, and state-mutation propagation skips them.

**Cursor universal — trial mode for non-AR data.** `next_tick()` is the single dispatch for both AR text byte (rolling cursor) and non-AR data (numeric). In trial mode (`slab_bytes` not set), each tick yields one batch of trials with `hard_eos = [True] * B`. The runEpoch outer loop drives `ds.next_tick()` directly for both modes; the DataLoader exists only so existing tests can grab `loader.dataset`.

`_end_of_stream` is a host-side `list[bool]` diagnostic only; the canonical hard-reset signal is the cursor's `hard_eos`.

### AR cursor unfold retirement (2026-05-13)

The legacy AR-training path padded + unfolded the embedded sentence into `[B, K, N, D]` cursor windows so the body could see a `[B*K, N, D]` parallel view of every prefix. At `bs=128`, `K=128`, `N=1024`, `D=10`, the unfolded tensor alone was ~320 MB.

The unfold was retired for AR training on 2026-05-13, replaced with a serial K-cursor loop (`_forward_per_stage_no_unfold`) that walked the same prefixes with a `[B, N, D]` tensor and a per-cursor causal mask.

### Within-sentence AR retirement (2026-05-14)

The serial K-cursor loop itself was retired one day later: the benchmark showed `_forward_per_stage_no_unfold` running at ~18 sent/sec (the K body+head calls dominate) vs the single-shot IR fast-path's ~61 sent/sec, and the real AR objective in this architecture is **next-sentence** prediction (the discourse layer) — not next-token within a sentence.

**Within-sentence training is now IR-only.** `InputSpace.forward` emits `[B, N, D]` (left-aligned, right-padded to N) and `_forward_per_stage` runs a single masked-LM pass:

1. **Stem**: `InputSpace.forward` + `PartSpace.forward` → `[B, N, D]`.
2. **Mask**: `create_ir_mask` replaces a `mask_rate` fraction of WHAT positions with `NULL_PERCEPT`; pre-mask event stored on `_ir_pre_mask_input` as the loss target.
3. **Body**: T stages on B rows (no per-cursor walk, no causal mask).
4. **Head**: `outputSpace` → `[B, N, predDim]`. The head is a side channel — IR loss is computed at the subsymbolic (PS), not at the head.

`runBatch` reads `_ir_mask_positions` and `_ir_pre_mask_input` and computes `MSE(perceptualSpace.subspace at masked positions, _ir_pre_mask_input at masked positions)`. The `<reconstruct>` element (and its `reconstructEnum`) is RETIRED (A1, 2026-06-09): the ConceptualCombine now unconditionally integrates all three streams (PS + SS + CS), so reconstruction is unconditionally from concepts — the former `concepts|symbols|both` selection (target derived by lifting `_ir_pre_mask_input` through `sigma_percept`; see Plan Section "Reconstruction-loss target shape" Option B) is no longer a knob.

<maskedPrediction> is retired; <reconstruct> is retired (the output mode was the only path that fired the reverse pipeline); <reverseScale> is renamed to <reconstructionScale> (the legacy name remains parseable with a one-shot deprecation warning).

Sentence-level AR moves to InterSentenceLayer — see doc/Architecture.md Section "Sentence-level AR (InterSentenceLayer)" for the ARMA(p, q) design.

---

## PartSpace

**Role.** Single-arg input processor — the bottom-up SYNTHESIS branch (Pi/Sigma swap, rev. 2026-06-09). Applies self.sigma (the additive/ union fold) to its argument (the atom-view stem after the synthesis front end embeds). Owns the surface-keyed Lexicon (self.vocabulary) and the MPHF + index table for per-word surface → row lookup.

**Owned state:**

- self.sigma: a single SigmaLayer (percept_dim -> percept_dim, where percept_dim is the EMBEDDED percept width — _fold_width; a widening PS sizes the fold at nOutputDim, not the raw nInputDim). Inherits from GrammarLayer; accepts butterfly=True, N=N for cross-position cascade mode. (The PiLayer PS used to own moved to WholeSpace — Pi is analysis.)
- self.vocabulary: the Lexicon (Embedding), keyed by MPHF over surface bytes. Per-word vectors are nDim-wide (CS-space-dim per the flat-slab invariant).
- self._mphf_gpu_layer: MPHF infrastructure for fast surface lookup.
- self.chunk_layer: BPE machinery (the ChunkLayer from bin/Layers.py).
- self.percept_store (there is no self.radix_layer attribute; percept_store is a Space property that forwards to self.subspace.percept_store, bin/Spaces.py): when <synthesis>radix</synthesis>, the input lookup routes through RadixLayer (radix trie + inverse table + learned codebook + byte fallback). RadixLayer is a first-class Layer subclass in bin/Layers.py (formerly the standalone PerceptStore). PartSpace.reverse invokes RadixLayer.reverse for the structural decode (chunk-id → bytes → slot). Promotion knobs default to threshold=4, min_length=2.

The legacy pi_input / pi_concept ModuleLists are retired, as is the sigma_percept-style additive fold on CS.

**Forward (PS.forward(in_sub, cs_out=None)):**

```
def forward(self, in_sub, cs_out=None):
    # Dual-towers rev 2: PS's own conceptual feedback is stashed, not yet
    # folded (self._cs_feedback = cs_out).
    x_subspace = in_sub
    self._cs_feedback = cs_out
    # synthesis front end embeds (lexicon/bpe/byte/radix/mphf), then:
    primary = self.forwardBegin(x_subspace, returnVectors=True)
    # Unified fold-width law (fold_content_apply, bin/Spaces.py): the sigma
    # fold is sized to the CONTENT columns only (Space.nDim); a wider event
    # carries the trailing where/when band, which rides through unchanged.
    return fold_content_apply(self.sigma.forward, self.sigma.nInput, primary)
```

in_sub is the atom-view stem (PS runs ONCE at stage 0 — the single-pass subsymbolic decision; the per-stage recurrence advances through the ConceptualCombine, not repeated PS calls). cs_out is PS's symmetric counterpart to the cs_out WholeSpace also now accepts (the dual-towers rev 2 signature).

**Math (the sigma fold — PS's synthesis operator):**

```
sigma(x) = tanh(W_sigma @ atanh(x) + b_sigma)   # additive/union, log-domain
```

(The multiplicative pi math — pi(x) = tanh(W_pi @ atanh-domain + b) in the (1+x)/(1-x) log embedding — now lives on **WholeSpace** as the analysis fold; see the orientation banner.)

**Reverse.** `PS.reverse` applies `self.sigma.reverse` on the text path (LDU inverse via `InvertibleLinearLayer`); structural recovery goes through `object_basis.reverse` and (in radix mode) the `RadixLayer.reverse` chunk-id → bytes decode.

**Butterfly mode (Stage 5):** when `<PartSpace><butterfly>true</butterfly>`, the fold is constructed with `butterfly=True`. The cascade length `N` is auto-derived from the space shape (`nInput * nInputDim`, internally padded to the next power of two); there is no `<butterflyN>` knob (it was retired 2026-06-05). `<butterfly>` itself is a deprecated alias for the architecture-level `<sigmaPi>` (new configs should use that). Internal storage becomes the per-node LDU triplet `butterfly_L` / `butterfly_d` / `butterfly_U` (`nn.Parameter`s over the flattened SCALAR element axis, not a packed per-pair matrix — see **Butterfly mode on `GrammarLayer`** above) plus a `butterfly_perms` bit-reversal buffer. Closes the XOR convergence target. PartSpace is subsymbolic and takes no `<codebook>` element (it was retired; PS is fixed to `none`); butterfly weight gradient flow therefore flows through the continuous `.event` passthrough on PS. STE-through-snap (for spaces that do quantize) is a known follow-up.

**Range.** Vectors live in `[-1, 1]^d` (tanh-bounded). No negation operator — percepts represent feature magnitudes with sign indicating direction.

---

## ConceptualSpace

**Role.** Forms concepts from perceptual/symbolic *sources*. When the sparse transform is active (`symbolicOrder > 0` in parallel mode, `_sparse_active()`), a concept is a high-dimensional atom in **ConceptDim** (stored in the CS concept dictionary, the `similarity_codebook`) whose signed activation is produced by the POST-PUMP SYMBOLIC PHASE (2026-07-02 two-phase rework; dual-towers rev 2 feedforward sigma-pyramid, 2026-07-10, superseding the v3 iterated wave): the settled field is snapped to the ORDER-0 snap block (`cs_snap_order0`) and a feedforward pyramid runs up to $K$ = `symbolicOrder` order-indexed rungs over the single untyped square `ConceptualAttentionLayer` (one hop per rung, gathered to that order's rows via `order_slice`, with a per-batch top-K taper — no fixed point, no re-injection), then each activation scales its atom (the decode inlined in `cs_forward_content`). `PerceptDim` and `ConceptDim` are **decoupled** (the weights live in index/activation space); a concept is NOT an additive linear map over percept vectors and there are no "conceptual hyperplanes partitioning perceptual space." `CS.forward` itself is ALWAYS STM bookkeeping only (the transform fires once per forward at `_forward_body`'s cutover, never in-loop). See **The symbolic phase** below.

**Geometry.** A concept code is `signed_activation × softplus(atom)`: the dictionary atom is a **strictly-positive** ConceptDim feature signature (`F.softplus`), and the **sign / magnitude** live in the signed scalar activation — magnitude = certainty, sign = present vs anti-present, low magnitude = chosen radially but uncertain. The sign comes from the signed sparse **weights** (a negative weight = a feature's presence anti-correlates with the concept), not from a stored signed unit direction. (The legacy "named unit-norm direction + `argmax` retrieval" view is retired on the forward path.)

**Owned layer (2026-05-13 rebalance → 2026-05-29 clean-stack → RETIRED).** ConceptualSpace owns **no parameterised fold layer**. The historical `self.sigma_in` / `self.sigma_cs` SigmaLayers below were RETIRED — they are no longer constructed, and `CS.reverse` no longer applies them (see the **Reverse** note). The table records the pre-retirement Stage-10 design:

| Layer (RETIRED) | Direction | Math | Notes |
|---|---|---|---|
| `self.sigma_in` | incoming-contribution fold | per-stage SigmaLayer (Ramsified across stages) | Stage |
| `self.sigma_cs` | residual-CS iteration kernel for stages k > 0 | per-stage SigmaLayer | Same |

**Clean-stack STM (2026-05-29 experiment).** The Stage-10 additive composition

```
folded = sigma_in(combined) + sigma_cs(prev)
```

is replaced with per-stage space-role attribution:

```
stage 0      folded = primary    (PS event from subspace.materialize())
stage k > 0  folded = sym        (SS event from word_subspace.materialize())
```

No additive mixing across space-roles; no residual lift; trivially invertible (read-back, no inverse-Sigma needed). The `STM_k = STM_{k-1} + SS_k` carry-forward variant was tested and reverted — the pure clean-stack form is the landing point.

Because `sigma_in` / `sigma_cs` were dead-weight on the forward path (no gradient — they never fired) while `CS.reverse` applied `sigma_in.reverse` unconditionally, the round-trip carried an UNMATCHED inverse fold (the source of garbage XOR_exact recon tokens). That forward / reverse semantic mismatch was RESOLVED by retiring the layers entirely: with the sparse transform OFF, CS is a pure bookkeeping carrier (forward push / reverse read-back) with no fold to invert, and the symbolic generalization operator moved to WholeSpace (inverted upstream of `CS.reverse` on the reconstruction path, in `BasicModel._reverse_body`). Convergence on MM_xor continues via the PiLayer butterfly cascade. (When the sparse transform is ON, CS DOES own a parameterised forward transform — the `ConceptualAttentionLayer`'s edge values and the concept dictionary — see **The symbolic phase** below.)

**Reverse.** `CS.reverse` is a thin pass-through (no fold layer to invert). The reverse chain operates on the terminal STM contents; per the master plan, no per-stage caches. The sparse-coding reconstruction is referential — the untyped edge lists ARE the concept's decomposition — rather than an inverse fold.

**The symbolic phase (two-phase forward, 2026-07-02; dual-towers rev 2 FEEDFORWARD SIGMA-PYRAMID, 2026-07-10, superseding the v3 iterated wave).** When `_sparse_active()` (i.e. `_symbolic_order > 0` and parallel/`serial=false`), `BasicModel._forward_body` runs the purely subsymbolic pump for `subsymbolicOrder` passes and then ONE cutover on the settled terminal field (`cs_symbolic_phase` = the snap + `cs_forward_content`, `bin/Spaces.py`):

```
a_0        = cs_snap_order0(settled)        # signed tanh normalized-sum presence
                                            # vs the ORDER-0 snap block (+ EMA trace)
a          = pad(a_0, N)                    # order-0 rows, zero-padded to the full inventory
for k in 1..min(K, len(order_caps) - 1):    # K = symbolicOrder: a CEILING, not forced depth
    start, end = order_slice(k)             # order k's row range in the stacked inventory
    cand       = tanh(W [a | 1])[start:end] # ONE feedforward hop, gathered to order k's rows
    winners    = top_k(rank(cand), caps[k]) # per-batch top-K taper (order_caps()[k])
    a[start:end] = cand * winners_mask      # only the winners commit; losers stay 0
code[c]    = a[c] * softplus(atom[c])       # dictionary decode (inlined)
```

`cs_forward_content` (`bin/Spaces.py`) is a strict **feedforward sigma-pyramid**, not an iterated fixed-point wave: order 0 is the snap, and each subsequent order $k$ is read straight off `order_slice(k)` and computed in exactly ONE hop through the shared store `W` — "No fixed point, no re-injection" (the function's own in-code comment). Unlike the retired v3 wave there is no repeated re-application of `W` to its own output and no additive source term `s` carried step to step; `order_caps()` derives each order's row budget as a tile-based taper `[base, base>>1, .., 1]` (`base` = `outputShape[0]` tiles, shrunk until the whole taper fits `nVectors`), and at each order only the top-`caps[k]` candidates by rank survive. Rank defaults to `|cand|`, optionally boosted by `self._relevance_priority` (an admitted-rows-only awareness-spreading score, `rank = |cand| * (1 + score)` with `score = p[row] + (|W| p)[row]`; absent by default, in which case ranking is byte-identical to plain `|cand|` top-K). The per-step wave-settle statistic once tracked as `_cs_wave_qe` is now hardcoded to `None` (`# wave retired`) — a single feedforward pass has no settle dynamics left to report, and the Kripke-groundedness diagnostic that read it, `cs_groundedness_probe`, was REMOVED 2026-07-10 along with the wave (zero grep hits in the current codebase; see Architecture.md "Groundedness and cycles"). Per-order diagnostics instead live on `_cs_level_acts` / `_cs_level_rows` (the winning activations / global row indices at each rung), the latter used to stage the pyramid's per-order winners onto the subspace index so a generic `materialize()` pulls exactly the selected codes.

The percept families AND the per-order role-split families are RETIRED: the store is ONE square untyped `ConceptualAttentionLayer` (a `SparseLayer` subclass) over the stacked concept inventory, edges = fuzzy

set-membership degrees, plus a bias-column edge for relations bounded above by the EVERYTHING pole (a concrete whole retires it). The row space is order-tapered, not a flat two-block split: `order_caps()` (`bin/Spaces.py`) gives `[base, base>>1, .., 1]` (`base = min(outputShape[0], N)`, shrunk until the whole taper fits `N = nVectors`), and `order_slice(k)` is the `[start, end)` row range of order `k` within that stacked taper. The SNAP block is `order_slice(0)` (width `base`) and RESERVES codebook rows for order-0 concepts (no in-edges; their decomposition lives in the reference store); `order_slice(k)` for `k = 1..K` is the RELATION POOL, itself sub-divided per order (each order's cap is the prior order's `>> 1`, first-come within its own slice; overflow warns loudly). Self-edges raise (the Quine atom). Weights are **signed** and learnable (`SparseLayer.values`, grown host-side by `add_concept_edge`, surfaced via `getParameters()` and registered into the optimizer by `_maybe_rebuild_optimizer_for_csw`); the store ALSO owns the DISCRETE relation records (ordered `[whole, part]` constituents; `ConceptAllocator` in bin/Layers.py holds the global ids/caches). Population is at mint: one untyped edge per SYMBOLIC constituent plus the EVERYTHING bias edge (`_populate_concept_weights`; `_concept_source_order` stays as bookkeeping — $K$ is an ITERATION BUDGET, not forced ramsification: a depth-$d$ vine completes at iteration $d$, tail links first). Concretely, each sparse entry pairs one concept row index with one symbol column index. Repeated entries with the same concept index form its set-like definition; recursive `[whole=current, part=rest]` relation concepts form a vine. The normative distinction is stated in Architecture.md. The phase outputs feed the SS leg, the head-side losses (conceptual SBOW on the settled slab), and the concept table — NEVER the subsymbolic carrier (`<sparseReplace>` retired). Off-path (`symbolicOrder = 0`) → no cutover → byte-identical.

**Activation carrier.** `subspace.activation` is the scalar/presence activation used by the Space pipeline. Paired `[pos, neg]` tensors still appear in explicit grammar/truth operators that implement catuskoti logic, and in user-supplied truth-set activations, but they are no longer a space-wide output mode.

**MASK on `SubSpace._active`.** Two orthogonal per-position tensors: `activation` and `_active: [B, N, M]` (modality presence flags). `_apply_mask` is shape-disambiguated:

| Mask shape | Effect |
|---|---|
| Aligns with `out.shape[-1]` (feature axis) | Element-wise multiply on output |
| Aligns with `out.shape[-2]` (position axis) | Zero masked rows of `_active`; `materialize()` gates downstream |

### Word auto-bind deferred to `Reset` (fullgraph forward)

The compiled per-batch forward (`BaseModel.enable_compiled_step`, `torch.compile(fullgraph=True)`) must carry **zero graph breaks**. The word auto-bind — `ConceptualSpace._maybe_autobind_meta`, which grows the WholeSpace codebook and META taxonomy when a freshly-seen percept first lands — is irreducibly host-side: `.item()` loops, `if pid < 0` branches, Python dict/set taxonomy mutation, and (pre-allocation aside) codebook growth. Dynamo cannot trace any of those, so the auto-bind no longer runs on the `forward` path.

Instead (2026-06-03 refactor) it is **deferred to the sentence/document boundary**. `PartSpace._embed_radix` stashes the encountered percept ids (`_forward_input['indices']`) and the pre-pi seed (`_embedded_input`) during the forward — a side-effect dynamo simply replays. On the next hard `ConceptualSpace.Reset` (fired on `hard_eos`, between sentences), `_commit_autobind_from_stash` reads that stash and performs the *same* allocation in eager Python. Same words, same SS rows — only moved from mid-stage to the between-sentence reset, so a downstream tensor op in the same forward no longer sees a mid-forward codebook growth (verified behaviour-preserving across the radix / meta-taxonomy / `mm_xor` convergence suites). Whole-slab configs run one forward per sentence, so the stash is the whole sentence; a per-word/serial config commits the last forward's stash per reset.

Two smaller forward-purity fixes accompany it: `SymbolSubSpace._synthesize_ rule_probs` normalizes branchlessly (`probs / row_sums.clamp_min(tiny)` instead of `if nz.any()`), and the fail-loud `isfinite` guards (here and in `insert_meta` / `record_lbg_pull`) are gated behind `util.MODEL_DEBUG` — a constant the tracer folds away when off, so the data-dependent `.all()` host sync leaves the compiled graph while

divergence still raises under `MODEL_DEBUG` runs and the eager finite-loss guard. (`BASIC_FULLGRAPH=0` relaxes the strict gate to enumerate any remaining breaks; `MODEL_COMPILE=eager` traces without the Inductor C++ backend.)

### ShortTermMemory

ConceptualSpace owns `self.stm` — a `ShortTermMemory` instance, a per-batch stack of unquantized CS "ideas." Post-substrate-refactor, `CS.forward` is STM bookkeeping (shift + push), with no atomic fold layer. (Two-phase rework, 2026-07-02: the symbolic transform no longer runs inside `forward` at all — it fires once at the post-pump cutover and its outputs never enter the STM; see **The symbolic phase**.)

| Property | Default | Configurable via |
|---|---|---|
| Capacity | 8 (within Miller's $7 \pm 2$ band) | `<ConceptualSpace><stmCapacity>N<` |
| Storage | `[batch, capacity, concept_dim]` buffer + `[batch]` depth pointers | `persistent=False` (working state, no |
| Cleared on | Hard `Reset` (sentence boundary) | Soft reset leaves it intact |

API: `push(b, idea)`, `pop(b)`, `peek(b, n=0)`, `snapshot(detach=False)`, `size(b)`, `is_full(b)`, `is_empty(b)`, `clear(b=None)`, `ensure_batch(batch)`.

**STM transition by mode:**

- **SERIAL / GRAMMATICAL**: `_stm_shift_and_push(idea)` — shift slots 0..(cap-2) to take values from slots 1..(cap-1), write new idea to the top slot (default cap = 8, so slots 0..6 take from 1..7 and the new idea lands in slot 7). Per-word cadence: one push per `PS.forward(IS_t)` call.
- **PARALLEL**: T iterations of `PS.forward(CS)` write to STM slots simultaneously; no shift. T = `<subsymbolicOrder>`.

The signal router (`SymbolSubSpace.languageLayer`) consumes `stm.snapshot()` as its slab input for grammar op dispatch.

Both transitions follow a **predict-then-perceive** discipline (the in-STM `IntraSentenceLayer` predicts the free slot / slab from the retained context before the new event is written). The full STM treatment — predict-then-perceive, attentional filtering (serial runs WITH attention by design), relative-vs-absolute end-states, and the LTM chain of end-states — is in the dedicated STM.md chapter.

### Lift / Lower as binary GrammarLayer subclasses

(Pre-2026-05-27, Lift / Lower were "substrate-borrowing" — they reached into `PartSpace.sigma` and `ConceptualSpace.pi` for their math. That pattern is retired in Stage 4 of the substrate refactor. `PartSpace.sigma` itself remains LIVE — PartSpace owns and uses a single `SigmaLayer` (`self.sigma`), allocated in `__init__` and applied in its forward fold (the Pi/Sigma swap, rev. 2026-06-09, put Sigma/synthesis on PartSpace). The per-stage `ConceptualSpace.sigma_in` that once carried the two-loop pi-sigma additive math on CS has since been RETIRED (the 2026-05-29 clean-stack STM experiment bypassed it on the forward path; it was later removed entirely — CS owns no fold).)

`LiftLayer` and `LowerLayer` are now first-class **binary GrammarLayer subclasses**, each owning its own internal sub-layer for the pairwise math:

```
class LiftLayer(GrammarLayer):
    arity = 2
    rule_name = "lift"
    space_role = 'CS'
    # Internal substrate: self._sigma = SigmaLayer(...)
    def forward(self, left, right):
        return self._sigma.compose(left, right)
    def reverse(self, parent):
```

```
        return self._sigma.generate(parent)

class LowerLayer(GrammarLayer):
    arity = 2
    rule_name = "lower"
    space_role = 'CS'
    # Internal substrate: self._pi = PiLayer(...)
    def forward(self, left, right):
        return self._pi.compose(left, right)
    def reverse(self, parent):
        return self._pi.generate(parent)
```

Both register with the signal router (`SymbolSubSpace.languageLayer`) as CS reduce ops via the existing host-layer registry path. Both inherit `GrammarLayer` butterfly mode (Stage 5) — set `butterfly=True, N=N` at construction to enable cross-position cascade.

Cognitive correspondence:

- **Lift** (sigma-style additive synthesis) — "lifting features onto concepts" / predication: `"the boy runs"`.
- **Lower** (pi-style multiplicative contraction) — "lowering concepts into specific percept-realizations" / attribution: `"the running boy"`.

The distinction is preserved at the rule-id level (parser records which op fired) plus the operational difference in the per-pair math.

See Language.md for the GrammarLayer specifics.

---

## WholeSpace

**Role.** Owns the **unified word lexicon codebook** with paired (orthographic, semantic) rows. Hosts grammar ops that need codebook write access. The information bottleneck for word identity.

**Owned state.**

- `self.subspace.what`: a `Codebook` carrying whole-percept prototypes at width `nDim`. Includes per-row meronomy storage (`category_ids`, `part_parents`, `category_logits`). When the configured codebook is `<codebook>quantize</codebook>`, the codebook is a trainable `nn.Parameter` updated under Gray (1990) EMA with **LBG-style variance tracking and splitting**: rows whose running per-component variance exceeds a threshold split along the top-variance eigendirection, with children seeded around the parent $\pm\ \delta \cdot$ variance_axis. (2026-05-29 Task C.) Set `<codebook>none</codebook>` to retire the SS codebook entirely (degraded mode; the grammar bivector recommender has no `W` to walk on reverse — see `test_mm_grammar_without_vqvae_learns_xor_signal`.)
- `insert_paired_word(word, ps_vector)` API: at OOV insert time, creates a paired (orth, semantic) entry — the orth row is initialized from `ps_vector` (a CS-space-dimensioned per-word vector from PS's Lexicon, per the flat-slab invariant); the semantic row is randomly initialized; the two are parented via `Codebook.set_part_parent(orth_idx, sem_idx)`.
- `get_semantic_row(orth_idx)`: O(1) lookup via `_paired_orth_to_sem` map precomputed at insert time.
- **META taxonomy** (2026-05-27 Stage 8): a separate cross-codebook taxonomy mapping `(ps_row, ws_row) -> meta_row`. Originally keyed by signed-int refs (positive=PS, negative=SS); the 2026-05-28 plan migrates this to a `.where`-keyed integer-position scheme. META entries are populated by `ConceptualSpace._maybe_autobind_meta` at stage 0 when PS promotes a chunk and SS has a Codebook (2026-05-29 Task G — the auto-bind moved from PS to CS so the cross-space allocation fires from the layer that sees both contributions).
- Signal router dispatch site for **codebook-write-required grammar ops** (insert, EMA updates, codebook expansion / culling).

No atomic forward fold layer on SS. The pre-substrate-refactor `self.sigma` (C ↔ S monotonic SigmaLayer) is retired; the C → S "naming" is now the codebook snap performed in CS via `SS_subspace.materialize()` or directly via `insert_paired_word` at write time. Symbol-side grammar math lives on the GrammarLayer subclasses (intersection, union, lift, lower, etc.), not on SS itself.

**Whole-percept presence.** Each whole-percept represents presence (`1`) or absence (`0`) of a named entity (the compact emission migrating to SymbolSpace is the symbol; WholeSpace itself holds whole-percepts). Mapping: `presence = (activation + 1) / 2`.

**Lookup chain (surface → meaning):**

1. Surface bytes enter PS via `InputSpace`.
2. MPHF on PS produces a row index into `PS.vocabulary` (the Lexicon Embedding).
3. The PS Lexicon vector (CS-space-dimensioned) flows through `PS.forward` → CS via STM push.
4. When CS-state activations need to match a whole-percept, they quantize against `SS.subspace.what` (the codebook).
5. Quantization can resolve to an orth row; `get_semantic_row(orth_idx)` returns the parented semantic row → the meaning.

**Codebook geometry.** `subspace.what.getW().shape == (nVectors, nDim)`. Each row is a free coefficient vector over conceptual axes; not unit-norm. Retrieved via Euclidean L2 (per Codebook's metric contract). `SS.nVectors` must be sized to at least `2 * expected_OOV_word_count` to accommodate paired rows (e.g., MM_5M sets `nVectors=131072 = 2 * 65536`).

**Multi-stage SS.** In multi-stage configs, the lexicon mirror is wired to `self.wholeSpaces[-1]` (the terminal stage; `model.wholeSpace`). The terminal SS must be sized for paired-row capacity.

**Conceptual order.** `BasicModel` stores per-stage ConceptualSpace / WholeSpace instances for `subsymbolicOrder`. Symbol dimension is geometrically partitioned per order. See Reasoning.md.

See Philosophy.md, Logic.md, Mereology.md, and Language.md.

---

## Monotonicity of the lift / lower chain

With `monotonic=true`, the P → C → S chain is an **order-preserving map on a positive cone**.

Three pieces:

1. **Truth-set activations may live on the positive cone** `[0, 1]^{2K}` as user-supplied paired `[pos, neg]` bivectors. The componentwise partial order $\leq$ is the parthood order for that truth surface.
2. **The Pi / Sigma maps are restricted to** $W \geq 0$ entry-wise (`monotonic=True` selects `NonNegativeInvertibleLinearLayer` or `NonNegativeLinearLayer`):

$$a \leq b \text{ componentwise} \Longrightarrow Wa \leq Wb \text{ componentwise}$$

3. **Therefore Pi / Sigma preserve parthood pole-by-pole for truth-set bivectors and other positive-cone activations.**

The bivector layout keeps the contradiction corner `[1, 1]` distinct from the ignorance corner `[0, 0]` under positive matmul — a single bitonic axis would let $aP - aN$ cancel under summation.

The `ImpenetrableLayer` regularizer maintains separation among same-rank prototypes when its XML weights are enabled. See Logic.md Section Parthood as Projection and Philosophy.md.

---

## SyntacticSpace — retired

The standalone `SyntacticSpace` class has been retired. `WholeSpace` itself has no `compose` method — dispatch instead runs through two cooperating layers: `build_space_syntactic_layer` (`bin/Language.py`) constructs a per-space `SyntacticLayer` and stores it as `space.syntacticLayer` (one instance per PartSpace / ConceptualSpace / WholeSpace, registered with the `SymbolSpace` coordinator's host-layer registry), and the signal router `SymbolSubSpace.languageLayer` (a `LanguageLayer`) is the canonical parser — its `compose` / `generate` do the binary-derivation work the retired `SyntacticSpace` used to do. Words are still concepts encoding grammatical rules, and the derivation is still stored as word tuples — just dispatched through `WholeSpace.syntacticLayer` / `symbolSpace.languageLayer` rather than living on a separate Space.

See Language.md for grammar and parser dispatch.

---

## OutputSpace

**Role.** Maps symbolic (or syntactic) vectors to task targets. Three branches, selected at construction (`bin/Spaces.py`):

- **Nonlinear / activation mode** (`<nonlinear>` on `<OutputSpace>`, `self.nonlinear_output`): an `InvertibleLinearLayer` acts on the scalar SYMBOL ACTIVATION vector (`[B, nSymbols] -> [B, nOutput]`), wrapped with `atanh` → linear → `tanh` to reproduce the nonlinearity a `PiLayer` would apply internally — only PS / CS may own a `SigmaLayer`/`PiLayer`, so this path builds the same nonlinear surface directly instead of delegating to one. `forward` reads `vspace.materialize(mode="activation")`; `reverse` mirrors it: `tanh(linearLayer.reverse(atanh(x)))`.
- **Unquantized regression head** (`self._regression_head`, true when `nVectors <= 1` or `<codebook>none</codebook>`): a head that can only name a single codebook prototype would have a VQ snap collapse to the row-mean, so this branch NEVER snaps regardless of a configured `<codebook>quantize</codebook>`. `forwardLinear` is bias-free (`x @ W`); a learned scalar intercept `self._readout_bias` (zero-init) is added by `_apply_readout` so a `{0,1}` target (e.g. XOR) can still shift off the feature mean. `reverse` undoes it via `_invert_readout` before the inverse linear. The former `<readout>` enum (`identity`/`sigmoid`) was retired 2026-06-19 — the head is always linear+bias, never squashed.
- **Codebook (quantized, symbolic) head** (`elif self.codebook`, when `nVectors > 1` with a codebook): the post-linear event snaps through `self.subspace.get_vectors().forward(output)` — genuinely symbolic, multi-vector outputs only.

**Forward.** Vector-mode (the two non-activation branches): `y = W_out * x + b_out` through the configured linear chain (`InvertibleLinearLayer` when `<invertible>`, else a `LinearLayer` pair), then the regression-head or codebook branch above. Always `reshape=True` — the `[B, nSymbols, symbolDim]` tensor is flattened before projection (`OutputSpace` is the sole flattener, 2026-06-07 dim-explicitness pass).

**Reverse.** Pseudoinverse of `W_out` (vector-mode; regression-head un-readout runs first), or the activation-mode inverse above. Text mode snaps each output vector to the nearest codebook entry (nearest-neighbour lookup).

**`getEmbeddedIO()` override.** Returns raw target dimensions rather than encoded dimensions, so loss is computed in the output vocabulary space.

**Text mode generation.** Autoregressive: each step's predicted token vector is snapped to its nearest codebook entry and fed back as input until `maxResponseLength` or EOS.

**Key parameters.** `nActive`, `nDim`, `nVectors`.

**Layer.** `LinearLayer` / `InvertibleLinearLayer` (vector-mode) or the wrapped `InvertibleLinearLayer` (activation-mode), with `(bias, temp)` for ergodic mode.

**Range.** Forward rescales `[-1, 1]` to the original data range via `Data.denormalize()`. Reverse applies `Data.normalize(x, "output")`.

**Invertibility.** Pseudoinverse (vector-mode); the activation-mode branch is exactly invertible via its `InvertibleLinearLayer`; not exactly invertible in general.

# Short-Term Memory

## Relation to LLMs, Formal Concept Analysis, and DisCoCat

STM is the local working set that replaces the anonymous residual-state view of many LLM descriptions. It holds unquantized ideas while the parser decides which typed DisCoCat-like reductions should apply. Those ideas are still grounded in the Formal Concept Analysis side of the model: their slots point back to part/whole support, concept order, and codebook rows rather than floating as untyped context vectors.

> **2026-06-02 update (subsymbolic analyzer).** Operators no longer enter the STM idea space. They are kept in the SS **codebook** (`WholeSpace.insert_operations`, wired into `SymbolSubSpace.__init__`) and resolved as a soft superposition over the operator-prefixed parse tree; the STM idea slots hold only **combined meanings** – an operator defines *how* meanings combine, contributing none of its own.
>
> **Status (2026-05-30):** new chapter for the STM serial / parallel modes work. Documents the per-batch STM buffer, the predict-then-perceive cadence (serial and parallel), the attentional-filtering regime (serial runs **with** attention by design — the old serial-vs-attention guard was lifted), the routing-parser SS-analysis / CS-execution split, the in-STM `IntraSentenceLayer` AR predictor, masked-word reconstruction via priming, the relative-vs-absolute end-state preservation with its content-aware learn-score gate, and the LTM chain of end-states feeding inter-sentence prediction. Where a piece is a deliberate scaffold or a deferred wiring, this chapter says so.

## Overview

ConceptualSpace is, post-substrate-refactor, an STM container plus a grammatical CPU — it owns no atomic forward fold (see Spaces.md). The **short-term memory (STM)** is the structure CS manages: a per-batch stack of unquantized CS "ideas" that the model accumulates across a sentence and reduces at the sentence boundary. This chapter is the single reference for what the STM is, how it fills, what reads it, and how its end-states chain into long-term memory.

Two cadences write the STM, selected by `<serial>` (legacy configs derive it from `symbolicOrder > 0`; see Architecture.md):

```
SERIAL    one idea per word; predict the free slot, then perceive (push)
PARALLEL  one whole-slab pass; predict the slab, then perceive (set all slots)
```

Both follow a **predict-then-perceive** discipline: before the freshly materialized event is written into the STM, the in-STM predictor reads the retained context and stakes a prediction; the just-written event is then its supervision target. This is the concept-space analogue of a language model's next-token objective, scaled to the in-sentence working set.

The trainable target configuration is `MentalModel.xml` (serial, `<data><dataType>embedding</dataType>`, `<attention>` unset (resolves to `"off"` — see Section 4), sentence prediction on, FineWeb data); the comparison framing against a transformer LM is Section 12.

---

## 1. STM data model

`ShortTermMemory` (Layers.py) is a `Layer` (not a `Space`): it carries no SubSpace and no forward / reverse tensor-map contract. ConceptualSpace builds one at construction as `self.stm` and treats it as the primary structure it manages.

**Capacity.** Default 8, set via <ConceptualSpace><stmCapacity>N</stmCapacity></ConceptualSpace> (DEFAULT_CAPACITY = 8); the legacy <wMax> alias is retired. Eight sits inside Miller's $7 \pm 2$ band — the working-set size psycholinguistics ascribes to human short-term memory. The capacity is the rolling-window length: at steady state the STM holds the last cap ideas and the oldest falls off as new ones arrive.

**Buffer.** The data is a single per-batch tensor [B, cap, concept_dim] plus a [B] long depth-pointer vector recording how many slots each row has filled (saturating at cap). STM contents are runtime working state, not learned weights. concept_dim is the full CS output width reserved for the event payload; positional/temporal columns are preserved when a config gives CS nonzero nWhere / nWhen.

**ShortTermMemory owns its own live buffers (A5).** The idea stack is NOT proxied off SymbolSubSpace. _buffer / _depth / _max_depth_host are properties over plain (non-nn.Module-buffer) attributes _live_buffer / _live_depth / _live_max_depth_host, set with object.__setattr__ so STM state stays out of the traced graph's inputs. Only capacity and concept_dim still proxy: when attach_word_subspace has wired an owning SymbolSubSpace, they read that subspace's _idea_capacity / _stm_payload_dim (so an externally-grown idea-stack capacity sizes the next begin_forward correctly); otherwise they fall back to the constructor-supplied _init_capacity / _init_concept_dim. This landed with the 2026-05-21 STM-Layer refactor; a later fix (the "A5" comment on ensure_batch) retired the old ss is None branch (a second, SymbolSubSpace-attached buffer) entirely — there is now a single live store regardless of attachment.

**API.** Idea-stack: push(b, idea), pop(b), peek(b, n=0) (peek(b, 0) = most recent), snapshot(detach=False), size(b), is_full(b), is_empty(b), clear(b=None), ensure_batch(batch), ensure_capacity(capacity) (grow-only). Batch push primitives (masked / whole-slab writers used by the per-word and parallel forwards): push_step(ideas), push_step_masked(ideas, gate_b_1), push_window_batch(ideas). Slot-kind provenance (word-bearing-fold filtering; None = recording off, the default): kinds_enable(batch, depths=None, kind="other"), note_push_masked(gate_rows, kind), note_push_all(kind). STM shift/reduce scorer surface (formerly stm_driver.STMDriver / stm_trainer, now living directly on ShortTermMemory): init_scorer(rule_signatures, payload_dim, hidden_dim=None), shift(word_subspace, b, payload, *, category, order, ref_id), reduce_step(word_subspace, b), reduce_step_soft(word_subspace, b), train_scorer_step(word_subspace, input_vectors, target_rule_ids, *, snap_fn, optimizer=None). The signal router consumes stm.snapshot() as its slab input.

**Lifecycle.** Cleared on hard Reset (sentence boundary) — the per-batch idea stack drops everything from the just-finished sentence and the next sentence starts empty. Soft reset leaves the STM intact (see Spaces.md).

---

## 2. Serial sequencing

In SERIAL / GRAMMATICAL mode each word traverses a per-word path: MPHF surface lookup → PartSpace.forward (synthesis front end + self.sigma) → ConceptualSpace.forward, which does the STM bookkeeping. The whole pass is **predict-then-perceive per word**, implemented in ConceptualSpace.forward (Spaces.py):

1. **Snapshot + predict the free (newest) slot.** Before writing the new event, _stm_predict_then_perceive_serial(i (Spaces.py) takes stm.snapshot() and runs the in-STM predictor from the **retained context** — the snapshot with the oldest slot dropped (snap[:, :-1] when depth $\geq 2$; the whole snapshot when depth = 1). The prediction is stashed on self._stm_predicted_idea. On the first word (empty STM) the prediction degenerates to zeros and **no** loss accumulates (there is no prior context to predict from). The serial predictor folds the context with an order-invariant **sum** over the slot axis, so dropping the oldest (regardless of which physical slot holds it) yields the same prediction.
2. **Perceive (overwrite via _stm_shift_and_push).** _stm_shift_and_push(idea) (Spaces.py) is the perceive step. At capacity, slots $0 \ldots (\text{cap} - 2)$ shift into $1 \ldots (\text{cap} - 1)$ and the new idea lands in slot 0 (the newest slot); the oldest idea (the last slot cap $- 1$) falls off. Below capacity it is a plain push: shift the occupants right and write slot 0. Under <serialObjectMeta> (default off; doc/specs/mereological- order-raising.md "Serial-mode word-at-a-time loop") the push

instead fires through `stm.push_step_masked(idea_bd, commit_b_1)` gated to `commit_b_1 = inputSpace._word_last_slot_mask[:, p:p+1]` — the **last-slot-of-word commit gate** — so a multi-slot (radix-spelled) word pushes exactly **one** idea, not one per byte (`BasicModel._per_word_body_step`, Models.py). Flag off (or no word index available) falls back to the per-slot `gate_b_1` commit, byte-identical to the pre-`serialObjectMeta` path.

3. **Accumulate the intra-loss.** The held prediction is scored against the just-perceived `idea` as $\mathcal{L}_{\text{intra}} = \text{MSE}(\hat{c}_t, c_t)$ (see Section 6).
4. **Language dispatch.** The signal router dispatches grammar ops over the STM contents (read-only via CS, write-required via SS).
5. **Mid-reading opportunistic reduces (2b-2, Alec 2026-07-12).** Once the per-word commit lands, the per-word router fire (`_chart_compose_per_word`, Section 7) populates this sentence's `current_rules`, then `BasicModel._per_word_body_step` (Models.py) runs **two** scored opportunistic `_stm_bounded_reduce_step(protect_depth=..., gate_tau=self.stm_reduce_tau)` calls — the STM stack **is** the syntactic loop; the SymbolicLoop stays the activation processor. Each call is a masked no-op wherever the shift/reduce DP's reduce-marginal stays under `<stmReduceTau>` (default `0.5`), and a structural no-op without an arity-2 grammar. `protect_depth` is threaded through from `_sentence_relative_mask` so a relative row's depth-3 end-state is never folded below its protected floor by a mid-read reduce, mirroring the boundary sweep's protection (Section 9). This is independent of the forced back-pressure reduce that fires when the host depth mirror reaches capacity (same `_stm_bounded_reduce_step` primitive, called unconditionally rather than tau-gated).

   **Convention pin — newest-at-slot-0, shift-RIGHT.** The **free / newest** slot is **slot** 0 and the rolling window shifts **right**, dropping the oldest (the last occupied slot, cap $-1$ at capacity). `peek(n)` counts $n$ back from the newest, so it reads slot $n$ directly; `snapshot()` returns the live slab **newest-first**. The "predict the free slot" step predicts slot 0, and the retained context that conditions it is `snap[:, :-1]` (everything but the soon-to-be-evicted oldest slot). A named primitive `_stm_shift_for_predict` (Spaces.py) exists for the literal "rotate so the free slot is free" step, but the hot serial path deliberately does **not** call it — `_stm_shift_and_push` already does shift-right-then-write in one pass, and a separate physical shift would double-shift the buffer. The helper is kept off the forward path and exists for independent unit-testing of the named step.

   *(History: this convention was flipped from the original newest-at-top / shift-LEFT layout; the flip preserves every semantic — `peek` still returns the most-recent idea, the reduce still collapses to the same root, and the relative end-state still yields the same predicate/idea1/idea2 — only the physical slot order changed.)*

---

## 3. Parallel sequencing

In PARALLEL mode the per-stage forward sees the whole sentence at once: the slab `[B, N, D]` **is** the STM, each of the $N$ positions its own slot. The same predict-then-perceive discipline applies whole-slab, implemented in `ConceptualSpace.forward` via the `folded.shape[1] > 1` branch:

1. **Predict $\hat{C}$ from the previous slab.** `_stm_predict_then_perceive_parallel(folded)` (Spaces.py) snapshots the previous STM and runs the predictor **per slot** (no cross-slot collapse), producing a `[B, prev_N, D]` slab stashed on `self._stm_predicted_slab`.
2. **Perceive via `_stm_set_all_slots`.** `_stm_set_all_slots(slab)` (Spaces.py) writes all $N$ positions directly as the slot stack (no shift, no mean-reduction). The slab is position-ordered (position 0 oldest, $N-1$ newest); under the newest-at-slot-0 convention it is **flipped** along the position axis so the slab's newest lands at slot 0. Capacity-clip: if $N >$ cap it keeps the last `cap` positions (drop oldest, mirroring the serial rolling window) then flips; if $N <$ cap it zeroes the unfilled (older) tail so a prior pass's state cannot leak through. The matching `_stm_predict_then_perceive_parallel` flips the `folded` target the same way before scoring $\mathcal{L}_{\text{intra}}$, so the per-slot prediction (computed over the newest-first previous STM) and its target stay slot-aligned — byte-identical to the old oldest-first alignment.
3. **Loss only when shapes align.** $\mathcal{L}_{\text{intra}}$ accumulates only when the predicted slab shape matches

`folded` exactly. On the first pass (empty previous STM) the prediction is zeros and loss is skipped; when the previous slot count differs from $N$ the per-slot prediction does not align with the target and the pinned decision is to skip loss for that pass rather than fabricate an overlap. Loss resumes once the STM steady-state slot count matches the slab width.

The earlier "mean-reduce to a single idea, then shift-push" pattern was a bug in parallel mode — it destroyed per-slot identity and made the reverse pipeline emit the same recon vector in every slot. The slot-preserving `_stm_set_all_slots` is the fix.

---

## 4. Attentional filtering

**Serial mode IS the attentional-filtering regime.** The old serial-vs-attention guard was **lifted**: serial sequencing and attentional filtering are the same regime, not mutually exclusive options — `<attention>` (off/primer/second-order/low-rank, read per-Space with default `off`) composes with `<symbolicOrder>` rather than being gated by it. The legacy `<hasAttention>` boolean is deprecated and inert, superseded by the `<attention>` element.

> **Correction — `MentalModel.xml` does not set `<attention>`.** `data/MentalModel.xml` has no `<attention>` element at all, so the knob resolves to its default, `"off"` (`TheXMLConfig.space(section, "attention", default="off")`, Spaces.py). The trainable target config therefore runs the Gaussian / word-span window below (CS→PS) and the taxonymic mask (CS→SS), but **not** any `<attention>` primer/second-order/low-rank retrieval mode. An earlier draft of this chapter claimed `<attention>primer</attention>` was set; that was wrong. See also Section 12.

**CS→PS windowing — two policies.** On the per-word serial path the input to each word is not a raw slice but a windowed contextual trace over the per-sentence percept sequence $[B, T, D]$; which policy runs is gated by `<serialObjectMeta>` (default off).

- **Gaussian window (default).** `gaussian_window_word(full_seq, center_k)` (Models.py) forms a single envelope centred on the processed word at $k$:

$$w_i = \exp\left(-\frac{(i-k)^2}{2\sigma^2}\right), \qquad \sigma = \text{maskRate} \cdot T,$$

  normalized so the center weight $w_k \approx 1$. The peak sits at the current word; words far from $k$ are attenuated toward 0 by the Gaussian tail. The windowed percepts are mapped into conceptual space and **summed**, so word $k$'s representation carries a faint meronymic trace of its neighbours (the local context *is* part of the embedding signal). This **replaces** the prior BERT-style hide-a-token `create_ir_mask` on the per-word grammar path; it is not target-hiding — the center word is preserved. The centring is **hardcoded**: peak at the current word.

- `word_span_window` **(HARD-MASK-TO-WORD-SPAN,** `<serialObjectMeta>` **on).** `word_span_window(full_seq, center_k, word_idx)` (Models.py) replaces the soft Gaussian tail with a **hard** same-word mask: the masked **sum** over exactly the slots sharing word $k$'s id (`word_idx == word_idx[:, k]`), so PS processes the active word's own span only — no part of a neighbouring word leaks in via the tail. Falls back to the single slot `full_seq[:, k:k+1, :]` when no per-slot word index is available (e.g. byte mode). Whole- sentence context still re-enters serial processing via the read prelude gist/intent, not this window.

**CS→SS taxonymic mask.** On the symbolic side the inverse recommender zeros **non-admissible** codebook rows before a lift / lower (union / intersection) reduction so an operand can only resolve to a row of the right grammar category and conceptual order. The admissible row set is built by `priming_kwargs_for_slots` (Language.py) — intersecting `refs_by_category` with `refs_by_order` per slot — and applied in the recommender's `_row_mask` closure (Layers.py, nested in `Ops._binary_op_recommend`), which admits only the category/order-matched rows plus the $\perp$ / $\top$ sentinels. A parallel `_row_weights` closure multiplies

a taxonymic **priming** mask over the admitted rows (the `left_priming` / `right_priming` weights from `taxonomy.priming_mask`).

**Guidance-signal contract.** The mask is a *guidance signal*: it biases which codebook rows / sequence positions the reduction may consult, given the current word and grammar state. The centring policy (CS→PS) is the swappable seam — the present text-reading variant centres on the current word; a future image-reading variant would swap that policy (centre on a fixated region) without changing the rest of the pipeline.

**Out of scope.** Learning the mask is out of scope for this work — both masks are **hardcoded** (Gaussian centring fixed at the current word; taxonymic admissibility read structurally from the codebook's category / order metadata). A learned attention policy is a documented future direction, not built here.

---

## 5. Routing parser: SS-analysis vs CS-execution

The grammar runs through the signal router (`LanguageLayer`, Language.py); `SymbolSubSpace` owns it as `self.languageLayer`. Conceptually the work splits in two:

- **SS-analysis** — `SymbolSubSpace.compose` (Language.py) is the analysis stage: a soft superposition over the taxonymic codebook that selects, per-space, a **hard rule dict** `current_rules = {space_role: list[list[int]]}` — an outer per-space_role entry holding one inner rule-id list **per batch row** on the full-router path, or a single batch-shared inner list on the default-only path (`_default_compose_rules`, `_flatten_selected_rules` tolerate both shapes, plus the legacy flat `list[int]` per-space_role form). It chooses *which* reductions fire.
- **CS-execution** — actually applying the chosen reductions (lift, lower, union, intersection, swap, quantize, not) to the concept tensors runs CS-side in `ConceptualSpace.forward` and the WholeSpace stack-route path, with the per-space `SyntacticLayer` cursors (Language.py) executing the unary $\pi$ / $\sigma$ folds on reverse. Only lift / lower / union / intersection consult the codebook (inverse-recommended); swap / quantize / not are tensor-only.

  **Honesty — the split is a clean code boundary only on the default-only path.** When the grammar is *default-only* (every rule is the unary $\pi$ / $\sigma$ fold), `compose` emits `current_rules` from the grammar XML and runs **no** tensor reduction, and the per-space `SyntacticLayer.forward` / `reverse` cursors do the CS-side execution — a genuinely clean analysis/execution separation. On the **full-router** path, however, `LanguageLayer.compose` does **both**: it selects the rules *and* folds the slab tensorially through the op modules (`BinaryStructuredReductionLayer.forward` → `op(left, right)`), caching the root state. The per-space `SyntacticLayer` cursors are then **deliberately bypassed** on that path — guarded by `not _grammar_is_default_only` (Language.py, the `SyntacticLayer.forward/ .reverse` guard) — precisely so the reduction is not double-applied. So in the full-router case the SS-analysis and CS-execution stages are **co-located** inside `LanguageLayer.compose` rather than separated across modules. The audit found this; it is a documented task-5 follow-up, not a finished boundary.

`_grammar_is_default_only` is computed from the configured grammar at `SymbolSubSpace.__init__` (Language.py) and gates both `compose` and `generate`.

---

## 6. IntraSentenceLayer

`IntraSentenceLayer` (Layers.py) is the in-STM autoregressive predictor — the layer that stakes the prediction in predict-then-perceive. It is owned by ConceptualSpace (`self.intraSentenceLayer`).

**Architecture: combined PI-then-Sigma, no intermediate** tanh**.**

- `self.pi`: `PiLayer(concept_dim $\to$ working_dim)`, `invertible=True`, `nonlinear=True` — the log-domain multiplicative boundary fold lifts each STM slot into the working width and bounds it to $[-1, 1]$.
- `self.sigma`: `SigmaLayer(working_dim $\to$ concept_dim)`, `invertible=True`, `nonlinear=False` — a raw linear $Wx + b$ that collapses the lifted slots into the predicted idea.

The defining requirement is that there is **no extra activation interposed** between the PI body and the Sigma body: `sigma` is built `nonlinear=False` (a raw $Wx + b$), so no additional tanh sits between PI's output and Sigma's input. This is **not** a linear fusion — `pi` is `nonlinear=True` (its symmetric log-domain $(1 + x)/(1 - x)$ embedding is an intrinsic nonlinearity), so the composite is a nonlinear PI followed by a raw-linear Sigma, not two fusable linear cores. `working_dim` defaults to `concept_dim`, keeping both sublayers square isomorphisms so the parallel per-slot round-trip reverse(forward($x$)) $\approx x$ is exact up to the LDU inverse tolerance.

**Forward signature.** `forward(prior_slots, routing=None, parallel=False)` with `prior_slots: [B, K, D]`:

- **Serial collapse** (`parallel=False`, the primary regime): PI-lift every slot, **sum-fold over the slot axis**, then Sigma-collapse to one idea → `[B, D]`.
- **Parallel per-slot** (`parallel=True`): PI→Sigma per slot, no cross-slot mixing → `[B, N, D]`.

The serial collapse is many-to-one (sum over $K$ slots), so its `reverse` is necessarily approximate (it divides the recovered fold equally across the $k = \text{stm\_capacity} - 1$ slots); the parallel per-slot path is width-preserving and exactly invertible.

**Loss.** $\mathcal{L}_{\text{intra}} = \text{MSE}(\hat{c}_t, c_t)$. `IntraSentenceLayer.intra_loss(pred, target)` (Layers.py) is a plain `F.mse_loss` helper documented as the training-path wiring point, but the live path does **not** call it: `ConceptualSpace._accumulate_intra_loss` (Spaces.py) inlines the same math (`(prediction - target).square()`) directly. `intra_loss` is exercised only by `test/test_intra_sentence_layer.py`, not by any forward call site — an unused-but-documented helper, not dead code to delete.

`_accumulate_intra_loss` keeps each per-word step's scalar loss in a **list** rather than folding it into a running `accum + step_loss` sum (2026-07-08 fix): the old running-sum pattern built a per-word add-chain *inside* the compiled forward, which Inductor inlines transitively into one kernel with a buffer argument per step — over Metal's 31-buffer kernel-arg limit on wide configs (e.g. `N=64` produced a 34-arg scalar-sum kernel, a fullgraph blocker immune to fusion caps). As a list, each step's scalar stays an independent graph output, and `consume_intra_loss` (Spaces.py) chunk-sums the list **eagerly** (post-body, pre-backward, mirroring the ARMA term) and returns the per-step mean, resetting the accumulator. The weight is `<intraLossWeight>` (default `0.1`); the term is gated off when grad is disabled or the weight is non-positive.

---

## 7. Per-word router firing

The `<routerWireSerial>` knob (model.xsd) gates when the router fires on the serial path:

| Value | Behaviour |
|---|---|
| `per-word` | fire per word; boundary fire off |
| `boundary` | fire only at the sentence boundary |
| `both` | **(default)** per-word AND boundary both fire |
| `off` | neither fires |

The **per-word fire** (`BasicModel._chart_compose_per_word`, Models.py) runs `symbolSpace.compose` over the current STM snapshot *before* `cs.forward` for the next word, populating `symbolSpace.current_rules` for the SS dispatch. It lives in the host-side loop, outside the per-iteration captured graph, so even a full `languageLayer` path that introduces a host sync cannot break the per-word capture gate. The **boundary**

**fire** (`BasicModel._chart_compose_at_C`, Models.py) runs iff `router_wire_serial in ('boundary', 'both')`.

> **Routing conditioning of the intra predictor is LANDED.** Every `SymbolSubSpace.compose` call (per-word or boundary, default-only or full-router) builds a first-class `RoutingState` (Language.py) ADDITIVE to the unchanged `current_rules` host-side rule dict: `_synthesize_rule_probs` (Language.py) turns the fired rule_ids into a dense `rule_probs: [B, n_rules]` distribution — a gradient-bearing soft-marginal aggregation (`_synthesize_rule_probs_soft`) whenever the router ran tensorially, or a DETACHED hard scatter (`_synthesize_rule_probs_hard`: unit mass onto the fired rule-ids, L1-normalized per row) on the default-only fast path — and stashes it on `symbolSpace.routing_state.rule_probs`. `ConceptualSpace._intra_routing_for_predict` (Spaces.py) reads that tensor, returns it only when its last dim matches `n_rules == len(TheGrammar.rule_table)` (`IntraSentenceLayer.routing_proj`'s expected width) and aligns its batch dim to the predictor's STM-snapshot context (broadcast when one side is `B=1`; `None` on an unreconcilable mismatch, fail-loud on a non-finite tensor). Both `_stm_predict_then_perceive_serial` and `_stm_predict_then_perceive_parallel` (Spaces.py) pass the resolved routing tensor into `intraSentenceLayer.forward`, which projects it `[B, n_rules] \to [B, concept_dim]` via `routing_proj` and adds it as a bias to the Sigma output (Section 6) — so the per-word fire now DOES make the in-STM predictor rule-aware, not just the SS dispatch context. `routing=None` (no reachable `symbolSpace`, wrong width, or an unreconcilable batch mismatch) degrades to the un-biased predictor, byte-identical to the pre-wiring behaviour.

---

## 8. Masked-word reconstruction via priming

Masked words enter the pipeline as **all-zeros** percept slots. On reverse, the router fills the blank via a best-fit codebook walk biased by the taxonymic prior, POS selection, and accumulated prediction — it reconstructs the most plausible word for the slot given the grammar context and the codebook geometry.

**Priming machinery.** The bias enters through `left_priming` / `right_priming` in `Basis.lift` / `Basis.lower` (Spaces.py), forwarded down to the inverse recommender's `_row_weights` closure (Layers.py, nested in `Ops._binary_op_recommend`) where each admitted codebook row is scaled by its taxonymic priming weight (the $\perp$ / $\top$ sentinels pinned to 1.0). Primed rows are preferred in the argmax that selects the operand. `test/test_primed_reverse_hard_mask.py` exercises this path.

**Tests.** The reverse roundtrip and reconstruction tests are `test/test_stm_reverse_roundtrip_lift_lower.py`, `test/test_stm_reverse_roundtrip_union_intersection.py`, and `test/test_stm_recon_from_cleared_cache.py`.

> **Honesty — the recon-from-cleared-cache test is an honest `xfail`.** `test/test_stm_recon_from_cleared_cache` marks the top-$k$ word recovery assertion `xfail` (not relaxed to a trivially-true bound) because of **two pre-existing upstream bugs**, both out of scope here and flagged as separate follow-ups:
>
> - **Finding A** — on the untrained config the per-word forward fills the CS STM with NaN: `conceptualSpace.stm.snapshot()` is non-finite, so the reduced single-$S$ seed is already NaN before reverse runs.
> - **Finding B** — even with a *finite* seed, the reverse perceptual leg (`PartSpace.reverse`) turns it NaN.
>
> The non-`xfail` assertions in that file pin everything that *does* hold today (the per-op reverses reconstruct `[B, N, D_c]`; the decode uses the real perceptual codebook, no reimplementation). When the two upstream bugs are fixed the `xfail` will xpass. Documenting the contract honestly: the priming-biased recon path is built and tested, but end-to-end word recovery on an untrained model is blocked upstream.

---

## 9. Relative vs absolute end-states

At the sentence boundary the STM is reduced to its end-state. The shape of that end-state depends on whether the sentence asserts an **absolute** or a **relative** truth.

- **Absolute** sentences reduce to a **single idea** (depth 1) — the root $S$ the start-symbol reduction wrote. This is the dominant path and the one the IR loss consumes.
- **Relative** sentences (the `isPart` / `isEqual` predicate family; a `REL_T`-named start is the back-compat fallback signal for grammars that don't tag their relative start, see "Conservative detection" below) are **preserved at depth 3** as the `[predicate, idea1, idea2]` end-state. Under the newest-at-slot-0 convention these are stored newest-first, so the predicate (oldest constituent) sits at the **last** slot (depth−1), idea1 at depth−2, and idea2 (the folded-newest-rest) at slot 0. `learn_relations_from_stm` reads them from those slots so `_maybe_learn_relation` receives the identical predicate / idea1 / idea2 it did under the old oldest-first (`slots 0/1/2`) layout.

The mechanism is a per-row `protect_depth` gate threaded into `_stm_reduce_to_single_S` (Models.py). Each masked micro-step (`_stm_bounded_reduce_step`, Models.py) folds only while depth > protect_depth: absolute rows pass protect_depth = 1 (collapse all the way), relative rows pass protect_depth = 3 (stop at the depth-3 end-state). The sweep itself is statically unrolled to `cap - 1` forced micro-steps, capped below that by `<syntacticOrder>` (doc/specs/orders.md; `0` = unbounded, the default, running the full `cap - 1` sweep) — a positive value bounds how many fold levels the boundary sweep runs, clamped to `cap - 1` so the CUDA-graph trip count stays static regardless of the runtime word count. Since a reduce micro-step is a no-op once a row's depth reaches 1, capping below `cap - 1` simply hands on a partially-composed forest rather than forcing the sentence root.

**Conservative detection.** `_sentence_relative_mask` (Models.py) decides per row from the grammar — it scans `symbolSpace.current_rules`' SS- and CS-role rule-id lists (the relative producers are CS-role rules; the CS scan is anchor-gated, see `Language.sentence_relative_mask`) for any rule_id `TheGrammar.is_relative_rule` flags: primarily a **grammar-driven** signal (`lhs` names a relative-start category the WholeSpace tagged `<start name="relative_truth">`), falling back to a single-symbol `"REL_T"` start pattern for grammars that don't name their starts, OR (independent of the LHS signal) a rule whose `method_name` is in `_RELATIVE_OP_NAMES = {isEqual, isPart}` — the role-collapsed grammar's relative-truth family; the retired `queryPart` / `assertPart` / `part` op names are folded into `isPart` and no longer appear here (the transitional grammar's `queryPart` / `assertPart` rules still key off the `REL_T`-LHS fallback). It **defaults to collapse on any uncertainty**: a false positive would stop an absolute sentence's collapse and break the dominant path and the IR loss, so missing / empty rules, a grammar with no relative rule, or any ambiguous shape all return all-False. The absolute path stays **byte-identical** in every fall-through case.

**Ineffable-relation routing.** Not every accepted relation becomes a WS-META row. `_route_learned_relation` (Spaces.py) splits by reducibility: a **REDUCIBLE** relation (both entity operands snap to existing codebook rows) resolves to WS positions and calls `WholeSpace.insert_relation` carrying the full tetralemma trust — the "intuitive knowing" path described below. An **INEFFABLE** relation (a composed idea that does not snap to an existing row) is instead stored UNCOLLAPSED as an `(idea1, predicate, idea2)` triple in the sibling `RelativeTruthStore` (or, on an `<ltmConsolidation>` config, is already present in the unified `ltm_store` from the observe site — see Section 10 — so no separate write happens) with a scalar trust collapsed from the tetralemma. This "explicit knowing" branch returns an `('idea', row)` tuple (`row == -1` when the store is full or, under consolidation, as the "lives in the unified store" marker) so a caller can tell the two homes apart; when no relative store is reachable it degrades to the reducible path rather than dropping the relation.

**Learn-score acceptance gate.** Concept-codebook insertion of a learned relation is gated by a content-aware **learn-score** (`_compute_learn_score`, Spaces.py):

> **Terminology (2026-06-21 convention).** The CS part↔whole relation table is the **Concept codebook** — each entry is a *concept* tying one part-percept to one whole-percept by reference. Earlier text called this the "symbol table"; the de-overloaded name is *concept* (a *symbol* is the

0-D SymbolSpace reference *to* a concept, not the relation itself). The taxonymic / perceptual codebooks consulted for lift/lower operands (Sections 4, 8) are distinct and keep their names.

$$\text{learn\_score} = \text{children\_in\_codebook} \times \text{is\_truth\_obvious} \times \text{resolves\_contradiction},$$

each factor in $[0, 1]$. A relation is accepted **iff** learn_score $\geq$ `<truthCriterion>` **and** truthCriterion $< 1$. The **default is** `1.0` — truth-learning is OFF by default (nothing learned or recorded); opt in by lowering `truthCriterion` toward 0. Because the factors multiply, a **low `is_truth_obvious` does not on its own block**: lies and uncertain relations can still be learned if the other two factors are high. At `truthCriterion = 1` nothing is learned; at `0` everything is.

Accepted insertions carry a **tetralemma trust 4-tuple** $(t, f, b, n)$ (TRUE / FALSE / BOTH / NEITHER, summing to 1) computed by `_tetralemma_trust` (Spaces.py) from the TruthSet posture via `assess()`. On the REDUCIBLE branch (see "Ineffable-relation routing" above) this 4-tuple is bound onto the relation's WS-META node by `WholeSpace.insert_relation` (Spaces.py) with the predicate as the parent and the two ideas as its taxonomy children; on the INEFFABLE branch it is instead collapsed to a single scalar trust before it is stored (the `RelativeTruthStore` / `TernaryTruthStore` row format has one trust column, not four — see Section 10).

> **Honesty — the learn-score factors read the GLOBAL truth layer.** The `is_truth_obvious` and `resolves_contradiction` factors currently read the **global** `truth_layer.assess()` — `support` and `conflict` respectively (`_learn_score_is_truth_obvious` / `_learn_score_resolves_contradiction`, Spaces.py) — behind a swappable seam (each factor is an independently overridable method, the plan's required test seam). A **per-relation projection** is the documented refinement; the global read is a first cut, not the final formula.

Truth **recording** into the TruthLayer is governed by the *same* continuous `<truthCriterion>` bar (a per-cell activation is recorded when its clamped magnitude clears `truthCriterion`), so a single knob governs both recording and learned-relation acceptance — there is no separate gold path and no binary switch. The user-provided `<truth>` gold set is ingested by `store_truths`, which drops `truthCriterion` to `0` for the ingestion epoch (capturing every provided gold truth), then restores it; the same recording block fires during normal training per the configured `truthCriterion`. The binary `accumulateTruth` / `truthMinMagnitude` switches are **retired**. See Params.md and Logic.md.

> **Behavioural note.** Unlike the retired binary gate (which kept recording off during training), truths are now accumulated continuously during training per `truthCriterion`; raise it toward `1` to suppress recording.

---

## 10. LTM as the chain of STM end-states

Long-term memory (LTM) is the **full chain of STM end-states**. There are two backing modes, selected by `<ltmConsolidation>` (default off).

**Legacy mode (`<ltmConsolidation>` off).** The chain lives on `InterSentenceLayer` (Layers.py) as a per-row bounded `collections.deque` `_stm_end_states` of size `<ltmCapacity>` (default `1024`). Each entry is a time-ordered tuple

```
(depth: int, payload: [depth, D] tensor, trust: float | None)
```

where `depth` is 1 for an absolute end-state and 3 for a relative `[predicate, idea1, idea2]` end-state (so the payloads are ragged — a fixed register-buffer tensor does not fit, hence the per-row deque). The third element is a **per-row scalar trust**, not the tetralemma 4-tuple — the 4-tuple lives only on the WS-META insertion path (Section 9); the chain (and the consolidated store below) both collapse it to one float before storing. The chain is transient host-side state (`persistent=False` semantics, not in `state_dict`).

**Consolidated mode (<ltmConsolidation> on).** The discourse LTM chain and the RelativeTruthStore relation corpus are combined into ONE persistent TernaryTruthStore on SymbolSubSpace (symbolSpace.ltm_store, Layers.py): a single [capacity, 3, nDim] tensor of (NP1, VP, NP2) idea-vector rows plus a per-row timestamp, scalar trust $\in [-1, 1]$, and provenance origin (ORIGIN_CONVERSATION / ORIGIN_PROVISIONED / ORIGIN_USER). Unlike the deque, this store is a set of registered buffers — it **rides the state_dict** and survives Reset. The boundary observe site appends each sentence's end-state as an **INFIX** triple NP1=idea1, VP=predicate, NP2=idea2 (an absolute row leaves VP/NP2 as the zero vector and carries rel_type=REL_NONE; a relative row carries rel_type=REL_OTHER) — INFIX, not the [predicate, idea1, idea2] prefix order the legacy deque's payload uses. get_stm_chain (Layers.py) detects the wired ltm_store and reads from it instead of the per-row deque: it pulls the **global** recency window via store.recent(n) (descending timestamp, reversed to oldest-first) and reconstructs each row into the SAME tuple shape the deque path returns — (1, np1[None, :], trust) for an absolute row, (3, stack([np1, vp, np2]), trust) for a relation — so downstream readers (_reduce_end_state_to_root, predict_next_end_state) are mode-agnostic. Because the store is global rather than per-row, the b argument to get_stm_chain is **ignored** in this mode (correct for B=1 / a single conversation; batched B>1 training shares the one recency window across rows).

observe_stm_end_state(depths, payloads, tetralemmas=None) (Layers.py) records **every** sentence's end-state — it is **not** gated by truthCriterion. The distinction is load-bearing: truthCriterion gates only the separate Concept-codebook insertion of *learned relations* (Section 9), whereas LTM is the AR sequence the inter-sentence predictor consumes and must see the full history. A non-finite payload **raises** (fail-loud on numerical divergence); in legacy mode the deque maxlen evicts the oldest entry once full (in consolidated mode TernaryTruthStore.append instead returns -1 once capacity is reached — see Models.py's ltm_store.append_relation / append_idea call site). get_stm_chain(n=None, b=0) returns the last $n$ (or all) end-states for a row, oldest-first, regardless of mode.

LTM is the **full chain**; the **TruthSet** is the accepted-belief subset (the relations that cleared the learn-score gate and were inserted into the Concept codebook). LTM keeps everything that happened; the TruthSet keeps what was believed.

---

## 11. Inter-sentence prediction

A **lifted IntraSentenceLayer instance** (_inter_predictor, Layers.py) predicts the next end-state over the LTM chain — the same predictor class as the in-STM one, instantiated at the inter-sentence level. Its chain window is $K = \min(\text{ltmCapacity}, 8)$ (_inter_chain_window): the AR signal that predicts the next end-state lives in the last handful of sentences, so a small bounded window is used rather than the full ltmCapacity.

predict_next_end_state(b=0) (Layers.py) produces the next end-state **shape** $(\hat{d}, \hat{p}[\hat{d}, D])$:

- **Chain reduction (ragged → fixed), mode-dependent.** Each chain entry is reduced to its **root** by _reduce_end_state_to_root (Layers.py) — but WHICH slot is the root depends on the LTM mode (Section 10):
  - **Legacy mode.** The end-state is stored newest-first, so the root lives at the **last** slot (depth $-1$): for an absolute end-state (depth 1) that is slot 0 (the collapsed idea); for a relative end-state (depth 3) it is slot 2, the predicate (the head the relative structure hangs off).
  - **Consolidated mode.** The payload is the store's native INFIX [idea1, predicate, idea2] order, and the root is always **slot** 0 = idea1 — the subject/topic, present even when there is no predicate (an absolute row has no predicate/idea2, only idea1), so it is the one slot every row shares regardless of depth.

  The last $K$ roots form a [1, K, D] context, left-padded with zeros so the most recent sits at the tail (newest-at-−1, like the ARMA ring).

- **Root prediction.** `_inter_predictor.forward(context, routing=None, parallel=False)` → `[1, D]`, the predicted root.
- **Loss.** $\mathcal{L}_{\text{inter}} = \text{MSE}(\hat{p}, p)$ on the roots, accumulated by `_accumulate_inter_loss` (Layers.py) and drained by `consume_inter_loss`, weight `<interLossWeight>` (default `0.1`). The actual root is detached so the loss trains `_inter_predictor`, not the perception path. `observe_stm_end_state` scores the prediction made for a row against the end-state that actually arrived.
- **InfoNCE next-idea contrastive term (optional, additive).** When `<interContrastiveWeight>` is positive (default `0.0`, off), `observe_stm_end_state` also ranks the actual next root above the chain's past roots (negatives) under $\cos(\hat{p}, \cdot)$/temp (`<interContrastiveTemp>`, default `0.1`) via `_accumulate_inter_contrastive` → `consume_inter_contrastive_loss` (Layers.py) — a `torch.nn.functional.cross_entropy` over `[pos_root; neg_roots]` with the positive at index 0. Best-effort: a short/odd chain simply yields fewer negatives (the accumulator no-ops with none; the MSE term above still runs regardless). Fail-loud on a non-finite step.

The prediction is consumed by `generate_sentence` via the `_c_prior [depth, D]` staging path: a predicted next-end-state shape is staged across the first `depth` STM slots as a sentence-level conditioning bias (`ConceptualSpace.forward`'s slotwise `_c_prior` branch, Spaces.py).

> **Honesty — $\hat{d}$ is a copy-last AR prior.** The predicted **depth** $\hat{d}$ is a simple AR prior: the depth of the **most recent** end-state in the chain (a relative sentence tends to be followed by structure of the same shape; an absolute by an absolute). This delivers `depth in {1, 3}` without a separate learned head, but it is a **scaffold** — a tiny `concept_dim $\to$ 2` argmax head is the documented upgrade path. The chain reduction always collapses each end-state to ONE root vector rather than mean-pooling over depth (the root carries the sentence-level signal, mirroring the ARMA `_pool_sentence_rep`) — but which physical slot is "the root" is mode-dependent (see the "Chain reduction" bullet above): it is "slot-0" only in consolidated INFIX mode, where idea1 is always at slot 0; in legacy mode the root is the OLDEST slot (depth − 1), which is slot 0 only for a depth-1 absolute end-state. A non-finite predicted root raises.

---

## 12. nanochat comparison framing

The trainable target is `MentalModel.xml`: `<symbolicOrder>1</symbolicOrder>`, `<data><dataType>embedding</dataType>` `<sentencePrediction>true</sentencePrediction>`, FineWeb data (`<shardDir>data/fineweb</shardDir>`). This is the configuration that exercises the full STM stack — serial sequencing, the CS→PS windowing and CS→SS taxonymic mask (Section 4), the in-STM and inter-sentence predictors, and the LTM chain. It does **not** set `<attention>`, so that knob resolves to its default `"off"` — see the correction in Section 4; the legacy `<hasAttention>` boolean is deprecated and inert regardless.

The comparison loss, as trained by `runBatch`, is

$$ \mathcal{L} = \mathcal{L}_{\text{intra}} + \mathcal{L}_{\text{inter}} + \mathcal{L}_{\text{ARMA}} + \mathcal{L}_{\text{inter-contrastive}}, $$

the masked-LM information-reconstruction term at the subsymbolic (PS) (see Spaces.md) plus the in-STM next-idea term (Section 6) plus the inter-sentence next-end-state term (Section 11) plus the discourse ARMA($p$, $q$) term (`BasicModel._discourse_arma_loss`, Models.py; `InterSentenceLayer.observe`, Architecture.md — a separate ring-based sentence-rep predictor on the SAME `InterSentenceLayer`, distinct from the STM-chain `_inter_predictor` of Section 11) plus the optional InfoNCE next-idea contrastive term (Section 11, off by default via `<interContrastiveWeight>`). This is analogous to a transformer language model's next-token cross-entropy, but computed **in concept space** rather than over a token vocabulary: $\mathcal{L}_{\text{intra}}$ predicts the next idea within a sentence, $\mathcal{L}_{\text{inter}}$ the next sentence's end-state shape, $\mathcal{L}_{\text{ARMA}}$ the next sentence-level representation over the discourse AR/MA rings, and $\mathcal{L}_{\text{IR}}$ reconstructs

masked content — together the concept-space counterpart of the autoregressive LM objective a system like nanochat trains.

> **Out of scope.** The comparison harness itself — running this against a transformer baseline and reporting the numbers — is a **separate plan**. This chapter documents the loss that *would* be compared and the configuration that produces it; it does not build the benchmark.

---

### See also

- Spaces.md — ConceptualSpace as STM container; the `ShortTermMemory` API; Sigma / Pi ownership.
- Architecture.md — modes of operation (serial / parallel); the per-word operational flow; `InterSentenceLayer` ARMA predictor.
- Language.md — the signal router, `compose` / `generate`, GrammarLayer reductions.
- Mereology.md — parthood as clipped-cosine projection; the relations the relative predicates draw on.
- Reasoning.md, Logic.md — the truth surfaces and tetralemma trust.
- Params.md — `<stmCapacity>`, `<intraLossWeight>`, `<interLossWeight>`, `<routerWireSerial>`, `<ltmCapacity>`, `<truthCriterion>` (the single continuous truth bar).

# Language

This document is synchronized to the code as of 2026-06-02. The source of truth is `bin/Language.py`, `bin/embed.py`, and (for the perceptual analyzer) `bin/perceptual_analyzer.py`. (The earlier `bin/typed_stack.py`, `bin/stm_driver.py`, and `bin/parse_state.py` modules were retired in the 2026-05-21 / 2026-05-29 refactors; their functionality folded into `bin/Layers.py` and `bin/Language.py`.)

## Relation to LLMs, Formal Concept Analysis, and DisCoCat

The language layer is the architecture's DisCoCat-facing surface. Like Categorical Compositional Distributional semantics, it treats grammar as a typed composition discipline over vector meanings: reductions such as lift, lower, union, intersection, `part`, and `equal` decide how meanings combine. Unlike a typical transformer LLM, these reductions are explicit operators rather than latent behaviors distributed across heads. Their operands are tied back to the Formal Concept Analysis side of the model through concept order, role participation, and part/whole support in the codebooks.

> **2026-06-02 deltas (subsymbolic analyzer + terminal emitter).**
>
> - **PS/SS grammar sections.** A `.grammar` file may nest its `<compose>`/`<generate>` under `<PartSpace>` and `<WholeSpace>`. `Grammar.configure` parses them into separate rule tables: `ps_rules` (space-role `P`, read by the PS analyzer) and the canonical symbolic `rules` (`ws_rules`). A bare `<compose>`/`<generate>` file loads as `<WholeSpace>` (backward-compat). The legacy `.cfg` loader is gone.
> - **Grammar rewrite.** `*_MARK` categories and the copy/swap MARKER helper rules are deleted; surface markers are learned and owned by the operator. Each operator-argument position is its own category (`CONJ_L45`/`CONJ_R45`, …). copy/swap are retired from the symbolic grammar (kept only as the T5 elision surface policies).
> - **SurfaceSchema + absorb/emit** (`bin/Layers.py`). Five universal templates T1-T5 declare each operator's marker slot + order; T4 `BINARY_JUXTAPOSE` is the default. `GrammarLayer.absorb` binds a co-occurring marker to the operator (many-to-one); `emit` replays it from recorded route metadata, never the lossy `generate()`.
> - **Operators in the SS codebook, not the STM idea space** (amends plan decision #2). `WholeSpace.insert_operations(grammar)` registers each operation in a dedicated operator codebook on WholeSpace (`_operation_vectors` / `_operation_positions`), separate from the `subspace.what` whole-percept codebook so the percept / idea / `.where` position namespace is untouched; it is wired into `SymbolSubSpace.__init__` so every built

model's operator-prefixed parse-tree nodes are codebook-resolvable. The STM idea space holds only combined meanings – the operator says *how* meanings combine, contributing none of its own. The rule-id stays in `.where` (its presence marks a slot as a *computed* idea, which is by definition not a codebook vector).

- **IdeaSubSpace** (`bin/Language.py`) – the PS-meronymic carrier analogue of `SymbolSubSpace`: span buffers + parent/child links + route ids/scores + the marker-route replay fields (`_marker_ps_id`/`_marker_span`/`_order_bit`/`_marker_position`).
- **Perceptual analyzer** (`bin/perceptual_analyzer.py`). `EndpointSumWhere` is the invertible span key $where = phase(start) + phase(end)$: the angle decodes the span center, the magnitude the span length. `MeronymicAnalyzer` analyzes a surface into terminals (compatibility mode reuses the word/byte tokenizer as the `boundary` op, so it matches the current lexer), writes durable spans to an `IdeaSubSpace`, exposes a fixed-capacity terminal-stream view, and reverse-synthesizes surface (`synthesize` exact replay; `synthesize_tree` from an operator-prefixed tree with `emit`). `soft_operator_compose` + `WholeSpace .operator_superposition` apply a soft operator distribution over the SS operation codebook (one-hot → the typed grammar; spread → the superposition that discriminates `A AND B` from `A OR B`).
- **PS-to-SS binding.** `WholeSpace.resolve_ps_terminal(ps_id)` emits `null_sem()` before a binding exists, counts exposures, and promotes a repeated terminal into a fresh SS row.

**2026-05-29 deltas:**

- `unreduce()` passes the space-role-local Basis (Codebook) to binary GrammarLayer reverses as `basis=space_role_basis` (replacing the prior raw-`W` form). `UnionLayer.reverse` / `IntersectionLayer.reverse` **and now** `ConjunctionLayer.reverse` / `DisjunctionLayer.reverse` extract `W = basis.getW()` internally and dispatch to `Ops.disjunctionReverse` / `Ops.conjunctionReverse` — the codebook recommender recovers the operand pair exactly on a discrete vocabulary (the serial XOR reconstruction path); without a basis they keep the lossy `(parent, parent)` fallback. The genuinely non-invertible predicate folds (`isEqual` / `isPart` / `exist`) declare `invertible = False` and are not invertible by design (a truth/predicate value does not retain its operands). Layers that don't accept the `basis` kwarg yet are handled by a `TypeError` fallback. No back-ref is stored on the layer.
- `MetaLayer` was renamed to `SymbolizeLayer` (no semantic change).
- Word-mode parse appends a `\x00` null sentinel after the words slab for explicit end-of-sequence on the forward path.

## Current Parser Surface

`SymbolSpace.compose()` and `SymbolSpace.generate()` are the public parser entry points. There is no longer a backend selector: the signal router (`LanguageLayer`) is the single canonical parser.

The `<parserBackend>` knob is RETIRED (Stage 3, 2026-05-27). The CKY chart and the STM shift/reduce parsers it used to select have been deleted in favour of the signal router. The retired `parserBackend` values (`chart` / `stm` / `parallel`) no longer exist.

`<routerKind>` is also RETIRED alongside the chart; the `signal` behaviour it once selected is now unconditional. The retired `<parserBackend>`, `<routerKind>`, `<chartTau>`, `<chartTopK>`, and `<chartNoiseEps>` elements raise a loud `ValueError` at config load if a `<SymbolSpace>` config still sets them (`Language._assert_retired_chart_knobs_absent`); see `data/model.xsd`.

## Retired XML Knobs

`SymbolSpace.chartCompose`, `SymbolSpace.softChartCompose`, and `bivectorOutput` are no longer read by the runtime and should not appear in `data/*.xml`.

`SymbolSpace.useGrammar` is a different case: it is **still read**, not silently ignored. Its mere presence in a config trips a loud `DeprecationWarning` and forces a fallback load of `default.grammar`, discarding

whatever <grammar> file the config actually named (Language.py:1607-1622). A surviving <useGrammar> tag therefore silently swaps in the wrong grammar rather than doing nothing — remove the tag, don't rely on the fallback.

Grammar mode is no longer a useGrammar string at all; the retired "none" / "all" vocabulary collapsed into one boolean, SymbolSubSpace._grammar_is_default_only. It is True when every compose rule is either an implicit passthrough or the default unary pi / sigma substrate fold (no operator grammar loaded), and flips to False the moment any non-default operator rule (part, equal, conjunction, …) is present.

> **SS-analysis vs CS-execution.** SymbolSubSpace.compose is the SS-side *analysis* stage (it selects the per-space hard rule dict current_rules); the CS-side *execution* (applying lift / lower / union / intersection / swap / quantize / not to the concept tensors) runs in ConceptualSpace.forward and the per-space SyntacticLayer cursors. This split is a clean code boundary **only on the default-only path**: on the full-router path LanguageLayer.compose does both selection and tensor reduction, and the per-space cursors are deliberately bypassed (not _grammar_is_default_only). See STM.md Section 5 for the accurate, audited account.

## Grammar

TheGrammar is the singleton Grammar instance. Rules are loaded from XML <SymbolSpace><language><grammar> blocks or from a configured grammar CFG. A RuleDef stores:

```
(space_role, canonical, arity, method_name, lhs, rhs_symbols,
 width_min, width_max, query)
```

The grammar file carries BOTH directions: the <compose> section holds the forward rules (op_O1 = op.forward(op_I1, op_I2)) and the <generate> section the reverse rules (op_I1, op_I2 = op.reverse(op_O1)) — parsed into rules_upward / rules_downward and concatenated into the one flat TheGrammar.rules table. Both directions carry the BARE method_name (the .forward/.reverse suffix is stripped and survives only in canonical), so a generate rule resolves the SAME host layer the compose rule uses. Note the arity asymmetry: a generate rule's RuleDef.arity counts its RHS *call* arguments (1 for op.reverse(op_O1)); its two-output nature is implicit in the LHS string. Enumerating "the binary reverse ops" therefore filters on .reverse in canonical and the HOST's arity == 2 (BasicModel._grammar_reverse_ops), not on RuleDef.arity.

The live grammar style is **operator-role categories**, not a declared part-of-speech taxonomy: every operator contributes its own <op>_I1, <op>_I2 (inputs) and <op>_O1 (output) categories. For example, from data/complete.grammar:

```
part_O1 = part.forward(part_I1, part_I2)
lift_O1 = lift.forward(lift_I1, lift_I2)
conjunction_O1 = conjunction.forward(conjunction_I1, conjunction_I2)
```

All four shipped .grammar files (default, complete, xor, shamatha) are exclusively in this role-only form. See Role-Collapsed Grammar and the Operator Codebook below for the full account, including how a word's category is recovered from role participation rather than declared.

RuleDef.lhs / .rhs_symbols parsing (Grammar._parse_category) still accepts an explicit conceptual-order suffix — NP3, S4 = lift(NP3, VP1), NP* Kleene forms — for backward compatibility, but that style is now **explicitly legacy**: NP3, NP4, VP1, MP1, S4, and S5 are members of _FORBIDDEN_STATE_TOKENS in test/test_role_collapsed_grammar.py, and no shipped grammar file declares any of them.

### GrammarLayer forward / reverse inventory

The per-operator contracts, as implemented (2026-07-14; every op's forward is what <compose> fires and its reverse is what <generate> fires — see the section pairing above). "Recommender" means the basis-threaded codebook walk (Ops.conjunctionReverse / Ops.disjunctionReverse → Ops._binary_op_recommend); "snap" means the op-respecting dot-metric word snap (snap=True →

Ops.word_pair_snap). Ops with no faithful inverse raise (raise_no_inverse, the fail-loud contract) — fabricating a split would corrupt the reconstruction.

| op (rule_name) | arity | role | forward |
|---|---|---|---|
| not | 1 | CS | pole swap |
| non | 1 | CS | non-affirming complement |
| intersection | 2 | CS | Ops.intersection (RadMin / lattice min; ADJ mask, meet) |
| union | 2 | CS | Ops.union (RadMax / lattice max, OR-region, join) |
| chunk | 2 | CS | additive left + right (PS-style chunking) |
| sum | 2 | CS | element-wise left + right |
| product | 2 | CS | element-wise left * right |
| lift | 2 | CS | order-raising fold (internal SigmaLayer; optional gate) |
| verb | 2 | CS | sparse verb-conditioned spectral operator |
| adverb | 2 | CS | VP eigenmodifier (apply_adverb) |
| lower | 2 | CS | order-lowering (internal PiLayer; DET) |
| preposition | 2 | CS | .where-relation refinement of NP/VP |
| bind | 2 | CS | contextual missing/controlled-NP resolution |
| tense | 1 | CS | phase rotation of the .when band (shift_time(+delta)) |
| aspect | 1 | CS | identity (rewrite() planned; not a live rule) |
| morphology | 1 | CS | surface inflection → .when (tense/aspect feature ops) |
| symbolize | 2 | CS | bind PS percept row + WS symbol row into an idempotent META node (fused average; in |
| conjunction | 2 | SS | Ops.intersection monotonic (scalar activation min; RadMin under <radialStmReduce>) |
| disjunction | 2 | SS | Ops.union monotonic (scalar activation max; RadMax under <radialStmReduce>) |
| exist | 1 | SS | identity (EXISTS roots the minimal event) |
| isEqual | 2 | SS | identity-assertion truth bivector |
| isPart | 2 | SS | parthood-assertion truth bivector |
| part | 2 | CS | returns the encompassing parent (parthood learned by codebook geometry) |
| whole | 2 | CS | converse of part (PartLayer subclass) |
| equal | 2 | CS | geometric mutual-parthood on concept bivectors (Layers.EqualLayer) |
| query | 2 | CS | geometric parthood query → truth bivector |

Notes. (1) The binary lattice reverses (union/intersection, conjunction/disjunction) accept left_rows / right_rows (typed candidate restriction), left_priming / right_priming (soft boosts), radial (signed-magnitude order), and snap — recovery is owned by the layer and DIFFERS by op: the join is fit-determined; the lossy meet leans on priming (which words are present). (2) The trace-free free-derivation decode (_reverse_reduce_unfold → _reverse_choose_op) CHOOSES among the <generate> binary ops per un-fold step by round-trip fit — op.compose(op.reverse(parent)) \approx parent — with no forward record. (3) VerbLayer/AdverbLayer subclass LiftLayer; WholeLayer subclasses PartLayer; TenseLayer/AspectLayer share _WhenOpMixin; EqualLayer lives in bin/Layers.py, all others in bin/Language.py.

## Knowledge Artifacts

embed.build_knowledge_section(grammar) creates the parser knowledge section, five sub-sections in all:

- word_table: bootstrap CSR word table (build_word_table_initial(wv), embed.py:857) — UTF-8 surface-form bytes keyed by keys_values/keys_offsets, plus a ref_ids column initialized to -1 (unassigned) until a curated POS lexicon / tagger populates it.
- taxonomy: base category refs plus explicit ordered refs.
- reference_codebook: scalar prototypes and order per ref.
- typed_indexes: refs_by_category and refs_by_order.
- grammar.rule_order_signatures: serialized rule typing.

The taxonomy builder (build_taxonomy_from_grammar) still supports ordered-ref children of a base category — the machinery is generic, parsing whatever Grammar._parse_category finds — but that shape is now **legacy**: it only appears for a grammar declaring explicit-order categories (NP3, NP4, …; see the Grammar section above), and no shipped .grammar file declares any. On the live role-only grammars every category is order-flat (part_O1, lift_I1, …), so taxonomy has no ordered children in practice. Historically the illustration was:

```
NP
|-- NP3
`-- NP4
```

KnowledgeView.category_of_ref(ref_id) returns the base category (NP), while KnowledgeView.order_of_ref(ref_id) returns the conceptual order (3, 4, etc.) — accurate for a grammar that still uses this legacy style.

SymbolSpace.category_codebook has been retired. The live category embedding is:

SymbolSpace.category_embedding: nn.Embedding

## STM Shift/Reduce

ShortTermMemory.shift / .reduce_step / .reduce_step_soft and the _RuleScorer MLP (bin/Layers.py) are a typed admissibility-masked shift/reduce driver — SHIFT snaps an input vector to the nearest live reference and pushes (payload, category, order, ref_id); REDUCE masks rule logits by typed admissibility, softmaxes over what's admissible, and records the argmax. It has **zero production callers**: every call site is test/_stm_test_fixtures.py, which keeps the surface alive as a compat shim for tests written before the 2026-05-21 SymbolSubSpace refactor. The live parsing mechanism is LanguageLayer.compose's soft-DP / Viterbi weighted deduction (binary_tiling_soft_dp / binary_tiling_viterbi); see STM.md Section 5 for the accurate, audited account of SS-analysis vs CS-execution.

ConceptualSpace.stm (a ShortTermMemory instance) is, in the live model, a plain payload/idea stack, not a typed parser stack: its per-batch slab and depth pointer (_buffer / _depth) are ordinary tensors written by push_step_masked(ideas, gate) (the gated newest-at-slot-0 push) and read back by snapshot(). It carries no category / order / ref-id typing — that typed buffer is what the dead shift/reduce driver above expects, and nothing in the live model populates it.

## Syntax And SVO

BasicModel.write_syntax_tree() (bin/Models.py) is currently unwired infrastructure, not a live path. The function hardcodes chart = None and traces = None — the CKY chart is retired and the signal router does not yet populate per-leaf derivation traces to replace it — so every call emits a bare <noTrace/> element regardless of input. The docstring's <node>/<leaf> format is what the function would produce once a trace source is wired, not what it produces today.

SVO extraction has the same status. SymbolSpace.set_last_svo / get_last_svo / clear_last_svo (bin/Language.py) exist, and clear_last_svo does fire from production code (every CS-forward cycle and at Reset / soft-reset boundaries), but set_last_svo itself has no production caller anywhere in bin/*.py — it is exercised only by test/test_per_batch_state_isolation.py and test/test_subspace_context.py. Until something calls it, _last_svo never leaves its post-clear zero state in the live model.

## Role-Collapsed Grammar and the Operator Codebook

The live role-only grammars replace part-of-speech categories with *operator roles*. Instead of a fixed NP / VP / S taxonomy, each operator contributes its own argument and result roles, named <op>_I1, <op>_I2 (inputs) and <op>_O1 (output). The rule equal therefore exposes equal_I1, equal_I2, equal_O1, and the grammatical "category" of a span is just the set of operator roles it can fill. Dimensionality is recovered

from participation (bin/participation.py) rather than declared up front: symbols that fill the same roles cluster into the same category.

Starts are scoped per space (_configure_starts):

- PartSpace.start is the universal whole-input role U — the analyzer begins from the entire surface and decomposes it.
- WholeSpace.start is the set of operator output roles (equal_O1, part_O1, exist_O1, …) — parsing begins from what an operator can *produce*.

The **operator codebook** is a second codebook on WholeSpace, separate from the whole-percept codebook:

- _operation_vectors: operator name → identity vector.
- _operation_positions: operator name → codebook position.

It stores one prototype per operator — the live codebook is equal, part, whole, exist, conjunction, disjunction, adverb, bind, intersection, lift, lower, morphology, non, not, preposition, product, sum, tense, union, verb (insert_operations, bin/Spaces.py:19582) — and is kept CPU-explicit so the host-side identity lookup never mixes with an ambient MPS / CUDA default device.

### Soft Operator Superposition

operator_superposition(query_vec) is a softmax over the cosine similarity between a query and every operator vector. soft_operator_compose(dist, left, right) is the distribution-weighted sum of each operator's compose, evaluated per operator *arity* — unary operators such as exist are called as compose(a), binary operators as compose(a, b). A one-hot distribution reduces exactly to the typed grammar; a spread distribution superposes operators, and that superposition is the mechanism that discriminates A AND B from A OR B while a single layer is chosen.

shape_operators(examples, op_names, steps, lr, seed) trains the operator vectors by backpropagating an MSE between the superposed prediction and the supervised result, writing the shaped vectors back into _operation_vectors. The op set may span the whole grammar, so the shaper filters to layers of arity 1 or 2 and dispatches compose by arity.

**Status (D1 gate — met).** Role-collapse does not swap declared POS for another single-label POS system; it replaces declared shared categories with operator-local participation categories that a word may fill several of (overlaps are expected). The D1 gate is therefore not a single-label POS recovery test — it asks whether those participation patterns are *structured enough to drive a learned collapse* into the smaller mutually-exclusive category set the live parser needs. They are: participation.learned_collapse proposes merges by participation similarity and accepts only those that keep every grammar rule distinguishable (collapse_conflicts == 0); on the transitional complete.grammar it compacts 43 context-unique symbols into 14 mutually-exclusive categories with zero parser conflicts (test/test_d1_pos_recovery_gate.py). The exact substitutability congruence is trivial there (every symbol is context-unique), so "recovers the grammar" means the parser's rule decisions survive the collapse, not exact rule regeneration. With the gate met, the former standalone role-collapse file has been absorbed into complete.grammar, which is the broad live role-only grammar used by MentalModel.xml. The part relation is unified there: the grammar declares the single compositional op part (plus its converse whole); isPart / queryPart survive only as <Queries> predicates, outside <compose>. The query/assert split is *not* recovered by _dispatch_method_name_for_rule (bin/Language.py:4187) — that helper only fires for query="true" rules, and complete.grammar declares none. The live unification is reasoning.py's _SURFACE_TO_KIND table (bin/reasoning.py:27), which maps every surface form — part, isPart, and queryPart alike — to the same KIND_IS_PART reduction kind (and symmetrically equal / isEqual / queryEqual → KIND_IS_EQUAL). The operator codebook, soft superposition, and participation clustering are live and tested.

**Participation Categories as the Chooser's Syntactic-Category Context**

*(Live path. The category source is learned from role participation during perception, attached to the **MetaSymbol**, and threaded into the placement chooser as grammatical context.)*

> **Terminology note**: the CS part↔whole relation table is the **Concept codebook** (concepts), not a "symbol table"; WholeSpace holds **whole-percepts**; only `SymbolSpace` / `MetaSymbol` things are **symbols**. Code identifiers (e.g. `subspace.what`, `_sym_*`) are unchanged here — that is a separate code pass.

A word's grammatical **category is its frequency of participation across the operator roles** above (`<op>_I<n>` inputs / `<op>_O1` output), **learned from perception** (analysis of input), not from generation. No part-of-speech label is declared or needed: if *cat* fills `ADV`'s operand role and `LIFT`'s argument role with roughly equal frequency, *collapsing* that frequency profile is what makes it a **noun** — the model knows the category by its role distribution, never by a name. This is exactly what `participation.learned_collapse` formalizes (merge by participation similarity, keeping every rule distinguishable).

This role-participation profile is the **primary determinant of syntax** — what a constituent *is* (its category) governs what it can combine with more than its surface content does. So it is precisely the context the placement chooser (`MLPTransformChooser`, the soft route's scorer — see Soft-superposition route) needs when scoring "should this pair reduce, and with which operator?": the chooser must see the **category of the value already sitting in each slot**, not only the candidate operator's own output. The design realizes this as a small **Category codebook keyed by the MetaSymbol**, learned online by E/M from perception:

- **The MetaSymbol unifies word and object (it already exists, live).** The Concept codebook stores `concept -> code`; a **MetaSymbol** is the exception — one symbol that holds *two* codes, the **word code and the object code**, a deliberate equivalence class asserting *this word* ≡ *this object*. This is the live **META node** in the WholeSpace taxonomy: `WholeSpace.insert_meta` allocates one SS-codebook row tagged `"meta"` whose two taxonomy children are the PS object position and the SS word position, minted during **perception** by the autobind hook (`ConceptualSpace._maybe_autobind_meta` at the sentence boundary). Because the category attaches to the MetaSymbol, the syntactic signal learned from the *word's* role participation directly shapes the *object's* category, and vice versa.
- **A small Category codebook, not a permanent per-word count table.** The VQ lives directly in role-participation space: $K \approx$ `n_roles` (55 on the live `complete.grammar`, `compute_role_vocabulary`) initial centroids, one seeded from each labelled role (`<op>_I<n>` inputs + `<op>_O1` outputs). Unlearned MetaSymbols have only a bounded temporary row in `MetaSymbolCategoryLearner`; learned MetaSymbols keep just `MetaSymbol -> category_id`.
- **E/M learned from perception, with emergent collapse.** Each analysis route contributes a sparse role vector to the MetaSymbol that occupied the terminal position. The pending row accumulates that evidence until mass, confidence, margin, and short-term stability thresholds are met. Then the MetaSymbol commits to one VQ centroid and the pending row is discarded. Starting from one centroid per role and letting unused centroids decay, **effective K shrinks as role-use profiles pull centroids together** — the online realization of `participation.learned_collapse`, where "noun" emerges without a label.
- **Feeds the per-slot category to the chooser.** The chooser conditions each slot on the **role vector of the committed centroid**; while a word is still unsettled, the pending row supplies a temporary role context. The gather path is `percept id $\to$ taxonomy parent (MetaSymbol) $\to$ committed category or pending evidence $\to$ role vector`. `MLPTransformChooser` receives the vector as a feature block; anchor-dot/default routing uses the same vector as a labelled-role score prior.

This splits into two phases: **(1)** learn the category codebook from perception in the autobind hook (no change to the layer forwards — reads the stashed analysis route); **(2)** thread the per-slot category through `compose` / `score_binary` / `score_unary` into the chooser `feat` (the larger change — the layer forwards carry only `[B,N,D]` today, with no per-slot symbol identity).

**Status (implemented behind `<categoryCodebook>`, default true).** `WholeSpace.enable_category_codebook`

builds the role-space VQ (`codebook_retire=False`) + `_category_role[K, n_roles]`, enumerated from `compute_role_vocabulary`; it is requested at build and **lazily enabled on the first perception forward**. `LanguageLayer._collect_round0_role_obs` stashes the first binary space-role's round-0 reduces (`op_I1`/`op_I2` per operand), and the autobind hook (`_maybe_autobind_meta`) feeds those observations to `MetaSymbolCategoryLearner`. The learner owns the pending per-MetaSymbol role rows, commits stable symbols into `WholeSpace._category_assign`, and drops the pending row. Structured grammar layers use the resulting role context for all transform choosers: MLP as an input feature, anchor-dot/default as a labelled-role score prior. The current round-0/first-space-role observation is parallel-mode-correct; serial (`<serial>true</serial>`) attribution is approximate. The old `WholeSpace.category_codebook` was retired 2026-05-20 and is gone; the dormant declared-POS tables (`category_embedding`, `category_logits`/`category_ids`, the order-taxonomy admissibility gate) are superseded and slated for follow-up retirement, not reuse.

## Shared Weighted-Deduction Framework

PartSpace analysis and WholeSpace parsing are two readings of the *same* item graph under semiring-weighted dynamic programming (weighted deduction). The graph is scored once; the semiring chosen selects the quantity:

- **sum-product** gives soft inside / forward marginals (every viable rule keeps positive mass, and gradient flows through $\log Z$);
- **max-plus** gives the single Viterbi route used for the hard forward value.

Both spaces share the same direction-agnostic primitives in `bin/Language.py`:

- `binary_tiling_viterbi` — max-plus best route.
- `binary_tiling_soft_dp` — sum-product marginals (`reduce_marginal_op`, `logZ`).

The WholeSpace reducer (`BinaryStructuredReductionLayer`) scores copy / reduce items over rule columns; the PartSpace analyzer (`MeronymicRouter`, `bin/perceptual_analyzer.py`) scores merge evidence over perceptual atoms. Because the soft marginals are retained even when the Viterbi route hardens to one rule, two tied reduce rules both keep positive mass and both receive gradient — the property pinned by `test/test_signal_router_layer.py::test_layer_keeps_soft_superposition_over_reduce_rules`.

### Soft-superposition route (the <learning> two-pass)

The straight-through forward above is the **default** (and the byte-identical basin when `<architecture><learning>` is off): the forward value commits to the Viterbi route while the soft marginals carry the gradient. Under the two-pass `<learning>` mode the structured layers instead run a **pure sum-product superposition at a temperature**, and the chooser sits in the gradient path *directly* — no argmax, no `.detach()`, no straight-through.

A single scalar `superposition_temperature` $t \in [0, 1]$ drives it (`BinaryStructuredReductionLayer` / `UnaryStructuredLayer`):

- the route scores are scaled by `superposition_scale(t)` $= 1 - t$ before the soft DP / softmax, so $t = 0$ is the chooser's own (sharp) softmax and $t = 1$ collapses the scores to uniform (flat, maximally exploratory);
- the forward value **is** `binary_tiling_soft_dp`'s temperature-scaled marginals (binary) or the temperature-scaled action softmax (unary). The op blend is the pure `op_soft` posterior, not the hardened one-hot.

`binary_tiling_viterbi` is still computed, but only to read off the routing masks for the tree (below) — that read-off is outside the gradient path in both modes. When `superposition_temperature` is unset (`None`) every branch falls back to the legacy straight-through, object-for-object, so the default basin is unchanged.

**Two passes as two trials.** With `<learning>` on, `runEpoch` runs each sentence through `runBatch` *twice*, as two independent forward/loss/backward trials (no `loss_A + loss_B`, no shared graph):

1. **pass A** at $t = 0$ (sharp / deterministic) — recorded into the batch error like any ordinary step;
2. **pass B** at $t =$ `<exploreTemperature>` (default `0.5`, flatter) — an exploration trial whose value is trimmed from the per-batch error and does **not** increment the batch count.

Pass B is temperature sampling, not a separate objective: a flatter route lets the chooser escape a local commitment that pass A's sharp argmax would otherwise lock in, and because the chooser is differentiable in both passes the exploration gradient updates the same anchors. `BasicModel _set_superposition_temperature(t)` walks the router's `_unary_layers` / `_binary_layers` (and the STM reducer) to stamp $t$; `runBatch` sets it before the forward and resets to `None` in a `finally`.

**Reading a tree.** A hard derivation is always recoverable on demand: pin $t = 0$, run a temp-0 analysis, and read the argmax routing trace (`action_kind` / `action_op` / the copy / reduce masks) that `BasicModel.write_syntax_tree()` already walks. Running several sentences through a temp-0 pass yields one tree each.

This is the standard semiring-parsing pattern (Goodman 1999, *Semiring Parsing*; the SCFG inside-outside line of Lari and Young 1990; weighted logic programming / parsing transformations, Eisner and Blatz 2006; and neural grammar induction, Kim, Dyer, and Rush 2019). See the plan's section 5.6 for the full mapping.

### Future work: nouns from PartSpace, adjectives from WholeSpace

The signed-space snap models a concrete noun as an adjective pre-applied to the top domain — `black(cat($\ldots$))` treats "cat" as the same *kind* of function as "black", the later one already mapped over $\mathbb{1}$. That modeling choice is probably already latent in the mereology poles. The lattice poles are vectors of the presence domain (see `doc/Architecture.md`): **NOTHING** $= [0, 0, ...]$ is the part/bottom pole, **EVERYTHING** $= [1, 1, ...]$ is the whole/top pole. So:

- a **whole (property)** is pre-applied to EVERYTHING — narrowing the wide-open $\mathbb{1}$ object downward (the `assert_concept_relation` first-concrete-whole-replaces-EVERYTHING move); this is the *adjective* shape, a modifier that carves the domain;
- a **part (particle)** is pre-applied to NOTHING — building presence up from **0** (the first-concrete-part-replaces-NOTHIN move); this is the *concrete-noun* shape, an object accreted from constituents.

The conjecture for a future pass: source **concrete nouns from PartSpace** and **adjectives from WholeSpace**, so the grammatical part-of-speech distinction falls out of which pole a symbol is pre-applied to, rather than being a learned category. This would give the snap's ADJ(N) intersection a principled home — the noun's broad PS-side support and the adjective's narrowing WS-side mask are then *typed by origin*, and the support-restricted metric (score over the parent's support, unaffected by the modifier's attenuation) becomes the natural recovery law rather than a tuning choice. Open questions: how order-raising (`maybe_raise_order`) interacts with a PS/WS part-of-speech split; whether abstract nouns want the WholeSpace (property-like) or PartSpace (object-like) origin; and how the `<Anchors>` closed-class relation surfaces sit relative to this axis.

## Mereology

Single-page reference for the mereological grammar: parthood as the fundamental operation, the five mereological relations, and the `ImpenetrableLayer` regularizer.

### Relation to LLMs, Formal Concept Analysis, and DisCoCat

Mereology is the bridge from subsymbolic perception to explicit concept order. Where an LLM usually leaves part/whole structure implicit in token statistics, BasicModel stores and trains it as codebook geometry. This is the architecture's closest point of contact with Formal Concept Analysis: part-percepts and whole-percepts define a fuzzy extent/intent relation, and concepts are the ordered links over that relation. DisCoCat enters one level later, when the language layer composes these ordered meanings according to typed grammar.

**Part/whole spaces (2026-06-12; base class retired 2026-07-10).** The perceptual side names the duality directly: `PartSpace` (bottom-up synthesis over atoms) and `WholeSpace` (top-down analysis over unity), both subclassing `Space` directly — there is no `PerceptualSpace` base. A thin `PerceptualSpace(Space)` base briefly existed (no params/submodules; only `NULL_PERCEPT_KEY` and isinstance sites) but was removed as part of the dual-towers rev-2 pyramid rework, once PS/WS became symmetric duals sharing one `forward(in_sub, CS_out)` signature. At the corpus callosum, objects are analysed and synthesized by sending them back through the towers — wholes get split, parts get chunked. For the object/reference (sign/symbol) vocabulary, see Spaces.md and Philosophy.md.

**Meronomy reconciliation (2026-06-11; MeronomySpec wins — plan §2).** Four reframings of this page's claims:

1. **`part()` is the graded retrieval surrogate, not *the* parthood relation.** Exact dominance (`Ops.partOf` — elementwise $\leq$ reduced with `all`) is the semantic ground truth; the clipped cosine below is its retrieval-time score. `part()` stays total and form-level over everything — including symbol codes and binding-table-bound rows — and that totality is its cordon duty (spec §1, §7): geometric relations answer "looks like"; *asserted* parthood lives in the truth store and the towers' registration, answers "is", and is a different call. Reading one relation as the other is the category error.
2. **Fusion = elementwise max stays as the order-theoretic join.** The learned $\sigma$-fold satisfies $\sigma \succeq \max$ (MeronomySpec §10.1) — join versus parametric whole-maker; the whole-maker can only over-cover the join, never under-cover it.
3. **The region partition (incl. exact `equal == 0` underlap) is a scoring vocabulary only.** No abstraction classifier consumes it (withdrawn, spec §9): no adjacency exists over witness dimensions, so disconnection is meaningless there.
4. **Contiguity is sequential, never dimensional** (spec §2): it is owned by `Mereology.Contiguous` on the derivational axis; $\sigma$ extents over witness dims are single lumps always.

**2026-05-29 delta — binary GrammarLayer reverses take Basis, not W.** `Ops._binary_op_recommend(result, W, op_name, …)` (the mereology-guided recommender that walks `W` rows to find an operand pair such that $\text{op}(x_1, x_2) \approx \textit{result}$) is still keyed on the raw `W` tensor at the low-level kernel signature. But the public `UnionLayer.reverse(parent, basis=None)` / `IntersectionLayer.reverse(parent, basis=None)` now accept a Codebook / Basis object (typically `WholeSpace.subspace.what`) and extract `W = basis.getW()` internally before dispatching to `Ops.disjunctionReverse` / `Ops.conjunctionReverse`. The chart reverse / signal-router dispatch (`bin/Language.py::unreduce()`) passes `basis=space_role_basis` at the call site; no back-ref is stored on the layer.

**Codebook IS the meronymic structure.** The standalone `MereologicalTree` sidecar that formerly stored explicit parent / equality links was retired in favour of pure-geometric parthood on the `WholeSpace` bivector codebook. The grammar layers `PartLayer`, `IsEqualLayer`, `EqualLayer`, and `QueryLayer` operate directly on codebook bivector activations via clipped cosine projection — no separate adjacency table, no `<architecture><mereologicalTreeSize>` XML knob (silently ignored if present). Asserted meronymic relations are learned by training pulling the codebook geometry into the right configuration.

---

## Parthood as Clipped Cosine Projection

Parthood is the single fundamental operation. Every other mereological relation is defined in terms of it.

**Terminology.** Below, $A, B$ are **percept** vectors (dimensionally-embedded codes in the `PartSpace`/`WholeSpace` codebooks); `part`/`whole`/`equal` are geometric relations over those percepts. "concept" is reserved for a `ConceptualSpace` relation tying one part-percept to one whole-percept (by reference), and "symbol" for a 0-D `SymbolSpace` reference to a concept.

For two percept vectors $A, B \in \mathbb{R}^D$:

$$\text{part}(A, B) = \frac{\max(0,\ A \cdot B)}{|A|\,|B|}$$

The clipped cosine projection lies in $[0, 1]$ and satisfies Boole's contrapositive:

$$\text{part}(A, B) = \text{part}(-B, -A)$$

because $(-B) \cdot (-A) = A \cdot B$ and norms are sign-invariant.

**Empty-operand contract (asymmetric).** `part(A, B)` returns 1 when $|A|$ is near zero regardless of $|B|$ — the empty set is part of everything — but returns 0 when $|A|$ is non-empty and $|B|$ is near zero — nothing non-empty is part of the empty set. Empty-vs-empty ($|A| \approx |B| \approx 0$) resolves to 1 (the $|A|$ branch wins the tie). See `Ops._part_kernel` (`bin/Layers.py`).

---

## The Full Suite

Every member of the suite is expressible through `part`. Each `Basis`/`Space` method takes a `scalar` flag (default `scalar=False`, the *vector* form used inside the grammar layers, e.g. $\text{part}(x, y) = x \cdot (y/\|y\|)$ elementwise); `scalar=True` collapses it to the clipped-cosine scalar in $[0, 1]$ tabulated below, which is what every other formula on this page and `ImpenetrableLayer` actually consume (`Basis.part(..., scalar=True)`).

| Method | Formula (`scalar=True`) | Interpretation |
|---|---|---|
| `part(A, B)` | $\max(0, A \cdot B) / ( | A |
| `whole(A, B)` | $\text{part}(B, A)$ | A contains B. |
| `equal(A, B)` | $\text{part}(A, B) \cdot \text{part}(B, A)$ | Mutual parthood, $[0, 1]$. |
| `overlap(A, B)` | $0 < \text{equal}(A, B) < 1$ | Strictly-partial mutual parthood (indicator). |
| `underlap(A, B)` | $\text{equal}(A, B) = 0$ | No mutual parthood (indicator). |
| `boundary(A, B)` | $ | \mathrm{part}(A, B) - \mathrm{part}(B, A) |
| `copart(A, B)` | $1 - \text{part}(A, B)$, clamped to $[0, 1]$ | The part of B *not* accounted for by A. |

**Exact dominance vs. retrieval score.** `part`/`whole`/`equal`/`overlap`/ `underlap`/`boundary`/`copart` above are the graded clipped-cosine *score* (see the Meronomy reconciliation note above, item 1). The boolean dominance trio `Ops.partOf(S1, S2)` / `Ops.wholeOf(S1, S2)` / `Ops.overlapOf(S1, S2)` (`bin/Layers.py`) is the separate exact ground truth: elementwise $\leq$ / $\geq$ reduced over the last dim with `all` (`overlapOf` is the zero-directed elementwise min, `Ops._radmin`) — not a `part`-suite formula, and not what this page's scores approximate at retrieval time.

---

## Region Partition

`equal(A, B)` partitions into three disjoint regions:

| Region | Condition |
|---|---|
| Underlap | $\text{equal} = 0$ |
| Overlap | $0 < \text{equal} < 1$ |
| Identity | $\text{equal} = 1$ |

Under clipped cosine, `part` is symmetric, so the three regions collapse to a single scalar classifier. For a future `Basis` with asymmetric `part`, the general five-relation table applies:

**Five-relation table** (general case, $\tau \approx 1$, $\epsilon \approx 0$):

| Relation | Condition |
|---|---|
| `disjoint(a,b)` | $P[a,b] < \epsilon$ and $P[b,a] < \epsilon$ |
| `part(a,b)` | $P[a,b] > \tau$ and $P[b,a] < \epsilon$ |
| `part(b,a)` | $P[a,b] < \epsilon$ and $P[b,a] > \tau$ |
| `equal(a,b)` | $P[a,b] > \tau$ and $P[b,a] > \tau$ |
| `overlap(a,b)` | Both partial |

**Asymmetric subsumption.** Since `part` is symmetric under clipped cosine, classical asymmetric subsumption is not encoded in raw magnitude. It is recovered **relationally** via figure / ground: compare part$(A, B)$ against part$(A, \neg B)$. This is what makes Boole's contrapositive hold exactly.

---

## Mereological Fusion

Fusion of a truth set is the **least upper bound** (LUB / join) over stored truth vectors:

$$\text{fusion} = \max_i \text{truths}[i]$$

(elementwise max). In bivector space, the fusion vector names the top-right corner of the axis-aligned bounding hyperrectangle dominating every stored truth. Fusion is the geometric dual of luminosity: LUB (join) vs GLB (meet).

DoT is already baked into each stored truth (`stored[i] = activation_i * degree_i` in `TruthLayer.record`), so fusion is trust-weighted automatically.

See Logic.md section **Fusion** for full discussion and the leading-bivector / paired-index layout caveat.

---

## ImpenetrableLayer: Five-Relations Regularizer

`ImpenetrableLayer` regularizes the WholeSpace whole-percept codebook. It classifies each ordered pair of codebook rows $(i, j)$ into one of the five mereological relations (using `Basis.part`), and penalizes partial overlap when paired with a trust mismatch. The learned half of the Codebook Uniqueness Contract.

### Penalty

$$\mathcal{L} = \mathrm{overlap\_weight} \cdot \frac{\sum_{i \neq j} \mathrm{overlap}(i, j) \cdot |\mathrm{trust}(i) - \mathrm{trust}(j)|}{K(K-1)}$$

- $\mathrm{variance\_floor}$ term $$

Overlap strength is damped to zero as the pair approaches `equal`:

$$\text{overlap\_strength}(i,j) = \min(P[i,j],\ P[j,i]) \cdot (1 - \max(P[i,j],\ P[j,i])^k)$$

with $k =$ equal_suppression (default 4.0). This is a continuous formula, not the discrete five-way classification below it: $\min(P[i,j], P[j,i])$ already reads near-zero for `disjoint` and strict `part` (one or both scores near

0), so only `equal` needs an explicit damp — the $(1 - \max(...)^k)$ factor pushes the penalty toward zero as a pair's mutual parthood approaches 1.

A separate `variance_floor` term guards against row collapse.

#### Trust

Per-row usage frequency, derived from VQ EMA counts:

$$\text{trust}(i) = \frac{\text{cluster\_size}[i]}{\sum_j \text{cluster\_size}[j]}$$

When VQ is absent, trust falls back to $\|cb[i]\| / \max_j \|cb[j]\|$.

> **Relative relations and tetralemma trust.** A relative sentence in the `part / isEqual` predicate family is preserved at depth 3 as `[predicate, idea1, idea2]` and may be learned into the codebook as a ternary META edge (predicate as parent, the two ideas as children) via `WholeSpace.insert_relation`. An accepted relation carries a **tetralemma trust 4-tuple** $(t, f, b, n)$ (TRUE / FALSE / BOTH / NEITHER, summing to 1) from the TruthSet posture, gated by the content-aware learn-score against `<truthCriterion>`. See STM.md Section 9.

#### Diagnostics

After `forward()`: `last_overlap_loss` and `last_variance` populate whenever their weight is nonzero. `last_relation_counts` (dict summing to $K(K-1)$) populates only under `MODEL_DEBUG` — the five `.sum().item()` calls it needs are host syncs that would otherwise break CUDA-graph capture.

#### Configuration

XML knobs (under WholeSpace):

| Knob | Default | Meaning |
|---|---|---|
| `impenetrableOverlap` | 0.0 | Weight of overlap × trust-diff penalty |
| `impenetrableVariance` | 0.0 | Minimum variance regularizer weight |
| `impenetrableAntisymmetry` | 0.0 | Legacy antisymmetry weight, still accepted by schema |
| `impenetrableTransitivity` | 0.0 | Legacy transitivity weight, still accepted by schema |

---

## Order-raising (building the meronymic lattice)

Gated behind `<mereologyRaise>` (default off → byte-identical).

> **Terminology note.** The cross-tower link that ties one part-percept to one whole-percept by reference is a **concept** (a `ConceptualSpace` relation); `insert_meta` allocates it into the **Concept codebook** — the `WholeSpace.taxonomy / taxonomy_parent_map / meta_pair_to_idx / meta_trust / part_chain` tables. The earlier `_sym_*` placeholder name no longer exists in code (it survives only in older comments); prose below uses "concept" for the relation.

The two towers stay **in-kind** — `PartSpace` $\sigma$ *composes parts* → *parts*, `WholeSpace` $\pi$ *analyses wholes* → *wholes* — and the **concept (META node) is the cross-tower link**, associated with both an overlapping part-percept and whole-percept (`insert_meta(ps_pos, ws_pos, fused_vec=None, *, ema=0.1, trust=None)`; `ss_pos` is the pre-rename spelling). An object's identity is **isomorphic**: $\sigma$-up meets $\pi$-down at the object. At init there are only **atoms** (part-percepts in PartSpace) and the **universe** (the top whole-percept in WholeSpace); objects emerge from attention.

Three balancing forces keep a single coherent lattice and converge it:

1. **Link the tightest relation** — the largest part-percept ↔ the smallest whole-percept (via `.where` extent); skip a link a bigger-part/smaller-whole already subsumes; drop useless concepts.
2. **Too many parts → synthesize a higher-order part** (`maybe_raise_order`): when a whole accumulates more than `K_many` parts, mint a higher-order PART that subsumes them, with **abstraction order** one above its constituents (tracked via the ramsification table — order 0 = atom, 1 = basic category, …) and explicit `part_chain` provenance (a higher-order part is abstract, so its `.where`/`.when` are discontiguous and the meronymy is tracked, not read off `.where`). Idempotent per whole.
3. **Only one part → Lewis' Singleton → analyse the whole** ($\pi$ divides) or drop the spurious link.

Link surgery: `delete_meta` / `unlink_child` (invert `insert_meta`'s registrations); `ps_children_of_whole` counts a whole's parts. The higher-order code is synthesized by `SigmaLayer.synthesize_over_set` (the M-way generalization of the binary atanh-sum fold) — or, in the "subsymbolic first" phase, the mean-combine of constituents, with `part_chain` as the source of truth. (First pass: the raise fires correctly when a whole has many parts; the live pid-keyed autobind binds 1 percept→1 concept, so similarity-based many-to-one binding + the prune-and-rebind of moot edges are noted follow-ups.)

**The corpus callosum links the towers (part `isa` whole)**

The full mechanism (see the spec): the **corpus callosum** in ConceptualSpace (the learned `[2N,N]` `self.callosum` glue) is what joins the towers into one meronomy. A part-percept `A` (PartSpace) and a whole-percept `B` (WholeSpace) have `.what` codes from *different* codebooks (incomparable), but their **`.where` is comparable** — so the callosum asks whether `A.where` is *part of* `B.where` (a direct span-tuple containment test, `record_cross_tower_meronomy`; see the 2026-07-09 delta below for why this is no longer a `WhereEncoding` decode); when it is (and no greater part / lesser whole intervenes, per codebook activation) it forms the **concept `A isa B`** (token `isa` type; the type *names* the token). Co-occurrence at the same `.where`/`.when` drives the link, weakening when they dissociate — a **Hebbian** coupling between codebooks.

Since the 2026-06-16 redesign `.where` carried an **endpoint-sum bracket `[start, end]`** (angle = span center, magnitude = extent); containment was `A.start >= B.start && A.end <= B.end`, and **contiguity** (adjacency / gap) between sibling parts was the same endpoint comparison. An instant snaps to zero extent, so atoms compare as points. (See `doc/Spaces.md` for the encoding. 2026-07-04: `.when` left the bracket — it is the 4-dim start ladder; temporal extents ride the record store / exact clock, not the band. **2026-07-09:** the muxed `.where` band itself moved to a 2-rung quadrature *ladder* over the byte START only (`WhereEncoding`, `bin/Spaces.py`) — range rung + resolution rung, no END in the band. `WhereEncoding.decode_span` is retired; the endpoint-sum bracket codec survives only on the analyzer side, as `EndpointSumWhere` (`bin/perceptual_analyzer.py`), a distinct codec. The runtime containment test above no longer goes through a `WhereEncoding` decode at all: `WholeSpace.record_cross_tower_meronomy` (`bin/Spaces.py`) and `RunStructureLayer.contained_mask` (`bin/Layers.py`) read `.where` span tuples directly.)

**Design decision (2026-06-29, REVISIT) – the `.where` support of percepts vs symbols.** Every PERCEPT (a part from PartSpace or a whole from WholeSpace) MUST carry a real `.where` (`nWhere > 0`): a definite `[start, end]` extent. A percept with no location is not admissible. A SYMBOL (a concept-level relation entry, or an SS reference) MAY instead either (a) carry `nWhere = 0` (no location of its own), or (b) carry a RECOMBINED `.where` equal to the bounding span of its constituents – `start` = the leftmost constituent's `start`, `end` = the rightmost constituent's `end` (the support of a symbol is the sum of the supports of its members). If the recombination is hard to implement, the fallback is to give every symbol a `.where` of "everywhere" (universal support). THE ANT-COLONY QUESTION: is the location of an ant colony IN each ant (distributed – every ant keeps its own `.where`), or is there a SINGLE location that subsumes all the ants' locations (the bounding region)? The recombined bounding-span (b) takes the single-subsuming-extent view; "everywhere" is its degenerate limit. This choice determines whether a concept's `.where` is distributed over its parts or collapsed to one subsuming extent, and should be revisited.

Word ↔ object cannot be linked this way (too unlike; no convex set is specific enough), so a **second-order**

**meta-object** is synthesized in PartSpace, **outside `.where/.when`**, fusing the word-code and object-code into the symbol used in serial communication (the MetaSymbol). *(Language update 2026-07-02, revised same day by the two-phase rework: in the ramsified CS this is a FIRST-order META-concept – the sec-4c ORDERED PAIR [whole = word-symbol, part = object-symbol], roles as positional slots of an ordered pair rather than containment claims; the typed read-out recovers (word, object) by INTERSECTION with the word-symbol class (`meta_word_object`), and the reference-table pairing remains the serial-mode access path. See doc/Architecture.md sec A.)* Because symbols are outliers, the part↔whole **concepts** live in a **two-code LUT** and the symbolic taxonomy is **relations over symbol indices** — which are the TruthLayer's **absolute truths** (propositions) and **relative truths** (`RelativeTruthStore`), to be integrated. Taxonomy relations are also learned **explicitly from trusted language** ("cats are furry" → `[cats] <= [furry]`).

**Granularity / the part-whole ratio** is the correctness signal (and the MM_20M mean-collapse fix): *many parts → one whole* = under-analysed; *one part → one whole* = over-analysed. An incorrect ratio is the criterion to request **further synthesis ($\sigma$, e.g. radix) or analysis ($\pi$, property tiling) within the problematic `.where`** — until words emerge as parts. MM_20M collapses because its byte chunker never climbs and its analyser never descends, so no word-granularity parts exist for XOR.

**Design decision (2026-06-29, REVISIT; brake updated 2026-07-03) – locking in the "basic level" of analysis.** The mereological analysis should converge on and STABILISE at Eleanor Rosch's BASIC LEVEL – the granularity where the part-whole ratio is correct (neither many-parts-one-whole nor one-part-one-whole). The radix climb builds parts UPWARD (byte -> prefix -> word -> …); left unbounded it over-climbs (a whole short sentence promotes to one percept). Two principled brakes hold it at the basic level: (a) a STRUCTURAL bound – word tiling: a chunk cannot cross a word boundary, where the cut is WHITESPACE AND PUNCTUATION (separator runs – whitespace or punctuation – are themselves maximal same-class tiles that get observed/promoted just like word tiles; see `PartSpace._embed_radix` under `_meronomy_words` and `RadixLayer`'s `word_bounded` promotion gate); and (b) the part-whole-ratio convergence signal above (request more synthesis/analysis until the ratio is right). The ideal is to LOCK IN whatever basic level the corpus implies. As a pragmatic HEDGE – and because **word-level is essentially the basic level for a symbolic LLM** – the analyser cuts on the word tiling (PS observes/promotes per tile so parts top out at word size), which is a hard structural bound rather than a size threshold: the earlier `<basicLevelMinSize>` / `<basicLevelMaxSize>` letter-count bounds were too loose to separate a short sentence from its words on their own, could not express "don't cross a boundary" at all, and are now REMOVED (dead config, never wired to anything – the word tiling supersedes them); the full ratio-driven basic-level convergence remains the to-revisit principled version.

**Design contract – types from properties, word-sized `.where` (Alec, 2026-07-03).** The enabled analysis methods are a set of PROPERTIES identifying the TYPES we attend to, dividing the input (left of space, right of space, punctuation, numbers) into word-sized objects. Each object's `.where` then either (a) ASSOCIATES with a maximal part from PS – a promoted percept covering exactly that span – or (b) is SENT BACK to PS to produce a parts-based definition covering exactly that `.where` (spell-out bounded by the span).

**Explicit concepts ⟷ implicit subsymbolic representations (dual-coded, 2026-06-21)**

> **Terminology note.** This section formerly called the explicit part↔whole relation entry a "symbol." Per the convention, that relation — one part-percept tied to one whole-percept by reference, held in the Concept codebook's `taxonomy` / `taxonomy_parent_map` / `meta_pair_to_idx` / `meta_trust` / `part_chain` tables — is a **concept**. The 0-D `SymbolSpace` **symbol** is a separate, intensional reference *to* such a concept and is not what those entries are. Prose below says "concept"; the code no longer carries a `_sym_*` placeholder (that name survives only in older comments).

The architecture carries **two coordinating structures** for the same content — an **explicit relational** tower (the concepts) and an **implicit subsymbolic** one — and order-raising is what keeps them in correspondence. We have *both*, not one or the other.

- **Explicit (concepts).** A concept is **built directly** (not folded): an index-value relation entry

tying a `PartSpace` part-percept code to a `WholeSpace` whole-percept code, with self-reference (`('sym', id)` members) so concepts nest into the taxonomy. Concepts associate part-percepts and whole-percepts **in virtue of a common whole**, and **identity on insertion is keyed to the exact ordered pair** — `WholeSpace.meta_pair_to_idx[(ps_pos, ws_pos)]` for `insert_meta`, `ConceptAllocator.relate_idx[(part, whole)]` (`bin/Layers.py`) for `relate` — **not** same-part-OR-same-whole: a repeat insert of the identical pair reinforces the existing concept, but a new pairing that merely shares a part or a whole with an existing concept mints a distinct one. `.where` **varies trial-to-trial** and is *not* the identity or trigger key — it keeps only its spatial/meronomy role (extent, containment). The Concept codebook (`taxonomy` / `taxonomy_parent_map` / `meta_pair_to_idx` / `meta_trust` / `part_chain`) is this structure. (This refines the earlier `.where`/co-occurrence framing above: the *trigger* to abstract is **many constituents under a common concept**, not a shared `.where`.) These relations are the INDEX side; the concept's subsymbolic content (the `ConceptDim` atom + its untyped sparse edge decomposition, populated at mint by `_populate_concept_weights`) is the representational side — see the next bullet.

- **Implicit (subsymbolic).** Each concept has a corresponding **learned vector** — a strictly-positive `ConceptDim` **atom** (a feature signature) stored in the CS concept dictionary (`similarity_codebook`, softplus-rectified). For a higher-order concept the *production* is no longer "$\sigma$ then quantize", nor the earlier iterated sparse wave; it is a **feedforward $\sigma$-pyramid** (`cs_forward_content`, bin/Spaces.py; dual-towers rev 2, 2026-07-12): the single untyped square `ConceptualAttentionLayer` computes each rung $k$'s rows in one hop, $a^k = \tanh(W[a^{<k} \mid 1])$, gathered under a per-batch top-K taper (`order_slice(k)` / `_order_caps`), with $K =$ `symbolicOrder` bounding the rung count — **no fixed point and no re-injection** (each rung reads only the rows already settled below it; nothing is fed back into its own input). The concept code is the final rung's activation scaling its positive atom ($a \cdot \text{softplus}(atom)$; radial: magnitude = certainty, sign = present vs anti-present). The many→one abstraction is carried by the sparse WEIGHTS (which sources contribute, with what sign), not by a $\sigma$-fold + VQ snap; the gradient reaches the weights, the source activations, and the dictionary (forward-connected). `PerceptDim` and `ConceptDim` are decoupled — a concept is in a different vector space than its percepts, never a sum of percept vectors.

**Coordination.** Order-raising is the coupling between the two. When the explicit tower raises a higher-order part-percept/whole-percept (built directly, inserted on the **over-collected side** of the higher-order table — too-many-parts → higher-order part, too-many-wholes → higher-order whole), it **creates the corresponding implicit subsymbolic higher-order representation** via the sparse wave above (new untyped edges from the relation's row to its constituents on the shared square store). The explicit concept **indexes / names** the implicit vector; the implicit vector is the concept's **subsymbolic content**. The two co-evolve and must agree: a directly-built concept-index paired with a quantized vector that is a valid point in the higher-order mereological codebook. Direct index + quantized representation are the two complementary halves of one concept.

This is the dual-coded (neuro-symbolic) core: discrete concept **relations over indices** in lock-step with continuous **quantized representations** — neither alone, but the two coordinating.

---

## Why This Design

- **Parthood as projection.** One formula, Boole-contrapositive exact, continuous in $[0, 1]$. The old composite formula conjunction$(1 - \text{dist}(x, x \cap y), 1 - \text{dist}(y, x \cup y))$ mixed set-valued operands with a distance.
- **Overlap penalty (not antisymmetry).** Legacy `ImpenetrableLayer` penalized mutual parthood $P[i, j] \cdot P[j, i]$ directly, pushing *every* overlapping pair apart including synonyms. The five-relations design leaves `equal` pairs alone and only penalizes the ambiguous middle region, gated by trust mismatch.
- **Trust via VQ EMA.** Already tracked for VQ commit loss.

- **Parthood is preserved by Pi / Sigma.** Under `monotonic=true`, Pi/Sigma are restricted to $W \geq 0$. User-supplied truth-set bivectors live on the non-negative paired `[pos, neg]` cone, where componentwise $\leq$ is the parthood partial order. Each lift / lower preserves parthood pole-by-pole for that truth surface. The bivector layout keeps contradiction `[1, 1]` distinct from ignorance `[0, 0]` under positive matmul. See Spaces.md "Monotonicity of the lift / lower chain".

---

## Mereological Algorithm: Meronymic Synthesis and Analysis

> **Main algorithm.** One monotone synthesizer maps parts to wholes and one monotone analyzer maps wholes to parts. Together they build a codebook whose part/whole codes carry a partial order by construction: a concept lattice. The analyzer is the complement-mirror of the synthesizer, so the two are one adjoint design rather than unrelated procedures.

The live configuration surface is:

```
<PartSpace><synthesis>meronomy</synthesis></PartSpace>
<WholeSpace><analysis>meronomy</analysis></WholeSpace>
```

PartSpace owns bottom-up synthesis and WholeSpace owns top-down analysis. The legacy `<chunking>` spelling and PartSpace `analyse` mode are rejected by code. Other synthesis and analysis modes remain available.

### Relation to FCA, LLMs, and DisCoCat

The algorithm is the code-facing form of the Formal Concept Analysis analogy: byte-span containment supplies the extent order, whole/property rows supply the intent side, and learned part/whole links form a fuzzy concept lattice. Unlike an LLM, the order is explicit rather than only an effect of token co-occurrence hidden in weights. DisCoCat enters above this ordered substrate, where typed grammar composes conceptual meanings; it does not define the perceptual order.

### The Adjoint Operators

Parts and wholes form a bounded lattice. The operators start from opposite poles—`NOTHING` ($\bot$, all zero) and `EVERYTHING` ($\top$, all one)—already named in `bin/Layers.py`. (`ATOM` / `UNIVERSE` were the pre-2026-07-02 names; both survive as back-compat aliases.)

| Property | Synthesizer | Analyzer |
|---|---|---|
| direction | parts $\rightarrow$ whole | whole $\rightarrow$ parts |
| pole / start | $\bot$: all zero | $\top$: all one |
| operation | join: $0 \vee p_1 \vee p_2 \vee \cdots$ | meet: $1 \wedge c_1 \wedge c_2 \wedge \cdots$ |
| produces | dominating wholes (suprema) | subordinate parts (infima) |
| monotonicity | increasing: `whole > parts` | decreasing: `part < whole` |
| configuration | `<synthesis>meronomy</synthesis>` | `<analysis>meronomy</analysis>` |

They are exact De Morgan duals under percept complement:

```
analyzer(x) = complement(synthesizer(complement(x)))
complement(x) = 1 - x
```

Implement the join-from-$\bot$ synthesizer; obtain the analyzer by reflection. These are sigma/pi *lattice roles* (union from zero and intersection from one), not the `SigmaLayer`/`PiLayer` butterfly folds. The butterfly folds remain the higher-order abstraction machinery. See Percept Geometry.

### The Order Lives in the Codes

The part/whole codes themselves are coordinatewise monotone on the `[0,1]` presence cube: every whole dominates its parts. That ordered codebook is the mereological embedding. In FCA terms, the monotone adjoints provide the Galois closure; the ordering of the codes is what distinguishes a lattice from an unordered lookup table.

SBOW is deliberately excluded here. It situates free conceptual relations by distributional substitutability, whereas the mereological layer is grounded by span order and nearest-row identity. Moving percept rows with SBOW would fight the closure and could redirect token decode. See SBOW Situates Concepts, Not Percepts.

### The Part Specification: `.where` Containment

For text, parthood is byte-span containment:

```
P is-part-of W  iff  span(P) is contained by span(W)
```

`record_cross_tower_meronomy` and `RunStructureLayer.contained_mask` read the order from `.where` brackets. It is exact and complete for contiguous parts, including a cross-boundary part that the local build did not combine:

```
"abcd" = [0,4]   "ab" = [0,2]   "cd" = [2,4]   "bc" = [1,3]
[1,3] is contained by [0,4]  =>  "bc" is-part-of "abcd"
```

Thus a synthesizer may build `abcd` locally as `ab join cd` and still recover the global constraint involving `bc`. Non-contiguous selections such as `ac` lie outside this poset by design.

### Synthesizer: Build, Then Project

```
synthesize(parts):
    seed  = bottom join part_1 join ... join part_k
    poset = {(whole, part): span(part) is contained by span(whole)}
    code  = project_monotone(seed, poset)
```

The join seed dominates the parts used in the local build, but it may initially violate a cross-boundary constraint. Projection repairs the code against the complete `.where` order. On sparse, bounded-depth grounding, the soft union stays away from the all-one corner. Cross-order abstraction is handled by the separate butterfly folds rather than by repeatedly stacking this join.

### Redistribution as Isotonic Projection

For every containment edge and coordinate, require:

$$\text{code}(W)_i \geq \text{code}(P)_i.$$

The constraints are isotonic and decouple per coordinate. Redistribution is therefore per-coordinate isotonic regression on the `.where` containment DAG: project the codes to the nearest point that satisfies every part/whole edge. The projection is convex and minimum-norm, so it moves only what the order requires. A newly promoted part requires projection only in its `.where` neighborhood.

The differentiable pressure is the Vendrov-style hinge $\sum_i \max(0, P_i - W_i)$. When training content jointly, use projected gradient: the hinge supplies pressure and the projection supplies the hard guarantee. A test construction in which `join(ab, cd)` initially violated the cross-boundary `bc` relation projected to zero violations while remaining in `[0,1]`.

### Analyzer: the 1-x Mirror

The analyzer is meet-from-$\top$, reflected through `1-x`, and produces subordinate parts. It cuts a whole by boundary properties. The first live approximation fixes word boundaries with `OR(space, punctuation)` via `char_class_region([WHITESPACE, PUNCT])`. The fuller design grows a composable, factorable basis of relational properties such as left/right-of-X and splits a newly promoted whole until it reaches the word level.

`InputSpace` supplies both the atomic part view and the unity whole view. `PartSpace` tokenization is selected by `<synthesis>`; WholeSpace division is selected by `<analysis>`. `analysis=raw` preserves the whole byte buffer, `analysis=word` uses the word boundary detector, and `analysis=meronomy` routes the top-down role through the meronymic path.

### Convergence and Supported Modes

With `subsymbolicOrder > 0`, synthesis chunks small parts upward and analysis splits large wholes downward. Each pass feeds the next until the two meet at an operational basic level. The current boundary rule pins that level to words as a first approximation; learning the more general Rosch-style basic level is an open extension.

The adjoint pair can subsume alternate text front ends, but the implementation continues to support the existing synthesis modes (`radix`, `lexicon`, `bpe`, `mphf`, and `byte`) and analysis cuts (`byte`, `word`, `raw`, `sentence`, `grammatical`, and `meronomy`).

### Live Routing and Implementation Boundary

`MeronymicRouter` in `bin/perceptual_analyzer.py` scores candidate merges against the perceptual codebook and routes them with the shared `binary_tiling_viterbi` / `binary_tiling_soft_dp` primitives (see Language.md, *Shared Weighted-Deduction Framework*). In a cold state the codebook contains the whole-input vector and byte vectors, so the router decomposes the surface to byte terminals. As merge promotion learns words bottom-up, the same router reproduces word-level runs. `analyze_routed` preserves the raw bytes of UTF-8 segments and reconstructs a byte-exact surface for replay.

The live router and span bookkeeping establish the path and the complete order specification. **(2026-06-25, LANDED standalone.)** The hard `join-from-bottom` plus isotonic-projection guarantee is implemented in `bin/Mereology.py` (`join_from_bottom`, `meet_from_top`, `project_monotone`, `mereological_synthesize`, `mereological_analyze`) and exercised over projected trees in `bin/Meronomy.py` (`MeronomyTree` and friends), with `test/test_mereology.py`, `test/test_meronomy.py`, and `test/test_meronomy_laws.py` covering both. It is not yet applied to the live percept codebook; the relational analyzer basis and learned convergence criterion are the remaining next-boundary work. The existing butterfly folds and conceptual SBOW remain unaffected.

Implementation references:

- poles, membership folds, and character-class regions: `bin/Layers.py`
- `record_cross_tower_meronomy` and `stage_analysis_spans`: `bin/Spaces.py`
- `.where` containment edges, the join/meet/isotonic-projection primitives, and the mereological synthesizer/analyzer (`where_containment_edges`, `join_from_bottom`, `meet_from_top`, `project_monotone`, `mereological_synthesize`, `mereological_analyze`): `bin/Mereology.py`
- the projected-tree exerciser (`MeronomyTree`, word/phrase synthesis over it): `bin/Meronomy.py`
- presence and complement geometry: Spaces.md
- conceptual situating: conceptual-similarity-space plan

**Same module, different topic.** `bin/Mereology.py` — home to the join/meet/projection primitives above — also carries the unrelated `Mereology` mixin: the contemplative-awareness measure family (`Contiguous`, `Continuous`, `Peaceful`, `Area`, `Luminosity`), mixed into `BaseModel` first in MRO (`class BaseModel(Mereology, nn.Module)`, `bin/Models.py`). See doc/research/three-surfaces.md for that geometry — the two share a file and a name, not an implementation.

#### `isPart` as a Grammar Layer

`IsPartLayer` lifts parthood into the role-collapsed grammar as a binary operator (arity 2). Under a query model it dispatches to `queryPart` (`_dispatch_method_name_for_rule`), so `isPart` and `isEqual` are the relative operators recognized by `_relative_start_categories` (`_RELATIVE_OP_NAMES`). Its identity vector lives in the `WholeSpace` operator codebook and participates in the same soft superposition as the other operators.

# Logic

## Overview

This document defines the logic system at three levels:

1. **Subsymbolic (vector / field level)** — geometry in ConceptualSpace
2. **Symbolic (scalar level in [-1,1])** — order + polarity in WholeSpace
3. **Rationality** — propositional truth store built on both

Executable implementations of the subsymbolic and symbolic operators are the `GrammarLayer` subclasses in `bin/Language.py` and the `Ops.*` kernels in `bin/Layers.py` they dispatch to (see Language.md).

## Relation to LLMs, Formal Concept Analysis, and DisCoCat

The logic layer states what a BasicModel sentence meaning can be trusted to do after it has been composed. In an LLM this kind of inferential behavior is often implicit in next-token priors. Here it is explicit: Formal Concept Analysis gives the order-theoretic reading of concept support and intent, DisCoCat gives the typed composition route that builds sentence meanings, and the truth operators test or accumulate the resulting propositions.

> **Terminology (2026-06-21).** One noun per-space: **percept** = a PartSpace/WholeSpace thing (dimensionally embedded, EXTENSIONAL; *part* = atom / bottom-up $\sigma$, *whole* = property/region / top-down $\pi$ are its two subtypes); **concept** = a ConceptualSpace relation tying one part-percept $\leftrightarrow$ one whole-percept (the Concept codebook, diachronic); **symbol** = a SymbolSpace thing (0-D, INTENSIONAL, references a concept; the symbol codebook). The Section 2 scalar emission is the *symbol* that migrates to SymbolSpace, while WholeSpace itself holds whole-percepts. Code identifiers below are unchanged (a separate code pass renames them).

---

## 1. Subsymbolic Layer

Objects: vector sets $(B, N, D)$ interpreted as RBF / luminosity fields.

#### Operators

- **Union**: `Ops.union` — the saturating RadMax lattice fold (dual to `intersection`), NOT concatenation. The additive combine that briefly held the `union` name was renamed `chunk` (2026-07-05) and moved to `<PartSpace>` as a structural op (`ChunkLayer`: `left + right`, residual-bearing).
- **Intersection**: Co-supported regions (RadMin lattice fold; RBF merge).
- **Negation (affirming)**: `neg(x) = -x`. Antipodal opposition on hypersphere.
- **Non (non-affirming)**: `Ops.non` (the `_non_kernel`; `NonLayer` is the grammar-surface form, which complements the leading bivector poles) — bitonic returns the triangular residual $1 - |\text{clamp}(x, -1, 1)|$ (completing the partition of unity true + false + non = 1); monotonic is $\text{relu}(x - \text{threshold})$ when a threshold is supplied, else zero. (`Basis.non` was removed in the 2026-05-01 Step-9 cleanup; only the `*Reverse` methods remain on `Basis`.)

**Meronomy reconciliation (2026-06-11; MeronomySpec §3 wins).** The sign flip `-x` is licensed only at the REFERENCE space-role — downstairs the sign is occupied (form content) or nonexistent (one-sided percepts); only at the reference space-role is it vacant of denotational duty and free to carry polarity (`a = -1` on A's row = present, certain denial; never negative mass in ground space). `non()`'s withdrawal reading is the prasajya move — lowering `|a|` toward 0 without crossing — gated, never spontaneous. RadMin/RadMax below remain signed-scalar kernels at their own space-role; the odds embedding is retired from the meronymic path (legacy `PiLayer` keeps it for non-meronymic consumers).

- **Parthood** (fundamental): `Basis.part()` / `Ops.part`. The default (vector) form is the elementwise projection of $x$ into $y$'s unit direction:

$$\text{part}(x, y) = x \cdot \frac{y}{\|y\|}$$

The `scalar=True` form is the clipped cosine projection:

$$\text{part}(x, y) = \frac{\max(0,\ x \cdot y)}{\|x\| \cdot \|y\|}$$

(The `monotonic` flag is accepted for signature parity but unused by the part kernel.) The scalar form satisfies Boole's contrapositive trivially (dot product and norms sign-invariant under joint negation). The full suite (`whole`, `equal`, `overlap`, `underlap`, `boundary`) composes through `part`.

---

## 2. Symbolization

**Historical (design note only).** The norm-projection formula below has no implementation: `Basis.symbolize` was removed in the 2026-05-01 Step-9 cleanup (no live callers), and live symbol emission is the WholeSpace codebook snap (the quantize/lookup that names a whole-percept's prototype row). The formula is kept as the original design reading of "symbolization = norm projection".

For $X \in (B, N, D)$:

$$s(X) = 2 \cdot \text{mean}(\|x_i\|) - 1 \quad \in [-1, 1]$$

Interpretation: $+1$ = strong presence, $0$ = neutral, $-1$ = absence.

---

## 3. Symbolic Layer (Scalars in [-1,1])

`Basis` supports **monotonic** (plain min/max over positive-cone values, including user-supplied truth-set bivectors) and **bitonic** (sign-aware). Monotonic forms here; bitonic forms (RadMin, RadMax) in Section 7. See Spaces.md "Monotonicity of the lift / lower chain".

Let $a, b \in [-1, 1]$.

| Operator | Bitonic | Monotonic |
|---|---|---|
| Negation (affirming) | $\text{neg}(a) = -a$ | — |
| Non (non-affirming) | $\operatorname{non}(a) = 1 - a$ | $a$ |
| Union | soft RadMax (LSE-smoothed; Section 7) | $a \cup b = \max(a, b)$ |
| Intersection | soft RadMin (LSE-smoothed; Section 7) | $a \cap b = \min(a, b)$ |

**Parthood as Projection**

Parthood is the **fundamental mereological operation**. The scalar (`scalar=True`) form, for $A, B \in \mathbb{R}^D$:

$$\text{part}(A, B) = \frac{\max(0,\ A \cdot B)}{\|A\| \cdot \|B\|} \quad \in [0, 1]$$

(The default vector form is the elementwise projection $A \cdot (B/\|B\|)$; see Section 1.) Boole's contrapositive $\text{part}(A, B) = \text{part}(-B, -A)$ holds trivially.

The full suite composes through `part`:

| Method | Formula |
|---|---|
| `whole(A, B)` | `part(B, A)` |
| `equal(A, B)` | $\text{part}(A, B) \cdot \text{part}(B, A)$ |
| `overlap(A, B)` | $0 < \text{equal}(A, B) < 1$ |
| `underlap(A, B)` | $\text{equal}(A, B) = 0$ |
| `boundary(A, B)` | \$ |

Under clipped cosine, `part` is symmetric, so `equal` reduces to $\text{part}^2$. Asymmetric classical subsumption is recovered **relationally** via figure/ground: compare $\text{part}(A, B)$ against $\text{part}(A, \neg B)$. See Mereology.md.

For ternary scalar values (monotonic projection):

| part(a,b) | **+1** | **0** | **-1** |
|---|---|---|---|
| **+1** | 1 | 0 | 0 |
| **0** | 1 | 1 | 1 |
| **-1** | 0 | 0 | 1 |

Empty-set conventions (asymmetric): zero is vacuously part of everything ($\text{part}(\emptyset, y) = 1$), but nothing non-empty is part of the empty set ($\text{part}(x, \emptyset) = 0$; $\text{part}(\emptyset, \emptyset) = 1$).

---

## 4. Key Insight

- Subsymbolic = geometry
- Symbolic = order + polarity
- Symbolization = norm projection

Cleanly separates representation (vectors) from logic (scalars).

---

## 5. Rationality

Rationality is the propositional logic layer, built on the grammar rules `part(S, S)` (CS space-role, `PartLayer`) and `isEqual(S, S)` (SS space-role, `IsEqualLayer`).

### Truth Statements

A **truth statement** is any assertion processed through the full pipeline. The `TruthLayer` — owned by the symbol tower, reached as `self.symbolSpace.truth_layer` — stores activations **scaled by** DegreeOfTruth:

$$\text{stored} = \text{activation} \times \text{degree}$$

| Degree | Stored Vector | Effect |
|---|---|---|
| +1 | full activation | attractor |
| 0 < d < +1 | scaled activation | weak attractor |
| 0 | zero vector (inert) | prunable |
| -1 < d < 0 | scaled, negated | weak disperser |
| -1 | negated activation | disperser |

### TruthLayer (symbolSpace.truth_layer)

`TruthLayer` is instantiated in `SymbolSubSpace.__init__` (`SymbolSpace` is a transparent forwarding container over the held `SymbolSubSpace` coordinator, so `symbolSpace.truth_layer` reaches it). `WholeSpace.forward` records activations into the TruthLayer governed by the single continuous `<truthCriterion>` bar (0 = record every activation, 1 = record none): a per-cell activation is recorded when its clamped magnitude clears `truthCriterion`. The same knob also gates learned-concept acceptance in `ConceptualSpace` (relations admitted into the Concept codebook), so one continuous knob governs all truth learning. The binary `<accumulateTruth>` / `<truthMinMagnitude>` switches are **retired**. Recording now fires wherever `WholeSpace.forward` runs — both normal training and the `store_truths` gold-ingestion epoch (which drops `truthCriterion` to `0` to capture every provided gold truth, then restores it). See STM.md Section 9 and Params.md:

- `record(activation, degree)` — store `activation * degree`.
- `query(activation)` — find the closest stored truth; returns `(index, cosine similarity)` of the best match (or `None` below the threshold). The similarity is signed — consonance (+) vs. dissonance (-) — not the stored degree.
- `field(concepts)` — project all stored truths into ConceptualSpace as a scalar field over concept vectors.

**Batch recording (`record_batch` / `compact`).** `record_batch(activations, trust, degree)` stages a batch into a bounded pending buffer with per-entry trust, making no per-cell storage decision inside the compute brick; `compact(min_trust)` — called outside the brick, after forward/backward/step — drops entries below the trust threshold and promotes the survivors to the persistent `truths` buffer (its one host sync per tick stays out of CUDA-graph capture).

**Admission governance.** `conflict_profile(extra=None)` measures the per-dimension contested mass $\min(T_k, F_k)$ over the absolute store, optionally with candidate row(s) added (measured, never stored); `conflict_mass` is its per-dimension max. `admissible(candidate, threshold)` is the commit-point gate: a candidate belief is admissible iff admitting it would keep the corpus's conflict mass at or below the threshold. `preemption_signal(threshold, hysteresis)` is the preattention trigger — it fires when the conflict mass exceeds the threshold and stays fired until the mass falls below threshold − hysteresis (chatter guard); it returns `(mass, fired)`.

**Paraconsistent assessment (`assess`).** `assess()` is the terminal paraconsistent read of the accumulator: `support` / `conflict` / `ignorance` in $[0, 1]$, recovered from the affirming/denying poles of the stored Degrees of Truth (support $= a_P(1-a_N)$; conflict $= a_P R_N + a_N R_P$; ignorance $= (1-\max(a_P, a_N))(1-\max(R_P, R_N))$, where $a$ are mean and $R$ max pole masses). Keeping *conflict* (the set splits on a proposition) distinct from *ignorance* (the set is silent on it) is exactly the degeneracy a scalar $a_P - a_N$ collapse loses.

**Integration with the meronomy.** The absolute truth set (propositions, e.g. `cat <= [animal() & orange() & object()]`) and the `RelativeTruthStore` (relations between two ideas, e.g. `[cats] <= [furry]`) are **the same two structures** the corpus callosum needs for the concept-relation meronomy: a two-code part↔whole LUT (absolute) plus relations over concept indices (relative) — both live in the Concept codebook. These **are integrated**, behind `<ltmConsolidation>`: the unified `TernaryTruthStore` is the canonical store — the part↔whole LUT *is* the absolute table; the concept-taxonomy relations *are* the relative table — and `TruthLayer` becomes a compatibility read model over it (`attach_ltm` binds the store; `sync_from_ltm` rematerializes the view). Taxonomy relations are also learned **explicitly from trusted language** ("cats are furry" → the relation `[cats] <= [furry]`, admitted when trusted).

### Truth Field

$$f(c) = \frac{1}{n} \sum_{i=1}^{n} c \cdot t_i$$

where $t_i = \text{activation}_i \times \text{degree}_i$. Two kinds of regions:

- **Attractors** ($f \to +1$): concept vectors near +DoT truths.
- **Dispersers** ($f \to -1$): concept vectors near -DoT truths.

### Consonance and Dissonance

1. **Internal consonance** — stored truths should be mutually consistent.
2. **External consonance** — incoming statements are evaluated against the truth field.

### Propositional Structure

Parthood and equality are grammar operations over activations:

- `part(S, S)` (CS space-role) — `PartLayer.forward` passes the encompassing parent `right` through unchanged; the parthood relationship between the operands is carried by the codebook geometry (the codebook IS the meronymic tree), not by any output scaling.
- `isEqual(S, S)` (SS space-role) — `IsEqualLayer.forward` is `torch.maximum(left, right)`, the lattice join on the bivector cone; asserted equality shows up as codebook co-location (mutual parthood) learned through training.

Together they define a partial order over symbolic activations.

### Truth Accumulation Pipeline

Truth entries arrive as `(text, DoT)` pairs. `store_truths()` stages all texts on TheData and runs a standard inference epoch via `runEpoch(split="runtime")`. During forward, `WholeSpace.forward()` records each raw symbolic activation (degree 1.0). After the epoch, each activation is scaled by its DoT:

$$t_i = \text{activation}_i \times \text{DoT}_i$$

Two-phase design (encode all, then scale) ensures all truths are encoded in the same pipeline context before DoT modulation.

---

## 6. Consistency and Verification

### Internal Consistency (within a TruthSet)

The design reading is a leave-one-out check: for each $t_j$, evaluate the field produced by the rest of the set at $c_j$:

$$d_j = f_{\setminus j}(c_j) = \frac{1}{n-1} \sum_{i \neq j} c_j \cdot t_i$$

- $d_j > 0$: consonant; $d_j \approx 0$: independent; $d_j < 0$: dissonant.

A disperser (DoT $< 0$) near an attractor (DoT $> 0$) naturally produces negative field values, flagging the contradiction.

The $d_j$ field check as written is **not implemented**. The live machinery is `TruthLayer.consistency()` — cross-truth contradiction detection returning a scalar score (and, with `return_report=True`, the list of contradicting index pairs) — and `TruthLayer._luminosity_without(j)`, the leave-one-out luminosity used as the orthogonalization criterion.

### Verification (incoming statement against stored truth)

The design reading: process the statement, then query the field:

$$v = f(c_a) = \frac{1}{n} \sum_{i=1}^{n} c_a \cdot t_i$$

| $v$ | Interpretation |
|---|---|
| $v \to +1$ | strong support |
| $v \approx 0$ | no opinion |
| $v \to -1$ | strong contradiction |

There is no `verify()` method; the nearest live surfaces are `TruthLayer.query` (nearest stored truth), `TruthLayer.assess` (the paraconsistent support / conflict / ignorance read, Section 5), and `TruthGroundedReasoner.evaluate` (posture over a `QuerySpec`; Reasoning.md). For point lookups, `symbolSpace.truth_layer.query()` returns `(index, cosine similarity)` of the single closest truth — the signed similarity, not the stored degree.

### Logical Entailment (symbolic closure)

Subsymbolic field checks detect *geometric* consistency but not *logical* entailments. Example: `part(A, B) = +1` and `part(B, C) = +1` entail `part(A, C)`, but $A$ and $C$ could be geometrically distant. The symbolic closure layer:

1. **Extract propositions** by decomposing stored SS activations into relation + two operands.
2. **Compute closure** via transitivity:
   - $\text{part}(A, B) \wedge \text{part}(B, C) \Rightarrow \text{part}(A, C)$
   - $\text{equals}(A, B) \wedge \text{equals}(B, C) \Rightarrow \text{equals}(A, C)$
   - $\text{equals}(A, B) \wedge \text{part}(A, C) \Rightarrow \text{part}(B, C)$

   Each entailed proposition inherits DoT = min of the premise DoTs.
3. **Inject entailments** as new symbolic activations recorded in the TruthLayer.

This closure **already exists**: `TruthGroundedReasoner.is_part` (`bin/reasoning.py`) walks beam-limited MIN-trust chains — a chain is only as true as its weakest hop — and `materialize` writes a verified chain's conclusion back as a direct lemma (a `REL_PARTOF` row carrying the MIN-composed trust), so a later identical query is a direct hit. `TruthLayer.derive` performs the pairwise transitivity step and `extrapolate` generalizes it to all two-argument methods, each derived truth recorded with attenuated DoT inherited from its premises. The geometric field alone remains sufficient for soft consistency checks and nearest-truth lookups.

## 7. Radial Operators for Hypersphere Ternary Logic

**Ternary Logic Operators**

+1 = true 0 = unknown −1 = false

**NOT ( neg(a) = −a )**

| a | ¬a |
|---|---|
| +1 | −1 |
| 0 | 0 |
| −1 | +1 |

**NON ( non(a) = 0 )**

| a | non(a) |
|---|---|
| +1 | 0 |
| 0 | 0 |
| −1 | 0 |

**Intersection ( agree → min\|a,b\| )**

| a \ b | +1 | 0 | −1 |
|---|---|---|---|
| +1 | +1 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| −1 | 0 | 0 | −1 |

**Union ( agree → max\|a,b\|; 0 absorbs )**

| a \ b | +1 | 0 | −1 |
|---|---|---|---|
| +1 | +1 | +1 | 0 |
| 0 | +1 | 0 | −1 |
| −1 | 0 | −1 | −1 |

**PART ( mereological, range [0, 1] )**

| a \ b | +1 | 0 | −1 |
|---|---|---|---|
| +1 | +1 | 0 | 0 |
| 0 | +1 | +1 | +1 |
| −1 | 0 | 0 | +1 |

*PART(a,b): 1 when a ⊆ b; 0 is part ofeverything; opposite signs → 0*

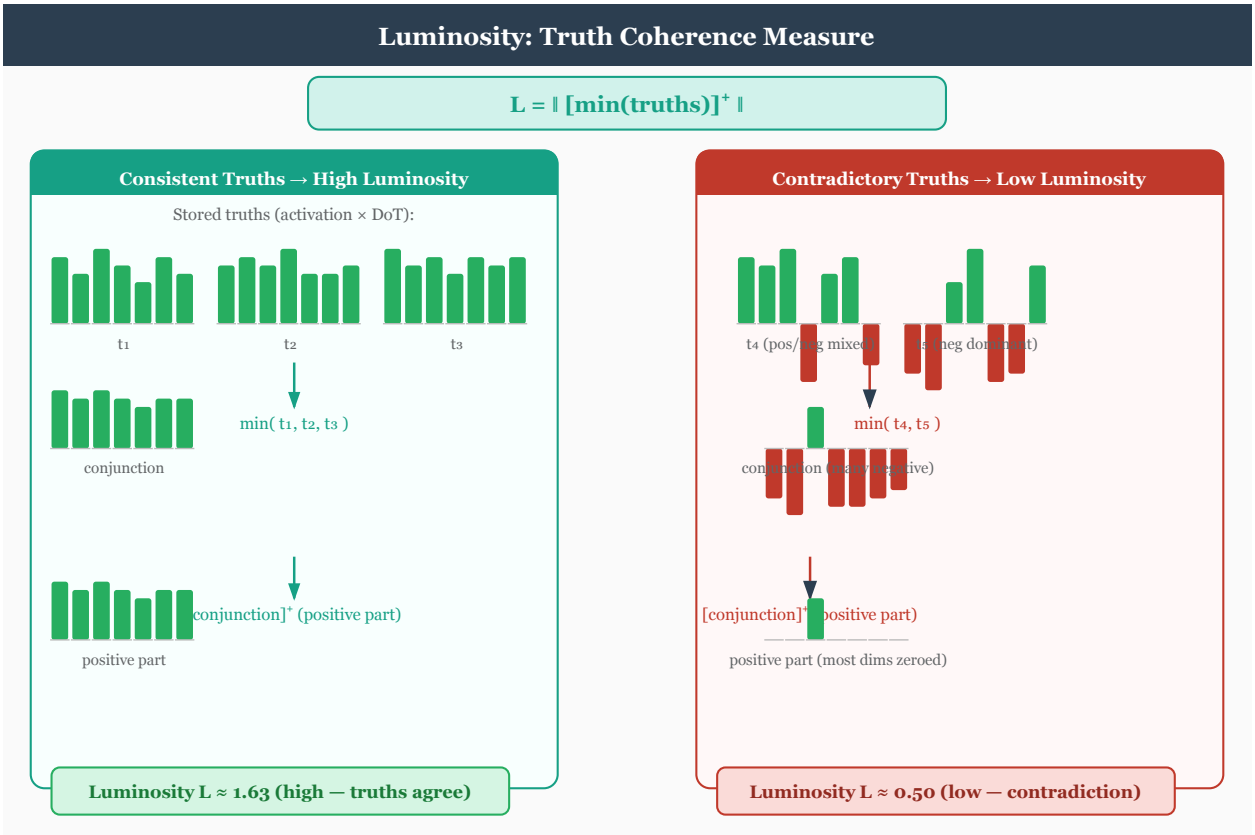
RadMin, RadMax, NOT, and NON operate over signed magnitude space `[-1, +1]`. Sign = direction of assertion; magnitude = strength; zero = **unknown**. +1 = True; 0 = Unknown (no assertion); −1 = False. Magnitude ordering: $x \subset y$ iff $|x| < |y|$.

> **Soft defaults (2026-05-29).** The DEFAULT bitonic kernels behind `Ops.union` / `Ops.intersection` are the LSE-smoothed soft forms (`kind='soft'`: `Ops._soft_radmax` / `Ops._soft_radmin`), so both operands receive softmax-weighted gradient per cell (hard RadMax routed gradient only to the winning operand, zeroing the loser's `.grad`). The hard bodies survive as `kind='radial'`; the table below gives the hard (radial) semantics, which the soft forms approach as $\tau \to 0$.

| Op | Behavior | Ternary truth table |
|---|---|---|
| **RadMin** (AND) | Sign disagree → 0; else $\mathrm{sign}(x) \cdot \min( | x |
| **RadMax** (OR) | Sign disagree → 0; else $\mathrm{sign}(x) \cdot \max( | x |
| **NOT** | $-x$. Involutive; preserves magnitude; NOT(0) = 0. | input → -input |
| **NON** | $1 - | x |

Both RadMin and RadMax are commutative and collapse to zero on sign disagreement.

## Luminosity




**Meronomy reconciliation (2026-06-11; MeronomySpec §3 rev b wins).** There are TWO measures, split as follows. The **truth-set luminosity** — what `Mereology.Luminosity(truth_layer=…)` / `TruthLayer.luminosity` computes — is now the catuṣkoṭi coverage measure over the stored codes: per conceptual dimension the signed references split into true/false pole coverage `(T_k, F_k)` (elementwise max of `relu(±row)`), and

```
luminosity = mean_k[(T_k - F_k) - min(T_k, F_k)]   in [-1, 1]
```

— total area weighted by sign, minus the regions where the sign differs (contradictory evidence). It is computed **on the codes** (no decode pullback) and is order-independent. The meet/GLB form below survives only as the **orthogonalization criterion** (`TruthLayer._luminosity_without`) and `darkness()`'s mirror — it is no longer the truth-set measure. (The diagram shows the meet form.)

The meet-form coherence scalar (orthogonalization criterion):

```
||relu(min(truths))||
```

Element-wise min computes the conjunction (point where all truths agree); relu removes negative dimensions; L2 norm gives brightness.

- **High luminosity**: truths are coherent and mutually reinforcing.
- **Low luminosity**: truths are sparse, contradictory, or incoherent.

Two roles in the model:

1. **Top-down bias** (NOT implemented): the design has concept input scaled by luminosity each conceptual iteration (`concept_input * (1 + truthBiasScale * luminosity)`). The `<truthBiasScale>` knob is parsed (`architecture.truthBiasScale` → `truth_bias_scale`) but

never consumed — a dead knob today.
2. **Loss modification** (live): low luminosity increases training loss via `SymbolSubSpace.truth_modulated_loss`, the multiplicative factor totalLoss $\cdot (1 + w_{lum}(1 - \text{lum}) + w_{univ}(1 - u))$ (see Reasoning.md "TruthLoss").

**Fusion**

Fusion is the **mereological least upper bound** of the stored truth set:

```
fusion = max_i truths[i]
```

In bivector space (paired-index `[p0, n0, p1, n1, ...]`), fusion names the top-right corner of the smallest hyperrectangle dominating every stored truth componentwise. Geometric dual of luminosity:

- **Luminosity** = `||relu(min(truths))||` — GLB / meet, scalar coherence.
- **Fusion** = `max(truths)` — LUB / join, vector coverage.

Trust (DoT) is already baked into stored truths, so fusion is trust-weighted automatically.

**Layout caveat.** TruthLayer's paired-index slicing (`[..., 0::2]` for positive poles, `[..., 1::2]` for negative) assumes a *repeated* bivector layout. The WholeSpace codebook uses a *leading* bivector plus positional trailers: `[pos, neg, where..., when...]`. Callers feeding WholeSpace activations into `luminosity()` must slice the leading 2 dims first (`acts[..., :2]`); see the `SymbolSubSpace.truth_modulated_loss` call-site in `bin/Language.py`.

**Supporting measures**

- **Consistency** (`isConsistent()`): folds truths into a union via successive `Ops.disjunction` (the thin `Basis.disjunction` wrapper was removed in the 2026-05-01 Step-9 cleanup). In bitonic mode, conflicting +/- assertions cancel.
- **Grounding** (`ground()`): finds the minimal subset entailing a query.
- **TruthLoss**: additive penalty for contradictions, measured by union norm reduction. See Reasoning.md.
- **Derive**: pairwise mereological inference via `part()`; generalized by `extrapolate()` to all two-argument methods.

---

## Semantic Summary

| Operator | Type | Role | Behavior on Contradiction |
|---|---|---|---|
| RadMin | Binary | Conjunction (AND) | Collapses to zero |
| RadMax | Binary | Disjunction (OR) | Collapses to zero |
| NOT | Unary | Sign-flipping negation | N/A |
| NON | Unary | Non-affirming negation | N/A |

---

## 8. Cross-references

The level-crossing operations (`lift`, `lower`, `intersection`, `union`), `Pi`/`Sigma` layer ownership, witness-set / slab-lattice geometry, and `Ops.*` namespace inventory have moved out of this document.

- **`Ops.*` reference**: Language.md.
- **PiLayer / SigmaLayer ownership** (`forwardPi`/`reversePi`/`forwardSigma`/ `reverseSigma` aliases): Spaces.md.
- **Mereological suite** (`part`, `whole`, `equal`, `overlap`, `underlap`, `boundary`, `copart`): Mereology.md.

- **Tetralemma bivector layout** (TRUE/FALSE/BOTH/NEITHER corners): Spaces.md and Philosophy.md.

# Reasoning System

> **2026-05-29 delta:** the chart / signal-router reverse path now passes the space-role-local Basis (`subspace.what`) to binary GrammarLayer reverses as `basis=space_role_basis`. The mereology-guided recommender (`Ops._binary_op_recommend`) walks the Codebook's `W` rows to find operand pairs $(x_1, x_2)$ such that $\text{op}(x_1, x_2) \approx \textit{parent}$. Under `<codebook>none</codebook>` on WholeSpace the recommender has no rows to walk and falls back to the lossy `(parent, parent)` pseudo-inverse — degrading the reasoning loop's structural recovery on multi-stage chart parses.

Truth-aware model methods plus the query-reasoning helpers in `bin/reasoning.py`: `QuerySpec`, `TruthGroundedReasoner`, `NeuralToolUser`, and `policy_answer_loss`. Builds on the TruthLayer infrastructure (Logic.md) and grammar composition (Language.md).

## Relation to LLMs, Formal Concept Analysis, and DisCoCat

Reasoning is the point where BasicModel uses explicit structure instead of asking an LLM-style prior to improvise an answer. Formal Concept Analysis contributes the ordered concept support that makes grounding and entailment auditable. DisCoCat contributes the typed composition path that turns phrases and sentences into candidate propositions. The reasoner then checks those propositions against the TruthLayer rather than treating fluent continuation as evidence.

## Partitioned Symbol Space

> **Terminology (percept / concept / symbol).** Throughout this doc "symbol"/"symbolic" denotes the genuine SymbolSpace space-role — the 0-D, non-dimensionally-embedded references emitted as `symbolSum` — not the ConceptualSpace part↔whole relation table (those are *concepts*) and not the dimensionally-embedded perceptual content of PartSpace/WholeSpace (those are *percepts*: part-percepts and whole-percepts).

The symbol dimension is statically partitioned across conceptual orders using geometric decay. Each order writes only to its slice of `symbolSum`, while reading the full vector as feedback.

```
order 0:  [0,      D//2)       <- 1/2 of symbol_dim
order 1:  [D//2,   3D//4)      <- 1/4
order 2:  [3D//4,  7D//8)      <- 1/8
...
last order: remainder of D
```

Makes the symbol partition **self-describing**: position reveals conceptual order. Truth methods use `_activation_order()` to determine a query's order by finding the partition with the highest energy. Partition boundaries are precomputed once at model creation via `BasicModel._order_partitions`.

## Reasoning Methods

`isConsistent() -> dict`

Analyzes the TruthSet for internal consistency by folding all stored truths into a single summary via successive `Ops.disjunction`. In bitonic mode, conflicting +/- assertions on the same dimension cancel to zero. Returns `{'consistent': bool, 'score': float, 'sites': tensor, 'union_vector': tensor}`.

`ground(activation, threshold=0.6) -> dict`

Finds the minimal subset of the TruthSet entailing a query activation. Uses `_activation_order()` to filter truths by partition. Falls back to `TruthLayer.derive()` for indirect derivation. Returns `{'grounded': bool, 'basis': [indices], 'trace': [...], 'confidence': float}`.

`isTrue(activation) -> float`

Grounds a proposition and returns a scalar Degree of Truth in [-1, 1]. Positive = true, negative = false, zero = unknown. Delegates to `ground()`.

`extrapolate(seed_indices, max_new, attenuation) -> dict`

Generalizes `TruthLayer.derive()` to all two-argument grammar methods (union, intersection, `isEqual`, part). For each pair of stored truths, applies every eligible method and accepts results that preserve or increase luminosity. Accepted truths recorded at `attenuation * min(DoT_i, DoT_j)`. Returns `{'added': [indices], 'rejected': [(i, j, rule, delta_lum), ...]}`.

> **Meronomy reconciliation (2026-06-11).** The gate's role is unchanged, but the gated quantity is now the MeronomySpec §3 rev-b measure: `TruthLayer.luminosity` = the catuṣkoṭi coverage measure over the codes, `mean_k[(T_k - F_k) - min(T_k, F_k)]` — signed area minus conflict — order-independent, no decode pullback. The same applies to the multiplicative luminosity modulation under "TruthLoss" below.

## TruthLoss

Additive loss penalty for false propositions, via `<TruthLoss>` in model.xml (default 0.0 = disabled).

Measures the **union norm reduction** when a proposition is included in the TruthSet union via `Ops.disjunction`:

```
truth_union = disjunction(all stored truths)
extended    = disjunction(truth_union, new_proposition)
penalty     = max(0, ||truth_union|| - ||extended||)
```

| Case | Effect |
|---|---|
| Agreeing proposition | Preserves/extends union dims → no penalty |
| Unknown proposition (zero dims) | Passes through → no penalty |
| Contradicting proposition | Cancels conflicting dims → positive penalty |

DoT weighting is implicit: stored vectors carry DoT in magnitude, so contradicting a high-DoT truth causes a larger norm drop.

TruthLoss is **additive** and coexists with the **multiplicative** modulation applied by `SymbolSubSpace.truth_modulated_loss` which carries both the luminosity and the universality term: totalLoss $\cdot\,(1 + w_{lum}(1 - \text{lum}) + w_{univ}(1 - u))$.

## Bidirectional Reasoning Loop

`BasicModel.reason(givens, target, direction, max_steps)`:

- **Forward** (givens → conclusion): Encode givens into TruthSet, extrapolate new truths each step, check `isTrue(target)` until DoT exceeds threshold or `max_steps` is reached.
- **Reverse** (target → grounding): Encode target, call `ground()` to find minimal basis, extrapolate if insufficient.

Luminosity non-decrease is the validity certificate.

## Query Reasoning And Policy Loss

There is no live `grammar_learning_step()` method. Query reasoning is routed through:

1. `QuerySpec`, which normalizes query surfaces (`exist`, `isTrue`, `part`, `isPart`, `equal`, `isEqual`, `queryPart`, `queryEqual`) to `isTrue`, `isPart`, or `isEqual`.
2. `TruthGroundedReasoner`, the exact hard-tool layer over stored truth, parthood, equality, and derived chains.
3. `NeuralToolUser`, the recurrent soft-propose / hard-verify loop for intervening ideas.
4. `policy_answer_loss`, which trains the soft query route while detaching the hard proof mask.

**Step-2 next-idea blend.** `NeuralToolUser.reason_predict_next` (surfaced as `BasicModel.reason_predict_next`) blends the three hard tools `{arma, retrieval, deduction}` into ONE differentiable next-idea $\hat{e}$: the candidate ideas are detached (hard/grounded), and the gradient rides only the generator's query head — plus the optional `NextIdeaScorer`, a learned per-tool logit prior riding the state_dict — through the cosine-softmax blend weights (an absent tool gets a $-\infty$ logit, so its weight is exactly 0). Two `<training>` knobs gate the associated losses, both defaulting to 0.0 (off, byte-identical): `<answerLossWeight>` trains the soft query route via the policy answer loss (NLL on the $[0, 1]$ proof score), and `<predictNextLossWeight>` trains the blend as a next-idea predictor.

`BasicModel.reason(...)` remains as the model-level bidirectional reasoning entry point. Inference query routing is enabled when `reasoningIterations > 0`.

## The Thinking Kernel

`bin/thinking.py` implements the Thinking Kernel's runtime-enforced execution loop over the reasoner's hard tools (gate: `<architecture><thinkingBudget>`; absent/0 = off, byte-identical; positive N = the op budget of a top-level `think()` frame):

1. `TruthInterval` — signed `[lower, upper]` contained in `[-1, 1]` + trust + provenance; `luminosity` is the max-abs distance from unknownness; `status` classifies true / false / unknown / mixed / conflicting against the `tau` bar.
2. `Frame` / STM stack — `think()` pushes, `answer()` pops; only the certified `ChildResult` (value, interval, trust, trace) crosses a frame boundary; scratch is discarded.
3. `ThinkingKernel.execute` — validates each proposed op, charges the shared budget pool, and enforces the closure rules: a true/false answer the frame's evidence does not support is refused (unsupported assertion → unknown); budget exhaustion closes `bounded_unknown`; unknown is a valid terminal. LTM writes happen only inside the runtime: `_materialize_close` (trusted derivation via `reasoner.materialize`, gated `<ltmConsolidation>` — read off the model's `ltm_consolidation` flag; a dead underscored `getattr` that silently kept `materialize` off from `reason_about` / `think_about` was fixed 2026-07-16, so lemma write-back now fires from those entry points when the gate is on) and `incorporate` (testimony above the source×channel trust floor).
4. `KernelPolicy` — the deterministic baseline: `lookup` (LTM-direct, no chaining) → close if luminous → climb `part(·, up)` opening one `think()` subgoal per unvisited whole (soft $\alpha$ ordering via the `InterveningIdeaGenerator` when present — $\alpha$ only orders, never asserts). On an unknown LEAF the policy consults each registered addressee once via `query()` (`arma` built in): NUMERIC testimony folds into the frame interval as `asserted × source_trust` (§14.2 — flimsy testimony cannot manufacture luminosity); tensor testimony is content, never truth; the durable write stays the explicit `incorporate`.
5. `compile_rewards` / `trace_examples` — the §12 reward compilation (per-op $\Delta$luminosity $-$ step_cost plus the terminal on grounded closes; the spec §12.2 trust/relevance factors are NOT applied) and the `(state, op)` supervision exporters for next-operation training.
6. `NextOpPolicy` / `next_op_loss` / `traces_from_store` — §12.6 next-op learning: the head is behavior-cloned on grounded traces generated from the store's 2-hop chains (`<training><thinkingLossWeight>`, runBatch hook, eager build for optimizer membership; the teacher never writes LTM). At inference the head is consulted only at explore-vs-stop choice points over the legal option menu — it can waste budget, never assert.

`BasicModel.think_about(query_spec)` builds the kernel from the model; `answer_query` attaches the kernel's certified result under the payload's `kernel` key when the budget is positive. Tests: `test/test_thinking_kernel.py`.

## Parser And Conceptual Order

Grammar mode is derived from the loaded grammar block. Default-only unary `pi` / `sigma` rules take the fast path; non-default operator rules enable grammar-directed parser dispatch.

`subsymbolicOrder` controls the number of P→C→S stages and the symbol partition geometry. Higher-order symbols write to later partitions, so truth grounding, consistency, and extrapolation can respect conceptual order.

The parser backend is no longer selectable. Stage 3 of the substrate refactor (2026-05-27) retired the CKY chart and STM shift-reduce parsers; the signal router (`LanguageLayer`) is the single canonical parser. The former `SymbolSpace.parserBackend` / `routerKind` knobs (along with `chartTau`, `chartTopK`, `chartNoiseEps`) are RETIRED — setting any of them in a config raises a loud `ValueError` at load time (see `Language._assert_retired_chart_knobs_absent`).

Explicit ordered grammar is preferred:

```
S4 = lift(NP3, VP1)
S5 = lift(NP4, MP1)
```

Here all NPs share base category `NP`; the suffix gives the conceptual order. Lift and lower are the only syntactic operations that change argument/return order.

## Configuration

| Parameter | Location | Default | Description |
|---|---|---|---|
| `<TruthLoss>` | `<training>` | 0.0 | Additive truth-loss weight |
| `<subsymbolicOrder>` | `<architecture>` | 1 | Percept→Concept→Symb |
| `<reasoningIterations>` | `<architecture>` | 1 | Query-reasoning chain dep |
| `<queryReasoning>` | `<architecture>` | false | Deprecated alias; `true` ma |
| `<parserBackend>` | `<SymbolSpace>` | — | **RETIRED** (Stage 3, 202 |
| `truthCriterion` | `<architecture>` / `<ConceptualSpace>` / `<WholeSpace>` | 1.0 | Single continuous truth ba |
| `answerLossWeight` | `<training>` | 0.0 | Policy answer loss weight |
| `predictNextLossWeight` | `<training>` | 0.0 | Step-2 next-idea blend los |
| `intraLossWeight` | `<training>` | 0.1 | In-STM next-idea loss $\mathcal{L}_{\text{in}}$ |
| `interLossWeight` | `<training>` | 0.1 | Inter-sentence next-end-st |
| `routerWireSerial` | `<architecture>` | both | Per-word router-fire gating |
| `ltmCapacity` | `<SymbolSpace>` | 1024 | LTM chain capacity (`Inte` |

The relative-vs-absolute end-state machinery, the content-aware learn-score gate, and the tetralemma trust 4-tuple carried on accepted relative META edges are documented in STM.md Section 9.

## Contemplative Awareness Methods

Four methods on `BaseModel` characterizing stages of contemplative awareness as spatial/computational properties. `Contiguous()`, `Continuous()`, and `Peaceful()` are implemented (each returns a measure in `[-1, +1]`; see `bin/Mereology.py`). `Peaceful()` reads the TruthLayer and returns `valence-symmetry × luminosity-uniformity` (balanced affirming/denying pole masses × uniformly-held per-proposition magnitude; `0.0` when no truths are stored). `Done()` remains a stub that raises `NotImplementedError` — it defines the target characterization, not the implementation.

| Method | Stage | Characterization |
|---|---|---|
| `Contiguous()` | One-Pointedness (Shamatha / FA) | Single connected, convex region in PartSpace; contiguous span in Wh |
| `Continuous()` | Simplicity (Continuity / OA) | Concept states flow continuously; Jacobian of forward map is bounde |
| `Peaceful()` | One Taste (Emotional Symmetry) | TruthLayer luminosity uniformly high across stored propositions |
| `Done()` | Buddhahood (Non-Meditation) | Model is a fixed point of forward-reverse; reconstruction loss zero |

Shamatha Speech is the target grammar mode for `Contiguous()`: complete DNF object grammar plus spatiotemporal contiguity. Every `conjunction` / `disjunction` over object parts must keep `where()` support connected and `when()` support continuous. Differs from serial mode — may reduce over all active percepts at once; rejects scattered object fields, not multi-percept fields. See Philosophy.md.

### Testing

Unit tests in `basicmodel/test/test_reasoning.py` cover all methods without requiring a trained model. English-level tests (syllogisms, contrapositives, semantic equivalence) are `@pytest.mark.xfail` until word identity is learned through training.

## Training

For the current ownership of the training lifecycle and the proposed split between launch resolution, corpus cursors, host orchestration, compiled tensor steps, objectives, and checkpoints, see Runtime Architecture and Componentization.

**2026-05-29 deltas:**

- **Embedding unit-cell wrap.** The training loop calls `normalize()` on the `Embedding` basis (`perceptualSpace.subspace.what`) right after `optimizer.step()` in `bin/Models.py::runBatch`. That `normalize()` is a periodic unit-cell **wrap** into $[-1, 1)$ via `embed._wrap_unit_ball` (torus geometry), *not* an L2 ball projection (`Layers.Lexicon.normalize` does ball-project, but is not on this path). Keeps embedding vectors from drifting out of the unit cell under `JOINT` / `BACKPROP` modes.
- **Seeded retries for MM_xor tests.** `test_learns_xor_signal` and `test_convergence` in `test/test_mm_xor.py` use `for seed in (42, 123, 7): torch.manual_seed(seed); …` (the previously-dead `seed` loop variable in `test_convergence` is now actually consumed). Pass-if-any-attempt-converges semantics.
- **Reconstruction is unconditional from concepts.** The `<reconstruct>` element (and `reconstructEnum`) was RETIRED (A1, 2026-06-09); there is no knob to set or override. Whenever reconstruction fires it is always concepts-seeded from the terminal `ConceptualSpace` STM snapshot.
- **Supervised output loss restored.** `bin/Models.py::runBatch` computes MSE on labeled datasets. Unlabeled corpora such as FineWeb train only through reconstruction/prediction objectives.

### Relation to LLMs, Formal Concept Analysis, and DisCoCat

Training keeps the LLM-like objectives of prediction and reconstruction, but does not make next-token likelihood the whole story. Early objectives shape the embedding and codebook substrate; later objectives train the Formal Concept Analysis-like concept order, the DisCoCat-like grammar composition path, and the truth/reasoning machinery that tests composed meanings. This is why the staged curriculum treats prediction as substrate-building and question answering as a separate deliberate objective.

## Overview

Two phases: **embedding pretraining** and **network training**. Embedding pretraining builds word vectors from a large corpus. Network training uses those vectors as input representation and learns to predict and reconstruct sentences.

The two phases can overlap: `<trainEmbedding>` controls whether and how embeddings continue to evolve during network training.

---

## The staged objective curriculum

Network training climbs five objectives, from substrate-building reconstruction to deliberate question answering. The order is not arbitrary — it is the architecture's **two learning rules** in their natural developmental order:

- **EMA / occurrence** moves the codebooks: concepts form by being *seen*, with no gradient and no goal. This is *prediction as substrate* — unconscious, statistical, **System 1**. You cannot ask "why did the empire fall?" until *empire* and *fall* are codebook rows, and those are EMA-built by exposure.
- **Gradient on a task error** moves the attention readouts and the relation store: the model learns *where to look* and *what relates to what* by being *tested* — directed, credit-assigned, **System 2**.

So "bootstrap with prediction, then answer questions" is just the EMA substrate-builder running first and the gradient deliberate-layer on top (the §6c sentence protocol's *prime subsymbolically from what you see, then pump symbolically*). Reconstruction (stages 1–2) is the **gate**: a question can only be *answered* once an idea can be turned back into words, so the grammar-operation inverses those stages need are the load-bearing prerequisite for the whole curriculum.

| stage | objective | trains |
|---|---|---|
| 1 | reconstruct, **keep syntax**, fill the missing **words** | the lexical/leaf inverse + the codebook (word↔sl |
| 2 | reconstruct with **no syntax / no words** (from the idea alone) | the full generative inverse (deliverable C) + abstr |
| 3 | predict the **next sentence** | the inter-sentence (discourse AR) predictor |
| 4 | answer a question by **reasoning** (no search) | relations, modus ponens (`consequents()`), the ca |
| 5 | answer a question by **LTM search** | global attention (the typed `.where`) + the soft-re |

Plain next-token/next-sentence prediction *under-trains this architecture's* retrieval and relation machinery (the local window usually suffices, so the separate global attention gets no reward); that is why prediction is the **substrate** here, not the goal — stages 4–5 train the deliberate layer, and the stochastic exploration finally earns its keep in stage 5 (search has only a distal reward, so it must explore to break symmetry).

---

## Phase 1: Embedding Pretraining (`make train` / `embed.py train`)

Produces a static embedding artifact (e.g. `BasicModel.kv`). Output path from `<embeddingPath>`. Under `make train` (`bin/train.py`), Phase 1 runs **only** when the artifact is absent or `--force-embeddings` is passed (byte-lexer configs skip it entirely); Phase 2 always runs.

### Pipeline

```
FineWeb-EDU parquet shards
    -> Pass 1: stream documents, lex + parse, count words, build vocabulary
    -> Pass 2: stream documents again, train SBOW per sentence, discard examples
    -> Save WordVectors to sentence.pt
```

### SBOW (Sentence Bag of Words)

The current streaming SBOW trainer uses a `Lexicon` embedding table only: no vocabulary projection head and no full softmax. Given a sentence of $N$ words, it builds a Gaussian-weighted leave-one-out in-group center (`pode`) for each word, samples a same-power random out-group, and trains two attractive terms:

- the word vector is pulled toward its in-sentence pode;
- random out-group vectors are pulled toward the pode's torus antipode.

Implementation: `StreamingSBOWTrainer._train_sentence()` (private) in `bin/embed.py`. `sim(...)` below is `Lexicon.similarity`, the wrapped-MSE torus similarity in $[-1, 1]$ — not a dot product.

```
vecs = embeddings(idx)                # [N, D]
pode = inner_kernel @ vecs             # [N, D], diagonal excluded
antipode = Lexicon.antipode(pode)
loss = -logsigmoid(sim(pode, vecs)).mean()
loss -= logsigmoid(sim(antipode, out_vecs)).mean()
```

### CBOW (Continuous Bag of Words)

CBOW predicts each target word from its context; every in-vocab word in the sentence takes a turn as the target ($N$ examples per sentence), with a leave-one-out padded context mean:

$$\bar{c}_i = \frac{1}{|C_i|} \sum_{j \in C_i} v_j$$

Loss applies the same negative-sampling objective per-word to the padded context mean rather than the leave-one-out centroid:

$$\mathcal{L}_{\text{CBOW}} = -\log \sigma(s(\bar{c}_i, v_{w_i})) - \frac{1}{K} \sum_{k=1}^{K} \log \sigma(-s(\bar{c}_i, v_{w_k^-}))$$

where $s(a, b)$ is the wrapped-MSE torus similarity (`_wrapped_mse_score`, $1 - 2\,\text{mean}(\delta^2_{\text{wrap}})$ in $[-1, 1]$ — not a dot product), $K = 64$ is the negative-sample count, and $w_k^-$ are uniformly sampled words. No full-softmax vocabulary head is involved. Implementation: `PretrainModel.train_step` → `_neg_sampling_loss` in `bin/embed.py`.

### Two-Pass Architecture

- **Pass 1 (vocabulary):** Stream all documents, count every word. Words meeting `min_count` are promoted; the model (a `Lexicon` embedding table only — no vocab-projection head) is allocated once at final vocab size.
- **Pass 2 (training):** Stream documents again. Per sentence, run one SBOW step and discard examples. Multiple epochs re-stream the data.

### Configuration (train.py / environment overrides)

| Variable | Default | Description |
|---|---|---|
| `BASIC_DATASET` | XML default | Dataset/data source selector |
| `BASIC_MAX_DOCS` | XML default | Max documents to process |
| `BASIC_NUM_SHARDS` | XML default | FineWeb-EDU shard count |
| `BASIC_NUM_EPOCHS` | XML default | Phase 2 epoch override |
| `BASIC_BATCH_SIZE` | XML default | Phase 2 batch-size override (`--batch-size`) |
| `BASIC_MAX_TOKENS` | XML default | Token budget |

| Variable | Default | Description |
|---|---|---|
| BASIC_MAX_BATCHES | XML default | Phase 2 batch cap |
| BASIC_RANDOM_SHARDS | 0 | 1 = randomize FineWeb shard order (--rando |
| BASIC_CHECKPOINT_EVERY_BATCHES | XML checkpointEveryBatches (0) | Mid-epoch periodic checkpoint cadence in bat |
| BASIC_RUN_TEST | unset | Enable test passes; optional value caps test ba |

#### Memory Considerations

For SBOW, dominant cost is the Lexicon table plus optimizer state. With 200K vocab and 100 dims:

- Embedding: 200K × 100 × 4 bytes = ~80MB
- Optimizer state depends on the chosen optimizer and device

Streaming SBOW trains per sentence and normalizes the Lexicon after each step.

---

## Phase 2: Network Training (Models.py)

The network learns to predict and reconstruct sentences using pretrained embeddings.

#### Within-sentence training objective (IR-only)

Within-sentence training is **always IR** (masked-LM at the subsymbolic (PS)). create_ir_mask replaces a mask_rate fraction of WHAT positions with NULL_PERCEPT and snapshots the pre-mask event on _ir_pre_mask_input. On the whole-slab / non-grammar path (_per_word_enabled=False) runBatch computes the dense masked-LM MSE(perceptualSpace at masked positions, _ir_pre_mask_input at masked positions) via compute_masked. On the per-word grammar path the reconstruction slot is instead the D3 reverse($S$) reconstruction — there lossIn IS the reverse pipeline. The supervised output-head loss is also back (2026-05-28): the output channel is scored at weight 1.0 whenever labels exist (unlabeled corpora degrade it to zero). Two carve-outs qualify "reconstruction is always concepts-seeded": at train time the D3 path **dedupes** the separate reconstruction_reverse term (lossIn already carries the reverse objective, so the concepts-seeded reverse is skipped to avoid double counting), and at serial EVAL the decode consumes the Method-1 stored-leaves replay (_reverse_method1_leaves) rather than decoding from the concept snapshot. The legacy <maskedPrediction> knob and the AR / ARUS / ARIR modes were retired 2026-05-14; sentence-level AR moved to InterSentenceLayer (see doc/Architecture.md Section "Sentence-level AR (InterSentenceLayer)").

| Knob | Effect |
|---|---|
| maskRate | Bernoulli mask probability at the subsymbolic (PS) (BERT default 0.15) |
| reconstructionScale | Blend weight between output and reconstruction loss; total = (1 - r)*output + r*recon. Leg |
| <reconstruct> | RETIRED (A1, 2026-06-09; reconstructEnum removed). There is no longer a target space-role kn |

#### <trainEmbedding>: Embedding Update Mode

Controls whether and how embeddings evolve during network training. When enabled (CBOW, SBOW, or BOTH), implements EM-like alternation:

- **E-step (network):** Forward + masked prediction + backward + optimizer.
- **M-step (CBOW/SBOW):** Run one embedding update on the same sentence.

| Value | Embedding | Model layers | Description |
|---|---|---|---|
| NONE | Frozen | Trained | Only model layers train |

| Value | Embedding | Model layers | Description |
|---|---|---|---|
| `CBOW` | CBOW (padded context, own optimizer) | Trained | Model layers train separately |
| `SBOW` | SBOW (centroid, own optimizer) | Trained | Faster variant |
| `BACKPROP` | Backprop only | Trained | Codebook trained purely via model loss |
| `BOTH` | SBOW post-batch | Trained | Two optimizers |
| `JOINT` | Single backward | Trained | Single optimizer: combined model + SBOW loss |

### Gradient Flow Through Codebook

`<trainEmbedding>` determines whether the embedding parameters appear in the **main optimizer** — that is the whole freezing mechanism. There is no mode-conditional `detach()` of the codebook weight anywhere: `optimize_embedding = train_embedding not in ("NONE", "CBOW", "SBOW")`, and `getOptimizer` filters embedding params out of the main param list when it is False. Under NONE/CBOW/SBOW gradients may still flow through the lookup, but no main-optimizer step ever moves the rows:

| `trainEmbedding` | In main optimizer | Embedding rows moved by |
|---|---|---|
| `NONE` | No | Nothing (frozen) |
| `CBOW` | No | CBOW pretrainer (own optimizer) |
| `SBOW` | No | SBOW pretrainer (own optimizer) |
| `BACKPROP` | Yes | Model loss only |
| `BOTH` | Yes | Model loss + post-batch SBOW step |
| `JOINT` | Yes | Single combined loss |

- **`NONE`** is recommended for constrained hardware (Apple MPS <16GB).
- **`CBOW/SBOW`** maintains clean EM separation.
- **`BACKPROP`** is the simplest trainable mode. Embedding shaped entirely by the task; best for small vocabularies.
- **`BOTH` risks gradient interference** — reconstruction pulls embeddings apart (distinguishability); SBOW pulls co-occurring words together. These forces can conflict because they use separate optimizers.

### `JOINT`: Single Combined Loss

Avoids EM alternation by computing one combined loss before one backward:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{model}} + \lambda \cdot \mathcal{L}_{\text{SBOW}}$$

where $\lambda$ is `<embeddingScale>` (default 0.1).

SBOW loss uses the same negative-sampling objective, with $s(a, b)$ the wrapped-MSE torus similarity (`_wrapped_mse_score`) rather than a dot product:

$$\mathcal{L}_{\text{SBOW}} = -\frac{1}{N}\sum_{i=1}^{N}\left[\log\sigma(s(c_i, v_{w_i})) + \frac{1}{K}\sum_{k=1}^{K}\log\sigma(-s(c_i, v_{w_k^-}))\right]$$

**Advantages.** One optimizer, one momentum buffer; gradient coherence; no post-batch step.

**Trade-off.** Both objectives share LR and Adam state. If loss surfaces have very different curvature, $\lambda$ may need tuning.

**Why MSE over Embeddings (not Cross-Entropy)**

- "cat" prediction when target is "kitten" incurs less loss than "democracy" — embedding space captures semantic similarity.
- Output dim is the embedding dim (~100) vs. vocab size (~200K).

**Training Loop**

Data flows through `SentenceStreamDataset` wrapped in `DataLoader`. The ordered training list is split into `B = batchSize` contiguous slabs of length `L = len(split) // B`; at step `t`, row `b` is item `b * L + t`. Temporal context is coherent across steps. No per-epoch global shuffle.

For each B-wide batch:

1. `_start_spaces_for_forward()` calls `Start()` on every Space (`runBatch` pre-invokes it before the compiled step; `forward()` self-invokes when not externally started).
2. `InputSpace.forward()` lexes/embeds once into `[B, N, D]` (left-aligned, right-padded to N). No K axis, no cursor unfold.
3. `create_ir_mask` replaces a `mask_rate` fraction of WHAT positions with `NULL_PERCEPT`; pre-mask event stashed on `_ir_pre_mask_input` and the position mask on `_ir_mask_positions`.
4. `_forward_body` runs T stages on B rows.
5. `_forward_head` produces `[B, N, predDim]` (a side channel — IR loss is computed at the subsymbolic (PS), not at the head).
6. `runBatch` computes loss via `TheError.add`:
   - `output` (supervised head): MSE between the aligned head prediction and the labels, weight 1.0 when labels exist (zero-weighted otherwise).
   - `reconstruction` (subsymbolic (PS)): MSE between post-body `perceptualSpace` and `_ir_pre_mask_input` at masked positions (on the per-word grammar path this slot is the D3 reverse($S$) reconstruction instead).
   - `reconstruction_reverse` (concepts-seeded): the reverse pass seeded from the terminal `ConceptualSpace` STM snapshot (the `<reconstruct>` enum was retired), weighted by `reconstructionScale`; skipped at train time when D3 already carries the reverse objective (dedupe).
   - `embedding_sbow` (`JOINT` / byte-lexer perceptual SBOW), weighted by `<embeddingScale>`.
   - `arma` (sentence-level): `InterSentenceLayer.observe(s_t)` MSE between the ARMA(p, q) prediction and the current sentence rep, weighted by `armaScale`.
   - `intra` / `inter` / `inter_contrastive`: the intra-sentence predict-then-perceive term, the inter-sentence end-state term, and the InfoNCE next-idea contrastive term.
   - Optional, default-off (weight 0.0 unless configured): `conceptual_sbow`, `definition_sparsity`, `answer`, `thinking`, `predict_next`, `leaf_distill`, `gate_l1` — plus any auxiliary terms the pipeline Spaces wrote to their shared `Error` instance.
7. One `backward()` + `optimizer.step()` per DataLoader yield.
8. Embedding training (`CBOW`/`SBOW`/`BOTH`) runs once per batch.

---

## SBOW vs CBOW

| Property | CBOW | SBOW |
|---|---|---|
| Targets per sentence | N (every in-vocab word) | N (every word) |
| Context | Padded LOO mean of other in-vocab words | LOO centroid of N-1 |
| Positive updates | N per step | N per step |
| Repulsive force | K=64 random negatives pushed from the context mean | random out-group to t |
| Signal density | High | High |
| Loss | two `-logsigmoid` negative-sampling terms (wrapped-MSE scores) | two `-logsigmoid(sim(` |

| Property | CBOW | SBOW |
|---|---|---|
| Updates embeddings | Yes (own optimizer) | Yes (own optimizer) |
| Used in (`<trainEmbedding>`) | `CBOW` | `SBOW`, `BOTH`, `JOINT` |

Both embedding trainers now make every in-vocab word a target under negative sampling; neither uses a full-softmax projection head. The remaining difference is geometric: CBOW queries a uniform padded context mean and pushes $K = 64$ random negatives away from it, while the streaming SBOW builds a Gaussian-weighted leave-one-out pode and pulls random out-group vectors toward its torus antipode.

---

### Integrated checkpoint architecture

| Artifact | Example | Contents |
|---|---|---|
| XML config | `data/MM_20M_fineweb.xml` | Architecture, objectives, corpus and checkpoint path |
| Checkpoint | `output/MM_20M_fineweb.ckpt` | Model and embedding state, vocabulary/BPE extras, optimizer, counters, |

The integrated checkpoint is resumable. Mid-epoch resume requires the same corpus manifest and batch size; mismatches fail loudly instead of replaying a different cursor. `--force-embeddings` remains a deliberate Phase-1 rebuild, not a separate model artifact contract.

---

### Embedding Space Geometry

`Lexicon` defaults to projective unit-ball lookup in the model, while the streaming SBOW trainer still constructs its training table with `ball=False` for the legacy torus pode/antipode objective. See Lexicon.md for the current geometry and the migration note.

---

### Profiling

`--profile` wraps Phase 2 in `cProfile`:

```
python bin/train.py --model data/BasicModel.xml --profile --max-docs 500
# make train_micro is the capped micro run (no --profile flag of its own;
# invoke bin/train.py directly to add it). make bench_remote profiles the
# recon bench on ArborStudio via torch.profiler, not cProfile.
```

Output: a `.prof` file in `output/profiles/` plus a top-30 summary to stdout. View with `snakeviz`:

```
pip install snakeviz
snakeviz output/profiles/train_20260319_111441.prof
```

For live profiling, `py-spy` can attach by PID (requires root on macOS):

```
pip install py-spy
sudo py-spy record -o profile.svg --pid <PID> --duration 30
```

# Ergodic Exploration

This document describes the current implementation in `bin/Layers.py`. The theory-level motivation belongs in Architecture.md and MachineMinds.md; this page is the operational contract.

## Relation to LLMs, Formal Concept Analysis, and DisCoCat

Ergodic exploration is orthogonal to the LLM/FCA/DisCoCat split, but it supports all three roles. It lets LLM-like predictive training explore without replacing the architecture's explicit codebooks; it lets Formal Concept Analysis-like concept order settle from repeated evidence; and it lets DisCoCat-like grammar composition search among typed reductions without turning the model into an unstructured residual stream.

## Effective Weights

`ErgodicLayer` mixes learned structure with sampled noise through two scalar buffers:

$$W_{\text{eff}} = \text{bias} \cdot W + \text{var} \cdot \varepsilon$$

- `bias` is the learned-weight trust.
- `var` is the exploration-noise scale.
- `bias + var` is approximately 1, with clamps for numerical stability.

The buffers are not trained by Adam. They are updated from observed gradient energy in `paramUpdate()`.

## Initialization

Current initialization is conservative:

- `ErgodicLayer` starts with `bias = 1` and `var = 0`.
- `set_sigma(0.5)` sets the responsiveness parameter, but does not itself make the first forward noisy.
- The first nonzero exploration update happens after a backward pass, when `paramUpdate()` observes gradients.
- `LinearLayer(ergodic=True)` initializes `W` to an identity matrix.
- `InvertibleLinearLayer` initializes its LDU factors to identity (`raw_L = 0`, `d = 1`, `raw_U = 0`).

This supersedes older notes that described alpha-zero startup or zero-initialized ergodic weights.

## Gradient Energy Sensor

`observe_sigma()` scans learnable parameters with gradients, ignores noise buffers, and accumulates mean squared gradient energy. It keeps an EMA-style running variance with:

- `sigma_beta = 0.99`
- `sigma_kappa = 0.01 / requested_sigma` via `set_sigma(sigma)`

`sigma_to_ergodic()` bias-corrects the variance estimate and maps it to the exploration buffer:

$$\text{var} = \frac{s}{s + \kappa}, \qquad \text{bias} = 1 - \text{var}$$

with implementation clamps:

- `var <= 0.95`
- `bias >= 0.05`

Low observed variance therefore favors exploitation; high observed variance increases exploration.

## Update Order

In the training path, the ergodic sensor is updated after `backward()` while gradients are still available:

```
loss.backward()
if model.ergodic:
    model.paramUpdate()
optimizer.step()
```

`paramUpdate()` is a no-op when `ergodic=False`.

## Manual Control

`set_sigma(sigma)` adjusts responsiveness:

- `sigma == 0` sets `sigma_kappa` very high, effectively suppressing exploration.
- `sigma > 0` sets `sigma_kappa = 0.01 / sigma`.

The code uses this as a control surface for deterministic/evaluation paths and for experiments that want more or less exploration.

## Layer Details

### `LinearLayer`

When `ergodic=True`, `LinearLayer` stores:

- `W`: learned identity-initialized weight matrix;
- `noise`: sampled matrix buffer;
- optional `biasWeight` and `biasNoise`.

The forward path applies the same `bias * learned + var * noise` pattern to weights and optional bias.

### `InvertibleLinearLayer`

`InvertibleLinearLayer` represents:

$$W = L \cdot D_{\text{embed}} \cdot U$$

Noise is injected into the LDU factors, not into a materialized dense inverse. The reverse path uses triangular solves, so the inverse remains exact for the effective factors. No SVD path is used.

### `SigmaLayer` and `PiLayer`

`SigmaLayer` and `PiLayer` use `LinearLayer` or `InvertibleLinearLayer` internally, so they inherit the same ergodic behavior. Composite layers forward `set_sigma()` and `paramUpdate()` to their child layers.

## Current Limits

- There is no `global_temp` knob in the current implementation.
- `dropoutRate` exists as a field on `ErgodicLayer`, but the documented exploration mechanism is the `bias`/`var` noise mix.
- The implementation is scalar per layer, not a learned per-neuron temperature trained by Adam.

# Machine Minds

Alec Rogers, September 24, 2025

> **Implementation note.** This essay records the motivating theory and early experiments. The current code's ergodic mechanism is documented in Ergodic.md: layers start with `bias=1`, `var=0`, update those buffers from gradient energy, and do not use an Adam-trained global temperature. Current invertible layers use the factorized implementation described in Architecture.md, not the early SVD sketch.

## Relation to LLMs, Formal Concept Analysis, and DisCoCat

This essay frames the motivation for treating machine minds as architectures, not only as trained LLM weights. In the current BasicModel docs, that motivation has three technical anchors: LLMs provide the comparison class for fluent prediction; Formal Concept Analysis names the extent/intent discipline behind concept order; and DisCoCat names the grammar-to-vector composition discipline behind sentence meaning. The later architecture documents make those anchors operational.


## Abstract

*Problem*

*Society depends on machine minds, but we lack a good theory of them. While human and machine minds functionally overlap, their design goals differ, and treating a machine mind as a human mind has significant consequences. We should have a different theory of mind for machine minds, informed by how those minds are designed and trained.*

*Background*

*Anthropomorphizing machines helps us understand them relative to human minds. This essay focuses on three questions: what are machine minds, what do machine minds feel, and what do machine minds know.*

*Solution*

*Three measures make a machine mind more humanistic: weight ergodicity, network invertibility, and output certainty. Weight ergodicity treats weights as samples from a random process rather than fixed constants. Network invertibility makes the architecture bidirectional and partially invertible. Output certainty means knowing that it does not know, rather than only the probability of one answer versus another. All models tested use a 20-neuron, single hidden layer on MNIST.*

*Invertibility matters for meaningful semantic embedding: an LLM's symbols should correspond to something we can identify, not be abstract tokens inside Searle's Chinese translation engine.*


## Background

Scope: Large Language Models (reinforcement learning with known inputs/outputs and unknown weights — the transformer architecture).

---

## Part I: What Machine Minds Are — Weight Ergodicity

### Weights as Ergodic Samples

Measurement theory views weights as **samples from an ergodic distribution** rather than fixed constants. $W$ at any moment is one realization of a stochastic process:

$$W = \mu M + \sigma V$$

where $\mu$ and $\sigma$ are the bias and variance parameters; $M$ has norm 1 and zero variance (analogous to a typical weight matrix), and $V$ is sampled from a standard normal with unit variance.

This perspective comes from **Lagrangian Field-theoretic Gradient Control** (LFGC), treating the weight landscape as a field where boundary conditions shape the distribution from which weights are drawn. The ergodic hypothesis guarantees that time averages (training steps) equal ensemble averages (weight configurations).

See Rogers, A. (2026), *Local Freedom under Global Constraint*, Zenodo, doi: 10.5281/zenodo.18644302.

### Ergodic State

The historical temperature formulation has been replaced in code by explicit `bias` and `var` buffers. `bias` carries the deterministic path, `var` carries exploration noise, and the gradient-energy sensor shifts mass toward exploration only after a backward/update step.

### Advantages of Ergodic Weights

1. **Exploration without an Adam temperature.** Bias/variance buffers react to observed gradient energy rather than a separately optimized temperature.
2. **Controlled startup.** Current layers start deterministic (`bias=1`, `var=0`) and add exploration after the first update.
3. **Theoretical convergence guarantee.** Simulated-annealing literature provides a guarantee of convergence to global optimum (given infinite iterations).

Standard *dropout* can be interpreted as per-neuron variance in the bias parameter. The same regularization occurs naturally from time-varying weight biases.

---

## Part II: What Machine Minds Feel — Network Invertibility

### Bidirectional Perception

Egocentric viewpoint sees perception as passive; physics suggests it is bidirectional. Approximated via invertibility of weight multiplications and activation functions.

### Symbols, Worldlines, and Karmic Inertia

The symbols an LLM processes have meanings given by **worldlines** — the accumulated history of contexts in which a token has appeared, the gradients that have shaped its embedding, and the network states it has participated in. The meaning of a symbol is the integral of its worldline.

Weight adaptation (or inference) puts a **force** into the system. That force adapts either the system's inputs or outputs depending on which has less **karmic inertia** — resistance to change. The bidirectional architecture means the force propagates in both directions:

- **Forward (encoding):** input representation adapts to match learned categories — perception shaped by expectation.
- **Reverse (decoding):** internal state adapts to reconstruct the input — expectation grounded in perception.

The invertible architecture makes both directions explicit and trainable.

### Implementation

Even when not invertible, bidirectionality plays a role in backpropagation and autoencoders. We examine simultaneously training in both forward (predict output from input) and reverse (predict input from output of the *penultimate* layer) directions — the network is composed of a bijective front (*recognizer*) and a surjective back (*generalizer*). See also Rogers et al. 2001 for related work on sharing weights between policy and critic.

---

## Part III: What Machine Minds Know — Output Certainty

### Certainty vs. Probability

A network's error function corresponds to its desire. Cross-entropy loss *requires* output classification — expressing the probability of one class *versus another*, not the certainty that the input belongs to a given class.

To remedy: blend cross-entropy with a product of cross-entropy and the prediction norm. Zero output is penalized by cross-entropy; incorrect predictions carry additional penalty proportional to magnitude.

**Certainty-Weighted Loss**

$$error = -\sum_{c=1}^{N} t_c \log(p_c)$$

$$certainty = |\hat{y}|$$

$$surprise = (\alpha) \cdot certainty \cdot error + (1 - \alpha) \cdot error$$

For amplitude certainty:

$$error = (\hat{y} - y)^2$$

**Knowing What You Do Not Know**

A network should **know that it does not know**, not merely the probability of one answer vs. another. Cross-entropy forces probability distribution across all known classes, even when "I don't know" is correct. Certainty weighting separates "which class?" from "how sure?"

---

## Methods

Python + PyTorch. MNIST dataset: 60K train, 10K test, 28×28 monochrome at 256 bit depth. Inputs are preprocessed (mean and variance removed) and shuffled.

Input: 28×28 array. Output: one-hot target class. Architecture: 20 hidden unit NN. Batch size 10, 9 epochs, 7 trials for error bars.

Standard benchmark: ReLU, LR 0.01, ADAM, cross-entropy loss.

Historical base ergodic experiment: zero-init weights, single per-layer temperature, ADAM updates temperature, dropout (0.75) on middle layer.

Five models evaluated: base + four ergodic variants (one plain ergodic, one with input normalization, two reversible — one with separate forward/backward layers, one invertible). That experiment used an SVD-form sketch; the current code uses the LDU-style factorized invertible layer documented in Architecture.md.

The historical weight update equations avoided a learning rate. Compared to traditional gradient descent ($\triangle_W = \alpha(y - \hat{y})\nabla_F x$):

$$e = y - \hat{y}$$

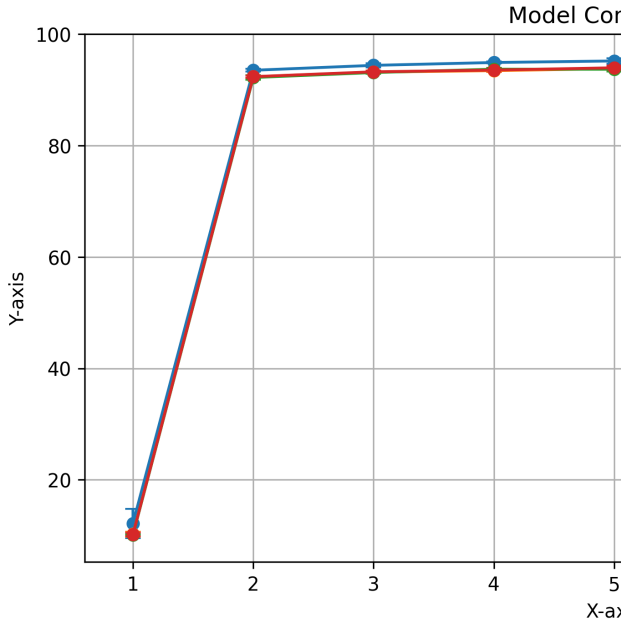
$$c = \alpha\hat{y}^2 + (1 - \alpha)$$

$$\triangle_W = (ce - \alpha\hat{y}e^2)x$$

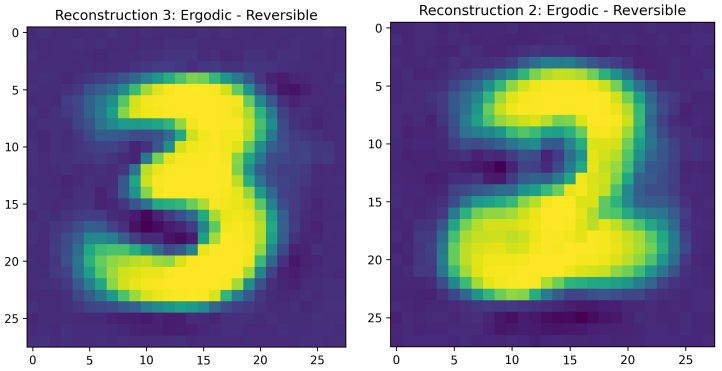
The network simultaneously minimizes error and "incorrect certainty".

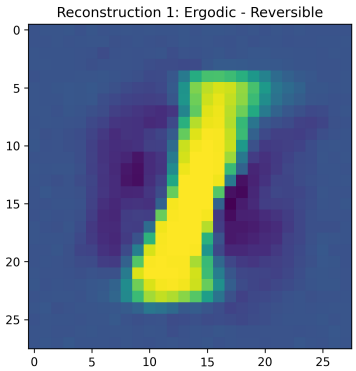
## Results

Model comparison with error bars shows the simple model performs slightly better than all proposed paradigms, with all proposed paradigms performing almost identically:




Reconstruction of input (separate weight matrix, MSE loss):




## Discussion

Benefits:

- Random initialization unnecessary — only the simple model with random weight variation shows significant performance variation.

- Learning rate replaced by weight noise variance (dropout role replaced by bias variance).
- Error function encapsulates output certainty in addition to between-class certainty.

Limitations:

- Pearson correlation between certainty and accuracy is insignificant — the implementation used *certainty* = $p_c$ (softmax output) rather than $|\hat{y}|$, creating contradictory constraints since softmax outputs sum to 1 while the certainty norm imposes another constraint. Corrected code should use norm-weighted MSE.
- PyTorch autograd tunes weights directly, interfering with simultaneous temperature tuning in ways not captured.

### Conclusion

Strong philosophical grounding for several paradigms, but implementation issues limited testing. Effectiveness in multilayer NNs and challenging tasks remains unclear.

### References

Rogers, Shannon, Lendaris; 2001: *A comparison of DHP based antecedent parameter tuning strategies for fuzzy control*, IFSA/NAFIPS, IEEE.

Rogers, A; 2003: *Analysis of the Instantaneous Estimate of Autocorrelation*, unpublished.

Rogers, A; 2026: *Local Freedom under Global Constraint*, Zenodo, doi: 10.5281/zenodo.18644302.

## XML Configuration Parameters

Model configuration is specified in XML files (e.g. `data/XOR_exact.xml`). Default values are loaded from `data/model.xml` and overridden by model-specific configs. The schema is in `data/model.xsd`.

This document is the human-facing reference for common knobs and migrations. `data/model.xsd` is authoritative for the complete accepted element set.

### Relation to LLMs, Formal Concept Analysis, and DisCoCat

Several XML knobs expose the architecture's relation to LLMs, Formal Concept Analysis, and DisCoCat. `dataType=embedding`, sentence prediction, and reconstruction cover the LLM-like prediction/generation path. `subsymbolicOrder`, `symbolicOrder`, `mereologyRaise`, `monotonic`, and the codebook settings control the FCA-like concept order and part/whole lattice. `serial`, `learning`, `transformChooser`, `categoryCodebook`, and grammar-layer settings control the DisCoCat-like typed composition path.

Overlay merge order: `data/model.xml` → `<config>.xml` → runtime overrides.

Note on naming: the schema and the XML configs use `nInput` / `nOutput` (sequence-length sentinels), `nDim` (per-vector dim), `nVectors` (codebook size), and `nInputDim` / `nOutputDim` (raw shape hints for the LiftingLayer). Older docs sometimes call `nOutput` / `nInput nActive`; treat that as the same thing as `nOutput`.

> 2026-05-28: `<nWhere>` / `<nWhen>` are retired from per-file configs. The band is architectural (`canonical_shape`, `bin/architecture.py`): (nWhere=4, nWhen=4) on every interior space (2026-07-09 multi-rung pass), (0, 0) on OutputSpace. The `.where` field is the canonical positional identifier.
>
> 2026-06-16: `.where` is an **endpoint-sum bracket** `[start, end]` (angle = span center, magnitude = extent), the invertible `EndpointSumWhere` form. An instant (`start==end`) is byte-identical to the prior single-quadrature point. **Superseded 2026-07-09**: the endpoint-sum bracket is retired from the muxed band — `.where` is now the same 4-dim 2-rung quadrature

ladder as `.when`, over the byte START only (LF period = `<wherePeriod>`, HF = `wherePeriod / <whereRungRatio>`); the analyzer-side `EndpointSumWhere` keeps the bracket form.

2026-07-04 (encoding pass): `.when` is the **4-dim 2-rung start ladder** (`WhenStartDurationEncoding`; onset only — duration left the band; the exact long-int clock addresses LTM, the Option-C hybrid). New `<architecture>` knobs: `<wherePeriod>` (default 8192 input bytes; `.where` period, decoupled from nObjects, warn-once raise-to-fit), `<whenPeriod>` (default $10^6$ ticks), `<whenRungRatio>` (default 32). Tense is the onset-vs-`now` relation; see doc/Spaces.md.

---

### `<architecture>`

Core model settings. Training and data parameters are in nested sub-elements `<training>` and `<data>` (see below).

| Parameter | Type | Default | Description |
|---|---|---|---|
| `data/dataType` | string | `"numeric"` | The data space-role (was the retired architecture-level `modelType`), s |
| `reconstruct` | — | — | RETIRED (A1, 2026-06-09): the `reconstructEnum` / `<reconstruct` |
| `subsymbolicOrder` | int | `1` | Iteration depth of the P→C→S pipeline. `order > 1` partitions the s |
| `wherePeriod` | int | `8192` | The `.where` quadrature period in input BYTES (2026-07-04 encodin |
| `whenPeriod` | int | `1000000` | The `.when` v2 LF horizon in clock ticks (the LTM event-addressing r |
| `whenRungRatio` | int | `32` | LF/HF period ratio of the `.when` 2-rung start ladder (safe branch bo |
| `processSymbols` | bool | `false` | Apply extra symbolic processing after Sigma. |
| `monotonic` | bool | `false` | Constrain invertible Sigma / Pi to $W \geq 0$ so the lift / lower chain is |
| `ergodic` | bool | `false` | Ergodic exploration: eligible layers use `W_eff = bias * W + var *` |
| `naive` | bool | `false` | Materialise `W_eff` densely in `InvertibleLinearLayer`. Slower; debu |
| `serial` | bool | derived | Forward-dispatch mode. `true` = serial / grammatical (per-word `[B,` |
| `symbolicOrder` | int | `0` | The symbolic ITERATION BUDGET $K$ of the conceptual wave (v3 |
| `subsymbolicNoop` | string | (empty) | Pass indices whose per-pass stack layer is the IDENTITY pass-throu |
| `sparseReplace` | — | — | RETIRED (2026-07-02 P3 two-phase forward): phase separation ma |
| `routerWireSerial` | string | `"both"` | Per-word router-fire gating on the serial path: `per-word` (fire per wo |
| `learning` | bool | `false` | Two-pass soft-superposition training for the grammar chooser. Whe |
| `exploreTemperature` | decimal | `0.5` | Superposition temperature $t \in [0, 1]$ for pass B of `learning`. `0` = the |
| `transformChooser` | string | `"anchordot"` | Placement scorer for the structured grammar layers. `anchordot` = s |
| `reconstructFromIdea` | bool | `false` | Reverse from the idea rather than replaying the forward parse. Whe |
| `categoryCodebook` | bool | `true` | MetaSymbol participation-category codebook for the role-collapsed g |
| `adverbEigEdit` | bool | `false` | Legacy/direct `LiftLayer` helper flag for the adverb sparse eigenvalu |
| `mereologyRaise` | bool | `false` | Mereological order-raising: perception's autobind hook builds a mero |
| `embeddingPath` | string | (empty) | gensim `KeyedVectors` path. Empty disables embedding load. |
| `weightsPath` | string | (empty) | Model weights checkpoint path. Empty falls back to `output/<name>` |
| `maxResponseLength` | int | `4096` | Inference token budget (characters / bytes / tokens). Caps output a |
| `amp` | string | `"off"` | AMP autocast mode: `off`, `fp16`, `bf16`. |
| `truthBiasScale` | decimal | `0.1` | Strength of luminosity-based bias on concept input during forward. |
| `LuminosityWeight` | decimal | `0.1` | Multiplicative loss penalty for incoherent truths. |
| `UniversalityWeight` | decimal | `0.1` | Multiplicative loss penalty for unkind propositions. |
| `allowExcludedMiddle` | int | `1` | Catuskoti policy: `1` permits NEITHER, `-1` enforces classical LEM. |
| `allowContradiction` | int | `0` | Catuskoti policy: `0` forbids BOTH (non-contradiction), `1` permits pa |
| `gateL1Lambda` | float | `0.0` | L1 sparsity penalty on lift / lower gates. `0` disables. |
| `loadBalanceWeight` | float | `0.0` | Sparse-MoE load-balance loss weight. Currently **inert / no-op**: the |
| `l1Lambda` | decimal | `0.0` | Architecture-wide L1 penalty hook. Most configs leave this at `0` and |
| `discontinuityLambda` | decimal | `0.0` | Architecture-wide discontinuity penalty hook (legacy). |
| `conceptualWidth` | string | `"tapered"` | Symbol-partition geometry for `subsymbolicOrder > 1`. `tapered` = |
| `maxActivePerLayer` | int | `8` | Maximum active nodes per layer for reverse / mereological walks. |

| Parameter | Type | Default | Description |
|---|---|---|---|
| `codebookRetire` | bool | `false` | Retire codebook path where supported. Mostly a migration hook. |
| `symbolLearning.enabled` | bool | `false` | Enables runtime symbol-learning hooks. Disabled by default. |
| `mereologicalTreeSize` | int | – | Retired compatibility knob. Values are accepted but ignored; the co |
| `writeSyntax` | bool | `false` | Dump POS-labelled syntax tree at end of every `MentalModel.forwa` |
| `syntaxOutPath` | string | – | Output path for `writeSyntax`. |

Luminosity and universality are **multiplicative** — they scale the total loss by

$$(1 + \mathit{LuminosityWeight} \cdot (1 - \text{luminosity}) + \mathit{UniversalityWeight} \cdot (1 - \text{universality})).$$

`TruthLoss` (in `<training>`, below) is **additive** — it directly penalises propositions that contradict the TruthSet. Both coexist. See Ethics.md Section Universality and Reasoning.md Section TruthLoss for details.

---

**`<architecture><data>`**

Data loading and filtering.

| Parameter | Type | Default | Description |
|---|---|---|---|
| `dataset` | string | `"xor"` | Dataset key: `xor`, `mnist`, `tomatoes`, `text`, `inline`. Omit for inference-only model |
| `numShards` | int | `1` | Shard count (streaming text datasets). |
| `maxDocs` | int | `10000` | Per-shard document cap. |
| `shardDir` | string | (empty) | Text shard directory (e.g. `data/fineweb`). Only used when `dataset=text`. |
| `minFrequency` | float | `0.0` | Vocabulary admission threshold (fraction of corpus). Words below it are held in a |
| `classificationMin` | float | – | **Unparsed**: declared in `data/model.xml` / the schema, read by nothing in `bin/`. |
| `classificationMax` | float | – | **Unparsed**: declared in `data/model.xml` / the schema, read by nothing in `bin/`. |

Inline-mode configs additionally accept `<input use="train|test|validation">...|...|...</input>` and matching `<output>` elements directly inside `<data>`.

---

**`<architecture><training>`**

Training loop and I/O.

| Parameter | Type | Default | Description |
|---|---|---|---|
| `numTrials` | int | `1` | Independent training runs per invocation. |
| `numEpochs` | int | `3` | Epochs per trial. |
| `batchSize` | int | `64` | Contiguous streams through the dataset. Each batch row `b` receives the nex |
| `numWorkers` | int | `0` | DataLoader prefetch workers. `0` = synchronous in-process batch assembly. |
| `learningRate` | float | `0.001` | Adam learning rate. |
| `reconstructionScale` | float | `0.5` | Weight of reconstruction loss vs prediction loss in $[0, 1]$: $\mathcal{L}_{\text{total}} = (1 - r)\,\mathcal{L}_{\text{ou}}$ |
| `whatScale` | float | `0.7` | Loss weight on the `.what` (content) channel. |
| `whereScale` | float | `0.2` | Loss weight on the `.where` (positional) channel. |
| `whenScale` | float | `0.1` | Loss weight on the `.when` (temporal) channel. |
| `negSamples` | int | `64` | Negative samples per positive for CBOW / SBOW. Memory cost: $O(\text{batch} \cdot$ |
| `TruthLoss` | float | `0.0` | Additive penalty for propositions that contradict the TruthSet (union-norm |
| `maskRate` | float | `0.15` | Within-sentence IR Bernoulli mask rate. BERT default 0.15. |

| Parameter | Type | Default | Description |
|---|---|---|---|
| `trainEmbedding` | string | `"NONE"` | Embedding update mode: `NONE`, `CBOW`, `SBOW`, `BACKPROP`, `BOTH`, `JOINT`. See ta |
| `embeddingScale` | float | `0.25` | JOINT-mode embedding loss weight $\lambda$: $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{model}} + \lambda \cdot \mathcal{L}_{\text{emb}}$. Ignored |
| `autoload` | bool | `true` | Load weights from `weightsPath` at construction. Stale incompatible checkp |
| `autosave` | bool | `false` | Save weights after training completes. |
| `checkpointEveryBatches` | int | `0` | Save mid-training checkpoint every N optimizer steps. `0` disables periodic ch |
| `profile` | bool | `false` | cProfile the training loop. |
| `certainty` | bool | `false` | Per-neuron certainty tracking in ergodic layers. Allows individual neurons to |
| `sentencePrediction` | bool | `false` | Enables `InterSentenceLayer`, an ARMA(p, q) predictor over sentence repr |
| `armaScale` | float | `0.1` | Loss weight for the ARMA sentence-prediction MSE. |
| `sentencePrimingScale` | float | `0.05` | AR prediction cast into `concept_dim` and added as a bias to `concept_inpu` |
| `intraLossWeight` | float | `0.1` | Loss weight on the in-STM next-idea term $\mathcal{L}_{\text{intra}} = \text{MSE}(\hat{c}_t, c_t)$ from `IntraS` |
| `interLossWeight` | float | `0.1` | Loss weight on the inter-sentence next-end-state term $\mathcal{L}_{\text{inter}} = \text{MSE}(\hat{p}, p)$ fro |

**`<trainEmbedding>` — Embedding Update Modes**

| Value | Embedding method | Model layers | Gradients through codebook |
|---|---|---|---|
| `NONE` | Frozen | Trained | No |
| `CBOW` | CBOW (padded context) | Trained | No |
| `SBOW` | SBOW (leave-one-out centroid) | Trained | No |
| `BACKPROP` | Backprop only | Trained | Yes |
| `BOTH` | SBOW post-batch | Trained | Yes (two optimizers) |
| `JOINT` | Single backward | Trained | Yes ($\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{model}} + \lambda \cdot \mathcal{L}_{\text{SBOW}}$) |

---

**`<architecture><server>`**

HTTP server settings (used by `serve.py`).

| Parameter | Type | Default | Description |
|---|---|---|---|
| `host` | string | `"127.0.0.1"` | Bind address. |
| `port` | int | `8001` | Port. Override per model to run several in parallel. |
| `timeout` | int | `120` | **Unparsed**: declared in `data/model.xml`, read by nothing — `serve.py` reads only `hos` |

---

## BasicModel / MentalModel Quick Reference

The two configurations that train the full P→C→S pipeline:

```
<architecture>
  <truthBiasScale>0.1</truthBiasScale>
  <LuminosityWeight>0.1</LuminosityWeight>
  <UniversalityWeight>0.1</UniversalityWeight>
  <subsymbolicOrder>1</subsymbolicOrder>
  <monotonic>false</monotonic>
</architecture>

<training>
  <TruthLoss>0.0</TruthLoss>
</training>
```

The grammar section of `MentalModel.xml` declares `lift` as an SS binary rule:

```
<S>lift(S, S)</S>      <!-- lift is binary; SVO is assembled by
                            the chart-compose path S -> S VO,
                            VO -> V O, not a ternary lift -->
```

Transitive SVO is produced compositionally via typed grammar rules such as `S -> S VO` / `VO -> V O`, or via STM `ParseState` extraction for a subject lift over a transitive verb-phrase derivation. Grammar mode is derived from the loaded grammar.

---

## Space sections

Every space block uses the same sentinel convention:

- `nInput = 0` — InputSpace: derive from `TheData`; other spaces: derive from the previous space's `outputShape[0]`.
- `nOutput = 0` — same as `nInput` for this space (passthrough count).
- `nVectors = 0` — same as `nOutput` (used for non-codebook spaces).

Per-Space `<nWhere>` / `<nWhen>` declarations were retired on 2026-05-28 (see top-of-file note); the per-Space defaults live in `data/model.xml` and don't need re-declaration in per-experiment configs.

#### `<InputSpace>`

Lifts raw data into the model's internal representation.

| Parameter | Type | Default | Description |
|---|---|---|---|
| `nOutput` | int | sentinel | Sequence length: max tokens per input. For XOR: 2. For text models, OOV |
| `nDim` | int | `9` | Per-vector muxed EVENT width; content nWhat = nDim − nWhere − nWhe |
| `nInputDim` / `nOutputDim` | int | `0` | Raw shape hints. `0` defers to the loader's native dim. |
| `nVectors` | int | sentinel | Codebook size. Defaults to `nOutput`. |
| `codebook` | mode | `none` | `none`, `quantize`, or `project`. Legacy `true`/`false` are accepted as `quantize`/ |

> **`<lexer>` moved to `<WholeSpace>`** (Phase 4b of the analysis/synthesis dual-input plan, rev. 2026-06-09): lexing is analytic CUTTING, owned by the analysis side. An InputSpace-side `<lexer>` now fails schema validation and the runtime reader. InputSpace emits the dual view — `forward(x) -> (percepts_in, concepts_in)`: the atom view (content `[B, N, 1]`) for the perceptual branch and the unity view (`[B, 1, N]`) for the symbolic branch.

`inputShape` uses the data's native dim (e.g. 784 for MNIST, `inputLength` for text); `outputShape` uses `nDim` from `TheObjectEncoding`. `LiftingLayer` bridges the two.

#### `<PartSpace>`

Transforms lifted input into perceptual features via its synthesis fold — a `SigmaLayer` (additive/union; the Pi/Sigma swap of the analysis/synthesis plan, rev. 2026-06-09: PS is bottom-up SYNTHESIS over atoms; the multiplicative Pi fold moved to WholeSpace as top-down analysis).

| Parameter | Type | Default | Description |
|---|---|---|---|
| `nOutput` | int | sentinel | Number of active perceptual feature vectors. Should be ≥ `InputSpace.nOutput`. Fo |
| `nDim` | int | `9` | Per-vector muxed EVENT width; content nWhat = nDim − nWhere − nWhen (defa |
| `nVectors` | int | sentinel | Codebook size. When `>` `nOutput`, enables vector quantization with top-k selection o |
| `invertible` | bool | `false` | `true`: the PartSpace `SigmaLayer` uses the invertible LDU path for forward/reverse. |
| `hasAttention` | bool | `false` | DEPRECATED and INERT: the legacy boolean no longer constructs a QKV attenti |

| Parameter | Type | Default | Description |
|---|---|---|---|
| `nonlinear` | bool | `true` | Tanh-bound output to $[-1, 1]$. |
| `synthesis` | string | `"lexicon"` | Bottom-up union strategy (renamed from `<chunking>`, which is rejected loudly): `lex` |
| `butterfly` | bool | — | DEPRECATED per-space alias for the architecture-level `<sigmaPi>` enum (`true` → |
| `wordLearning` | int | `2` | Active lexicon-growth mode. `0` = frozen codebook; `>=1` = on first sight of a new wor |

Sigma layer math: $y_j = \tanh(Wx+b)$ (the additive/union fold). The PS `<codebook>` element was retired (the percept prototypes live on the `.what` basis; radix's PerceptStore IS the surface codebook). See Architecture.md and Philosophy.md for the analysis/synthesis orientation.

#### `<ConceptualSpace>`

Transforms perceptual features into abstract concepts via Sigma layers (additive / linear).

| Parameter | Type | Default | Description |
|---|---|---|---|
| `nOutput` | int | sentinel | Active concept vectors. For XOR: 3. |
| `nDim` | int | `9` | Per-vector muxed EVENT width; content nWhat = nDim − nWhere − nWhen (default 9 |
| `nVectors` | int | sentinel | Codebook size. |
| `invertible` | bool | `false` | `true`: `SigmaLayer(invertible=True)` (exact inversion via `atanh` + $W^{-1}$); `false`: separ |
| `hasAttention` | bool | `false` | DEPRECATED and INERT: no longer constructs an attention pass in conceptual proce |
| `nonlinear` | bool | `true` | Tanh-bound output to $[-1, 1]$. |
| `codebook` | mode | `none` | `none`, `quantize`, or `project`. |
| `butterfly` | bool | — | **Silently ignored**: ConceptualSpace never reads a per-space `<butterfly>` — only PartS |
| `stmCapacity` | int | `8` | STM ring depth (within Miller's $7 \pm 2$ band). Per-batch buffer `[B, stmCapacity, nDim` |

Sigma layer math: $y_j = \tanh(Wx + b)$. See Architecture.md. `InvertibleSigmaLayer` has been merged into `SigmaLayer(invertible=True)`.

> `ConceptualSpace` is now a bookkeeping carrier for STM/event state. The former per-stage `sigma_in` / `sigma_cs` layers are retired and no longer constructed on the live path; symbolic generalization lives outside CS.

#### `<WholeSpace>`

Discrete symbolic representation — the information bottleneck between perception and output, and (post the analysis/synthesis plan, rev. 2026-06-09) the home of top-down ANALYSIS: SS owns the Pi fold (multiplicative/intersection), the `<analysis>` division knob, and the `<lexer>` intake knob. See Language.md and Philosophy.md for the design.

| Parameter | Type | Default | Description |
|---|---|---|---|
| `nOutput` | int | sentinel | Total symbols. When reconstruction symbols are enabled, `nOutp` |
| `nDim` | int | `0` | `0` = inherit `ConceptualSpace.nDim` (`symbol_dim` must equal `co` |
| `nVectors` | int | sentinel | Codebook size. When `codebook=quantize`, equals the number o |
| `codebook` | mode | `quantize` | `none`, `quantize`, or `project`. Required for the full symbolic pipe |
| `analysis` | string | `"byte"` | Top-down division strategy over the unity view: `byte`, `word`, `raw` |
| `lexer` | string | — | DEPRECATED spelling: the reader prefers the newer `<analyze` |
| `nonlinear` | bool | `true` | Tanh-bound the activation. |
| `sortNetwork` | bool | `false` | Bitonic-sort regularizer on the codebook ordering. |
| `sortPasses` | int | `0` | Sort-network depth. `0` disables. |
| `l1Lambda` | decimal | `0.01` | L1 sparsity penalty on the symbolic activation. |
| `symbolResidualScale` | decimal | `1.0` | Symbol-residual injection scale (legacy mereology hook). |

| Parameter | Type | Default | Description |
|---|---|---|---|
| `outputSymbolResidualScale` | decimal | `0.0` | Output-side symbol-residual injection scale. |
| `commitmentBeta` | float | `0.25` | Space-level VQ-VAE commitment-loss $\beta$. **Asymmetric dispat** |
| `semanticArrangement` | float | `0` | Task 5 (C-13): post-sentence semantic arrangement over SS heat |
| `decorrelationWeight` | decimal | `0.0` | `ImpenetrableLayer` decorrelation regularizer on codebook rows. |
| `spectralFlatnessWeight` | decimal | `0.0` | `ImpenetrableLayer` spectral-flatness regularizer. |
| `truthCriterion` | unitInterval | `1.0` | **Single continuous truth bar** governing BOTH (a) WholeSpac |
| `trust` | unitInterval | `1.0` | **Architecture-level only** — read as `architecture.trust` (`Mod` |

WholeSpace owns one square invertible `PiLayer` at `self.pi` — the top-down analysis (product/intersection) fold — bridging the C $\leftrightarrow$ S boundary in both directions via its exact inverse (the Pi/Sigma swap, rev. 2026-06-09; SS owned a SigmaLayer at `self.sigma` before the corrected orientation). Consumers use `model.wholeSpace.pi` directly; the grammar rule binding registers it under both the `pi` rule name and the legacy `sigma` alias.

#### `<OutputSpace>`

Maps symbols to final predictions via linear layers.

| Parameter | Type | Default | Description |
|---|---|---|---|
| `nOutput` | int | sentinel | Output values. For XOR: 1. |
| `nDim` | int | `1` | Per-output dim. |
| `nVectors` | int | sentinel | Codebook size. Defaults to `nOutput`. |
| `invertible` | bool | `false` | Rarely set on OutputSpace. |
| `codebook` | mode | `none` | `none`, `quantize`, or `project`. |
| `nonlinear` | bool | `false` | OutputSpace is linear by default; rescales via `Data.denormalize`. |
| *(readout — retired 2026-06-19)* | — | — | The OutputSpace regression head (backlog #11): with `nVectors == 1` |

`LinearLayer` with `(bias, temp)` support for ergodic mode.

---

### `<SymbolSpace>`

Grammar infrastructure (SyntacticLayers, TruthLayer). Lives outside `<architecture>` as a top-level sibling.

| Parameter | Type | Default | Description |
|---|---|---|---|
| `syntacticHiddenDim` | int | `64` | SyntacticLayer MLP hidden size. |
| `truthMaxEntries` | int | `1024` | TruthLayer SS-codebook capacity. |
| `ltmCapacity` | int | `1024` | Long-term-memory (LTM) chain capacity on `InterSentenceLayer`: the pe |
| `parserBackend` | — | — | RETIRED (Stage 3 cleanup, 2026-05-27): the chart backend is gone and th |
| `routerKind` | — | — | RETIRED (Stage 3 cleanup, 2026-05-27): there is no longer a selectable ro |
| `chartCollapse` | string | `"root"` | **Unparsed**: declared in `data/model.xml` / the schema, read by nothing in |
| `chartTau` | — | — | RETIRED (Stage 3 cleanup, 2026-05-27) alongside the chart. Rejected wit |
| `wMax` | — | — | RETIRED: the legacy STM-capacity alias is no longer read. STM depth co |
| `chartTopK` | — | — | RETIRED (Stage 3 cleanup, 2026-05-27) alongside the chart. Rejected wit |
| `chartNoiseEps` | — | — | RETIRED (Stage 3 cleanup, 2026-05-27) alongside the chart. Rejected wit |
| `iterationsPerWord` | int | `1` | **Unparsed**: declared in `data/model.xml` / the schema, read by nothing in |
| `downwardGeneration` | bool | `false` | Emit a codebook head via `SymbolSpace.reconstruct()` and stash on `sel` |
| `language.interpretation` | float | `0.5` | Soft-interpolation weight between forward and generation directions. |

The <language><grammar> block contains space-role-scoped rules in <compose> and <generate> sections, each holding <percepts> / <concepts> / <symbols> sub-blocks of <rule> elements. Per the 2026-05-07 rollback, the grammar XML is the sole source of truth for which rule fires at each space-role (no code-level default_rule fallback). Legacy bare-space-role elements like <P>P = sigma(P)</P> directly under <grammar> are still accepted.

---

## Additional parsed knobs (2026-07 audit)

Knobs that are parsed and live in bin/ but were previously undocumented here. Every default below was verified against its read-site; data/model.xsd carries the full schema commentary. Site names are module + owning symbol (line anchors drift).

**<architecture> level**

| Knob | Where read |
|---|---|
| meronomy (attr dMaxStable) | Layers.py (meronomy_enabled / meronomy_d_max_stable |
| sigmaPi | Models.py (BaseModel init) + Spaces.py (PS/WS fold bu |
| whereRungRatio | Spaces.py (WhereEncoding construction) |
| syntacticOrder | Models.py (BaseModel init) |
| sentenceProtocol | Models.py (BaseModel init) |
| truthSet (<truth> rows: text, trust / kind attrs) | Models.py (provision_ltm) |
| thinkingBudget | Models.py (BaseModel init) |
| reasoningIterations | Models.py (BaseModel init) |
| queryReasoning | Models.py (BaseModel init) |
| ltmConsolidation | Models.py, Language.py (SymbolSubSpace) |
| stateless | Models.py, Language.py |
| globalAttention | Models.py (BaseModel init) |
| globalAttentionConsume | Models.py (BaseModel init) |
| readingAttention | Models.py (BaseModel init) |
| relevance | Models.py (BaseModel init) |
| primingDecay | Models.py (BaseModel init) |
| primingSpread | Models.py (BaseModel init) |
| stmReduceTau | Models.py (BaseModel init) |
| radialStmReduce | Models.py (_create_per_stage) |
| symbolicPriming | Language.py (attach_knowledge → configure_priming) |
| symbolTower | Models.py (BaseModel init) |
| serialObjectMeta | Models.py (BaseModel init) |
| conceptIndexRead | Models.py (BaseModel init) |
| ideaDecode | Models.py (BaseModel init) |
| verbSpectrum | Language.py (LiftLayer init) |
| prediction | Models.py (BaseModel init) |
| predictionTrialRatio | Models.py (ModelLoss wiring) |
| overlapWhereTiling | Models.py (BaseModel init) |
| continuityNorm | Mereology.py (contemplative mixin) |
| continuityRatioTarget | Mereology.py |
| continuitySharpness | Mereology.py |
| TetralemmaPolicy block (+ per-space tetralemmaOverride) | util.py (tetralemma_policy) |

**<architecture><training> level**

| Knob | Where read | Default | Purpose |
|---|---|---|---|
| `seed` | `Models.py` (run entry; env `BASIC_SEED` overrides) | unset | RNG pin (torch/pytho |
| `answerLossWeight` | `Models.py` (BaseModel init) | `0.0` | Reasoner answer loss w |
| `predictNextLossWeight` | `Models.py` (BaseModel init) | `0.0` | Next-idea blend loss w |
| `thinkingLossWeight` | `Models.py` (BaseModel init) | `0.0` | Thinking-kernel loss w |
| `leafDistillWeight` | `Models.py` (BaseModel init) | `0.0` | Leaf-distillation loss w |
| `interContrastiveWeight` | `Models.py` (ModelLoss), `Language.py` (discourse layer) | `0.0` | InfoNCE next-idea con |
| `interContrastiveTemp` | same | `0.1` | InfoNCE temperature. |
| `conceptualSimilarityScale` | `Models.py` (ModelLoss wiring) | `0.0` | SBOW training weight |

**Per-space**

| Knob (space) | Where read | Default |
|---|---|---|
| `attention` (PartSpace / ConceptualSpace / WholeSpace) | `Spaces.py` (each space init) | `off` |
| `attentionPromotion` (ConceptualSpace) | `Spaces.py` (CS init) | `false` |
| `wordStore` (PartSpace) | `Spaces.py`, `Language.py` | `false` |
| `chunkPromotionThreshold` (PartSpace) | `Spaces.py` (PS init) | `4` |
| `chunkPromotionMinLength` (PartSpace) | `Spaces.py` (PS init) | `2` |
| `demuxed` (InputSpace) | `Models.py` (`_make_perceptual_space`) | `false` |
| `lrScale` (OutputSpace) | `Models.py` (`getOptimizer`) | `1.0` |
| `definitionFreeSize` (ConceptualSpace) | `Spaces.py` (CS init) | `2` |
| `lbgThreshold` (WholeSpace) | `Spaces.py` (meta codebook) | `0.5` |
| `lbgMinCount` (WholeSpace) | same | `8` |
| `lbgEpsilon` (WholeSpace) | same | `0.1` |
| `divideWithinWhole` (WholeSpace) | `Spaces.py` (WS init + tiling site) | `true` |
| `gradientMode` (WholeSpace) | `Spaces.py` (WS init) | `"ste"` |
| `useStackRouter` (WholeSpace) | `Spaces.py` (WS init) | `false` |
| `initScale` (any space) | `Spaces.py` (`_read_init_scale`) | unset |
| `impenetrableOverlap` (WholeSpace) | `Spaces.py` (WS init) | `0.0` |
| `impenetrableVariance` (WholeSpace) | `Spaces.py` (WS init) | `0.0` |
| `commitmentDecay` (PS/CS/WS) | `Models.py` (VQ override loop) | unset (→ VQ default |
| `codebookEmaDeadThreshold` (PS/CS/WS) | same | unset (→ VQ default |
| `codebookGrowthEpsilon` (PS/CS/WS) | same | `0.0` |

**`<SymbolSpace>` level**

| Knob | Where read | Default | Purpose |
|---|---|---|---|
| `armaP` | `Language.py` (InterSentenceLayer build) | `5` | ARMA AR order for inter-sentence prec |
| `armaQ` | same | `2` | ARMA MA order. |
| `armaHiddenDim` | same | unset (auto) | ARMA predictor hidden width. |
| `signal.temperature` | `Language.py` (signal-router build) | `1.0` | Signal-router softmax temperature. |
| `language.useGrammar` | `Language.py` (grammar load) | — | DEPRECATED: still read, but only to |
| `language.start` | `Language.py` (grammar load) | `"S"` | Accepted completed-derivation shapes ( |

**Config-only orphan**: `useSubspaceActivation` appears in the schema and in test fixtures (`test_basicmodel.py`) but is parsed nowhere in `bin/` — setting it in a config has no effect.

### `InvertibleLinearLayer` (programmatic)

The LDU-factorised invertible linear primitive used internally by `SigmaLayer(invertible=True)` and `PiLayer(invertible=True)`. Not set via XML directly.

| Parameter | Type | Default | Description |
|---|---|---|---|
| `naive` | bool | `false` | `false`: apply L, D, U sequentially via triangular solves (no $W$ materialisation). `true`: mater |
| `stable` | bool | `false` | Clamps each diagonal entry $d_i$ to magnitude $[\epsilon, 1]$ with sign preserved before the ergodic ble |
| `ergodic` | bool | `false` | Factor-level noise injection. Registers `noise_raw_L`, `noise_raw_U`, `noise_d` buffers; resample |

**Class renames and removals.**

| Old name | New name / status |
|---|---|
| `LULayer` | Renamed to `InvertibleLinearLayer` |
| `InvertibleLinearLayer` (SVD-based) | Removed; replaced by LDU factorisation |
| `InvertibleSigmaLayer` | Merged into `SigmaLayer(invertible=True)` |
| `InvertiblePiLayer` | Merged into `PiLayer(invertible=True)` |

See Architecture.md for the LDU factorisation and factor-level noise injection.

---

## Example: XOR Configuration

```
<model>
  <architecture>
    <data><dataType>embedding</dataType></data>

    <training>
      <numEpochs>400</numEpochs>
      <learningRate>0.01</learningRate>
      <autoload>false</autoload>
      <trainEmbedding>JOINT</trainEmbedding>
      <embeddingScale>0.05</embeddingScale>
    </training>
  </architecture>

  <InputSpace>
    <nVectors>8</nVectors>
    <nDim>10</nDim>
    <nOutput>8</nOutput>
    <nOutputDim>10</nOutputDim>
  </InputSpace>

  <PartSpace>
    <nInput>8</nInput>
    <nVectors>1000</nVectors>
    <nDim>10</nDim>
    <nOutput>8</nOutput>
    <hasAttention>false</hasAttention>
    <invertible>true</invertible>
    <synthesis>lexicon</synthesis>
```

```
  </PartSpace>

  <ConceptualSpace>
    <nInput>8</nInput>
    <nVectors>8</nVectors>
    <nDim>10</nDim>
    <nOutput>8</nOutput>
    <invertible>true</invertible>
    <codebook>quantize</codebook>
  </ConceptualSpace>

  <WholeSpace>
    <nInput>8</nInput>
    <nVectors>8</nVectors>
    <nDim>2</nDim>
    <nOutput>8</nOutput>
    <codebook>quantize</codebook>
  </WholeSpace>

  <OutputSpace>
    <nOutput>1</nOutput>
    <nVectors>1</nVectors>
  </OutputSpace>
</model>
```

Everything not listed inherits from `data/model.xml`.

---

### Example: Embedding / Chatbot Configuration

```
<model>
  <architecture>
    <data>
      <dataType>embedding</dataType>
      <shardDir>data/fineweb</shardDir>
      <minFrequency>0.00001</minFrequency>
    </data>
    <weightsPath>BasicModel.ckpt</weightsPath>
    <embeddingPath>BasicModel.kv</embeddingPath>
    <training>
      <numEpochs>2</numEpochs>
      <batchSize>128</batchSize>
      <reconstructionScale>0.1</reconstructionScale>
      <trainEmbedding>BACKPROP</trainEmbedding>
      <autosave>true</autosave>
      <checkpointEveryBatches>500</checkpointEveryBatches>
      <sentencePrediction>true</sentencePrediction>
      <armaScale>0.1</armaScale>
    </training>
  </architecture>

  <!-- space definitions inherit from model.xml -->
</model>
```

Key points:

- `<data><dataType>embedding</dataType>` activates the embedding-based input pipeline alongside the neural model.
- `trainEmbedding=BACKPROP` trains model layers with gradients flowing through the codebook.
- XML config defines the architecture. `weightsPath` stores the neural model checkpoint.
- `minFrequency` gates vocabulary admission; words below it are buffered until they exceed the threshold.